COMPUTER SCIENCE · VOLUME ONE

Python *Programming*

A BEGINNER TEXTBOOK FOR SCHOOL STUDENTS

```python
# Your journey starts here
def greet(name):
    print(f"Hello, {name}!")
    return "Ready to learn Python"
```

Twelve chapters. 180 mixed-quiz questions. Four mini-projects. One final project. A complete foundation in programming — from your first print() statement to building a working quiz game.

STUDENT EDITION · 2026

>>>

About This Book

Licence, support, and feedback

Free to read

Python Programming is a free book. You never need to pay to read it, print it, or hand it to a friend, a classroom, or a study group.

Licence

© 2026 Mohamed Ahmed Fayad. This work is licensed under the Creative Commons Attribution-NonCommercial 4.0 International licence (CC BY-NC 4.0). You are welcome to share and adapt it for any non-commercial purpose, as long as you give credit and say if you made changes. Full licence text: creativecommons.org/licenses/by-nc/4.0

Support the next book

> If this book helped you learn, a small donation helps keep it updated and pays for more free lessons. Donating is always optional.
>
> Donate at: **ko-fi.com/1to10**

Found a mistake?

> No book is perfect. If you spot a typo, an example that does not run, or a question with the wrong answer, please tell us so the next version can fix it.
>
> Report it here: **moh.fayad14@gmail.com**

Edition

Student Edition · Version 1.0 · October 2026. A newer version may be available where you downloaded this book.

The examples in this book are written for Python 3.10 and later and run in IDLE or any Python editor. This book is an independent educational resource and is not affiliated with or endorsed by any company or product named in it.

Table of Contents

How to Use This Book 7

The Structure of Each Lesson . . . 7
Mini-Projects . . . 7
How to Study . . . 8
What You Will Build . . . 8

Introduction to Python 9

Lesson 1.1: What is Python? . . . 9
Lesson 1.2: Installing Python and IDLE . . . 11
Lesson 1.3: Your First Program . . . 13
Mixed Quiz 1 . . . 15
Chapter 1 Challenges . . . 16
Chapter Recap . . . 17
Common Mistakes . . . 17

Variables and Data Types 18

Lesson 2.1: Variables . . . 18
Lesson 2.2: Numbers: Integers and Floats . . . 20
Lesson 2.3: Strings . . . 22
Lesson 2.4: Booleans . . . 24
Lesson 2.5: Converting Between Types . . . 26
Mixed Quiz 2 . . . 28
Chapter 2 Challenges . . . 29
Chapter Recap . . . 30
Common Mistakes . . . 30

Operators and Expressions 31

Lesson 3.1: Arithmetic Operators . . . 31
Lesson 3.2: Operator Precedence . . . 33

Lesson 3.3: Comparison Operators . . . 35
Lesson 3.4: Logical Operators . . . 36
Mixed Quiz 3 . . . 38
Chapter 3 Challenges . . . 39
Chapter Recap . . . 40
Common Mistakes . . . 40

Mini-Project 1: Simple Calculator **41**

How It Works . . . 41
The Code . . . 41
Things to Try . . . 42

Input and Output **43**

Lesson 4.1: Getting Input with input() . . . 43
Lesson 4.2: Print Formatting . . . 45
Lesson 4.3: f-strings (Formatted Strings) . . . 46
Lesson 4.4: Putting It Together: A Small Conversation . . . 48
Mixed Quiz 4 . . . 50
Lesson 4.5: Importing Modules . . . 51
Chapter 4.5 Challenges . . . 56
Chapter 4 Challenges . . . 56
Chapter Recap . . . 57
Common Mistakes . . . 57

Making Decisions — Conditionals **58**

Lesson 5.1: The if Statement . . . 58
Lesson 5.2: if / else . . . 60
Lesson 5.3: if / elif / else . . . 61
Lesson 5.4: Nested Conditionals . . . 64
Lesson 5.5: Combining Conditions . . . 65
Mixed Quiz 5 . . . 67
Chapter 5 Challenges . . . 68
Chapter Recap . . . 69

Common Mistakes . . . 69

Loops **70**

Lesson 6.1: for Loops and range() . . . 70

Lesson 6.2: while Loops . . . 72

Lesson 6.3: break and continue . . . 74

Lesson 6.4: Nested Loops . . . 76

Lesson 6.5: Summing and Counting . . . 78

Mixed Quiz 6 . . . 80

Chapter 6 Challenges . . . 82

Chapter Recap . . . 82

Common Mistakes . . . 82

Mini-Project 2: Number Guessing Game **83**

How It Works . . . 83

The Code . . . 83

Things to Try . . . 84

Functions **84**

Lesson 7.1: Defining a Function . . . 84

Lesson 7.2: Parameters and Arguments . . . 86

Lesson 7.3: Returning Values . . . 88

Lesson 7.4: Default Arguments . . . 90

Lesson 7.5: Variable Scope . . . 92

Lesson 7.6: Why Use Functions? . . . 94

Mixed Quiz 7 . . . 95

Chapter 7 Challenges . . . 97

Chapter Recap . . . 97

Common Mistakes . . . 98

Lists and Tuples **98**

Lesson 8.1: Creating and Indexing Lists . . . 98

Lesson 8.2: Slicing Lists . . . 100

Lesson 8.3: Modifying Lists . . . 102
Lesson 8.4: Looping Over a List . . . 105
Lesson 8.5: Tuples . . . 106
Mixed Quiz 8 . . . 108
Chapter 8 Challenges . . . 109
Chapter Recap . . . 110
Common Mistakes . . . 110

Dictionaries and Sets **111**

Lesson 9.1: Creating and Using Dictionaries . . . 111
Lesson 9.2: Dictionary Methods . . . 113
Lesson 9.3: When to Use a Dictionary . . . 115
Lesson 9.4: Sets . . . 117
Mixed Quiz 9 . . . 118
Chapter 9 Challenges . . . 120
Chapter Recap . . . 120
Common Mistakes . . . 120

Mini-Project 3: Contact Book **121**

How It Works . . . 121
The Code . . . 121
Things to Try . . . 122

Working with Strings **123**

Lesson 10.1: Useful String Methods . . . 123
Lesson 10.2: String Slicing . . . 125
Lesson 10.3: Strings Are Immutable . . . 127
Lesson 10.4: Checking String Contents . . . 129
Mixed Quiz 10 . . . 130
Chapter 10 Challenges . . . 132
Chapter Recap . . . 132
Common Mistakes . . . 132

File Handling **133**

Lesson 11.1: Opening and Closing Files . . . 133
Lesson 11.2: The with Statement . . . 135
Lesson 11.3: Reading Files . . . 137
Lesson 11.4: A Practical Example: High Score Table . . . 139
Mixed Quiz 11 . . . 141
Chapter 11 Challenges . . . 143
Chapter Recap . . . 143
Common Mistakes . . . 143

Errors and Exceptions **144**

Lesson 12.1: Reading Error Messages . . . 144
Lesson 12.2: try / except . . . 146
Lesson 12.3: finally and else . . . 148
Lesson 12.4: Raising Your Own Exceptions . . . 150
Mixed Quiz 12 . . . 152
Chapter 12 Challenges . . . 153
Chapter Recap . . . 154
Common Mistakes . . . 154

Mini-Project 4: Word Frequency Analyzer **155**

How It Works . . . 155
The Code . . . 155
Things to Try . . . 156
What You Have Learned . . . 157

Final Project: Build a Quiz Game **157**

Step 1: Design the Quiz . . . 157
Step 2: Define the Questions . . . 157
Step 3: Write the Asking Function . . . 158
Step 4: Put It All Together . . . 159
Step 5: Extension Challenges . . . 160

Glossary **161**

Index **164**

Answer Key **167**

HOW TO USE THIS BOOK

How to Use This Book

This textbook is written for school students who are learning to program for the first time. You do not need any previous experience with computers beyond knowing how to use a mouse and a keyboard. Every idea is introduced step by step, with plenty of code examples that you can type and run yourself. The book uses Python 3 because it is free, friendly for beginners, and used by professional programmers around the world.

Each chapter is built the same way so that you always know what to expect. Reading the chapter is only half of the work — the other half is doing. Programming is a skill, and like playing an instrument or a sport, it grows through practice. The more code you write, the more natural it becomes.

The Structure of Each Lesson

Every lesson follows the same pattern. Knowing this pattern will help you study efficiently because you will know exactly where to look when you need to review something.

- **Lessons** break each topic into small, focused ideas. Each lesson has explanations and one or more code examples.
- **Lesson Check** is a set of 4 multiple-choice questions right after each lesson. If you get one wrong, go back and re-read the lesson before moving on.
- **Practice Questions** are 10 questions after each lesson check. They start easy and get harder, ending with code-writing tasks. They include multiple choice, True/False, code output prediction, bug finding, and short answer.
- **Challenges** appear at the end of each chapter. These are harder thinking questions that ask you to combine ideas and think creatively. Full solutions are in the Answer Key.
- **Mixed Quiz** at the end of each chapter tests your understanding with 15 questions of mixed types.
- **Chapter Recap** lists the key ideas in one place so you can quickly review what you have learned.
- **Common Mistakes** shows the errors beginners make most often so you can avoid them.

Mini-Projects

Every three chapters, you will find a **Mini-Project** that combines everything you have learned so far into a complete, working program. These projects are not just exercises — they are real programs you can use, modify, and show off. There are four mini-projects:

- **Mini-Project 1: Simple Calculator** (after Chapter 3) — a calculator that adds, subtracts, multiplies, and divides.
- **Mini-Project 2: Number Guessing Game** (after Chapter 6) — the computer picks a random number and you try to guess it.
- **Mini-Project 3: Contact Book** (after Chapter 9) — store, look up, list, and delete contacts using a dictionary.

- **Mini-Project 4: Word Frequency Analyzer** (after Chapter 12) — read a text file and count how many times each word appears.

Each mini-project comes with the full code listing and a set of extension challenges. Type the code in, run it, then try the extensions to make it your own.

How to Study

Read each lesson slowly. When you reach a code example, do not just read it — open Python on your computer, type the code in yourself, and run it. Change small things and watch what happens. Predicting what a program will do before you run it is one of the fastest ways to learn. If you get an error, that is good news: errors are how computers teach you what is allowed and what is not.

Try the Lesson Check and Practice Questions without looking back at the lesson. If you get a question wrong, return to the relevant part and read it again. Wrong answers are not failures — they are signposts telling you exactly which idea needs another look. The Answer Key at the back of the book gives you the correct answers with detailed explanations so you can check your work.

The **Glossary** at the back of the book explains every technical word used in the chapters. Whenever a word feels unfamiliar, look it up — it only takes a moment, and you will understand the chapter much better afterwards. There is also an **Index** at the very end of the book that lets you quickly find where any topic, function, or keyword is explained.

What You Will Build

In addition to the four mini-projects, the final chapter is a larger project: you will build a complete multiple-choice quiz game using everything you learned throughout the book. By the time you finish it, you will have written a real program that you can show to your friends and family — and that you can keep improving with your own questions and features. By the end of the book, you will have built five complete programs: a calculator, a guessing game, a contact book, a word analyzer, and a quiz game.

> **TIP**
>
> Keep a notebook beside your computer. Whenever a new word or idea appears, write it down in your own words. The Glossary in this book is a starting point, but the explanations that mean the most are the ones you write yourself.

CHAPTER 1

Introduction to Python

Python is one of the most popular programming languages in the world. It is used to build websites, control robots, analyse scientific data, train artificial intelligence models, and much more. In this chapter you will learn what Python is, where it came from, how to install it on your computer, and how to write your very first program.

Lesson 1.1: What is Python?

A **programming language** is a way of giving instructions to a computer. Just as you might write a recipe for a friend so they can bake a cake, a program is a recipe for a computer so it can perform a task. Python is one such language, and it was designed from the start to be easy for humans to read and write.

Python was created in 1991 by a Dutch programmer named Guido van Rossum. He named the language not after the snake, but after the British comedy group *Monty Python* — which is why the official Python documentation sometimes contains jokes and references to spam, eggs, and knights. The language quickly became popular because its code looks almost like plain English, which makes it easier for beginners to learn than many other languages.

Today Python is used in many different fields. Scientists use it to analyse experimental data. Engineers use it to build websites like Instagram and Spotify. Teachers use it in classrooms because it is gentle enough for young students but powerful enough for serious work. Learning Python gives you a tool you can use for almost any project you can imagine.

Why Learn Python?

- **Easy to read** — Python code looks a lot like English, so beginners spend less time fighting the language and more time solving problems.
- **Free and open** — Anyone can download, install, and use Python without paying anything. You can even read and modify the source code.
- **Huge community** — Millions of programmers use Python, so when you get stuck, you can almost always find help online.
- **Versatile** — The same language can build a small calculator script in five minutes or a giant web application used by millions of people.

Lesson 1.1 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. Python was created by which programmer?

A. Linus Torvalds
B. Bill Gates
C. Dennis Ritchie
D. Guido van Rossum

2. Python was named after:

A. A species of snake B. An ancient Greek word
C. A type of computer D. The Monty Python comedy group

3. Which of these is NOT a reason Python is popular for beginners?

A. It only runs on Windows B. It is easy to read
C. It is free and open D. It has a huge community

4. Guido van Rossum is from which country?

A. The United States B. Germany
C. Japan D. The Netherlands

Lesson 1.1 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. Who created Python?

A. Linus Torvalds B. Bill Gates
C. Dennis Ritchie D. Guido van Rossum

2. Python was named after:

A. A species of snake B. An ancient Greek word
C. A type of computer D. The Monty Python comedy group

3. Which is NOT a reason Python is good for beginners?

A. Easy to read B. Free and open
C. Large community D. Only runs on Windows

4. In what year was Python first released?

A. 1985 B. 2000
C. 1991 D. 2010

5. Guido van Rossum is from which country?

A. USA B. Germany
C. Netherlands D. Japan

6. True or False: Python was named after a species of snake.

☐ True ☐ False

7. What does this code print?

PYTHON

```
print("Hello, World!")
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
print(Hello, World!)
```

Bug: _____

9. In one sentence, explain what a programming language is.

10. Write a print statement that displays: Python is awesome!

Lesson 1.2: Installing Python and IDLE

Before you can write Python programs, you need to install Python on your computer. Python runs on Windows, macOS, and Linux, and the installation process is similar on all three systems. The official website python.org always has the latest version available for free download. Use a currently supported version of Python 3 — the examples in this book are designed for Python 3.10 and later, but you should install a version that still receives security updates. Check python.org for the latest supported versions.

When you install Python, you also get a program called **IDLE**, which stands for *Integrated Development and Learning Environment*. IDLE is a simple text editor combined with a Python interpreter, so you can type code, run it, and see the result all in the same window. For the first few chapters of this book, IDLE is all you need.

Installing on Windows

1. Go to python.org/downloads and click the yellow button that says "Download Python".
2. Open the downloaded file. **Before** you click "Install Now", check the box at the bottom that says *Add Python to PATH*. This is very important — without it, Windows will not be able to find Python.
3. Click "Install Now" and wait for the installation to finish.
4. Open the Start menu, type IDLE, and press Enter. A window with the Python shell should appear.

Installing on macOS

1. Go to python.org/downloads and click the yellow button to download the macOS installer.
2. Open the downloaded .pkg file and follow the prompts.
3. Open the Applications folder, then the Python 3 folder, and double-click IDLE to start it.

Installing on Linux

Most Linux distributions already include Python. Open a terminal and type python3 --version to check. If it is not installed, use your package manager, for example sudo apt install python3 idle3 on Ubuntu or Debian.

> **TIP**
>
> If you cannot install Python on your own computer, you can use a free online Python environment such as replit.com or trinket.io. You will not need to install anything — just open the website and start typing code.

Lesson 1.2 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What does IDLE stand for?

A. Interactive Direct Language Editor
B. Integer Decimal Logic Engine
C. Integrated Development and Learning Environment
D. Internal Data Link Environment

2. When installing Python on Windows, what must you check before clicking "Install Now"?

A. The box that says "Install for all users"
B. The box that says "Add shortcuts"
C. The box that says "Install pip"
D. The box that says "Add Python to PATH"

3. On which operating system is Python available?

A. Windows only
B. Windows, macOS, and Linux
C. macOS only
D. Linux only

4. If you cannot install Python on your computer, what can you do?

A. Give up and use a different language
B. Use a free online Python environment like replit.com
C. Buy a new computer
D. Python cannot be used without installing it

Lesson 1.2 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. Where should you download Python?

A. python.com
B. microsoft.com
C. google.com
D. python.org

2. What does IDLE stand for?

A. Interactive Data Language Editor
B. Internet Development Link Engine
C. Integrated Development and Learning Environment
D. Internal Direct Loop Executor

3. What is the PATH variable used for?

A. Storing files
B. Setting the desktop background
C. Telling the OS where to find programs
D. Connecting to the internet

4. Which operating systems can run Python?

A. Windows only
B. macOS only
C. Linux only
D. Windows, macOS, and Linux

5. What file is used to install Python on Windows?

A. setup.bin
B. python-installer.exe
C. A .exe installer downloaded from python.org
D. install.bat

6. True or False: IDLE comes bundled with Python when you install it.

☐ True ☐ False

7. What does this command print in a terminal?

PYTHON

```
python --version
```

Output: _____

8. A student opens a terminal and types: python --version but gets "command not found". What is the most likely cause?

PYTHON - has a bug

```
python --version
```

Bug: _____

9. In one sentence, explain what IDLE is.

10. Write the terminal command that shows the installed Python version.

Lesson 1.3: Your First Program

It is a tradition in programming that your first program in any new language should display the message **"Hello, World!"** on the screen. In Python, this entire program is just one line:

PYTHON · hello.py

```
print("Hello, World!")
```

Type this line into IDLE and press Enter. You should immediately see the words Hello, World! appear on the next line, without the quotation marks. Congratulations — you have just written and run your first Python program.

Let us break down what is happening here. The word print is a **function** — a named block of code that does something useful. The function called print takes whatever you give it inside the parentheses and displays it on the screen. The text "Hello, World!" is a **string**, which is programmer-speak for a piece of text. The quotation marks tell Python "this is text, not code".

You can print any text you like — just remember the quotation marks:

PYTHON · greetings.py

```
print("My name is Alex.")
print("I am learning Python.")
print("Python is fun!")
```

Each print call puts its message on its own line. You will learn later how to control this behaviour, but for now, just remember one print = one line of output.

Comments: Notes to Yourself

A **comment** is a piece of text inside your program that Python completely ignores. Comments exist for humans, not for the computer. You write a comment by starting a line with the hash symbol #. Everything from the # to the end of the line is ignored by Python.

PYTHON · comments.py

```
# This is a comment. Python ignores it.
print("Hello, World!") # You can also put comments after code.
# print("This line will NOT run.")
```

Comments are useful for two reasons. First, they let you explain what your code does, which is helpful when you come back to it after a few days. Second, they let you temporarily turn off a line of code without deleting it, which is handy when you are testing things. Programmers call this *commenting out* a line.

Saving Your Program in a File

So far you have been typing code directly into the Python shell, which runs each line as soon as you press Enter. For longer programs, you will want to save your code in a file so you can run it again later. In IDLE, click **File → New File** to open a new editor window. Type your code there, then click **File → Save** to save it with a name like hello.py. The .py extension tells the computer that this is a Python file. To run the program, click **Run → Run Module** or press **F5**.

> **TIP**
>
> Give your files clear, descriptive names such as greetings.py or quiz_game.py. Avoid spaces in filenames — use underscores instead. So quiz_game.py is good, quiz game.py is not.

Lesson 1.3 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What does the following code print? print("Hello, World!")

A. print("Hello, World!")
B. Hello World
C. An error
D. Hello, World!

2. What symbol starts a comment in Python?

A. //
B. /*
C. --
D. #

3. What happens if you write print(Hello) without quotation marks?

A. It prints Hello
B. It prints the word Hello without quotes
C. Python raises an error because Hello is not a defined variable
D. It prints nothing

4. What file extension should Python files use?

A. .txt
B. .py
C. .python
D. .p

Lesson 1.3 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. Which function displays text on the screen?

A. input()
B. print()
C. show()
D. display()

2. What symbol starts a comment in Python?

A. //
B. --
C. /*
D. #

3. What extension do Python files use?

A. .pyt
B. .python
C. .py
D. .p

4. What must text be wrapped in to make a string?

A. Parentheses ()
B. Square brackets []
C. Quotation marks
D. Curly braces {}

5. What happens when you run a .py file in IDLE?

A. It is deleted
B. It is printed on paper
C. Python runs the code line by line
D. It becomes a zip file

6. True or False: Comments are ignored by Python when the program runs.

☐ True ☐ False

7. What does this code print?

PYTHON

```
print("Hello")
print("World")
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
print "Hello"
```

Bug: _____

9. In one sentence, explain what a comment is and why it is useful.

10. Write a one-line program that prints your name.

Mixed Quiz 1

This quiz mixes different question types: multiple choice, True/False, short answer, and code writing. Try to answer all of them without looking back at the chapter first — that is how you really learn. Answers are in the Answer Key at the back of the book.

1. Who created the Python programming language?

A. Linus Torvalds
B. Guido van Rossum
C. Bill Gates
D. Ada Lovelace

2. What does the print() function do?

A. Sends a document to a printer
B. Reads input from the keyboard
C. Displays text on the screen
D. Saves a file to disk

3. What does the # symbol do in Python?

A. It marks the start of a comment
B. It creates a hashtag for social media
C. It multiplies two numbers
D. It marks an error in your code

4. What file extension do Python files use?

A. .pyt
B. .py
C. .python
D. .p

5. What is the name of the simple programming environment that comes bundled with Python?

A. Python Studio
B. PyCharm
C. IDLE
D. Visual Studio

6. In what year was Python first released?

A. 1985 B. 1991
C. 2000 D. 2010

7. Which key do you press in IDLE to run a saved program?

A. F1 B. F5
C. Ctrl+R D. Ctrl+Enter

8. True or False: Python was named after a species of snake.

☐ True ☐ False

9. True or False: Comments are ignored by Python when the program runs.

☐ True ☐ False

10. True or False: You must always put text inside quotation marks when you want to print it.

☐ True ☐ False

11. True or False: A single print() call can display text on more than one line of output by default.

☐ True ☐ False

12. In your own words, explain what a programming language is.

13. Give two reasons why comments are useful in a Python program.

14. Write a Python program that prints three lines: your name, your favourite subject, and one thing you hope to learn from this book.

15. Write a single line of Python that uses print() and also includes a comment on the same line explaining what the code does.

Chapter 1 Challenges

These questions are harder than the quiz. They ask you to combine ideas from this chapter, think creatively, and apply what you learned in new ways. Do not worry if you cannot solve them all on your first try — the point is to make you think. The full solutions are in the Answer Key at the back of the book.

Challenge 1. Write a Python program that prints your name on one line and your favourite hobby on the next, with a comment above each line explaining what it does.

Hint: You need two print() calls and two comments.

Challenge 2. What do you think happens if you run print("Hello") and then print("World") in the same program? How many lines of output do you get, and why?

Hint: Think about what print adds at the end of each call.

Challenge 3. A friend tells you that comments slow down your program because the computer has to read them. Is this correct? Explain why or why not.

Hint: Remember what the chapter said about what Python does with comments.

Challenge 4. You want to print the text: She said "Python is fun!" — including the quotation marks. How would you write the print statement? (Hint: think about which quote marks to use.)

Hint: You can use single quotes on the outside if the text contains double quotes.

Challenge 5. Write a program that uses exactly one print() call to display three lines of text: "Line 1", "Line 2", and "Line 3".

Hint: The chapter mentioned a special character that creates a new line.

Chapter Recap

Here are the most important ideas from this chapter. If you can explain each one in your own words, you are ready to move on.

- Python is a popular, beginner-friendly programming language created in 1991 by Guido van Rossum.
- You can download Python for free from python.org. Installing it also gives you IDLE, a simple environment for writing and running code.
- The print() function displays text on the screen.
- Strings are pieces of text. They must be wrapped in quotation marks.
- Comments start with # and are ignored by Python.
- Save your code in files ending in .py and run it with F5 in IDLE.

Common Mistakes

Every beginner makes these mistakes. Knowing them in advance will save you hours of debugging.

COMMON MISTAKE

Forgetting the quotation marks around text
Writing print(Hello) instead of print("Hello") will cause an error, because Python will try to look up a variable named Hello instead of treating it as text. Always wrap text in quotes.

COMMON MISTAKE

Using the wrong kind of quotation marks
Python accepts both single quotes ' and double quotes ", but you must use the same kind to open and close the string. "Hello' is wrong; "Hello" or 'Hello' is right.

COMMON MISTAKE

Forgetting the parentheses with print
In Python 3, print is a function and must always be followed by parentheses. Writing print "Hello" works in Python 2 but causes an error in Python 3.

COMMON MISTAKE

Typing in the wrong window
If you type print("Hello") in the editor window without running it, nothing will happen. Make sure you press F5 or click **Run → Run Module** to actually execute the code.

CHAPTER 2

Variables and Data Types

In the previous chapter, you wrote programs that printed fixed messages. Real programs usually need to remember things — a player's score, the temperature outside, the name of a student. In Python, you store information in **variables**. This chapter explains how to create variables, what kinds of data they can hold, and how to convert between different kinds.

Lesson 2.1: Variables

A **variable** is a named box that holds a value. You create a variable by giving it a name and using the equals sign = to put a value inside it. This action is called **assignment**. Once a variable exists, you can use its name anywhere you would use its value.

PYTHON · variables.py

```
name = "Alex"
age = 14
print(name)
print(age)
```

When you run this program, Python prints Alex on the first line and 14 on the second. The variable name now holds the string "Alex", and the variable age holds the number 14. You can change what is inside a variable at any time by assigning a new value:

PYTHON · reassignment.py

```
score = 0
print(score) # 0
score = 10
print(score) # 10
score = score + 5
print(score) # 15
```

Notice the line score = score + 5. This looks strange at first, but it makes perfect sense once you remember that = means "put the value on the right into the box on the left". Python first works out score + 5 (which is 10

+ 5 = 15), and then stores that result back into score. The old value of score is replaced.

Rules for Naming Variables

You can name a variable almost anything you like, but Python does enforce a few rules. Following these rules from the start will save you a lot of frustration later.

- Names can contain letters, digits, and underscores _, but cannot start with a digit. player1 is fine; 1player is not.
- Names cannot contain spaces. Use underscores instead: player_score, not player score.
- Names are case-sensitive. age, Age, and AGE are three different variables.
- You cannot use Python keywords (such as if, for, def, while) as variable names. Note that print is NOT a keyword — it is a built-in function — but you should still avoid using it as a variable name, because doing so would stop the print() function from working.

> **TIP**
>
> Choose names that describe what the variable holds. score is much clearer than s. player_name is much clearer than x. Long, descriptive names are better than short, mysterious ones.

Lesson 2.1 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. Which of these is a valid Python variable name?

A. 2nd_place
B. _score
C. player score
D. for

2. What does the line score = score + 5 do?

A. It creates a new variable called score + 5
B. It adds 5 to the current value of score and stores the result back in score
C. It compares score to score + 5
D. It causes an error

3. After running x = 10 then x = 20, what is the value of x?

A. 10
B. 30
C. An error
D. 20

4. Which of these is NOT a valid variable name?

A. player-name
B. player1
C. player_name
D. PlayerName

Lesson 2.1 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. Which is a valid variable name?

A. 2name
B. name-2
C. class
D. name_2

2. How do you assign a value to a variable?

A. x == 5
B. x := 5
C. x = 5
D. x => 5

3. What is the value of x after: x = 5 then x = 10?

A. 5 B. 10
C. 15 D. error

4. Which variable name is INVALID?

A. age B. 2nd_place
C. _score D. player1

5. What does this code do: name = "Ana"?

A. Prints Ana B. Deletes Ana
C. Nothing D. Creates a variable called name holding the string Ana

6. True or False: A variable can change its value while the program runs.

☐ True ☐ False

7. What does this code print?

PYTHON

```
x = 4
y = x + 3
print(y)
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
2cool = 10
```

Bug: _____

9. In one sentence, explain what a variable is.

10. Create a variable called age that stores the number 12.

Lesson 2.2: Numbers: Integers and Floats

Python has two main kinds of numbers. **Integers** (type int) are whole numbers like 3, -7, or 0. **Floating-point numbers** (type float), usually just called *floats*, are numbers with a decimal point like 3.14, -0.001, or 2.0. The difference matters because some operations behave differently depending on whether you are working with integers or floats.

PYTHON · numbers.py

```
a = 10 # int
b = 3 # int
c = 3.0 # float
print(a / b) # 3.333... (always returns a float)
print(a // b) # 3 (integer division, rounds down)
print(a % b) # 1 (modulo: the remainder after division)
print(a ** b) # 1000 (10 to the power of 3)
```

The code above shows several operators: + and - for addition and subtraction, / regular division, // integer division (also called *floor division*), % modulo (the remainder), and ** exponent (to the power of). The double-slash // always rounds down, even for negative numbers: -10 // 3 gives -4, not -3.

You can check the type of any value or variable using the type() function:

PYTHON · types.py

```
print(type(42)) # <class 'int'>
print(type(3.14)) # <class 'float'>
print(type("Hi")) # <class 'str'>
```

Lesson 2.2 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What is the type of the value 42?

A. float
B. str
C. int
D. bool

2. What does 10 // 3 evaluate to?

A. 3.333...
B. 1
C. 3
D. 4

3. What does 10 % 3 evaluate to?

A. 1
B. 3
C. 3.333...
D. 0

4. What does 2 ** 3 evaluate to?

A. 6
B. 9
C. 23
D. 8

Lesson 2.2 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What type is the value 7?

A. str
B. int
C. float
D. bool

2. What type is the value 3.14?

A. int
B. float
C. str
D. bool

3. What does 7 // 2 produce?

A. 3.5 B. 3
C. 4 D. 1

4. What does 10 % 3 produce?

A. 3 B. 0
C. 1 D. 10

5. What does 2 ** 3 produce?

A. 6 B. 8
C. 9 D. 5

6. True or False: In Python, 7 / 2 gives 3.5 (a float).

☐ True ☐ False

7. What does this code print?

PYTHON

```
print(7 % 3)
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
print(5 +)
```

Bug: _____

9. In one sentence, explain the difference between int and float.

10. Write code that prints the remainder when 17 is divided by 5.

Lesson 2.3: Strings

A **string** (type str) is a sequence of characters — in other words, a piece of text. You create a string by wrapping text in either single quotes ' or double quotes ". Both work the same way; just be consistent so your code looks tidy. If your text contains an apostrophe, like "It's a lovely day", use double quotes on the outside so the apostrophe does not end the string early.

PYTHON · strings.py

```
first = "Grace"
last = "Hopper"
full = first + " " + last
print(full) # Grace Hopper
```

The plus sign + joins two strings together. Programmers call this **concatenation**. Notice that we had to add a space between the first and last names — Python does not add one for you. You can also repeat a string using the multiplication sign *:

```
print("ha" * 3) # hahaha
print("-" * 20) # --------------------
```

Lesson 2.3 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. Which symbol is used to join (concatenate) two strings?

A. *
B. &
C. .
D. +

2. What does "ha" * 3 produce?

A. "ha ha ha"
B. "ha3"
C. "hahaha"
D. An error

3. Which of these strings contains an apostrophe correctly?

A. 'It's a lovely day'
B. "It's a lovely day"
C. Both A and B work
D. Neither works

4. What is the type of the value "Hello"?

A. int
B. float
C. str
D. bool

Lesson 2.3 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. How do you join two strings together?

A. With the + operator
B. With the * operator
C. With the - operator
D. With the / operator

2. What does "Ha" * 3 produce?

A. Ha3
B. Ha Ha Ha
C. HaHaHa
D. error

3. Which of these is a valid string?

A. 'Hello'
B. Hello
C. (Hello)
D. [Hello]

4. What is the result of "Py" + "thon"?

A. Py thon
B. PyPython
C. error
D. Python

5. Which quote style is valid for a string?

A. Only single quotes
B. Both single and double quotes
C. Only double quotes
D. Only backticks

6. True or False: You can multiply a string by a number to repeat it.

☐ True ☐ False

7. What does this code print?

PYTHON

```
print("La" * 3)
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
print('It's a nice day')
```

Bug: _____

9. In one sentence, explain what string concatenation is.

10. Write code that prints Hello repeated 4 times with no spaces.

Lesson 2.4: Booleans

A **boolean** (type bool) is a value that can only be **True** or **False**. Notice the capital letters — in Python, these are not strings, they are special keywords. Booleans are essential for making decisions in programs, as you will see in Chapter 5.

PYTHON · booleans.py

```
is_raining = True
is_sunny = False
print(is_raining) # True
print(type(is_raining)) # <class 'bool'>
```

Booleans often come from comparisons. Python can compare two values and tell you whether the comparison is true or false:

PYTHON · comparisons.py

```
print(5 > 3) # True
print(5 < 3) # False
print(5 == 5) # True (note the double ==)
print(5 != 4) # True (!= means "not equal to")
print(10 >= 10) # True
```

NOTE

A single equals sign = is for **assignment** (putting a value into a variable). A double equals sign == is for **comparison** (asking "are these two things equal?"). Mixing them up is one of the most common mistakes in programming — get into the habit of using the right one from day one.

Lesson 2.4 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What are the only two values a boolean can have?

A. True and False
B. Yes and No
C. 1 and 0
D. On and Off

2. What does 5 == 5 evaluate to?

A. 5
B. False
C. "yes"
D. True

3. What does 5 != 4 evaluate to?

A. False
B. True
C. An error
D. 5

4. Which comparison checks if x is greater than or equal to 10?

A. x >= 10
B. x > 10
C. x => 10
D. x >> 10

Lesson 2.4 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. Which are the two Boolean values?

A. Yes and No
B. 1 and 0
C. On and Off
D. True and False

2. What does 5 > 3 produce?

A. 5
B. 3
C. False
D. True

3. What does 4 == 4 produce?

A. True
B. 4
C. False
D. error

4. What does 10 != 10 produce?

A. True
B. 10
C. error
D. False

5. What is the type of True?

A. bool
B. str
C. int
D. float

6. True or False: In Python, true and false (lowercase) are valid Boolean values.

☐ True ☐ False

7. What does this code print?

PYTHON

```
print(7 > 9)
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
print(5 = 5)
```

Bug: _____

9. In one sentence, explain what a Boolean value is.

10. Write code that prints whether 12 is greater than 5.

Lesson 2.5: Converting Between Types

Sometimes you have a value of one type but you need it as another. For example, the input() function (which you will meet in Chapter 4) always gives you a string, even if the user typed a number. To turn that string into a real number, you use a **type conversion function**:

PYTHON · convert.py

```
age_text = "14"
age_num = int(age_text)
print(age_num + 1) # 15

price = 9.99
whole = int(price) # 9 (int() chops off the decimal part)

number = 42
text = str(number) # "42"
print("The answer is " + text)
```

The three conversion functions you will use most are int() (convert to integer), float() (convert to float), and str() (convert to string). If the conversion is not possible — for example, int("hello") — Python raises an error, which you will learn to handle in Chapter 12.

Function	Converts to	Example	Result
int()	integer	`int("42")`	`42`
int()	integer (chops decimal)	`int(3.99)`	`3`
float()	float	`float("3.14")`	`3.14`
float()	float	`float(5)`	`5.0`
str()	string	`str(42)`	`"42"`
bool()	boolean	`bool(0)`	`False`

Table 2.1 — Common type-conversion functions and examples.

Lesson 2.5 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What does int("42") return?

A. The string "42"
B. The integer 42
C. The float 42.0
D. An error

2. What does int(3.99) return?

A. 4
B. 3
C. 3.99
D. An error

3. What does str(42) return?

A. The integer 42
B. The float 42.0
C. The string "42"
D. An error

4. What happens if you try int("hello")?

A. It returns 0
B. It raises a ValueError
C. It returns None
D. It returns the string "hello"

Lesson 2.5 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What does int("5") produce?

A. "5"
B. 5.0
C. 5
D. error

2. What does str(7) produce?

A. 7
B. 7
C. error
D. "7"

3. What does float("3.5") produce?

A. "3.5"
B. 3.5
C. 3
D. error

4. What does bool(0) produce?

A. True
B. 0
C. False
D. error

5. What does int(3.9) produce?

A. 4
B. 3.9
C. error
D. 3

6. True or False: You cannot add a string and a number directly; you must convert first.

☐ True ☐ False

7. What does this code print?

PYTHON

```
print(int("8") + 2)
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
print("Age: " + 10)
```

Bug: _____

9. In one sentence, explain what type conversion is.

10. Write code that converts the string "15" to an integer and prints it plus 5.

Mixed Quiz 2

This quiz mixes different question types: multiple choice, True/False, short answer, and code writing. Try to answer all of them without looking back at the chapter first — that is how you really learn. Answers are in the Answer Key at the back of the book.

1. Which of these is a valid Python variable name?

A. 2cool
B. player score
C. player_score
D. for

2. What is the type of the value 3.14?

A. int
B. float
C. str
D. bool

3. What does the expression 10 % 3 evaluate to?

A. 3
B. 1
C. 3.33
D. 0

4. What does "ha" * 3 produce?

A. "hahaha"
B. "ha ha ha"
C. an error
D. "ha3"

5. Which operator means "is equal to" in Python?

A. =
B. ==
C. !=
D. ===

6. What is the type of the value True?

A. str
B. int
C. bool
D. truth

7. What does type(3.14) return?

A. <class 'int'>
B. <class 'float'>
C. <class 'str'>
D. <class 'num'>

8. True or False: In Python, age and Age are the same variable.

☐ True ☐ False

9. True or False: The expression 10 // 3 evaluates to 3.

☐ True ☐ False

10. True or False: You can convert the string "hello" to an integer using int("hello").

☐ True ☐ False

11. True or False: The variable name _score is valid in Python.

☐ True ☐ False

12. Explain the difference between = and == in Python.

13. Why is it a bad idea to name a variable x or y? What kind of names should you use instead?

14. Create three variables: name (a string), age (an integer), and height (a float). Then print all three on separate lines.

15. Write a program that creates a variable holding the string "42", then converts it to an integer, adds 8 to it, and prints the result.

Chapter 2 Challenges

These questions are harder than the quiz. They ask you to combine ideas from this chapter, think creatively, and apply what you learned in new ways. Do not worry if you cannot solve them all on your first try — the point is to make you think. The full solutions are in the Answer Key at the back of the book.

Challenge 1. What is the value of x after this code runs? x = 5 x = x + 3 x = x * 2 x = x - 4 Explain each step.

Hint: Work through each line one at a time, updating x.

Challenge 2. Write a program that asks the user for their birth year (as a string from input), converts it to an integer, and prints how old they will be in 2030.

Hint: You need int() to convert, then subtract birth year from 2030.

Challenge 3. What is the type of the result of 10 / 2? Is it int or float? Explain why.

Hint: Think about what the chapter said about the / operator.

Challenge 4. A variable called age holds the string "14". Write code that adds 1 to the actual number and stores the result in a variable called next_year. Then print "Next year you will be 15" using an f-string and the variable.

Hint: You need to convert, add, and then use an f-string to print.

Challenge 5. What do you think print(type(type(42))) prints? Explain your reasoning.

Hint: First work out what type(42) returns, then what type() of THAT returns.

Chapter Recap

Here are the most important ideas from this chapter. If you can explain each one in your own words, you are ready to move on.

- A variable is a named container for a value. You create one with name = value.
- Variable names must follow rules: letters, digits, underscores only; cannot start with a digit; cannot be a Python keyword.
- Python's main data types are int (whole numbers), float (decimal numbers), str (text), and bool (True or False).
- Use type() to find out what type a value is.
- Type conversion functions int(), float(), str(), and bool() convert values from one type to another.
- = is for assignment; == is for comparison.

Common Mistakes

Every beginner makes these mistakes. Knowing them in advance will save you hours of debugging.

COMMON MISTAKE

Using a single = when you mean ==
Writing if age = 14: will give an error. Use if age == 14: to compare values.

COMMON MISTAKE

Starting a variable name with a digit
2nd_place = "Maya" is illegal. Use second_place instead.

COMMON MISTAKE

Forgetting that input() returns a string
If you ask the user for their age, age = input() gives you a string. To do maths with it, convert it first: age = int(input()). You will see this pattern a lot in Chapter 4.

COMMON MISTAKE

Trying to concatenate a string and a number
print("Age: " + 14) causes an error. Convert the number first: print("Age: " + str(14)), or use an f-string (Chapter 4).

CHAPTER 3

Operators and Expressions

An **expression** is any piece of Python code that produces a value. For example, 2 + 3 is an expression — it produces the value 5. An **operator** is a symbol that does something to one or more values, like + for addition or * for multiplication. In this chapter you will learn all the operators you need to write useful Python programs.

Lesson 3.1: Arithmetic Operators

You already met the basic arithmetic operators in Chapter 2. Here they are again, all together, so you can see them as a family. Try each one in IDLE until you feel comfortable with how it behaves.

PYTHON · arithmetic.py

```
a = 10
b = 3
print(a + b) # 13 addition
print(a - b) # 7 subtraction
print(a * b) # 30 multiplication
print(a / b) # 3.333... true division (always float)
print(a // b) # 3 floor division (drops the remainder)
print(a % b) # 1 modulo (the remainder)
print(a ** b) # 1000 exponent (10 to the power of 3)
```

Two operators deserve a closer look. The modulo operator % is surprisingly useful. It tells you the *remainder* after division. For example, 10 % 3 is 1 because 10 divided by 3 gives 3 with 1 left over. A common use of modulo is to check whether a number is even or odd: a number is even if number % 2 == 0, and odd otherwise.

The exponent operator ** raises a number to a power. So 2 ** 10 is 2 raised to the 10th power, which is 1024. You can also take roots with fractional powers: 9 ** 0.5 is 3.0 (the square root of 9).

Operator	Name	Example	Result
`**`	Exponent	`2 ** 3`	`8`
`*, /, //, %`	Multiplication, division, floor div, modulo	`10 % 3`	`1`
`+, -`	Addition, subtraction	`5 + 3`	`8`
`==, !=, <, >, <=, >=`	Comparison operators	`5 == 5`	`True`
`not`	Logical NOT	`not True`	`False`
`and`	Logical AND	`True and False`	`False`

`or`	Logical OR	`True or False`	`True`

Table 3.1 — Python operators from highest to lowest precedence.

Lesson 3.1 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What does 10 % 3 evaluate to?

A. 3 B. 3.33
C. 1 D. 0

2. What does 2 ** 4 evaluate to?

A. 8 B. 6
C. 16 D. 24

3. Which operator gives the integer part of division (dropping the remainder)?

A. / B. %
C. // D. **

4. What does -7 // 2 evaluate to?

A. -3 B. -4
C. -3.5 D. -2

Lesson 3.1 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What does the / operator do?

A. Regular division (returns float) B. Floor division
C. Multiplication D. Exponentiation

2. What does the // operator do?

A. Regular division B. Modulo
C. Power D. Floor (whole-number) division

3. What does the % operator return?

A. The remainder B. The quotient
C. The product D. The power

4. What does the ** operator do?

A. Exponentiation (power) B. Multiplication
C. Modulo D. Floor division

5. What is the result of 9 / 3?

A. 3.0 B. 3
C. 0 D. 1

6. True or False: The % operator gives the remainder of a division.

☐ True ☐ False

7. What does this code print?

PYTHON

```
print(2 ** 4)
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
print(10 / 0)
```

Bug: _____

9. In one sentence, explain the difference between / and //.

10. Write code that prints 5 squared.

Lesson 3.2: Operator Precedence

When you write an expression with several operators, like 2 + 3 * 4, Python needs to know which operation to do first. The rules that decide this are called **operator precedence**. The rules in Python match the ones you learned in maths class: multiplication and division happen before addition and subtraction, and exponentiation happens before everything else.

PYTHON · precedence.py

```
print(2 + 3 * 4) # 14, not 20 (multiplication first)
print((2 + 3) * 4) # 20 (parentheses override)
print(2 ** 3 ** 2) # 512, not 64 (right-to-left: 2 ** 9)
print(-2 ** 2) # -4 (exponent before unary minus)
```

When in doubt, use parentheses. (2 + 3) * 4 is always clearer than relying on the precedence rules. Parentheses make your intent obvious to anyone reading your code — including you, three weeks later.

Augmented Assignment

A very common pattern is updating a variable based on its current value: score = score + 10. Python has a shorthand for this called **augmented assignment**:

PYTHON · augmented.py

```
score = 0
score += 10 # same as score = score + 10
score += 5 # score is now 15
score -= 3 # score is now 12
score *= 2 # score is now 24
score //= 5 # score is now 4
```

Lesson 3.2 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What is the value of 2 + 3 * 4?

A. 20 B. 24
C. 9 D. 14

2. Which operator has the HIGHEST precedence?

A. ** B. +
C. * D. //

3. What is the value of (2 + 3) * 4?

A. 14 B. 20
C. 24 D. 9

4. What does score += 10 do?

A. It creates a new variable B. It adds 10 to score without changing score
C. It causes an error D. It is the same as score = score + 10

Lesson 3.2 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. In the expression 2 + 3 * 4, what is computed first?

A. 2 + 3 B. 3 * 4
C. They happen at once D. It is random

2. What is the result of 2 + 3 * 4?

A. 14 B. 20
C. 24 D. 9

3. How do you force 2 + 3 to be computed first in 2 + 3 * 4?

A. Use quotes B. Use a comma
C. You cannot D. Use parentheses: (2 + 3) * 4

4. What does += do?

A. Just adds B. Just assigns
C. Adds then assigns D. Compares

5. What is the result of (2 + 3) * 4?

A. 14 B. 9
C. 24 D. 20

6. True or False: Parentheses can change the order in which operations are performed.

☐ True ☐ False

7. What does this code print?

PYTHON

```
print(10 - 2 - 3)
```

Output: _____

8. A student wants to add 2 and 3 first, then multiply by 4, but this prints 14 instead of 20. What is wrong?

PYTHON - has a bug

```
print(2 + 3 * 4)
```

Bug: _____

9. In one sentence, explain operator precedence.

10. Write code that uses parentheses to compute the average of 4, 8, and 12, then prints it.

Lesson 3.3: Comparison Operators

Comparison operators compare two values and produce a boolean (True or False). You already saw == (equal to) and != (not equal to). The full set is shown below.

- == equal to 5 == 5 → True
- != not equal to 5 != 4 → True
- < less than 3 < 5 → True
- > greater than 5 > 3 → True
- <= less than or equal to 5 <= 5 → True
- >= greater than or equal to 4 >= 5 → False

Comparison operators work on strings too, but the comparison is alphabetical (technically, by Unicode code point). Uppercase letters come before lowercase letters, so "Apple" < "apple" is True. Be careful with this when comparing user input.

Lesson 3.3 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What does 5 == 5 evaluate to?

A. 5
B. False
C. True
D. "equal"

2. Which operator means "not equal to"?

A. =!
B. !=
C. ==!
D. <>

3. What does "apple" < "banana" evaluate to?

A. False
B. An error
C. True
D. None

4. What does 5 <= 5 evaluate to?

A. False
B. True
C. An error
D. 5

Lesson 3.3 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. Which operator checks if two values are equal?

A. ==
B. =
C. !=
D. >=

2. What does != mean?

A. Equal to
B. Greater than
C. Not equal to
D. Less than

3. What does 5 <= 5 produce?

A. False
B. 5
C. True
D. error

4. What does 8 >= 10 produce?

A. True
B. 8
C. 10
D. False

5. What is the result of 3 < 7?

A. False
B. 3
C. 7
D. True

6. True or False: The == operator returns a Boolean (True or False).

☐ True ☐ False

7. What does this code print?

PYTHON

```
print(10 != 10)
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
if x = 5:
    print("yes")
```

Bug: _____

9. In one sentence, explain the difference between = and ==.

10. Write code that prints True if 9 is greater than 4, and False otherwise (use a comparison).

Lesson 3.4: Logical Operators

Logical operators combine boolean values. There are three: and, or, and not. They let you build more complicated conditions out of simpler ones.

PYTHON · logical.py

```
age = 16
has_id = True

# Both must be true
can_enter = age >= 18 and has_id
print(can_enter) # False

# At least one must be true
is_weekend = False
is_holiday = True
no_school = is_weekend or is_holiday
print(no_school) # True

# Reverses a boolean
print(not True) # False
```

The and operator only gives True if both sides are true. The or operator gives True if at least one side is true. The not operator flips a boolean: not True is False, and not False is True.

TIP

You can combine several conditions in one expression, but use parentheses to make the order clear. (age >= 13 and age <= 19) or has_parent_permission is much easier to read than the same expression without parentheses.

Lesson 3.4 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What does True and False evaluate to?

A. True
B. None
C. False
D. An error

2. What does True or False evaluate to?

A. True
B. False
C. None
D. An error

3. What does not True evaluate to?

A. True
B. False
C. None
D. An error

4. What does (5 > 3) and (10 > 20) evaluate to?

A. False
B. True
C. An error
D. None

Lesson 3.4 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. Which is a logical operator in Python?

A. plus
B. and
C. with
D. also

2. What does True and False produce?

A. False B. True
C. error D. None

3. What does True or False produce?

A. True B. False
C. error D. None

4. What does not True produce?

A. False B. True
C. error D. None

5. What does (5 > 3) and (2 > 1) produce?

A. False B. 5
C. True D. 2

6. True or False: The expression not False is True.

☐ True ☐ False

7. What does this code print?

PYTHON

```
print(True or False)
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
print(5 > 3 and)
```

Bug: _____

9. In one sentence, explain what the and operator does.

10. Write code that prints True only if 10 is greater than 5 and 3 is less than 7.

Mixed Quiz 3

This quiz mixes different question types: multiple choice, True/False, short answer, and code writing. Try to answer all of them without looking back at the chapter first — that is how you really learn. Answers are in the Answer Key at the back of the book.

1. What is the value of 2 + 3 * 4?

A. 20 B. 14
C. 24 D. 9

2. What does 17 % 5 evaluate to?

A. 3 B. 2
C. 3.4 D. 12

3. What does True and False evaluate to?

A. True B. False
C. None D. An error

4. What is 2 ** 3 ** 2?

A. 64 B. 512
C. 12 D. 36

5. Which line is equivalent to score = score + 5?

A. score + 5 B. score += 5
C. score++ 5 D. score =+ 5

6. True or False: In Python, the expression 5 > 3 > 1 is valid and evaluates to True.

☐ True ☐ False

7. True or False: The expression not (5 > 3) evaluates to True.

☐ True ☐ False

8. True or False: The multiplication operator * has higher precedence than the exponent operator **.

☐ True ☐ False

9. What is 2 + 3 * 4 - 1?

A. 13 B. 14
C. 23 D. 11

10. What is the result of True or False and False?

A. True B. False
C. An error D. None

11. True or False: 10 != 10 evaluates to False.

☐ True ☐ False

12. True or False: The expression 5 < 10 evaluates to True.

☐ True ☐ False

13. Explain in your own words what the modulo operator % does, and give one real-world example of when you might use it.

14. Why does Python have an order of precedence for operators? What would happen if it did not?

15. Write an expression that evaluates to True if the variable n is an even number between 1 and 100 (inclusive), and False otherwise.

Chapter 3 Challenges

These questions are harder than the quiz. They ask you to combine ideas from this chapter, think creatively, and apply what you learned in new ways. Do not worry if you cannot solve them all on your first try — the point is to make you think. The full solutions are in the Answer Key at the back of the book.

Challenge 1. What is the value of 2 + 3 * 4 - 1? Explain the order of operations.

*Hint: Remember: * before + and -, then left to right.*

Challenge 2. Write a single expression that is True if a number n is divisible by both 3 AND 5 (like 15, 30, 45). Use modulo and the and operator.

Hint: n % 3 == 0 and n % 5 == 0.

Challenge 3. What does 3 > 2 > 1 evaluate to? This is called a chained comparison. Explain how Python interprets it.

Hint: Python reads it as (3 > 2) and (2 > 1).

Challenge 4. Without running the code, predict what print(2 ** 3 ** 2) outputs. Explain why.

*Hint: ** is right-associative, so the right side is done first.*

Challenge 5. A shop gives a 10% discount if your total is over 100. Write a single expression that computes the final price: if total > 100, the price is total * 0.9; otherwise, it is just total. (Hint: Python has an inline "ternary" operator: value_if_true if condition else value_if_false.)

*Hint: The ternary operator is: result = total * 0.9 if total > 100 else total*

Chapter Recap

Here are the most important ideas from this chapter. If you can explain each one in your own words, you are ready to move on.

- Arithmetic operators: + - * / // % **.
- % gives the remainder; // gives the integer part of division; ** raises to a power.
- Python follows maths precedence rules: ** first, then * / // %, then + -. Use parentheses when in doubt.
- Augmented assignment (+= -= *= /=) is a shortcut for updating a variable.
- Comparison operators (== != < > <= >=) produce booleans.
- Logical operators and, or, not combine booleans.

Common Mistakes

Every beginner makes these mistakes. Knowing them in advance will save you hours of debugging.

COMMON MISTAKE

Confusing = with ==
if x = 5: is an error. if x == 5: is a comparison. Make this distinction a habit early.

COMMON MISTAKE

Mixing up integer and true division
10 / 2 gives 5.0 (a float), not 5 (an int). Use // if you need an integer result.

COMMON MISTAKE

Forgetting operator precedence
When you write 2 + 3 * 4, Python does the multiplication first. Add parentheses if you want the addition first: (2 + 3) * 4.

COMMON MISTAKE

Using and / or incorrectly
if age > 13 and < 19: is invalid Python. You must write if age > 13 and age < 19: — each side of and needs a complete comparison.

MINI-PROJECT

Mini-Project 1: Simple Calculator

You have learned about variables, arithmetic operators, input, output, and conditionals. Now let us put them together to build a simple calculator. The calculator will ask the user for two numbers and an operation (add, subtract, multiply, or divide), then display the result. This is your first complete program that combines multiple concepts.

How It Works

The calculator asks the user for two numbers and an operation choice. Based on the choice, it performs the calculation and prints the result. It also handles the case where the user tries to divide by zero.

The Code

Here is the complete calculator program. Read through it, type it into IDLE, and run it. Then try modifying it — for example, add a power operation or a modulo operation.

PYTHON · calculator.py

```
# Simple Calculator
print("=== Simple Calculator ===")

# Get two numbers from the user
num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))

# Show the operation choices
print("\nChoose an operation:")
print("1. Add (+)")
print("2. Subtract (-)")
print("3. Multiply (*)")
print("4. Divide (/)")

choice = input("Enter 1-4: ")

# Perform the calculation
if choice == "1":
    result = num1 + num2
    print(f"{num1} + {num2} = {result}")
elif choice == "2":
    result = num1 - num2
    print(f"{num1} - {num2} = {result}")
elif choice == "3":
    result = num1 * num2
    print(f"{num1} * {num2} = {result}")
elif choice == "4":
    if num2 == 0:
        print("Error: Cannot divide by zero!")
    else:
        result = num1 / num2
        print(f"{num1} / {num2} = {result}")
else:
    print("Invalid choice. Please run again.")
```

Things to Try

Once you have the basic calculator working, try these extensions:

- Add a power operation (choice 5) that uses ** to calculate num1 to the power of num2.
- Add a modulo operation (choice 6) that uses % to show the remainder.
- Wrap the whole program in a while loop so the user can do multiple calculations without restarting. Add an option to quit.
- Use round(result, 2) to display the result with at most 2 decimal places.

CHAPTER 4

Input and Output

So far, every value in your programs has been written directly into the code. Real programs need to talk to the user — ask questions, display results, and format output nicely. This chapter teaches you how to read what the user types and how to display information in a clean, readable way.

Lesson 4.1: Getting Input with input()

The input() function pauses your program and waits for the user to type something. When the user presses Enter, whatever they typed is returned to your program as a string. You can show a prompt message by passing it as an argument:

PYTHON · greeting.py

```
name = input("What is your name? ")
print("Hello, " + name + "!")
```

When you run this program, the message *What is your name?* appears on the screen. The program waits. You type your name and press Enter. Python stores what you typed in the variable name and then continues to the next line, which prints a personalised greeting.

COMMON MISTAKE

input() always returns a string, even if the user typed a number. If you ask age = input("Your age? ") and the user types 14, then age holds the string "14", not the integer 14. You cannot do maths with it until you convert it.

Reading Numbers

To read a number from the user, wrap input() in int() or float():

PYTHON · age.py

```
age_text = input("How old are you? ")
age = int(age_text)
next_year = age + 1
print("Next year you will be", next_year)
```

Many programmers combine the two steps into one line: age = int(input("How old are you? ")). This is fine for short programs, but be aware that if the user types something that is not a number, like "hello", the program will crash with a ValueError. You will learn how to handle that gracefully in Chapter 12.

Lesson 4.1 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What type does input() always return?

A. str
B. int
C. float
D. It depends on what the user types

2. How do you convert user input to an integer?

A. int(input())
B. int_input()
C. input().int()
D. input(int)

3. What does input("What is your name? ") display?

A. Nothing — it just waits for input
B. An error
C. The user's name
D. The prompt "What is your name? " then waits for input

4. If the user types 42 in response to input(), what type is the result?

A. int
B. float
C. str
D. bool

Lesson 4.1 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What type does input() always return?

A. int
B. float
C. str
D. bool

2. How do you show a message before the user types?

A. Use print() first always
B. You cannot
C. Use comment()
D. Pass it to input(): input("Name: ")

3. What does input() do?

A. Prints text
B. Deletes a file
C. Stops and waits for the user to type a line
D. Closes the program

4. If the user types 10 in response to input(), what is the type of the result?

A. int
B. float
C. str
D. bool

5. How do you turn the user input "5" into the integer 5?

A. str(input())
B. float(input())
C. You cannot
D. int(input())

6. True or False: input() always returns a string, even if the user types a number.

☐ True ☐ False

7. If the user types Bob, what does this print?

PYTHON

```
name = input("Name: ")
print("Hi", name)
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
age = input()
print(age + 1)
```

Bug: _____

9. In one sentence, explain why you often need int() around input().

10. Write code that asks the user "How old are you?" and prints back their age plus 1.

Lesson 4.2: Print Formatting

The print() function is more powerful than you might think. You can pass it several arguments separated by commas, and it will print them all on the same line, separated by spaces:

PYTHON · print_args.py

```
name = "Maya"
score = 87
print("Player", name, "scored", score, "points.")
# Output: Player Maya scored 87 points.
```

The default separator is a single space. You can change it with the sep argument. You can also change what print adds at the end of the line — by default, a newline — using the end argument:

PYTHON · print_sep_end.py

```
print("a", "b", "c", sep="-") # a-b-c
print("Loading", end="... ") # Loading... (no newline)
print("Done") # Done appears on same line
```

Lesson 4.2 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What is the default separator used by print() between arguments?

A. A comma
B. A space
C. A newline
D. No separator

2. How do you change what print() adds at the end of the line?

A. Use the sep argument
B. Use the end argument
C. Use the stop argument
D. You cannot change it

3. What does print("a", "b", "c", sep="-") output?

A. a-b-c
B. a b c
C. abc
D. a, b, c

4. What does print("Hello", end="") do differently from print("Hello")?

A. It prints nothing
B. It does not add a newline at the end
C. It adds an extra space
D. It causes an error

Lesson 4.2 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What does sep do in print()?

A. Ends the line
B. Stops printing
C. Sorts the items
D. Sets the separator between arguments

2. What does end do in print()?

A. Ends the program
B. Sets what is printed at the end (default newline)
C. Deletes the line
D. Sorts items

3. What is the default separator in print()?

A. Space
B. Comma
C. Newline
D. Nothing

4. What does print("a", "b", sep="-") print?

A. a b
B. a-b
C. ab
D. a b-

5. What does print("Hi", end="") do?

A. Prints Hi with no newline after
B. Prints nothing
C. Prints Hi twice
D. Causes an error

6. True or False: You can change what print() adds at the end of a line using the end parameter.

☐ True ☐ False

7. What does this code print?

PYTHON

```
print("a", "b", "c", sep="-")
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
print("Hi" sep="-")
```

Bug: _____

9. In one sentence, explain what the sep parameter does.

10. Write a print() call that prints 1, 2, 3 separated by commas (like 1,2,3).

Lesson 4.3: f-strings (Formatted Strings)

The cleanest way to combine text and values in modern Python is the **f-string**. An f-string is a normal string prefixed with the letter f. Inside an f-string, anything between curly braces { } is treated as Python code — its

value is computed and inserted into the string.

PYTHON · fstring.py

```
name = "Alex"
age = 14
print(f"Hello, {name}! You are {age} years old.")
# Output: Hello, Alex! You are 14 years old.

price = 19.99
qty = 3
print(f"Total: ${price * qty:.2f}")
# Output: Total: $59.97
```

The expression {price * qty:.2f} shows a powerful feature: after the expression, you can put a colon and a **format specifier**. Here .2f means "format as a float with 2 decimal places". This is perfect for displaying money, percentages, or any value where you want a fixed number of decimal places.

- {value:.2f} — float with 2 decimal places: 3.14
- {value:.0f} — float rounded to 0 decimal places: 3
- {value:10} — string padded to width 10: "hello "
- {value:>10} — right-aligned in width 10
- {value:^10} — centred in width 10

TIP

Use f-strings for almost all your output. They are clearer than the old "%s" % value style and faster than the ".format() method. They have been the recommended way to format strings in Python since version 3.6.

Lesson 4.3 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. How do you write an f-string in Python?

A. "Hello {name}"f
B. f"Hello {name}"
C. "f-Hello {name}"
D. format("Hello {name}")

2. What does f"My name is {name}" do if name = "Alex"?

A. It prints: My name is {name}
B. It causes an error
C. It prints: My name is + Alex
D. It prints: My name is Alex

3. What does f"{3.14159:.2f}" produce?

A. 3.14159
B. 3.1
C. 3
D. 3.14

4. Can f-strings contain expressions (like arithmetic)?

A. Yes, any valid Python expression
B. No, only variable names
C. Only simple addition
D. Only string operations

Lesson 4.3 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. How do you write an f-string?

A. f"Hello {name}" B. "Hello {name}"
C. f(Hello {name}) D. format("Hello {name}")

2. What does f"Hi {name}" do if name is "Sam"?

A. Prints Hi {name} B. Causes an error
C. Prints Hi name D. Prints Hi Sam

3. What does f"{3 + 4}" produce?

A. "7" B. "3 + 4"
C. 7 D. error

4. What does f"{price:.2f}" do if price is 3.5?

A. Rounds to integer B. Formats to 2 decimal places: "3.50"
C. Adds 2 D. Causes an error

5. Which is the correct f-string?

A. f"Hello {name}" B. f"Hello (name)"
C. f"Hello [name]" D. f"Hello "

6. True or False: f-strings let you put variables and expressions directly inside a string using {}.

☐ True ☐ False

7. What does this code print?

PYTHON

```
name = "Ana"
print(f"Hi {name}!")
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
name = "Ana"
print("Hi {name}")
```

Bug: _____

9. In one sentence, explain what an f-string is.

10. Write an f-string that prints: My age is 12 (use a variable age = 12).

Lesson 4.4: Putting It Together: A Small Conversation

Here is a small program that asks the user a few questions and uses their answers in the reply. Notice how input(), int(), arithmetic, and f-strings all work together.

PYTHON · chatbot.py

```
# A friendly little chatbot
print("Hi! Let me get to know you.")

name = input("What is your name? ")
age = int(input("How old are you? "))
city = input("Which city do you live in? ")

years_to_100 = 100 - age

print(f"Nice to meet you, {name}!")
print(f"You will turn 100 in the year {2026 + years_to_100}.")
print(f"{city} is a wonderful place to live.")
```

Try running this program and answering with your own details. Then modify it: ask the user for their favourite colour, their favourite number, or what they want to be when they grow up. The more you experiment, the more you will understand.

Lesson 4.4 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. In the chatbot example, what does int(input(...)) do?

A. It reads a string and converts it to an integer in one step
B. It reads an integer directly
C. It prints an integer
D. It causes an error

2. What happens if the user types "hello" when the program does int(input("Age? "))?

A. It raises a ValueError
B. It returns 0
C. It returns None
D. It returns the string "hello"

3. What does f"{2026 + years_to_100}" compute?

A. The string "2026 + years_to_100"
B. The sum of 2026 and years_to_100, inserted into the string
C. An error
D. Nothing

4. Why is the int() conversion necessary in the chatbot program?

A. Because Python requires all variables to be integers
B. Because it makes the program faster
C. It is not necessary; it is just a style choice
D. Because input() returns a string and you cannot subtract a string from an integer

Lesson 4.4 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. In a chatbot, what function reads the user reply?

A. print()
B. input()
C. reply()
D. get()

2. Why do we convert input() to int for a number question?

A. Because input() returns a string
B. To make it print
C. To slow the program
D. It is optional

3. If the user types 5, what is printed by: age = int(input()) then print(age * 2)?

A. 55 B. 10
C. 5 * 2 D. error

4. Which line asks the user for their name?

A. print("Name: ") B. name = print("Name: ")
C. input = name() D. name = input("Name: ")

5. In a chatbot, what is the typical order?

A. Input, then print, then nothing B. Respond, then input
C. Random D. Print, then input, then respond

6. True or False: A chatbot usually uses input() to get the user reply and print() to respond.

☐ True ☐ False

7. If the user types 3, what does this print?

PYTHON

```
n = int(input())
print(n + 7)
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
age = input()
print("Next year:", age + 1)
```

Bug: _____

9. In one sentence, explain why a chatbot needs input() and print().

10. Write a tiny chatbot that asks "What is your name?" and prints "Hello, " plus the name.

Mixed Quiz 4

This quiz mixes different question types: multiple choice, True/False, short answer, and code writing. Try to answer all of them without looking back at the chapter first — that is how you really learn. Answers are in the Answer Key at the back of the book.

1. What type does input() always return?

A. int B. float
C. str D. Whatever the user typed

2. How do you convert user input to an integer?

A. int_input() B. input(int)
C. int(input()) D. input().to_int()

3. What does the following print? print("a", "b", "c", sep="-")

A. a b c B. abc
C. a-b-c D. a, b, c

4. How do you write an f-string?

A. f"Hello {name}" B. "Hello {name}"f

C. "f-Hello {name}" D. format("Hello {name}")

5. What does print(f"{3.14159:.2f}") display?

A. 3.14159 B. 3.14

C. 3.1 D. 3

6. True or False: You can change what print adds at the end of the line using the end argument.

☐ True ☐ False

7. True or False: The default separator used by print between arguments is a comma.

☐ True ☐ False

8. True or False: f-strings can contain any valid Python expression inside the curly braces.

☐ True ☐ False

9. What is the value of name after name = input("Name: ") if the user types Maya and presses Enter?

A. The integer 4 B. The string "Maya"

C. The string "Name: Maya" D. None of the above

10. Which f-string correctly formats price as a dollar amount with two decimals?

A. f"${price}" B. f"${price:2}"

C. f"${price:.2f}" D. f"${price:float.2}"

11. True or False: The input() function pauses the program until the user presses Enter.

☐ True ☐ False

12. True or False: You can pass multiple arguments to print() separated by commas.

☐ True ☐ False

13. Explain why age = input("Age? ") followed by age + 1 will cause an error, and how to fix it.

14. Give one situation where you would use the end argument of print().

15. Write a program that asks the user for the price of an item and the quantity they want to buy, then prints the total cost formatted as dollars with two decimal places (e.g. $15.99).

Lesson 4.5: Importing Modules

So far, you have used only built-in functions like print(), input(), and len(). These are available automatically. But Python has many more useful tools stored in **modules** — separate files of code that you can load into your program using the import keyword.

A **module** is a file containing Python code that you can reuse. Think of it as a toolbox: you do not need to build the tools yourself — you just open the toolbox and take what you need. Python comes with dozens of built-in modules for maths, random numbers, dates, file handling, and much more.

The import Statement

To use a module, you write import followed by the module name. This loads the module and makes its functions available through the module name with a dot:

PYTHON · math_example.py

```
import math

# Use math.sqrt() to calculate a square root
result = math.sqrt(25)
print(result) # 5.0

# Use math.pi for the value of pi
print(math.pi) # 3.141592653589793

# Use math.ceil() to round up
print(math.ceil(3.2)) # 4

# Use math.floor() to round down
print(math.floor(3.8)) # 3
```

Notice the pattern: module_name.function_name(). The dot tells Python "look inside this module for this function." You have already seen this pattern with string and list methods like "hello".upper() and [1, 2].append(3).

The random Module

The random module is one of the most fun and useful modules. It lets you generate random numbers, which is essential for games, simulations, and quizzes:

PYTHON · random_example.py

```
import random

# Random integer between 1 and 6 (like a dice roll)
roll = random.randint(1, 6)
print(f"You rolled a {roll}")

# Random floating-point number between 0 and 1
chance = random.random()
print(f"Random decimal: {chance}")

colours = ["red", "green", "blue", "yellow"]
choice = random.choice(colours)
print(f"The winner is: {choice}")
```

The random module is used in many real programs. Game developers use it for dice rolls and card shuffling. Teachers use it to create random quiz questions. Scientists use it for simulations. Whenever you need unpredictability, import random is your friend.

Useful Built-in Functions

Python also has many built-in functions that do not need to be imported. Here are the most useful ones you should know:

PYTHON · builtins.py

```
# len() — counts the number of items
print(len("Hello")) # 5 (5 characters)
print(len([1, 2, 3])) # 3 (3 items)
print(len("Python")) # 6

# round() — rounds a number to a given number of decimals
print(round(3.14159, 2)) # 3.14
print(round(2.7)) # 3

# abs() — absolute value (removes the minus sign)
print(abs(-5)) # 5
print(abs(5)) # 5

# min() and max() — smallest and largest value
print(min(3, 7, 1, 9)) # 1
print(max(3, 7, 1, 9)) # 9
print(min([10, 20, 5])) # 5

# sum() — adds up all items in a list
print(sum([1, 2, 3, 4, 5])) # 15

# sorted() — returns a new sorted list
numbers = [3, 1, 4, 1, 5, 9]
print(sorted(numbers)) # [1, 1, 3, 4, 5, 9]

# .count() — counts how many times an item appears
print("hello".count("l")) # 2
print([1, 2, 2, 3, 2].count(2)) # 3

# .find() — finds the position of a substring (-1 if not found)
print("hello".find("l")) # 2 (first occurrence)
print("hello".find("z")) # -1 (not found)

# set() — create an empty set (NOT {} which makes a dict)
empty_set = set()
print(type(empty_set)) # &lt;class 'set'&gt;
```

TIP

You do not need to memorise all of these. The important thing is to know they exist, so when you need to find the biggest number in a list, you remember "there was a function for that" and look it up.

Escape Sequences

Sometimes you need to put special characters inside a string that would normally be impossible to type. For example, how do you put a quotation mark inside a string that is wrapped in quotation marks? Or how do you create a new line in the middle of a string?

Python uses **escape sequences** — a backslash \ followed by a letter — to represent these special characters:

Sequence	Meaning	Example
\n	New line	`print("Line 1\nLine 2")`
\t	Tab	`print("A\tB") # A B`
\'	Single quote	`print('It\'s OK')`
\"	Double quote	`print("She said \"Hi\"")`
\\	Backslash	`print("C:\\Users")`

Table 4.1 — Common escape sequences.

The most common escape sequence is \n (new line). You have already seen it in file handling, where f.write("Hello\n") writes "Hello" followed by a new line. Without the \n, the next write would continue on the same line.

PYTHON · escapes.py

```
# \n creates a new line
print("First line\nSecond line\nThird line")

# \t creates a tab (useful for alignment)
print("Name\tAge")
print("Alex\t14")
print("Maya\t15")

# \\\\ produces a single backslash
print("File path: C:\\Users\\Alex")
```

You will use escape sequences throughout your programming career. The \n new line is the most common, but knowing the others will save you from confusing errors when your strings contain quotation marks or backslashes.

Lesson 4.5 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What keyword loads a module into your program?

A. load
B. include
C. import
D. module

2. How do you use the sqrt function from the math module?

A. sqrt(25)
B. import sqrt(25)
C. math.sqrt(25)
D. math:sqrt(25)

3. What does len("Hello") return?

A. "Hello" B. 6
C. 5 D. H

4. What does random.randint(1, 6) return?

A. A random float between 0 and 1 B. A random integer between 1 and 6 (inclusive)
C. The number 6 D. A random string

Lesson 4.5 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What keyword loads a module?

A. load B. import
C. include D. module

2. How do you access sqrt in the math module?

A. sqrt(25) B. math.sqrt(25)
C. math->sqrt(25) D. import.sqrt(25)

3. What does len([1, 2, 3, 4]) return?

A. [1, 2, 3, 4] B. 4
C. 3 D. 1

4. What does round(3.7) return?

A. 3 B. 4
C. 3.7 D. 3.5

5. What does sum([10, 20, 30]) return?

A. [10, 20, 30] B. 10
C. 60 D. 3

6. True or False: You must import random before using random.randint().

☐ True ☐ False

7. What does this print?

PYTHON

```
import math
print(math.sqrt(16))
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
import random
roll = random.randint(1, 6)
print(roll)
```

Bug: _____

9. In one sentence, explain what a module is.

10. Write a program that rolls a 6-sided dice and prints the result.

Chapter 4.5 Challenges

These questions are harder than the quiz. They ask you to combine ideas from this chapter, think creatively, and apply what you learned in new ways. Do not worry if you cannot solve them all on your first try — the point is to make you think. The full solutions are in the Answer Key at the back of the book.

Challenge 1. Write a program that imports the random module and simulates rolling two dice. Print both rolls and their sum.

Hint: Use random.randint(1, 6) twice, then add them.

Challenge 2. What does this code print? Explain why. import math print(round(math.pi, 2))

Hint: math.pi is 3.14159... and round(x, 2) rounds to 2 decimals.

Challenge 3. Write a program that asks the user for a number, then prints its square root using math.sqrt(). Handle the case where the number is negative by printing a friendly message.

Hint: Use try/except or an if statement to check for negative numbers.

Challenge 4. What is the difference between random.random() and random.randint(1, 10)? Explain what each returns.

Hint: One returns a float, the other returns an integer.

Challenge 5. Write a program that picks a random name from a list of 5 friends and prints "The winner is: !" using random.choice().

Hint: Create a list, then call random.choice() on it.

Chapter 4 Challenges

These questions are harder than the quiz. They ask you to combine ideas from this chapter, think creatively, and apply what you learned in new ways. Do not worry if you cannot solve them all on your first try — the point is to make you think. The full solutions are in the Answer Key at the back of the book.

Challenge 1. Write a program that asks the user for two numbers, adds them, and prints the result in the format "5 + 3 = 8" using an f-string.

Hint: Use two input() calls, convert both to int, then use an f-string.

Challenge 2. What happens if you run print("a", "b", "c", sep="", end="!") ? Predict the exact output and explain.

Hint: sep="" means no separator; end="!" means "!" instead of newline.

Challenge 3. Write a program that asks the user for their name and age, then prints: "Hi Alex! You are 14 years old. In 10 years you will be 24." Use f-strings and int() conversion.

Hint: Two inputs: name (string) and age (needs int conversion).

Challenge 4. A program does: x = input("Enter a number: ") and then tries print(x * 3). If the user types 5, what is printed? Is it 15 or something else? Explain why.

*Hint: Remember: input() returns a string. What does "5" * 3 do?*

Challenge 5. Write a single print() call that produces exactly this output on one line: Hello*World (using the sep argument).**

*Hint: You need two arguments and sep="***".*

Chapter Recap

Here are the most important ideas from this chapter. If you can explain each one in your own words, you are ready to move on.

- input(prompt) reads a line of text from the user and always returns a string.
- Convert input to a number with int(input(...)) or float(input(...)).
- print() accepts multiple arguments separated by commas. Use sep to change the separator and end to change what is printed at the end of the line.
- f-strings (e.g. f"Hello {name}") are the modern, clean way to combine text and values.
- Format specifiers like :.2f control how numbers are displayed (e.g. two decimal places).

Common Mistakes

Every beginner makes these mistakes. Knowing them in advance will save you hours of debugging.

COMMON MISTAKE

Forgetting to convert input to a number
age = input("Age? ") gives a string. Adding 1 with age + 1 will crash. Always convert when you need a number: age = int(input("Age? ")).

COMMON MISTAKE

Mixing up sep and end
sep goes *between* the arguments; end goes after the last one. Mixing them up gives confusing output.

COMMON MISTAKE

Forgetting the f prefix
"Hello {name}" prints literally Hello {name}. You need f"Hello {name}" to insert the variable.

COMMON MISTAKE

Trying to concatenate a string with a number using +
"Total: " + 5 is an error. Use an f-string (f"Total: {5}") or convert with str().

CHAPTER 5

Making Decisions — Conditionals

The programs you have written so far run straight from top to bottom, doing the same thing every time. Real programs need to make decisions: if the user is old enough, let them in; otherwise, turn them away. Python uses **conditional statements** — if, elif, and else — to do this. This is one of the most important ideas in programming, so read carefully.

Lesson 5.1: The if Statement

An if statement runs a block of code only when a condition is true. The syntax is simple: the word if, followed by a condition, followed by a colon :, followed by an indented block of code. The indentation (usually four spaces) tells Python which lines belong to the if block.

PYTHON · if_basic.py

```
age = 18
if age >= 18:
    print("You are old enough to vote.")
    print("Do not forget to register!")

print("Program finished.")
```

If age is 18 or more, all three lines run. If age is less than 18, the two indented lines are skipped, and only Program finished. is printed. The indentation is not optional — it is how Python knows which lines are inside the if block.

Figure 5.1 — How an if / else statement flows

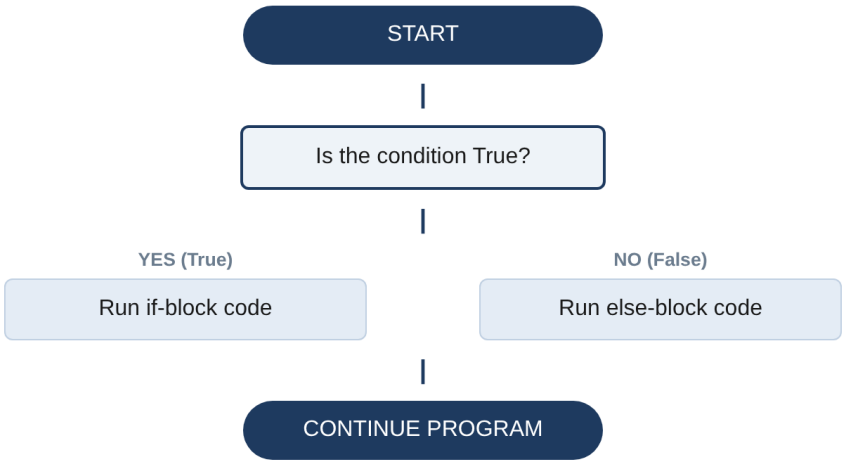


Figure 5.1 — How an if / else statement chooses which block of code to run.

Lesson 5.1 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What symbol must follow the condition in an if statement?

A. ;
B. :
C. ,
D. ->

2. How does Python know which lines belong to the if block?

A. Curly braces
B. Parentheses
C. Indentation
D. The word "end"

3. What happens if the if condition is False?

A. The program crashes
B. The indented block runs anyway
C. The indented block is skipped
D. Python asks the user what to do

4. How many spaces should you indent a block?

A. 2
B. 4
C. It does not matter as long as it is consistent
D. 8

Lesson 5.1 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What must come at the end of an if line?

A. A semicolon
B. A period
C. A colon
D. A comma

2. How does Python know which lines belong to the if?

A. By indentation
B. By braces {}
C. By commas
D. By quotes

3. What happens if the if condition is False?

A. The block runs anyway
B. The program crashes
C. It prints None
D. The block is skipped

4. Which is the correct if statement?

A. if x > 5:
B. if (x > 5)
C. if x > 5
D. if x > 5;

5. How many spaces are usually used for indentation?

A. 1
B. 2
C. 4
D. 8

6. True or False: The body of an if statement must be indented.

☐ True ☐ False

7. What does this code print?

PYTHON

```
x = 10
if x > 5:
    print("big")
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
if x > 5
    print("big")
```

Bug: _____

9. In one sentence, explain what an if statement does.

10. Write an if statement that prints "Adult" when age is 18 or more.

Lesson 5.2: if / else

Often you want to do one thing when the condition is true and a different thing when it is false. The else keyword lets you do this. The else block runs only when the if condition is false.

PYTHON · if_else.py

```
temperature = int(input("What is the temperature? "))

if temperature > 25:
    print("It is hot outside. Drink water!")
else:
    print("It is not too hot. Enjoy the day!")
```

Run this program twice — once with 30 and once with 15 — and notice that exactly one of the two messages is printed. The else block is a catch-all: it runs for every case that does not match the if.

Lesson 5.2 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What keyword do you use for the "otherwise" branch?

A. elseif
B. otherwise
C. elif
D. else

2. In an if/else, how many blocks can run?

A. 0
B. Exactly 2
C. Both
D. Exactly 1

3. What happens if the if condition is False in an if/else?

A. Neither block runs
B. The if block runs
C. The else block runs
D. Both blocks run

4. Can you have an if without an else?

A. No, they must always come in pairs
B. Only inside functions
C. Only in Python 3
D. Yes, the else is optional

Lesson 5.2 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What does else mean in an if/else?

A. Another condition
B. A loop
C. The block that runs when the if is False
D. A comment

2. How many else can one if have?

A. Zero or one
B. Many
C. At least two
D. Exactly three

3. What comes after the word else?

A. A condition
B. A semicolon
C. Nothing
D. A colon

4. In if/else, how many blocks can run?

A. Both
B. Neither
C. Up to three
D. Exactly one

5. Which is the correct if/else?

A. if x: ... else: ...
B. if x: ... else ...
C. else: ... if x:
D. if x else: ...

6. True or False: In an if/else, exactly one of the two blocks will run.

☐ True ☐ False

7. What does this code print?

PYTHON

```
x = 3
if x > 5:
    print("big")
else:
    print("small")
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
if x > 5:
    print("big")
else
    print("small")
```

Bug: _____

9. In one sentence, explain what the else block does.

10. Write an if/else that prints "Pass" if score is 50 or more, else prints "Fail".

Lesson 5.3: if / elif / else

When you have more than two cases, use elif (short for "else if"). You can have as many elif blocks as you like. Python checks them in order, from top to bottom, and runs the first one whose condition is true. If none of them is true, the else block runs (if there is one).

PYTHON · grades.py

```
score = int(input("Enter your test score (0-100): "))

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:
    grade = "F"

print(f"Your grade is {grade}.")
```

Notice that we do not need to write score < 90 in the second condition. If we reach the elif score >= 80 line, we already know that score < 90, because the if above was false. This is called **first-match branching** — Python checks each condition in order and runs only the first one that is True. This makes your code cleaner.

TIP

Order matters. Always put your conditions from most specific to most general, because Python stops at the first match. If you wrote if score >= 60: first, every score from 60 to 100 would get a D, and the other branches would never run.

Lesson 5.3 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What keyword means "else if" in Python?

A. elseif
B. else if
C. elsf
D. elif

2. In an if/elif/else chain, how many branches can run?

A. All of them
B. Exactly two
C. It depends on the conditions
D. At most one

3. If the if condition is True, does Python check the elif conditions?

A. Yes, always
B. No, it skips them
C. Only if there is no else
D. Only the first elif

4. Why do you not need to write score < 90 in the elif after checking if score >= 90?

A. Because if we reach the elif, we already know the if was False (score < 90)
B. Because Python automatically adds the check
C. Because it does not matter
D. You DO need to write it

Lesson 5.3 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What does elif stand for?

A. Element if
B. Else if
C. Extra if
D. End if

2. How many elif can one if have?

A. Only one
B. Exactly two
C. Zero or more
D. None

3. When does an elif run?

A. Always
B. Never
C. Only if all earlier conditions were False and the elif condition is True
D. Only if the else ran

4. What comes after the word elif?

A. Just a colon
B. A semicolon
C. A condition and a colon
D. Nothing

5. In if/elif/else, how many blocks can run?

A. All of them
B. Exactly two
C. None
D. At most one

6. True or False: Once a branch in if/elif/else is True, the rest are skipped.

☐ True ☐ False

7. What does this code print?

PYTHON

```
x = 5
if x > 10:
    print("a")
elif x > 3:
    print("b")
else:
    print("c")
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
if x > 10:
    print("a")
elif x > 3
    print("b")
```

Bug: _____

9. In one sentence, explain how if/elif/else is checked.

10. Write an if/elif/else that prints "hot" if temp is above 30, "warm" if above 20, else "cold".

Lesson 5.4: Nested Conditionals

You can put an if statement inside another if statement. This is called **nesting**. It is useful when you need to check a second condition only after the first one is true.

PYTHON · nested.py

```
age = int(input("Your age? "))
has_id = input("Do you have ID? (yes/no) ")

if age >= 18:
    if has_id == "yes":
        print("Welcome. You may enter.")
    else:
        print("You are old enough, but you need ID.")
else:
    print("Sorry, you must be 18 or older.")
```

Nesting is powerful, but deep nesting makes code hard to read. As a rule of thumb, if you find yourself nesting more than two or three levels deep, think about whether you can simplify the logic — sometimes combining conditions with and / or is cleaner than nesting if statements.

Lesson 5.4 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What is a "nested" if statement?

A. An if inside another if
B. An if with many elif branches
C. An if without an else
D. An if that never runs

2. Why should you avoid deeply nested if statements?

A. They are faster
B. They cause errors
C. They make code hard to read
D. They are not allowed in Python

3. What is an alternative to nesting if statements?

A. Using a for loop instead
B. Using a while loop instead
C. There is no alternative to nesting
D. Combining conditions with and/or in a single if

4. In a nested if, when does the inner if get checked?

A. Always
B. Only when the outer if condition is True
C. Only when the outer if is False
D. Never

Lesson 5.4 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What is a nested conditional?

A. An if inside another if
B. A loop inside an if
C. A function inside an if
D. A comment inside an if

2. Why use nested conditionals?

A. To loop
B. To check a second condition only when the first is True
C. To print
D. To define a variable

3. How does Python know which if is nested?

A. By braces
B. By color
C. By length
D. By indentation

4. What is wrong with too many nested ifs?

A. The code becomes hard to read
B. Nothing
C. They run slower
D. They cause errors

5. In a nested if, when does the inner if run?

A. Always
B. Never
C. Only when the outer if is True
D. Only when the outer if is False

6. True or False: You can put an if/else inside another if statement.

☐ True ☐ False

7. What does this code print?

PYTHON

```
x = 10
if x > 5:
    if x > 8:
        print("very big")
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
if x > 5:
if x > 8:
        print("big")
```

Bug: _____

9. In one sentence, explain what a nested conditional is.

10. Write a nested if that prints "valid" only when age is 13 or more AND age is 100 or less.

Lesson 5.5: Combining Conditions

You can use the logical operators and, or, and not inside an if condition to test several things at once. This is often cleaner than nesting.

PYTHON · combined.py

```
age = int(input("Your age? "))
has_id = input("Do you have ID? (yes/no) ")

if age >= 18 and has_id == "yes":
    print("Welcome. You may enter.")
elif age >= 18 and has_id != "yes":
    print("You are old enough, but you need ID.")
else:
    print("Sorry, you must be 18 or older.")
```

Lesson 5.5 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What does if age >= 18 and has_id == "yes": check?

A. That age is at least 18 OR the person has ID
B. That age is exactly 18
C. That age is at least 18 AND the person has ID
D. That the person has ID

2. What does if not is_raining: mean?

A. It is always raining
B. It might rain
C. It is NOT raining
D. It is raining heavily

3. What does True or False evaluate to?

A. False
B. True
C. None
D. An error

4. Which is clearer: nesting ifs or using and?

A. Using and is usually clearer for independent conditions
B. Nesting is always clearer
C. They are equally clear
D. Neither is clear

Lesson 5.5 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. Which operator means "both must be True"?

A. or
B. and
C. not
D. xor

2. Which operator means "at least one must be True"?

A. and
B. not
C. ==
D. or

3. What does (x > 5) and (x < 10) check?

A. x is between 5 and 10
B. x is greater than 10
C. x is less than 5
D. x is anything

4. What does not (x > 5) check?

A. x is greater than 5
B. x is 5
C. x is anything
D. x is 5 or less

5. Which is True: (3 > 2) or (5 > 9)?

A. True
B. False
C. error
D. None

6. True or False: You can combine two conditions with and or or inside an if.

☐ True ☐ False

7. What does this code print?

PYTHON

```
x = 7
if x > 5 and x < 10:
    print("in range")
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
if x > 5 and < 10:
    print("ok")
```

Bug: _____

9. In one sentence, explain the difference between and and or.

10. Write an if that prints "teen" when age is 13 to 19 inclusive (use and).

Mixed Quiz 5

This quiz mixes different question types: multiple choice, True/False, short answer, and code writing. Try to answer all of them without looking back at the chapter first — that is how you really learn. Answers are in the Answer Key at the back of the book.

1. Which symbol must follow the condition in an if statement?

A. ; B. :
C. , D. ->

2. How does Python know which lines belong to the if block?

A. By curly braces { } B. By parentheses ( )
C. By indentation D. By the word "end"

3. What keyword do you use for "else if" in Python?

A. elseif B. elif
C. else if D. elsf

4. What does this print? if 5 > 3: print("A") else: print("B")

A. A B. B
C. AB D. Nothing

5. Which condition correctly checks that x is between 1 and 10 (inclusive)?

A. 1 <= x <= 10 B. x > 1 and x < 10
C. x >= 1 or x <= 10 D. 1 < x < 10

6. True or False: You can have multiple elif blocks in a single if statement.

☐ True ☐ False

7. True or False: The else block always runs if the if condition is false, even if an elif already ran.

☐ True ☐ False

8. True or False: Indentation in Python is just for looks — Python ignores it.

☐ True ☐ False

9. What is the output of this code?

```
x = 7
if x > 5:
    print("A")
elif x > 3:
    print("B")
else:
    print("C")
```

A. A
B. B
C. C
D. AB

10. In if a and b:, when is the block run?

A. Only when a is True
B. Only when b is True
C. Only when both a and b are True
D. When either a or b is True

11. True or False: You can nest an if statement inside another if statement.

☐ True ☐ False

12. True or False: A single if statement without else is valid Python.

☐ True ☐ False

13. Explain the difference between nesting if statements and using and in a single condition. When might you prefer one over the other?

14. What is first-match branching, and how does it make elif chains cleaner to write?

15. Write a program that asks the user for a number and tells them whether the number is positive, negative, or zero.

Chapter 5 Challenges

These questions are harder than the quiz. They ask you to combine ideas from this chapter, think creatively, and apply what you learned in new ways. Do not worry if you cannot solve them all on your first try — the point is to make you think. The full solutions are in the Answer Key at the back of the book.

Challenge 1. Write a program that asks the user for a number and prints "Positive", "Negative", or "Zero". Use if/elif/else.

Hint: Three branches: > 0, < 0, and == 0.

Challenge 2. What is wrong with this code? if age = 18: print("Adult")

Hint: Think about the difference between = and ==.

Challenge 3. Write a program that asks the user for a score (0-100) and prints a letter grade: A (90+), B (80+), C (70+), D (60+), F (below 60). Use if/elif/else. Why does the order matter?

Hint: Check from highest to lowest. If you check 60+ first, everyone gets D.

Challenge 4. Without using "and", write an if statement that prints "Yes" only when x > 0 AND x < 10. (Hint: use nested ifs.)

Hint: Put an if inside another if.

Challenge 5. A movie theater charges $10 for adults (18+), $7 for children (under 13), and $8 for teens (13-17). Write a program that asks for the age and prints the ticket price.

Hint: Three age ranges, so you need if/elif/else.

Chapter Recap

Here are the most important ideas from this chapter. If you can explain each one in your own words, you are ready to move on.

- if condition: runs the indented block only if the condition is True.
- else: runs when the if condition is False.
- elif lets you check multiple conditions in order. Python runs the first match.
- Indentation (4 spaces) defines which lines belong to a block.
- You can nest if statements or combine conditions with and, or, not.

Common Mistakes

Every beginner makes these mistakes. Knowing them in advance will save you hours of debugging.

COMMON MISTAKE

Forgetting the colon :
Every if, elif, and else line must end with a colon. Forgetting it is the most common syntax error for beginners.

COMMON MISTAKE

Mixing tabs and spaces
Python 3 does not allow mixing tabs and spaces in the same block. Configure your editor to insert four spaces when you press Tab, and you will never have this problem.

COMMON MISTAKE

Using = instead of ==
if x = 5: is an error. Use if x == 5: to compare. This mistake will keep biting you for years — get into the habit of catching it now.

COMMON MISTAKE

Wrong order of elif conditions
Always order conditions from most specific to most general. If you put if score >= 60 before if score >= 90, no one will ever get an A.

CHAPTER 6

Loops

Computers are excellent at doing the same thing over and over again, very fast, without getting bored. A **loop** is a way to tell Python to repeat a block of code. Python has two main kinds of loops: for loops (which repeat a known number of times) and while loops (which repeat until a condition changes). This chapter teaches both.

Lesson 6.1: for Loops and range()

A for loop repeats a block of code once for each item in a sequence. The most common sequence is range(), which generates a series of numbers.

PYTHON · for_basic.py

```
for i in range(5):
    print(f"Counting: {i}")

# Output:
# Counting: 0
# Counting: 1
# Counting: 2
# Counting: 3
# Counting: 4
```

Notice that range(5) gives the numbers 0 through 4 — not 1 through 5. Programmers count from zero, and range(n) always gives exactly n numbers starting from 0. This takes some getting used to, but it is consistent throughout Python.

The variable i in the loop is called the **loop variable**. You can name it anything you like — i, num, count, or anything else. Each time through the loop, the loop variable takes the next value from the sequence, and the indented block runs with that value.

The range() Function in Detail

range() can take one, two, or three arguments:

- range(n) — numbers 0, 1, 2, ..., n-1
- range(a, b) — numbers a, a+1, ..., b-1 (starts at a, stops just before b)
- range(a, b, step) — numbers a, a+step, a+2*step, ... (up to but not including b)

PYTHON · range.py

```
for n in range(1, 6): # 1, 2, 3, 4, 5
    print(n, end=" ")
print() # newline

for n in range(0, 10, 2): # 0, 2, 4, 6, 8 (even numbers)
    print(n, end=" ")
print()

for n in range(10, 0, -1): # 10, 9, 8, ..., 1 (counting down)
    print(n, end=" ")
print()
```

Lesson 6.1 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. How many times does for i in range(5): run?

A. 4 B. 6
C. 5 D. 0

2. What numbers does range(3) produce?

A. 1, 2, 3 B. 0, 1, 2, 3
C. 0, 1, 2 D. 1, 2

3. What numbers does range(1, 4) produce?

A. 1, 2, 3, 4 B. 0, 1, 2, 3
C. 2, 3, 4 D. 1, 2, 3

4. What numbers does range(0, 10, 2) produce?

A. 0, 2, 4, 6, 8, 10 B. 2, 4, 6, 8, 10
C. 1, 3, 5, 7, 9 D. 0, 2, 4, 6, 8

Lesson 6.1 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What does range(5) produce?

A. 1, 2, 3, 4, 5 B. 0, 1, 2, 3, 4, 5
C. 5 D. 0, 1, 2, 3, 4

2. What does range(2, 6) produce?

A. 2, 3, 4, 5, 6 B. 6, 5, 4, 3, 2
C. 2, 6 D. 2, 3, 4, 5

3. What does range(1, 10, 2) produce?

A. 1, 2, 3, 4, 5 B. 2, 4, 6, 8, 10
C. 1, 10, 2 D. 1, 3, 5, 7, 9

4. How many times does this loop run: for i in range(3)?

A. 3 B. 2
C. 4 D. 1

5. What keyword starts a for loop?

A. loop B. each
C. for D. repeat

6. True or False: range(5) includes 0 but not 5.

☐ True ☐ False

7. What does this code print?

PYTHON

```
for i in range(3):
    print(i)
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
for i in range(5)
    print(i)
```

Bug: _____

9. In one sentence, explain what range(a, b) produces.

10. Write a for loop that prints the numbers 1 to 5 (each on its own line).

Lesson 6.2: while Loops

A while loop repeats a block of code as long as a condition is True. It is the right choice when you do not know in advance how many times you need to repeat.

PYTHON · while_basic.py

```
count = 1
while count <= 5:
    print(f"Loop iteration {count}")
    count += 1

print("Loop finished!")
```

This program does the same thing as for i in range(1, 6). The big difference is that *you* are responsible for updating the variable that the condition depends on. If you forget the line count += 1, the loop will run forever — a mistake called an **infinite loop**. To stop an infinite loop in IDLE, press **Ctrl+C**.

Figure 6.1 — How a while loop flows

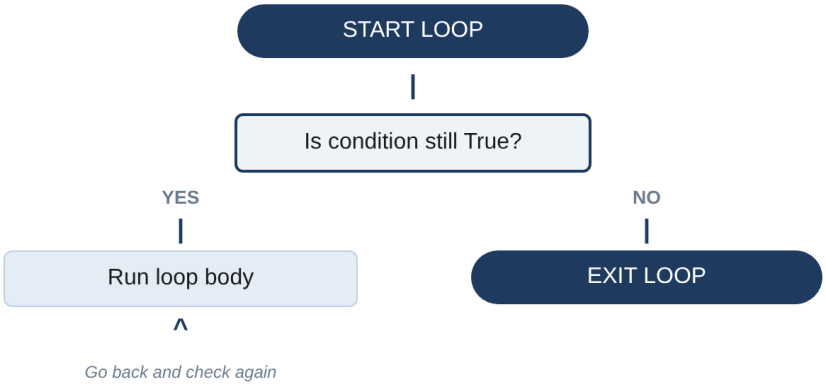


Figure 6.1 — A while loop keeps checking its condition and running its body until the condition becomes False.

Lesson 6.2 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What keyword creates a while loop?

A. while
B. for
C. loop
D. repeat

2. When does a while loop stop?

A. After 10 iterations
B. When the user presses Enter
C. When its condition becomes False
D. It never stops

3. What must you do inside a while loop to avoid an infinite loop?

A. Update the variable that the condition depends on
B. Use a break statement
C. Use a for loop instead
D. Nothing; while loops always stop

4. How do you stop an infinite loop in IDLE?

A. Press Ctrl+C
B. Close the window
C. Press Ctrl+Z
D. Restart the computer

Lesson 6.2 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What does a while loop need to keep running?

A. A False condition
B. A True condition
C. A number
D. A string

2. What must you do inside a while loop to avoid an infinite loop?

A. Update the variable used in the condition
B. Print every time
C. Use a string
D. Nothing

3. What is an infinite loop?

A. A loop that runs once
B. A loop that never ends
C. A loop with no body
D. A loop with no condition

4. Which is the correct while loop?

A. while (x < 5)
B. while x < 5
C. while x < 5;
D. while x < 5:

5. When does a while loop check its condition?

A. Only once at the start
B. Only at the end
C. Never
D. Before each iteration

6. True or False: A while loop may run zero times if its condition is False at the start.

☐ True ☐ False

7. What does this code print?

PYTHON

```
n = 0
while n < 3:
    print(n)
    n = n + 1
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
n = 0
while n < 3:
    print(n)
```

Bug: _____

9. In one sentence, explain how a while loop decides to stop.

10. Write a while loop that prints 5, 4, 3, 2, 1 (each on its own line).

Lesson 6.3: break and continue

Sometimes you need to escape a loop early, or skip the rest of the current iteration. Python gives you two keywords for this: break and continue.

PYTHON · break.py

```
# break: exit the loop immediately
while True:
    answer = input("Type 'quit' to stop: ")
    if answer == "quit":
        break
    print(f"You typed: {answer}")

print("Goodbye!")
```

PYTHON · continue.py

```
# continue: skip the rest of this iteration
for n in range(10):
    if n % 2 == 0:
        continue # skip even numbers
    print(n) # only odd numbers get printed: 1 3 5 7 9
```

Use break and continue sparingly. They make loops harder to follow because the flow of execution is no longer strictly top-to-bottom. Often, rewriting the loop with a clearer condition produces code that is easier to understand.

Lesson 6.3 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What does break do inside a loop?

A. Skips to the next iteration
B. Exits the loop immediately
C. Restarts the loop
D. Pauses the loop

2. What does continue do inside a loop?

A. Skips the rest of this iteration and moves to the next
B. Exits the loop
C. Restarts the program
D. Nothing

3. Which keyword exits a loop entirely?

A. continue
B. stop
C. exit
D. break

4. Should you use break and continue often?

A. Yes, always
B. No, use them sparingly because they make loops harder to follow
C. Only in for loops
D. They are not allowed in Python

Lesson 6.3 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What does break do in a loop?

A. Skips one turn
B. Stops the loop immediately
C. Restarts the loop
D. Prints the index

2. What does continue do in a loop?

A. Stops the loop
B. Skips the rest of this iteration and goes to the next
C. Restarts the program
D. Prints the index

3. Where can break be used?

A. Only in while loops
B. Only in for loops
C. Anywhere
D. In both for and while loops

4. What happens after break runs?

A. The loop continues
B. The loop ends and code after the loop runs
C. The program ends
D. Nothing

5. Which keyword skips just the current iteration?

A. break
B. pass
C. skip
D. continue

6. True or False: break exits the entire loop, while continue only skips the current iteration.

☐ True ☐ False

7. What does this code print?

PYTHON

```
for i in range(5):
    if i == 3:
        break
    print(i)
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
for i in range(5):
    if i == 2
        continue
    print(i)
```

Bug: _____

9. In one sentence, explain the difference between break and continue.

10. Write a for loop over range(10) that prints each number but uses break to stop when i equals 5.

Lesson 6.4: Nested Loops

You can put one loop inside another. This is called a **nested loop**. The inner loop runs completely for every iteration of the outer loop. Nested loops are commonly used to work with two-dimensional structures like grids, tables, or coordinate systems.

PYTHON · nested.py

```
# Print a 3x3 multiplication grid
for row in range(1, 4):
    for col in range(1, 4):
        product = row * col
        print(f"{product:4}", end="")
    print() # newline after each row

# Output:
# 1 2 3
# 2 4 6
# 3 6 9
```

The outer loop runs 3 times (row = 1, 2, 3). For each row, the inner loop runs 3 times (col = 1, 2, 3), so the body of the inner loop runs 9 times in total. Notice how print() with no arguments just prints a newline — handy for moving to the next row.

Lesson 6.4 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What is a nested loop?

A. A loop inside another loop
B. A loop that runs forever
C. A loop with no body
D. A loop that calls itself

2. In a nested loop, how many times does the inner loop run per outer iteration?

A. Once
B. It depends on the inner loop's range
C. Never
D. Twice

3. If the outer loop runs 3 times and the inner loop runs 4 times, how many total iterations of the inner body?

A. 7
B. 3
C. 12
D. 4

4. What does print() with no arguments do?

A. Prints a blank line
B. Nothing
C. Causes an error
D. Prints "None"

Lesson 6.4 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What is a nested loop?

A. A loop with no body
B. A loop inside another loop
C. A loop that never runs
D. A loop with break

2. In nested loops, which loop runs slower (less often)?

A. The inner loop
B. The outer loop
C. They run the same number of times
D. Neither

3. If the outer loop runs 3 times and the inner runs 4 times each, how many total inner iterations?

A. 7
B. 3
C. 12
D. 4

4. What is a common use of nested loops?

A. Printing a table or grid B. Defining a variable
C. Closing a file D. Reading one number

5. How is the inner loop shown in code?

A. With deeper indentation B. With braces
C. With quotes D. With a comma

6. True or False: The inner loop fully completes for each pass of the outer loop.

☐ True ☐ False

7. What does this code print?

PYTHON

```
for i in range(2):
    for j in range(2):
        print(i, j)
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
for i in range(2):
for j in range(2):
        print(i, j)
```

Bug: _____

9. In one sentence, explain what a nested loop is.

10. Write a nested loop that prints a 3x3 grid of stars (3 lines, each with *).**

Lesson 6.5: Summing and Counting

A very common use of loops is to add up a series of numbers or count how many things meet a condition. The pattern is always the same: create a variable before the loop to hold the running total, then update it inside the loop.

PYTHON · sum_count.py

```
# Add up the numbers from 1 to 100
total = 0
for n in range(1, 101):
    total += n
print(f"The sum is {total}") # 5050

# Count how many even numbers there are from 1 to 20
count = 0
for n in range(1, 21):
    if n % 2 == 0:
        count += 1
print(f"There are {count} even numbers.") # 10
```

TIP

The variable that holds the running total must be created *before* the loop starts, with a sensible initial value (usually 0). If you put total = 0 inside the loop, you reset it to 0 on every iteration, which is not what you want.

Lesson 6.5 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. Where should you create the "total" variable when summing in a loop?

A. Inside the loop
B. After the loop
C. It does not matter
D. Before the loop starts

2. What is the initial value of a sum variable usually?

A. 1
B. 0
C. 100
D. It depends

3. What does total += n do?

A. It creates a new variable
B. It is the same as total = total + n
C. It compares total to n
D. It causes an error

4. If you put total = 0 INSIDE the loop, what happens?

A. total is reset to 0 on every iteration, so the sum is wrong
B. It works fine
C. It causes an error
D. It makes the loop infinite

Lesson 6.5 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What is an accumulator variable?

A. A variable that prints
B. A variable that loops
C. A variable that is a string
D. A variable that builds up a total

2. What is a common starting value for a sum accumulator?

A. 1
B. 10
C. "start"
D. 0

3. What is a common starting value for a count accumulator?

A. 0 B. 1
C. 10 D. "count"

4. What does total += 5 do?

A. Adds 5 to total B. Subtracts 5
C. Multiplies by 5 D. Sets total to 5

5. Which loop pattern sums numbers 1 to 5?

A. total = 5; for i in range(5): total = i B. for i in range(5): print(i)
C. total = 0; for i in range(1, 6): total += i D. total = 0; for i in range(5): total = 0

6. True or False: To count how many items pass a test, start a counter at 0 and add 1 each time the test passes.

☐ True ☐ False

7. What does this code print?

PYTHON

```
total = 0
for i in range(1, 4):
    total += i
print(total)
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
for i in range(1, 4):
    total = 0
    total += i
print(total)
```

Bug: _____

9. In one sentence, explain the accumulator pattern.

10. Write code that prints the sum of the numbers from 1 to 10.

Mixed Quiz 6

This quiz mixes different question types: multiple choice, True/False, short answer, and code writing. Try to answer all of them without looking back at the chapter first — that is how you really learn. Answers are in the Answer Key at the back of the book.

1. How many times does for i in range(5): run?

A. 4 B. 5
C. 6 D. 0

2. What numbers does range(1, 5) produce?

A. 1, 2, 3, 4, 5
B. 1, 2, 3, 4
C. 0, 1, 2, 3, 4
D. 2, 3, 4, 5

3. What does break do inside a loop?

A. Skips to the next iteration
B. Exits the loop immediately
C. Restarts the loop from the beginning
D. Pauses the loop for one second

4. What does continue do inside a loop?

A. Skips the rest of this iteration and moves to the next
B. Ends the loop
C. Restarts the program
D. Nothing, it is a comment

5. What will for n in range(0, 10, 2): print(n) print?

A. 0 1 2 3 4 5 6 7 8 9
B. 0 2 4 6 8
C. 2 4 6 8 10
D. 0 2 4 6 8 10

6. True or False: A while loop will run forever if its condition never becomes False.

☐ True ☐ False

7. True or False: You can put a for loop inside another for loop.

☐ True ☐ False

8. True or False: range(10) produces the numbers 1 through 10.

☐ True ☐ False

9. How many times does this loop run? for i in range(2, 8): print(i)

A. 6
B. 7
C. 8
D. 10

10. What is the value of total after this code runs? total = 0; for n in range(1, 6): total += n

A. 5
B. 10
C. 15
D. 21

11. True or False: A for loop can loop over the items of a list directly, without using range().

☐ True ☐ False

12. True or False: The break keyword only exits the innermost loop it is inside.

☐ True ☐ False

13. Explain what an infinite loop is and how to stop one in IDLE.

14. When would you choose a while loop over a for loop? Give one example.

15. Write a program that uses a loop to print the multiplication table for the number 7, from 7 × 1 = 7 up to 7 × 10 = 70.

Chapter 6 Challenges

These questions are harder than the quiz. They ask you to combine ideas from this chapter, think creatively, and apply what you learned in new ways. Do not worry if you cannot solve them all on your first try — the point is to make you think. The full solutions are in the Answer Key at the back of the book.

Challenge 1. Write a program that uses a for loop to print the first 10 square numbers: 1, 4, 9, 16, 25, ... (Hint: use i ** 2).

*Hint: Loop from 1 to 10 and print i ** 2.*

Challenge 2. What does the following code print? for i in range(1, 4): for j in range(1, 4): print(i, j). Predict the output before running it.

Hint: The inner loop runs completely for each outer iteration.

Challenge 3. Write a program that asks the user for a number and prints a countdown from that number to 0 using a while loop.

Hint: Use while n >= 0: print(n); n -= 1.

Challenge 4. Write a program that adds up all the even numbers from 1 to 100 using a for loop and an if statement with modulo. Print the total.

Hint: Use range(1, 101), check if i % 2 == 0, and add to total.

Challenge 5. A password checker asks the user for a password. If they get it wrong, ask again. They get at most 3 tries. Write this program using a while loop and a counter.

Hint: Use a counter variable and check it in the while condition.

Chapter Recap

Here are the most important ideas from this chapter. If you can explain each one in your own words, you are ready to move on.

- for loops repeat a block once for each item in a sequence.
- range(n) produces numbers 0 through n-1. range(a, b) produces a through b-1. range(a, b, step) adds a step.
- while loops repeat as long as a condition is True.
- Always update the variable in a while condition, or you will get an infinite loop.
- break exits a loop immediately; continue skips to the next iteration.
- For summing or counting, create the total variable *before* the loop and update it *inside* the loop.

Common Mistakes

Every beginner makes these mistakes. Knowing them in advance will save you hours of debugging.

COMMON MISTAKE

Forgetting to update the variable in a while loop
If the condition variable never changes, the loop runs forever. Always include a line like count += 1 inside the loop body.

COMMON MISTAKE

Off-by-one errors with range()
range(1, 10) gives 1 through 9, not 1 through 10. If you want to include 10, use range(1, 11).

COMMON MISTAKE

Resetting the total inside the loop
Putting total = 0 inside the loop resets it on every iteration. Define the total *before* the loop and only update it inside.

COMMON MISTAKE

Using break when you mean continue
break ends the loop entirely; continue just skips the rest of the current iteration. Think carefully about which one you actually want.

MINI-PROJECT

Mini-Project 2: Number Guessing Game

Now that you know loops, conditionals, and the random module, let us build a number guessing game. The computer picks a random number between 1 and 100, and the player tries to guess it. After each guess, the computer says "too high" or "too low" until the player gets it right. This project combines loops, conditionals, and random numbers into a fun game.

How It Works

The program generates a random number using random.randint(1, 100). Then it enters a while loop that keeps asking the player for guesses until they get the right answer. Each wrong guess gives a hint (higher or lower). The program also counts how many guesses the player needed.

The Code

Here is the complete guessing game. Type it in and play it a few times. Then try the extensions below.

PYTHON · guessing_game.py

```
import random

# Pick a random number between 1 and 100
secret = random.randint(1, 100)
guesses = 0

print("=== Number Guessing Game ===")
print("I am thinking of a number between 1 and 100.")

# Keep asking until the player guesses correctly
while True:
    guess = int(input("Your guess: "))
    guesses += 1

    if guess == secret:
        print(f"Correct! You got it in {guesses} guesses!")
        break
    elif guess < secret:
        print("Too low! Try a higher number.")
    else:
        print("Too high! Try a lower number.")
```

Things to Try

Once you have the game working, try these extensions:

- Give the player a maximum number of guesses (like 7). If they run out, print "Game over! The number was X." and end the game.
- After the game ends, ask "Play again? (yes/no)" and use a second while loop to let them play multiple rounds.
- Add difficulty levels: Easy (1-50), Medium (1-100), Hard (1-500). Ask the player to choose at the start.
- Keep track of the best score (fewest guesses) across multiple rounds and display it at the end.

CHAPTER 7

Functions

A **function** is a named block of code that does one specific job. You have already used built-in functions like print(), input(), and int(). In this chapter you will learn to write your own. Functions let you organise your code, avoid repeating yourself, and make your programs easier to understand.

Lesson 7.1: Defining a Function

You create a function with the def keyword, followed by the function name, parentheses, and a colon. The body of the function is indented, just like the body of an if statement or a loop. To run the function, you write its name followed by parentheses — this is called **calling** the function.

PYTHON · function_basic.py

```
def greet():
    print("Hello there!")
    print("Welcome to Python.")

# Call the function
greet()
greet()
# "Hello there!" prints twice
```

Defining a function does not run it. The code inside def greet(): just sits there waiting. The function runs only when you call it by name. This is why you can call greet() as many times as you like — the function definition is reusable.

Lesson 7.1 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. Which keyword defines a function in Python?

A. function
B. func
C. define
D. def

2. What do you do to RUN a function?

A. Just define it
B. Press F5
C. Call it by name with parentheses
D. Nothing; it runs automatically

3. What is wrong with: def greet() print("Hi")

A. Missing quotation marks
B. print should be capitalized
C. Missing colon after the parentheses
D. Nothing is wrong

4. If you define a function but never call it, what happens?

A. The function body never runs
B. The function body runs once
C. Python prints a warning
D. The program crashes

Lesson 7.1 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. Which keyword defines a function?

A. func
B. def
C. function
D. define

2. What comes after the function name in a definition?

A. Parentheses ()
B. Braces {}
C. Brackets []
D. A comma

3. What must end the def line?

A. A semicolon
B. A period
C. A colon
D. A comma

4. How do you run a function?

A. Just write its name
B. Use the run keyword
C. Call it by name with parentheses
D. Restart Python

5. What is the body of a function?

A. The def line
B. The indented lines that run when called
C. The function name
D. The file name

6. True or False: Defining a function does not run its body until you call it.

☐ True ☐ False

7. What does this code print?

PYTHON

```
def greet():
    print("Hi")
greet()
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
def greet()
    print("Hi")
```

Bug: _____

9. In one sentence, explain what a function is.

10. Write a function called hello that prints "Hello there!" and then call it.

Lesson 7.2: Parameters and Arguments

A **parameter** is a variable listed in the function definition. An **argument** is a value you pass to the function when you call it. Parameters let your function work with different data each time it is called, instead of always doing exactly the same thing.

PYTHON · function_params.py

```
def greet(name): # name is a parameter
    print(f"Hello, {name}!")

greet("Alex") # "Alex" is an argument
greet("Maya") # different argument, same function
greet("Sam")
```

A function can take more than one parameter. Separate them with commas in both the definition and the call:

PYTHON · two_params.py

```
def describe_pet(name, animal):
    print(f"{name} is a {animal}.")

describe_pet("Rex", "dog") # Rex is a dog.
describe_pet("Whiskers", "cat") # Whiskers is a cat.
```

Lesson 7.2 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. In def greet(name):, what is "name"?

A. An argument
B. A variable
C. A keyword
D. A parameter

2. In greet("Alex"), what is "Alex"?

A. A parameter
B. A keyword
C. An argument
D. A function

3. How many parameters can a function have?

A. Exactly 1
B. At most 2
C. Any number (including 0)
D. Exactly 3

4. What does def describe_pet(name, animal): expect when called?

A. One argument
B. Three arguments
C. Two arguments
D. No arguments

Lesson 7.2 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What is a parameter?

A. A value passed in when calling
B. A loop
C. A comment
D. A name in the def line that receives a value

2. What is an argument?

A. A parameter name
B. A loop counter
C. The value passed to a function when calling it
D. A keyword

3. In def f(x):, what is x?

A. An argument
B. A keyword
C. A parameter
D. A loop

4. In the call f(5), what is 5?

A. An argument
B. A parameter
C. A keyword
D. A def

5. What happens if you call a function that expects 1 parameter with no arguments?

A. It raises a TypeError
B. It runs fine
C. It prints 0
D. It runs forever

6. True or False: A parameter is the variable in the definition; an argument is the value you pass when calling.

☐ True ☐ False

7. What does this code print?

PYTHON

```
def add(n):
    print(n + 1)
add(4)
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
def add(n):
    print(n + 1)
add()
```

Bug: _____

9. In one sentence, explain the difference between a parameter and an argument.

10. Write a function square(n) that prints n times n, and call it with 6.

Lesson 7.3: Returning Values

Functions can also **return** a value to the code that called them. Use the return keyword, followed by the value you want to send back. The caller can then store that value in a variable or use it in an expression.

PYTHON · return.py

```
def add(a, b):
    return a + b

result = add(3, 4)
print(result) # 7
print(add(10, 20)) # 30 (return value used directly)
print(add(1, 2) + add(3, 4)) # 10
```

The difference between print and return is one of the most important ideas in this chapter, and it confuses many beginners. print displays something on the screen for the human to read. return hands a value back to the program so it can be used in further calculations. A function that only prints its result cannot be used in expressions, because it does not actually give anything back.

PYTHON · print_vs_return.py

```
# BAD: prints but does not return
def bad_double(x):
    print(x * 2)

result = bad_double(5) # prints "10", but result is None
print(result + 1) # ERROR: None + 1

# GOOD: returns the value
def good_double(x):
    return x * 2

result = good_double(5) # nothing printed, but result is 10
print(result + 1) # 11
```

Lesson 7.3 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What does the return keyword do?

A. Sends a value back to the caller
B. Prints a value
C. Restarts the function
D. Ends the program

2. What happens when Python reaches a return statement?

A. The function continues to the next line
B. The function exits immediately
C. The program pauses
D. An error occurs

3. What does a function return if it has no return statement?

A. 0
B. An error
C. The last variable
D. None

4. What is the difference between print and return?

A. They are the same
B. return displays text; print sends a value
C. print displays text; return sends a value back to the program
D. print is faster

Lesson 7.3 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What does return do?

A. Sends a value back to the caller and stops the function
B. Prints a value
C. Loops
D. Defines a function

2. What is the difference between return and print?

A. print shows on screen; return sends a value back to the code
B. return shows on screen; print does not
C. They are the same
D. Neither does anything

3. What does a function without return give back?

A. None
B. 0
C. ""
D. error

4. What can you do with a returned value?

A. Only print it
B. Nothing
C. Only store it
D. Store it in a variable, use it in math, pass to another function

5. How many values can a single return statement send back (basic case)?

A. Two
B. Many
C. Only one
D. Zero or one

6. True or False: After return runs, the rest of the function body is skipped.

☐ True ☐ False

7. What does this code print?

PYTHON

```
def double(x):
    return x * 2
print(double(5))
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
def double(x):
    print(x * 2)
result = double(5)
print(result + 1)
```

Bug: _____

9. In one sentence, explain the difference between return and print.

10. Write a function add(a, b) that returns the sum of a and b, and print the result of add(3, 4).

Lesson 7.4: Default Arguments

Sometimes a parameter almost always has the same value. You can give a parameter a **default value** in the function definition. If the caller does not supply that argument, the default is used. If they do supply one, it overrides the default.

PYTHON · defaults.py

```
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}!")

greet("Alex") # Hello, Alex!
greet("Maya", "Hi") # Hi, Maya!
greet("Sam", "Howdy") # Howdy, Sam!
```

When a function has multiple parameters, all parameters with default values must come *after* all parameters without defaults. This is so Python can match arguments to parameters in order without ambiguity.

Lesson 7.4 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What is a default argument?

A. An argument that is always 0
B. An argument that cannot be changed
C. A parameter that has a default value if the caller does not provide one
D. The first argument

2. In def greet(name, greeting="Hello"):, what happens if you call greet("Alex")?

A. An error
B. greeting uses the default "Hello"
C. greeting is empty
D. name is ignored

3. Where must parameters with default values go?

A. Before parameters without defaults
B. In any order
C. Only at the start
D. After parameters without defaults

4. Can you override a default value?

A. No, it is fixed
B. Yes, by providing a different argument
C. Only inside the function
D. Only in Python 3

Lesson 7.4 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What is a default argument?

A. A value used when the caller does not pass one
B. A required argument
C. A loop counter
D. A returned value

2. How do you write a default value for a parameter x of 5?

A. def f(5 = x):
B. def f(x: 5):
C. def f(x = 5):
D. def f(x -> 5):

3. If def f(x=3): and you call f(), what is x?

A. 0
B. None
C. error
D. 3

4. If def f(x=3): and you call f(7), what is x?

A. 3
B. None
C. error
D. 7

5. Where must parameters with defaults go in the parameter list?

A. First
B. Anywhere
C. Last (after non-default ones)
D. They cannot mix

6. True or False: A parameter with a default can still be given a different value when calling.

☐ True ☐ False

7. What does this code print?

PYTHON

```
def greet(name="friend"):
    print("Hi", name)
greet()
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
def f(x=5, y):
    print(x, y)
```

Bug: _____

9. In one sentence, explain what a default argument is.

10. Write a function greet(name="there") that prints "Hello, " plus name, and call it with no arguments.

Lesson 7.5: Variable Scope

A variable created inside a function only exists inside that function. This is called **local scope**. Once the function finishes, the variable is destroyed. Variables created outside any function are called **global**, and they can be read from inside a function but should not be modified from inside one.

PYTHON · scope.py

```
def calculate():
    result = 5 + 3 # result is local to calculate
    return result

answer = calculate()
print(answer) # 8
print(result) # ERROR: result is not defined here
```

TIP

Scope is a feature, not a bug. It means you can use the same variable name (like result or i) inside many different functions without them interfering with each other. Each function has its own private set of variables.

Lesson 7.5 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. A variable created inside a function is called:

A. Global
B. Static
C. Local
D. Constant

2. Can you use a local variable outside its function?

A. No, it only exists inside the function
B. Yes
C. Only in the same file
D. Only if it is returned

3. What happens to a local variable when the function ends?

A. It is destroyed
B. It stays in memory
C. It becomes global
D. It causes an error

4. A variable defined outside all functions is called:

A. Local
B. Global
C. Static
D. Temporary

Lesson 7.5 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What is a local variable?

A. A variable created inside a function
B. A variable used everywhere
C. A variable in a file
D. A built-in name

2. What is a global variable?

A. A variable created inside a function
B. A loop counter
C. A returned value
D. A variable created outside all functions, usable throughout the file

3. Can a function read a global variable (without global keyword)?

A. No
B. Only sometimes
C. Only if it returns it
D. Yes

4. What happens if you assign to a variable named like a global inside a function?

A. It changes the global
B. It creates a new local variable
C. It causes an error
D. Nothing

5. Which keyword lets you modify a global inside a function?

A. local
B. nonlocal
C. global
D. var

6. True or False: A variable created inside a function cannot be used outside that function.

☐ True ☐ False

7. What does this code print?

PYTHON

```
x = 5
def f():
    print(x)
f()
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
def f():
    y = 5
f()
print(y)
```

Bug: _____

9. In one sentence, explain the difference between local and global variables.

10. Write a function that creates a local variable message = "hi" and prints it, then call the function.

Lesson 7.6: Why Use Functions?

Functions are not just a way to make code shorter — they are a way to make code *understandable*. A well-named function like calculate_average() tells the reader what the code does without them having to read every line. Functions also let you **reuse** code: write once, use many times. And they make **testing** easier — you can test one function at a time, in isolation, instead of trying to test the whole program at once.

- **Organisation** — break a big program into small, named pieces.
- **Reuse** — write once, call many times.
- **Readability** — calculate_total() is clearer than a wall of arithmetic.
- **Testing** — test each function separately.

Lesson 7.6 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. Which is NOT a benefit of using functions?

A. Code reuse
B. Better readability
C. Easier testing
D. Makes code run slower

2. Why is well-named code like calculate_average() better than inline arithmetic?

A. It is faster
B. It is more readable — the name explains what the code does
C. It uses less memory
D. It is required by Python

3. Can functions call other functions?

A. No
B. Only in Python 3
C. Only if they return a value
D. Yes

4. What is "code reuse"?

A. Writing code as fast as possible
B. Writing a function once and calling it many times instead of copying code
C. Using code from the internet
D. Recompiling code

Lesson 7.6 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. Which is a benefit of functions?

A. Slower programs
B. Code reuse
C. More bugs
D. Longer code

2. Why do functions improve readability?

A. They hide details behind a name
B. They make code longer
C. They remove all variables
D. They add comments

3. Why are functions easier to test?

A. They never have bugs
B. They do not need testing
C. They run slower
D. You can test one task in isolation

4. What is "code reuse"?

A. Writing the same code over and over
B. Using the same function in many places instead of copying code
C. Deleting code
D. Commenting code

5. Which is a sign of a well-named function?

A. The name is one letter
B. The name is unrelated
C. The name describes what it does
D. The name is very long

6. True or False: Using functions can make your code easier to read and easier to test.

☐ True ☐ False

7. What does this code print?

PYTHON

```
def greet(name):
    print("Hi", name)
greet("Sam")
greet("Ana")
```

Output: _____

8. A student copies the same 5 lines of code three times. What is the problem?

PYTHON - has a bug

```
print("Welcome!")
print("Welcome!")
print("Welcome!")
```

Bug: _____

9. In one sentence, explain why functions are useful.

10. Write a function called cheer that prints "Go team!" and call it twice.

Mixed Quiz 7

This quiz mixes different question types: multiple choice, True/False, short answer, and code writing. Try to answer all of them without looking back at the chapter first — that is how you really learn. Answers are in the Answer Key at the back of the book.

1. Which keyword defines a function in Python?

A. function
B. def
C. func
D. define

2. What is the difference between a parameter and an argument?

A. They are the same thing
B. Parameters are in the definition; arguments are in the call
C. Arguments are in the definition; parameters are in the call
D. Parameters are optional; arguments are required

3. What does the return keyword do?

A. Prints a value
B. Restarts the function
C. Sends a value back to the caller
D. Ends the program

4. What is the output? def f(x): return x*2 print(f(5))

A. 5
B. 10
C. 25
D. None

5. A variable created inside a function is called:

A. Global
B. Local
C. Static
D. Constant

6. True or False: A function can have multiple return statements.

☐ True ☐ False

7. True or False: Once a function returns a value, the rest of the function body still runs.

☐ True ☐ False

8. True or False: Parameters with default values must come before parameters without default values.

☐ True ☐ False

9. What is the output of: def f(x): return x * 2 print(f(f(3)))?

A. 6
B. 9
C. 12
D. 8

10. What happens if you define a function but never call it?

A. The program crashes
B. Python prints a warning
C. The function body never runs
D. The function runs automatically when the program starts

11. True or False: A variable created inside a function can be used outside that function.

☐ True ☐ False

12. True or False: A function with no return statement returns None by default.

☐ True ☐ False

13. Explain the difference between print and return inside a function. Why does it matter?

14. Why is it useful to break a long program into many small functions? Give two reasons.

15. Write a function called is_even that takes a number as a parameter and returns True if it is even and False otherwise. Then call it with the number 7 and print the result.

Chapter 7 Challenges

These questions are harder than the quiz. They ask you to combine ideas from this chapter, think creatively, and apply what you learned in new ways. Do not worry if you cannot solve them all on your first try — the point is to make you think. The full solutions are in the Answer Key at the back of the book.

Challenge 1. Write a function called square(n) that returns n * n. Then call it with 7 and print the result.

Hint: The function takes a parameter and returns its square.

Challenge 2. What is the output of: def f(x): return x * 2 then print(f(f(3)))? Explain step by step.

Hint: Work from the inside out: first f(3), then f() of that result.

Challenge 3. Write a function called is_even(n) that returns True if n is even and False if n is odd. Then use it in a loop to print all even numbers from 1 to 20.

Hint: Use n % 2 == 0 in the return statement.

Challenge 4. A function does: def mystery(a, b): return a + b. What happens if you call mystery("Hello", " World")? What about mystery(5, 3)? Explain why the same function works for both.

Hint: Think about what + does for strings vs integers.

Challenge 5. Write a function called calculate_bmi(weight, height) that calculates BMI (weight / height ** 2) and returns it. Then ask the user for their weight and height, call the function, and print the result rounded to 1 decimal place.

Hint: BMI = weight / (height squared). Use f-string formatting for the result.

Chapter Recap

Here are the most important ideas from this chapter. If you can explain each one in your own words, you are ready to move on.

- Define a function with def name(parameters): and an indented body.
- Parameters receive arguments when the function is called.
- return sends a value back to the caller. The caller can use it in expressions.
- print displays text; return hands back a value. They are not the same.
- Parameters can have default values, e.g. def f(x, y=10):.

- Variables created inside a function are local — they only exist inside that function.

Common Mistakes

Every beginner makes these mistakes. Knowing them in advance will save you hours of debugging.

COMMON MISTAKE

Forgetting to call the function
Defining def greet(): does not run the function. You must call it with greet(). Forgetting the parentheses is a common typo.

COMMON MISTAKE

Confusing print and return
print shows output to the user; return gives a value back to the program. If you want to use the result in further calculations, you need return.

COMMON MISTAKE

Forgetting the colon after the def line
def greet() is a syntax error. It must be def greet():.

COMMON MISTAKE

Using a local variable outside its function
A variable defined inside a function does not exist outside it. If you need a value from inside a function, return it.

CHAPTER 8

Lists and Tuples

So far, each variable has held a single value. But real programs often need to work with collections of values — a list of student names, a sequence of temperatures, a set of scores. Python has several built-in data structures for this. This chapter covers **lists** (changeable, ordered collections) and **tuples** (unchangeable, ordered collections).

Lesson 8.1: Creating and Indexing Lists

A **list** is an ordered collection of values. You create a list with square brackets, separating items with commas:

PYTHON · list_create.py

```
fruits = ["apple", "banana", "cherry", "date"]
numbers = [10, 20, 30, 40, 50]
mixed = ["hello", 42, True, 3.14] # mixed types are allowed
empty = [] # an empty list
```

You access individual items by their **index** — their position in the list. Python counts from zero, so the first item is at index 0, the second at index 1, and so on. You can also count from the end using negative indices: -1 is the last item, -2 is the second-to-last, and so on.

PYTHON · list_index.py

```
fruits = ["apple", "banana", "cherry", "date"]
print(fruits[0]) # apple
print(fruits[1]) # banana
print(fruits[-1]) # date (last item)
print(fruits[-2]) # cherry (second-to-last)
```

Figure 8.1 — List indexing (positive and negative)

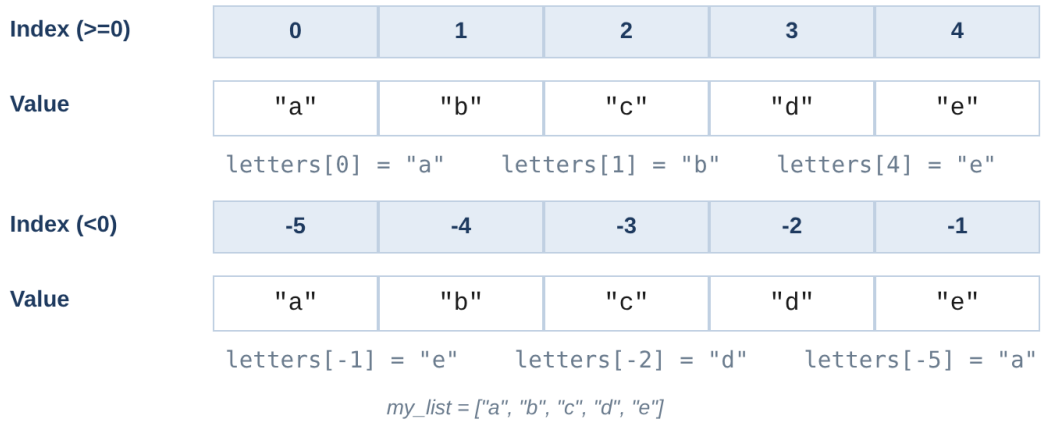


Figure 8.1 — List indexing. Positive indices count from the start; negative indices count from the end.

Lesson 8.1 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. How do you create a list in Python?

A. (1, 2, 3)
B. [1, 2, 3]
C. {1, 2, 3}
D. <1, 2, 3>

2. What is the index of the first item in a list?

A. 0
B. 1
C. -1
D. It depends

3. What does [10, 20, 30][1] return?

A. 10
B. 30
C. 20
D. 1

4. What does [10, 20, 30][-1] return?

A. 10 B. -1
C. 30 D. An error

Lesson 8.1 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. How do you create a list in Python?

A. With () B. With []
C. With {} D. With <>

2. What is the index of the first item in a list?

A. 1 B. -1
C. 0 D. 10

3. For nums = [10, 20, 30], what is nums[0]?

A. 20 B. 10
C. 30 D. error

4. For nums = [10, 20, 30], what is nums[-1]?

A. 10 B. 30
C. 20 D. error

5. What does len([5, 6, 7]) return?

A. 2 B. 3
C. 7 D. 0

6. True or False: In Python, list indices start at 0.

☐ True ☐ False

7. What does this code print?

PYTHON

```
a = [10, 20, 30]
print(a[1])
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
a = [10, 20, 30]
print(a[3])
```

Bug: _____

9. In one sentence, explain what a list is in Python.

10. Create a list called colors with the values red, green, blue, then print the second item.

Lesson 8.2: Slicing Lists

A **slice** is a sub-list extracted from a list. You write it with two indices separated by a colon: list[start:stop]. The slice includes items from start up to (but not including) stop. If you leave out start, Python uses 0. If you leave out stop, Python uses the end of the list.

PYTHON · list_slice.py

```
letters = ["a", "b", "c", "d", "e"]
print(letters[1:4]) # ['b', 'c', 'd']
print(letters[:3]) # ['a', 'b', 'c']
print(letters[2:]) # ['c', 'd', 'e']
print(letters[:]) # a copy of the whole list
print(letters[-2:]) # ['d', 'e'] (last two)
```

You can also add a step with a second colon: list[start:stop:step]. For example, letters[::2] gives every second item. letters[::-1] is a handy idiom that reverses the list.

Lesson 8.2 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What does [1, 2, 3, 4, 5][1:3] return?

A. [1, 2, 3]
B. [2, 3]
C. [2, 3, 4]
D. [1, 2]

2. What does [1, 2, 3][:2] return?

A. [1, 2]
B. [1, 2, 3]
C. [2, 3]
D. []

3. What does [1, 2, 3][::-1] return?

A. [1, 2, 3]
B. [3, 2, 1]
C. [1, 3]
D. An error

4. In slicing [start:stop], is the stop index included?

A. No, it stops before stop
B. Yes
C. Sometimes
D. Only if start is 0

Lesson 8.2 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What does a[1:4] return for a = [10, 20, 30, 40, 50]?

A. [20, 30, 40]
B. [10, 20, 30]
C. [20, 30, 40, 50]
D. [30, 40]

2. What does a[::-1] produce?

A. The same list
B. The first item
C. The list reversed
D. The last item

3. What does a[:3] give for a = [10, 20, 30, 40]?

A. [10, 20, 30]
B. [20, 30, 40]
C. [10, 20, 30, 40]
D. [10]

4. What does a[2:] give for a = [10, 20, 30, 40]?

A. [10, 20] B. [30, 40]
C. [20, 30, 40] D. [40]

5. In a slice a[start:stop], which index is included?

A. stop B. start
C. both D. neither

6. True or False: Slicing a list returns a new list; the original is not changed.

☐ True ☐ False

7. What does this code print?

PYTHON

```
a = [1, 2, 3, 4, 5]
print(a[1:3])
```

Output: _____

8. A student wants the last 2 items of a = [1, 2, 3, 4] but a[3:4] only gives [4]. What is wrong?

PYTHON - has a bug

```
a = [1, 2, 3, 4]
print(a[3:4])
```

Bug: _____

9. In one sentence, explain what list slicing does.

10. Given nums = [10, 20, 30, 40, 50], write code that prints the last three items using a slice.

Lesson 8.3: Modifying Lists

Lists are **mutable**, which means you can change them after they are created. You can change an item by assigning to its index, add items with append or insert, and remove items with remove or pop.

PYTHON · list_modify.py

```
shopping = ["milk", "eggs", "bread"]
shopping[0] = "almond milk" # change an item
print(shopping) # ['almond milk', 'eggs', 'bread']

shopping.append("butter") # add to the end
print(shopping) # [..., 'butter']

shopping.insert(1, "cheese") # insert at position 1
print(shopping) # ['almond milk', 'cheese', 'eggs', ...]

shopping.remove("eggs") # remove by value
popped = shopping.pop() # remove and return last item
print(popped) # butter
print(shopping)
```

Useful List Methods

Method	What it does	Example
`append(x)`	Add x to the end	`lst.append(5)`
`insert(i, x)`	Insert x at index i	`lst.insert(0, 5)`
`remove(x)`	Remove first occurrence of x	`lst.remove(5)`
`pop()`	Remove and return last item	`lst.pop()`
`sort()`	Sort the list in place	`lst.sort()`
`reverse()`	Reverse the list in place	`lst.reverse()`
`len(lst)`	Return the number of items (a function, not a method)	`len(lst)`

Table 8.1 — The most commonly used list methods.

Lesson 8.3 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. Which method adds an item to the END of a list?

A. add()
B. insert()
C. append()
D. push()

2. After running lst = [1, 2, 3] then lst.append(4), what does lst contain?

A. [4, 1, 2, 3]
B. [1, 2, 3]
C. [1, 2, 3, 4]
D. None

3. What does [1, 2, 3].pop() do?

A. Removes and returns the last item
B. Removes the first item
C. Adds an item
D. Sorts the list

4. After running lst = [3, 1, 2] then lst.sort(), what does lst contain?

A. [3, 2, 1]
B. [3, 1, 2]
C. None
D. [1, 2, 3]

Lesson 8.3 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What does append() do?

A. Removes an item
B. Sorts the list
C. Adds an item to the end of the list
D. Inserts at the start

2. What does insert(1, "x") do?

A. Inserts "x" at index 1
B. Removes index 1
C. Sorts the list
D. Prints "x"

3. What does remove(20) do?

A. Removes index 20
B. Sorts the list
C. Removes the first 20 found
D. Adds 20

4. What does pop() do with no argument?

A. Adds an item
B. Removes and returns the last item
C. Sorts the list
D. Prints the list

5. What does sort() do?

A. Sorts the list in place (lowest to highest)
B. Reverses the list
C. Copies the list
D. Prints the list

6. True or False: append() adds an item to the end of the list.

☐ True ☐ False

7. What does this code print?

PYTHON

```
a = [1, 2]
a.append(3)
print(a)
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
a = [3, 1, 2]
b = a.sort()
print(b)
```

Bug: _____

9. In one sentence, explain what append() does.

10. Create a list nums = [3, 1, 2], then sort it and print the result.

Lesson 8.4: Looping Over a List

One of the most common things you will do with a list is loop over it — that is, do something with each item in turn. A for loop is perfect for this:

PYTHON · list_loop.py

```
fruits = ["apple", "banana", "orange"]
for fruit in fruits:
    print(f"I like {fruit}s!")

# Output:
# I like apples!
# I like bananas!
# I like oranges!
```

If you need both the index and the value, use the enumerate() function. It gives you index-value pairs as you loop:

PYTHON · enumerate.py

```
fruits = ["apple", "banana", "cherry"]
for index, fruit in enumerate(fruits):
    print(f"{index + 1}. {fruit}")

# Output:
# 1. apple
# 2. banana
# 3. cherry
```

Lesson 8.4 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. How do you loop over a list?

A. for item in list:
B. for item = list:
C. foreach item in list:
D. loop item in list:

2. What does enumerate(["a", "b"]) give you?

A. Index-value pairs: (0, "a"), (1, "b")
B. Just the items
C. Just the indices
D. The items in reverse

3. In for item in fruits:, what is "item"?

A. The index
B. The actual value from the list
C. The list itself
D. A count

4. Can you modify list items while looping?

A. No
B. Only with while loops
C. Only if you use range()
D. Yes, but be careful

Lesson 8.4 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. Which loop works best for going through a list?

A. while item in list
B. for item in list
C. for i = list
D. repeat list

2. What does enumerate() give you?

A. Only the items
B. Only the index
C. Both the index and the item
D. The list reversed

3. In for x in [1, 2, 3]:, what is x on the first pass?

A. 0
B. 2
C. 3
D. 1

4. How many times does for x in [10, 20, 30] run?

A. 2
B. 3
C. 10
D. 30

5. Which line uses enumerate correctly?

A. for i, x in lst:
B. for i in enumerate(lst, x):
C. for i, x in enumerate(lst):
D. enumerate(lst) = for x

6. True or False: You can loop over a list directly with for item in my_list.

☐ True ☐ False

7. What does this code print?

PYTHON

```
for x in [10, 20, 30]:
    print(x)
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
for x in [1, 2, 3]
    print(x)
```

Bug: _____

9. In one sentence, explain what enumerate() does.

10. Given fruits = ["apple", "banana", "cherry"], write a for loop that prints each fruit on its own line.

Lesson 8.5: Tuples

A **tuple** is like a list, but **immutable** — once you create it, you cannot change it. You write a tuple with parentheses instead of square brackets:

PYTHON · tuple.py

```
point = (3, 4) # a 2D point
rgb = (255, 128, 0) # an orange colour
single = (42,) # a tuple with one item (note the comma)

print(point[0]) # 3
print(point[1]) # 4
point[0] = 5 # ERROR: tuples cannot be changed
```

Use tuples when you have a small, fixed collection of related values that should not change — coordinates, RGB colours, dates. Use lists when you need a collection that can grow, shrink, or be modified. Many Python functions return tuples because they cannot be accidentally changed by the caller.

Lesson 8.5 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. How do you create a tuple?

A. [1, 2, 3]
B. {1, 2, 3}
C. <1, 2, 3>
D. (1, 2, 3)

2. Can you change a value in a tuple after it is created?

A. Yes
B. Only the first value
C. No, tuples are immutable
D. Only in Python 3

3. What is the main difference between a list and a tuple?

A. Lists are mutable; tuples are immutable
B. Lists can only hold numbers
C. Tuples are faster
D. There is no difference

4. When would you use a tuple instead of a list?

A. When you need to add items
B. When the data should not change (like coordinates)
C. When you need to sort
D. Never

Lesson 8.5 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What symbol is used to create a tuple?

A. []
B. ()
C. {}
D. <>

2. Can you change a tuple after it is created?

A. Yes
B. Only once
C. Only the first item
D. No, tuples are immutable

3. What does t[0] give for t = (10, 20, 30)?

A. 20
B. 30
C. 10
D. error

4. Which is a typical use of a tuple?

A. A list you change often
B. A loop counter
C. Storing fixed data like coordinates (x, y)
D. A print statement

5. What does len((1, 2, 3)) return?

A. 3
B. 1
C. 2
D. 0

6. True or False: Once created, you cannot add, remove, or change items in a tuple.

☐ True ☐ False

7. What does this code print?

PYTHON

```
t = (1, 2, 3)
print(t[1])
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
t = (1, 2, 3)
t[0] = 9
```

Bug: _____

9. In one sentence, explain the difference between a list and a tuple.

10. Create a tuple point with values 5 and 7, then print the first value.

Mixed Quiz 8

This quiz mixes different question types: multiple choice, True/False, short answer, and code writing. Try to answer all of them without looking back at the chapter first — that is how you really learn. Answers are in the Answer Key at the back of the book.

1. What does ["a", "b", "c"][1] return?

A. "a"
B. "b"
C. "c"
D. 1

2. What does [1, 2, 3, 4][-1] return?

A. 1
B. 4
C. -1
D. An error

3. Which method adds an item to the end of a list?

A. add()
B. insert()
C. append()
D. push()

4. What is the result of [1, 2, 3, 4, 5][1:3]?

A. [1, 2, 3]
B. [2, 3]
C. [2, 3, 4]
D. [1, 2]

5. Which is true about tuples?

A. Tuples are mutable
B. Tuples are immutable
C. Tuples can only hold numbers
D. Tuples are the same as lists

6. True or False: You can change a value in a list after it is created.

☐ True ☐ False

7. True or False: You can change a value in a tuple after it is created.

☐ True ☐ False

8. True or False: The expression list[::-1] reverses the list.

☐ True ☐ False

9. What is the length of [1, 2, 3, 4, 5]?

A. 4
B. 5
C. 6
D. 0

10. What does [1, 2, 3] + [4, 5] produce?

A. [1, 2, 3, 4, 5]
B. [5, 7, 3]
C. [[1, 2, 3], [4, 5]]
D. An error

11. True or False: The append() method adds an item to the end of a list.

☐ True ☐ False

12. True or False: Lists can contain items of different types (e.g. a string and an integer in the same list).

☐ True ☐ False

13. Explain the difference between a list and a tuple. When would you choose one over the other?

14. What is the difference between lst.append(5) and lst = lst + [5]?

15. Create a list of your three favourite foods. Then write a for loop that prints each food on its own line, preceded by the word "Love:".

Chapter 8 Challenges

These questions are harder than the quiz. They ask you to combine ideas from this chapter, think creatively, and apply what you learned in new ways. Do not worry if you cannot solve them all on your first try — the point is to make you think. The full solutions are in the Answer Key at the back of the book.

Challenge 1. Write a program that creates a list of 5 numbers, then prints the largest and smallest using max() and min().

Hint: max() and min() are built-in functions that work on lists.

Challenge 2. What does [1, 2, 3] + [4, 5] produce? What about [1, 2] * 3? Explain both.

*Hint: + concatenates; * repeats.*

Challenge 3. Write a program that asks the user for 3 names, stores them in a list, then prints them in reverse order.

Hint: Build the list with append(), then reverse with [::-1] or .reverse().

Challenge 4. A tuple is (3, 4) representing a point. Write a function called distance(point1, point2) that calculates the distance between two points. (Hint: distance = ((x2-x1)2 + (y2-y1)**2) ** 0.5)**

Hint: Unpack the tuples: x1, y1 = point1.

Challenge 5. Without using the sum() function, write a loop that adds up all the numbers in [10, 20, 30, 40, 50] and prints the total.

Hint: Create a total variable before the loop, then add each item.

Chapter Recap

Here are the most important ideas from this chapter. If you can explain each one in your own words, you are ready to move on.

- Lists are ordered, mutable collections created with square brackets.
- Indexing starts at 0. Negative indices count from the end.
- Slicing with [start:stop] gives a sub-list.
- Useful methods: append, insert, remove, pop, sort, reverse.
- Use for item in list to loop over items. Use enumerate() to get both index and value.
- Tuples are like lists but immutable (cannot be changed).

Common Mistakes

Every beginner makes these mistakes. Knowing them in advance will save you hours of debugging.

COMMON MISTAKE

Index out of range
If a list has 3 items, valid indices are 0, 1, 2 (and -1, -2, -3). Accessing index 3 raises an IndexError. Remember that index numbering starts at 0.

COMMON MISTAKE

Confusing append with +
lst.append(5) adds 5 to the list. lst + [5] creates a *new* list with 5 at the end; it does not change lst unless you assign the result back to it.

COMMON MISTAKE

Forgetting that slices make a copy
new_lst = lst[:] creates a separate copy. Modifying new_lst does not affect lst. But new_lst = lst just gives the same list a second name — modifying one modifies the other.

COMMON MISTAKE

Trying to modify a tuple
Tuples are immutable. If you need to change values, use a list instead.

CHAPTER 9

Dictionaries and Sets

Lists and tuples use numbers as indices. But sometimes you want to look up a value using a meaningful name instead of a position. A **dictionary** lets you do exactly that — it stores **key-value pairs**, where each key maps to one value. This chapter also covers **sets**, which are collections of unique items with no order.

Lesson 9.1: Creating and Using Dictionaries

A dictionary is created with curly braces. Each entry is a key, a colon, and a value, with entries separated by commas. You look up a value by putting its key in square brackets — just like indexing a list, but with a key instead of a number.

PYTHON · dict_basic.py

```
student = {
    "name": "Alex",
    "age": 14,
    "grade": "A",
    "city": "Cairo"
}

print(student["name"]) # Alex
print(student["age"]) # 14
print(student["grade"]) # A
```

Keys must be unique — if you assign to a key that already exists, the old value is replaced. Keys must also be **immutable**: strings, numbers, and tuples are allowed, but lists are not (because lists can change). Values can be anything, including other dictionaries or lists.

PYTHON · dict_modify.py

```
student["age"] = 15 # change an existing value
student["email"] = "alex@x.io" # add a new key-value pair
print(student)
```

Lesson 9.1 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. How do you create a dictionary?

A. [1: "a"] B. (1, "a")
C. {"a": 1} D. <"a": 1>

2. What does student["name"] do?

A. Looks up the value for "name" B. Adds a key
C. Removes "name" D. Sorts the dict

3. What happens if you assign to an existing key?

A. An error B. A new key is added
C. Nothing D. The old value is replaced

4. Which of these CANNOT be a dictionary key?

A. "name" B. 42
C. (1, 2) D. [1, 2]

Lesson 9.1 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What symbol creates a dictionary?

A. [] B. ()
C. {} D. <>

2. What does a dictionary store?

A. Only numbers B. Only strings
C. Key-value pairs D. Only lists

3. For d = {"a": 1, "b": 2}, what is d["b"]?

A. 1 B. 2
C. "b" D. error

4. How do you set a key "x" to 5 in dict d?

A. d["x"] = 5 B. d.add("x", 5)
C. d.x = 5 D. d.append("x", 5)

5. What happens if you look up a key that is not in the dictionary?

A. Returns 0 B. Returns None
C. Adds the key D. Raises KeyError

6. True or False: Dictionary keys must be unique.

☐ True ☐ False

7. What does this code print?

PYTHON

```
d = {"name": "Ana", "age": 12}
print(d["name"])
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
d = {"a": 1}
print(d["b"])
```

Bug: _____

9. In one sentence, explain what a dictionary is.

10. Create a dictionary scores with key "math" set to 90, then print the value for "math".

Lesson 9.2: Dictionary Methods

Three dictionary methods are especially useful: keys() returns all the keys, values() returns all the values, and items() returns all the key-value pairs as tuples. These are perfect for looping over a dictionary.

PYTHON · dict_methods.py

```
student = {"name": "Alex", "age": 14, "grade": "A"}

for key in student.keys():
    print(key)
# name
# age
# grade

for key, value in student.items():
    print(f"{key}: {value}")
# name: Alex
# age: 14
# grade: A
```

To safely look up a key that might not exist, use the get() method instead of square brackets. get() returns None (or a default you specify) if the key is missing, instead of raising an error.

PYTHON · dict_get.py

```
student = {"name": "Alex", "age": 14}
print(student.get("name")) # Alex
print(student.get("email")) # None (no error!)
print(student.get("email", "n/a")) # n/a (custom default)

# Direct lookup raises an error if key is missing:
print(student["email"]) # KeyError!
```

Lesson 9.2 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What does dict.keys() return?

A. All the values
B. All the keys
C. All the pairs
D. The first key

2. What does dict.items() return?

A. Just keys
B. Just values
C. The number of items
D. Key-value pairs as tuples

3. What does dict.get("missing") return if the key does not exist?

A. An error
B. None
C. 0
D. The string "missing"

4. Which is safer: d["key"] or d.get("key")?

A. d["key"]
B. They are the same
C. Neither is safe
D. d.get("key") — it does not crash on missing keys

Lesson 9.2 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What does d.keys() return?

A. The values
B. Both
C. Nothing
D. The keys

2. What does d.values() return?

A. The keys
B. Both
C. Nothing
D. The values

3. What does d.items() return?

A. Only keys
B. (key, value) pairs
C. Only values
D. Nothing

4. What does d.get("x") return if "x" is not in d?

A. Raises an error
B. 0
C. None (or a default)
D. The whole dict

5. How do you loop over both keys and values?

A. for k, v in d:
B. for k, v in d.items():
C. for k in d.items(v):
D. for d in k, v:

6. True or False: get() returns None (instead of raising an error) when the key is missing.

☐ True ☐ False

7. What does this code print?

PYTHON

```
d = {"a": 1, "b": 2}
print(list(d.keys()))
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
d = {"a": 1, "b": 2}
for k, v in d:
    print(k, v)
```

Bug: _____

9. In one sentence, explain what d.get(key) does differently from d[key].

10. Given d = {"x": 1, "y": 2}, write a loop that prints each key and value on one line.

Lesson 9.3: When to Use a Dictionary

Dictionaries are the right choice whenever you have a natural mapping from one thing to another. Some typical uses:

- **Phone book** — names to phone numbers.
- **Word counts** — words to how many times they appear.
- **Configuration** — setting names to their values.
- **Caching** — function arguments to already-computed results.
- **Records** — fields like "name", "age", "email" to their values.

TIP

A common beginner pattern is using parallel lists: names = ["Alex", "Maya"]; ages = [14, 15]. A dictionary like {"Alex": 14, "Maya": 15} is almost always clearer and less error-prone. Reach for a dictionary whenever you find yourself writing parallel lists.

Lesson 9.3 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. Which is a good use case for a dictionary?

A. A list of numbers in order
B. A single number
C. A true/false value
D. A phone book (names to numbers)

2. What is a common beginner mistake that dictionaries fix?

A. Using too many variables
B. Using too many comments
C. Using too many loops
D. Using parallel lists to map one thing to another

3. In a dictionary, what is unique?

A. Values
B. Both
C. Neither
D. Keys

4. Can dictionary values be lists?

A. Yes
B. No
C. Only if they are strings
D. Only in Python 3

Lesson 9.3 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. When is a dictionary better than two parallel lists?

A. When you look up values by a key often
B. When order matters most
C. When you always loop by index
D. Never

2. What is a "parallel list" problem?

A. Two lists where index i in one matches index i in the other
B. A list inside a list
C. A list of strings
D. A sorted list

3. Why is a dict safer than parallel lists?

A. Dicts are always smaller
B. Dicts cannot have errors
C. Keys are easier to read than index numbers
D. Dicts run faster always

4. Which data fits a dictionary well?

A. A phone book (name to number)
B. A list of unknown length
C. One number
D. A sorted list of numbers

5. Which is a downside of parallel lists?

A. Easy to get the indices out of sync
B. Too fast
C. Always wrong
D. Cannot store strings

6. True or False: A dictionary is a good choice when you need to look up values by a key, like a phone book.

☐ True ☐ False

7. What does this code print?

PYTHON

```
phone = {"Ana": "555", "Bob": "123"}
print(phone["Bob"])
```

Output: _____

8. A student uses two lists: names = ["Ana", "Bob"], ages = [12, 13]. They add a name but forget to add an age. What can go wrong?

PYTHON - has a bug

```
names = ["Ana", "Bob"]
ages = [12]
```

Bug: _____

9. In one sentence, explain when you should use a dictionary instead of parallel lists.

10. Build a dictionary that maps "apple" to 3 and "banana" to 5, then print the value for "apple".

Lesson 9.4: Sets

A **set** is an unordered collection of **unique** items. You create it with curly braces (just like a dictionary, but with no colons). Sets are useful when you want to remove duplicates from a list, or when you want to check whether something is in a collection very quickly.

PYTHON · set.py

```
colors = {"red", "green", "blue", "red"} # duplicate ignored
print(colors) # {'red', 'green', 'blue'} (order not guaranteed)

# Convert a list with duplicates to a set to remove them
numbers = [1, 2, 2, 3, 3, 3, 4]
unique = set(numbers)
print(unique) # {1, 2, 3, 4}

# Convert back to a list if you need ordering
unique_list = list(unique)

# Membership test is very fast
print("red" in colors) # True
print("yellow" in colors) # False
```

Sets support mathematical set operations like union (|), intersection (&), and difference (-). These can be handy when you need to combine or compare collections.

PYTHON · set_ops.py

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print(a | b) # {1, 2, 3, 4, 5, 6} union
print(a & b) # {3, 4} intersection
print(a - b) # {1, 2} difference
```

Lesson 9.4 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. How do you create a set?

A. [1, 2, 3]
B. (1, 2, 3)
C. {1, 2, 3}
D. <1, 2, 3>

2. What is special about sets?

A. They only hold unique items
B. They are ordered
C. They can only hold numbers
D. They are mutable like lists

3. What does set([1, 1, 2, 2, 3]) produce?

A. [1, 2, 3]
B. {1, 1, 2, 2, 3}
C. An error
D. {1, 2, 3}

4. What does {1, 2} & {2, 3} produce?

A. {1, 2, 3} B. {1, 3}
C. {2} D. An error

Lesson 9.4 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What is special about a set?

A. It keeps order B. It only stores unique items
C. It stores pairs D. It is immutable

2. What symbol creates a set?

A. [] B. ()
C. {} D. <>

3. What does {1, 1, 2, 2, 3} produce?

A. {1, 1, 2, 2, 3} B. {1, 2, 3}
C. {3} D. error

4. What does a | b do for sets?

A. Union (items in either) B. Intersection (items in both)
C. Difference (in a but not b) D. Concatenation

5. What does a & b do for sets?

A. Union B. Intersection
C. Difference D. Power

6. True or False: A set automatically removes duplicate values.

☐ True ☐ False

7. What does this code print?

PYTHON

```
print({1, 2, 2, 3})
```

Output: _____

8. A student wants an empty set and writes {}. What is wrong?

PYTHON - has a bug

```
x = {}
```

Bug: _____

9. In one sentence, explain what a set is.

10. Given a = {1, 2, 3} and b = {2, 3, 4}, write code that prints their union.

Mixed Quiz 9

This quiz mixes different question types: multiple choice, True/False, short answer, and code writing. Try to answer all of them without looking back at the chapter first — that is how you really learn. Answers are in

the Answer Key at the back of the book.

1. How do you create an empty dictionary?

A. { }
B. [ ]
C. ( )
D. dict[]

2. What does student["age"] do?

A. Adds a new key called age
B. Looks up the value associated with the key "age"
C. Removes the key "age"
D. Returns the index of "age"

3. What does dict.get("missing") return if "missing" is not a key?

A. An error
B. The string "missing"
C. None
D. False

4. What does set([1, 1, 2, 2, 3]) produce?

A. [1, 2, 3]
B. {1, 2, 3}
C. {1, 1, 2, 2, 3}
D. [3, 2, 1]

5. Which of these CANNOT be a dictionary key?

A. "name"
B. 42
C. (1, 2)
D. [1, 2]

6. True or False: Dictionary keys must be unique.

☐ True ☐ False

7. True or False: Sets are ordered — items stay in the order you added them.

☐ True ☐ False

8. True or False: You can have a list as a dictionary value.

☐ True ☐ False

9. What does len({"a": 1, "b": 2, "c": 3}) return?

A. 1
B. 3
C. 6
D. An error

10. What does {"a", "b"} & {"b", "c"} produce?

A. {"a", "b", "c"}
B. {"b"}
C. {"a", "c"}
D. An error

11. True or False: Looking up a missing key with square brackets (e.g. d["missing"]) raises a KeyError.

☐ True ☐ False

12. True or False: You can store a dictionary inside another dictionary.

☐ True ☐ False

13. Give one real-world example where a dictionary would be a better choice than two parallel lists. Explain why.

14. What is the difference between d.get("key") and d["key"] when the key does not exist?

15. Create a dictionary that maps the names of three friends to their ages. Then write a loop that prints each friend's name and age in the format "Name is Age years old."

Chapter 9 Challenges

These questions are harder than the quiz. They ask you to combine ideas from this chapter, think creatively, and apply what you learned in new ways. Do not worry if you cannot solve them all on your first try — the point is to make you think. The full solutions are in the Answer Key at the back of the book.

Challenge 1. Write a program that counts how many times each word appears in the string "the cat sat on the mat the cat". Print each word and its count.

Hint: Use a dictionary: split the string, loop over words, increment counts.

Challenge 2. What is the output of: d = {}; d["a"] = 1; d["a"] = 2; print(d)? Explain.

Hint: Keys are unique; assigning again overwrites.

Challenge 3. Write a program that asks the user for 5 numbers, stores them in a set, and prints how many UNIQUE numbers were entered.

Hint: A set automatically removes duplicates; len() gives the count.

Challenge 4. Create a dictionary that maps student names to their scores on a test. Then write a loop that prints only the students who scored above 80.

Hint: Loop with .items() and use an if statement.

Challenge 5. What does dict.get("missing", "N/A") do? How is it different from dict.get("missing")?

Hint: The second argument is a custom default value.

Chapter Recap

Here are the most important ideas from this chapter. If you can explain each one in your own words, you are ready to move on.

- Dictionaries store key-value pairs in curly braces: {"name": "Alex"}.
- Look up values with dict[key]. Use dict.get(key) to avoid errors on missing keys.
- Keys must be unique and immutable (strings, numbers, tuples).
- .keys(), .values(), and .items() are used for looping.
- Sets are unordered collections of unique items, created with curly braces.
- Sets support union, intersection, and difference operations.

Common Mistakes

Every beginner makes these mistakes. Knowing them in advance will save you hours of debugging.

COMMON MISTAKE

Using a list as a dictionary key
Keys must be immutable. {[1, 2]: "x"} raises a TypeError. Use a tuple (1, 2) instead.

COMMON MISTAKE

Looking up a missing key with square brackets
student["email"] raises KeyError if email is not a key. Use student.get("email") if you are not sure the key exists.

COMMON MISTAKE

Confusing {} (dict) with [] (list)
{} creates an empty dictionary, not an empty list. To create an empty list, use [].

COMMON MISTAKE

Forgetting that sets are unordered
Do not rely on the order of items in a set. If order matters, use a list instead.

MINI-PROJECT

Mini-Project 3: Contact Book

You now know about dictionaries, lists, and functions. Let us build a contact book that stores names and phone numbers. The user can add contacts, look up a phone number by name, list all contacts, and delete a contact. This project shows how dictionaries are perfect for storing key-value data like a phone book.

How It Works

The program uses a dictionary called contacts where each key is a name and each value is a phone number. A while loop shows a menu of options and keeps running until the user chooses to quit. Each option is handled by an if/elif branch.

The Code

Here is the complete contact book program. It is longer than the previous mini-projects, but each part should be familiar to you.

PYTHON · contact_book.py

```
# Contact Book
contacts = {}

print("=== Contact Book ===")

while True:
    print("\n1. Add a contact")
    print("2. Look up a contact")
    print("3. List all contacts")
    print("4. Delete a contact")
    print("5. Quit")

    choice = input("Choose 1-5: ")

    if choice == "1":
        name = input("Enter name: ")
        phone = input("Enter phone: ")
        contacts[name] = phone
        print(f"Added {name}.")

    elif choice == "2":
        name = input("Enter name to look up: ")
        if name in contacts:
            print(f"{name}: {contacts[name]}")
        else:
            print(f"{name} not found.")

    elif choice == "3":
        if len(contacts) == 0:
            print("No contacts yet.")
        else:
            print("\n--- All Contacts ---")
            for name, phone in contacts.items():
                print(f" {name}: {phone}")

    elif choice == "4":
        name = input("Enter name to delete: ")
        if name in contacts:
            del contacts[name]
            print(f"Deleted {name}.")
        else:
            print(f"{name} not found.")

    elif choice == "5":
        print("Goodbye!")
        break

    else:
        print("Invalid choice. Try again.")
```

Things to Try

Once you have the contact book working, try these extensions:

- Save the contacts to a file using open("contacts.txt", "w") when the user quits, and load them back when the program starts.
- Store more information per contact: use a dictionary of dictionaries, where each contact has a name, phone, and email.
- Add an option to edit an existing contact's phone number.
- Sort the contacts alphabetically when listing them. Hint: use sorted(contacts.keys()).

CHAPTER 10

Working with Strings

You have been using strings since Chapter 1. Now it is time to learn what else they can do. Strings are one of the most useful data types in Python — almost every program deals with text in some way. This chapter covers the most important string methods, slicing, and the fact that strings are immutable.

Lesson 10.1: Useful String Methods

Python strings come with a rich set of **methods** — functions attached to the string that perform common operations. You call them with a dot: string.method(). Here are the ones you will use most often.

PYTHON · string_methods.py

```
sentence = " Hello, World! "

print(sentence.upper()) # " HELLO, WORLD! "
print(sentence.lower()) # " hello, world! "
print(sentence.strip()) # "Hello, World!" (remove whitespace)
print(sentence.replace(",", "!")) # " Hello! World! "
print(sentence.strip().split(",")) # ['Hello', ' World!']
print("hello".capitalize()) # "Hello"
print("hello world".title()) # "Hello World"
```

Two especially useful methods are split() and join(). split() breaks a string into a list of smaller strings; join() does the opposite — it glues a list of strings together into one. They are perfect for processing text.

PYTHON · split_join.py

```
# split: string -> list
csv_line = "Alex,14,Cairo,Math"
fields = csv_line.split(",")
print(fields) # ['Alex', '14', 'Cairo', 'Math']

# join: list -> string
words = ["Python", "is", "fun"]
sentence = " ".join(words)
print(sentence) # "Python is fun"

# join with a different separator
path = "/".join(["home", "alex", "documents"])
print(path) # "home/alex/documents"
```

Lesson 10.1 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What does "Hello".upper() return?

A. "hello"
B. "Hello"
C. An error
D. "HELLO"

2. What does " hi ".strip() return?

A. " hi "
B. " hi"
C. "hi"
D. "hi "

3. What does "a,b,c".split(",") return?

A. "abc"
B. ["abc"]
C. ["a", "b", "c"]
D. "a b c"

4. What does "-".join(["a", "b", "c"]) return?

A. "a-b-c"
B. "abc-"
C. ["a-b-c"]
D. "a b c"

Lesson 10.1 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What does "hi".upper() return?

A. "hi"
B. "Hi"
C. "HI"
D. error

2. What does "HI".lower() return?

A. "HI"
B. "Hi"
C. "hi"
D. error

3. What does " hi ".strip() return?

A. " hi "
B. "hi"
C. " hi"
D. "hi "

4. What does "a-b-c".split("-") return?

A. ['abc']
B. 'abc'
C. error
D. ['a', 'b', 'c']

5. What does "-".join(["a", "b", "c"]) return?

A. "abc" B. "a,b,c"

C. error D. "a-b-c"

6. True or False: replace("a", "b") returns a new string with all "a" replaced by "b".

☐ True ☐ False

7. What does this code print?

PYTHON

```
print("Hello".upper())
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
s = "hello"
s.upper()
print(s)
```

Bug: _____

9. In one sentence, explain what split() does.

10. Given s = " Python ", write code that prints s with no leading or trailing spaces.

Lesson 10.2: String Slicing

Strings can be sliced just like lists, using the same [start:stop:step] syntax. Slicing gives you a new string containing only the characters you asked for.

PYTHON · string_slice.py

```
word = "Python"
print(word[0]) # "P"
print(word[-1]) # "n"
print(word[0:3]) # "Pyt" (first 3 characters)
print(word[1:4]) # "yth"
print(word[:3]) # "Pyt"
print(word[3:]) # "hon"
print(word[::-1]) # "nohtyP" (reversed!)
```

A common task is to extract part of a string based on its position. For example, to get the first letter of a name, use name[0]. To get the last letter, use name[-1]. To get everything except the first letter, use name[1:].

Lesson 10.2 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What does "Python"[0] return?

A. "P" B. "ython"
C. 0 D. An error

2. What does "Python"[1:4] return?

A. "Pyt" B. "yth"
C. "ython" D. "Pyth"

3. What does "Python"[-1] return?

A. "n" B. "P"
C. "-1" D. An error

4. What does "Python"[::-1] return?

A. "nohtyP" B. "Python"
C. "Pytoh" D. An error

Lesson 10.2 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. For s = "hello", what is s[0]?

A. "e" B. "h"
C. "hello" D. error

2. For s = "hello", what is s[1:3]?

A. "he" B. "ell"
C. "hel" D. "el"

3. For s = "hello", what is s[-1]?

A. "h" B. "e"
C. error D. "o"

4. What does s[::-1] do?

A. Returns the first char B. Returns the last char
C. Causes an error D. Reverses the string

5. For s = "hello", what is s[:2]?

A. "he" B. "el"
C. "hel" D. "h"

6. True or False: String slicing uses the same [start:stop] syntax as list slicing.

☐ True ☐ False

7. What does this code print?

PYTHON

```
print("hello"[1:4])
```

Output: _____

8. A student wants the last character of s = "hello" but writes s[5]. What is wrong?

PYTHON - has a bug

```
s = "hello"
print(s[5])
```

Bug: _____

9. In one sentence, explain what s[start:stop] does for a string.

10. Given s = "abcdef", write code that prints the last 2 characters using a slice.

Lesson 10.3: Strings Are Immutable

Strings are **immutable** — once a string is created, you cannot change its individual characters. Trying to assign to an index raises an error. To "change" a string, you create a new one based on the old one and assign it back to the variable.

PYTHON · immutable.py

```
name = "Alex"
name[0] = "E" # ERROR: strings are immutable

# The right way: create a new string
name = "E" + name[1:] # "Elex"
print(name)

# Or use replace
name = "Alex".replace("A", "E") # "Elex"
print(name)
```

This might seem annoying at first, but it actually makes your code safer. If two variables refer to the same string, you cannot accidentally change it for one of them. Methods like upper() and replace() do not modify the original string — they return a new one.

PYTHON · methods_return_new.py

```
name = "alex"
name.upper() # this does NOTHING to name
print(name) # "alex" (unchanged!)

name = name.upper() # this works — reassign
print(name) # "ALEX"
```

Lesson 10.3 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. Are strings mutable or immutable?

A. Mutable
B. Sometimes mutable
C. Immutable
D. Only in functions

2. What happens if you try s[0] = "X" on a string?

A. It raises a TypeError
B. It works
C. It changes s to "X"
D. Nothing

3. What does "hello".upper() do to the variable s if s = "hello"?

A. Returns "HELLO" but s is still "hello"
B. Changes s to "HELLO"
C. Causes an error
D. Nothing

4. How do you "change" a string?

A. Use the modify() method
B. Use the change() function
C. Strings can be changed directly
D. You cannot; you create a new one and reassign

Lesson 10.3 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What does "immutable" mean for strings?

A. Strings can be changed
B. Strings are numbers
C. Strings cannot be changed after creation
D. Strings are lists

2. What does s = "hi"; s = s.upper() do?

A. Changes the original string in place
B. Creates a new uppercase string and rebinds s to it
C. Nothing
D. Causes an error

3. What happens if you try s[0] = "H" on s = "hi"?

A. Raises TypeError
B. Works fine
C. Prints "Hi"
D. Causes a NameError

4. Why do string methods return new strings?

A. Because they are slow
B. Because strings are immutable
C. Because they are lists
D. No reason

5. What does "cat".replace("c", "b") return?

A. "cat"
B. "bat"
C. "b"
D. error

6. True or False: Strings are immutable, so any change creates a brand-new string.

☐ True ☐ False

7. What does this code print?

PYTHON

```
s = "hi"
t = s.upper()
print(s, t)
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
s = "hi"
s[0] = "H"
print(s)
```

Bug: _____

9. In one sentence, explain what it means that strings are immutable.

10. Given s = "hello", write code that returns a new string with the first letter uppercase using slicing.

Lesson 10.4: Checking String Contents

Several string methods return booleans, which makes them perfect for use in if statements. They let you check what kind of characters a string contains.

PYTHON · string_check.py

```
text = "Hello123"
print(text.isalpha()) # False (contains digits)
print(text.isdigit()) # False (contains letters)
print(text.isalnum()) # True (letters and digits only)
print(text.isupper()) # False
print(text.islower()) # False
print("123".isdigit()) # True
print(" ".isspace()) # True

# Checking for substrings with in
sentence = "The quick brown fox"
print("fox" in sentence) # True
print("cat" in sentence) # False
print(sentence.startswith("The")) # True
print(sentence.endswith("fox")) # True
```

Lesson 10.4 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What does "123".isdigit() return?

A. False
B. True
C. "123"
D. An error

2. What does "Hello".islower() return?

A. False
B. True
C. None
D. An error

3. What does "fox" in "The quick brown fox" return?

A. True
B. False
C. 3
D. An error

4. What does "Hello".startswith("He") return?

A. False
B. "He"
C. True
D. An error

Lesson 10.4 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What does "abc".isalpha() return?

A. True B. False
C. "abc" D. error

2. What does "123".isdigit() return?

A. False B. "123"
C. True D. error

3. What does "Hello".startswith("He") return?

A. False B. True
C. "He" D. error

4. What does "a" in "cat" return?

A. False B. True
C. "a" D. error

5. What does "hello123".isalpha() return?

A. True B. "hello123"
C. False D. error

6. True or False: The in operator checks if a character or substring is inside a string.

☐ True ☐ False

7. What does this code print?

PYTHON

```
print("Python".startswith("Py"))
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
s = "abc"
print(s.isalpha("a"))
```

Bug: _____

9. In one sentence, explain what isdigit() does.

10. Given s = "abc123", write code that prints True if s is all letters, else False.

Mixed Quiz 10

This quiz mixes different question types: multiple choice, True/False, short answer, and code writing. Try to answer all of them without looking back at the chapter first — that is how you really learn. Answers are in the Answer Key at the back of the book.

1. What does "hello".upper() return?

A. "Hello" B. "HELLO"
C. "hello" D. An error

2. What does "a,b,c".split(",") return?

A. "abc" B. ["a", "b", "c"]
C. ["abc"] D. "a b c"

3. What does " hi ".strip() return?

A. " hi " B. "hi"
C. " hi" D. "hi "

4. What does "Python"[::-1] return?

A. "Python" B. "nohtyP"
C. "Ptoh" D. An error

5. Strings in Python are:

A. Mutable B. Immutable
C. Sometimes mutable D. Only mutable inside functions

6. True or False: The expression "cat" in "catalogue" evaluates to True.

☐ True ☐ False

7. True or False: name.upper() changes the variable name to uppercase.

☐ True ☐ False

8. True or False: The replace() method returns a new string; it does not change the original.

☐ True ☐ False

9. What does "hello world".split() return?

A. "hello world" B. ["hello", "world"]
C. ["h", "e", "l", "l", "o"] D. "helloworld"

10. What does "-".join(["a", "b", "c"]) return?

A. "a-b-c" B. "abc-"
C. ["a-b-c"] D. "a b c"

11. True or False: The expression "hello"[1] evaluates to "e".

☐ True ☐ False

12. True or False: The method startswith() returns a boolean.

☐ True ☐ False

13. Explain in your own words what it means for strings to be "immutable". Why does this matter?

14. What is the difference between split() and join()? Give an example of each.

15. Write a program that asks the user for a sentence, then prints (a) the sentence in all uppercase, (b) the number of words in the sentence, and (c) the sentence reversed.

Chapter 10 Challenges

These questions are harder than the quiz. They ask you to combine ideas from this chapter, think creatively, and apply what you learned in new ways. Do not worry if you cannot solve them all on your first try — the point is to make you think. The full solutions are in the Answer Key at the back of the book.

Challenge 1. Write a program that asks the user for a sentence and prints the number of words, the number of characters (including spaces), and the sentence in title case.

Hint: Use split(), len(), and .title().

Challenge 2. What does "-".join("hello") return? Think carefully — "hello" is a string, not a list.

Hint: join() can take any iterable, including a string (which is a sequence of characters).

Challenge 3. Write a program that checks if a password is valid: at least 8 characters, contains at least one digit, and contains at least one uppercase letter. Print "Valid" or "Invalid".

Hint: Use len(), any(c.isdigit() for c in pw), and any(c.isupper() for c in pw). Or loop manually.

Challenge 4. What is the output of: print("Hello".replace("l", "w"))? Explain what replace does.

Hint: replace() returns a new string with all occurrences replaced.

Challenge 5. Write a program that asks the user for a word and prints whether it is a palindrome (reads the same forwards and backwards, like "racecar").

Hint: Compare the word to its reverse: word == word[::-1].

Chapter Recap

Here are the most important ideas from this chapter. If you can explain each one in your own words, you are ready to move on.

- String methods like upper(), lower(), strip(), replace(), split(), and join() perform common text operations.
- Strings can be sliced like lists: word[1:4], word[::-1].
- Strings are immutable — methods return new strings; they do not change the original.
- Methods like isalpha(), isdigit(), and startswith() return booleans for use in conditions.
- The in operator checks whether a substring exists inside a string.

Common Mistakes

Every beginner makes these mistakes. Knowing them in advance will save you hours of debugging.

COMMON MISTAKE

Forgetting that string methods return new strings
s.upper() does not change s. You must capture the result: s = s.upper().

COMMON MISTAKE

Confusing split and join
split goes from string to list; join goes from list to string. They are opposites. Note also that join is called on the separator, not on the list: ", ".join(words), not words.join(", ").

COMMON MISTAKE

Trying to modify a string character
s[0] = "X" is an error. Build a new string instead: s = "X" + s[1:].

COMMON MISTAKE

Forgetting that indices start at 0
s[1] is the *second* character, not the first. This is the same off-by-one trap that catches every beginner.

CHAPTER 11

File Handling

So far, every value your programs use disappears when the program ends. Real programs need to save data — to a settings file, a high score table, a list of to-dos. This chapter teaches you how to read from and write to text files on your computer.

Lesson 11.1: Opening and Closing Files

To work with a file, you first need to **open** it. The built-in open() function takes the filename and a **mode** telling Python what you want to do with the file. The most common modes are "r" for reading, "w" for writing (which erases any existing content), and "a" for appending (adding to the end of an existing file).

PYTHON · file_basic.py

```
# Open a file for writing
file = open("greeting.txt", "w")
file.write("Hello, file!\n")
file.write("This is the second line.\n")
file.close() # always close when you are done!

# Open the same file for reading
file = open("greeting.txt", "r")
content = file.read()
print(content)
file.close()
```

Notice the \n at the end of each line. This is a special character called a **newline**. It tells the file (and the screen) to start a new line. Without it, all the text would be on one line. When you read the file back, the newlines are preserved.

COMMON MISTAKE

Always close your files when you are done with them. If you forget, the data you wrote might not actually be saved to disk, because Python keeps a small buffer in memory. The next lesson shows a much safer way to handle this.

Lesson 11.1 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. Which mode opens a file for reading?

A. "r" B. "w"
C. "a" D. "x"

2. Which mode opens a file for writing (erasing existing content)?

A. "w" B. "r"
C. "a" D. "read"

3. What does f.write("Hello") do?

A. Reads "Hello" from the file B. Closes the file
C. Writes "Hello" to the file D. Prints "Hello" to the screen

4. What must you do after opening a file?

A. Nothing B. Print it
C. Rename it D. Close it when done

Lesson 11.1 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. Which mode opens a file for reading?

A. 'r' B. 'w'
C. 'a' D. 'x'

2. Which mode opens a file for writing (overwriting)?

A. 'r' B. 'w'
C. 'a' D. 'rb'

3. Which mode opens a file for appending (adding to the end)?

A. 'r' B. 'w'
C. 'a' D. 'x'

4. What function opens a file?

A. read() B. open()
C. file() D. start()

5. What should you do when you finish with a file?

A. Close it with close() B. Delete it
C. Print it D. Nothing

6. True or False: You should always close a file after you finish using it.

☐ True ☐ False

7. What does this code print if data.txt contains the word Hi?

PYTHON

```
f = open("data.txt", "r")
print(f.read())
f.close()
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
f = open("data.txt", "r")
print(f.read())
```

Bug: _____

9. In one sentence, explain what open() does.

10. Write code that opens notes.txt for writing, writes "Hello", and closes the file.

Lesson 11.2: The with Statement

Forgetting to close files is such a common mistake that Python gives you a safer way: the with statement. It opens the file for you and automatically closes it when the block ends — even if an error occurs. Use with for all file operations.

PYTHON · with_statement.py

```
with open("greeting.txt", "w") as file:
    file.write("Hello, file!\n")
    file.write("This is safer.\n")
# file is automatically closed here

print("File written and closed.")
```

When Python leaves the with block, the file is automatically closed for you — even if an error occurred inside the block. The variable file still exists after the block ends, but the file itself is already closed, so you should not try to read from or write to it. This is one of the cleanest patterns in Python — use it for all your file work.

Append Mode: Adding to a File

Mode "a" (append) lets you add new content to the end of an existing file without erasing what is already there. This is perfect for log files, visitor lists, or score tables:

PYTHON · append.py

```
# Append a new visitor to the file
with open("visitors.txt", "a", encoding="utf-8") as f:
    f.write("Lisa\n")
    f.write("Bob\n")

# Read the file back to see all visitors
with open("visitors.txt", "r", encoding="utf-8") as f:
    print(f.read())
```

TIP

The encoding="utf-8" argument tells Python to use UTF-8 text encoding. This is important on Windows, where the default encoding might not handle special characters (like accents or emoji) correctly. It is a good habit to always specify encoding="utf-8" when opening text files.

NOTE

When you open a file with just a filename like "visitors.txt", Python creates or looks for the file in the **current working directory** — the folder where your Python program is saved. If you are using IDLE, this is usually the same folder as your .py file. You can also use a full path like "C:/Users/Alex/visitors.txt" on Windows or "/home/alex/visitors.txt" on macOS/Linux.

Lesson 11.2 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What does the with statement do?

A. Runs code multiple times
B. Creates a new file
C. Automatically closes the file when the block ends
D. Makes the file read-only

2. Which is safer: with open(...) or open(...) + close()?

A. with open(...) — it closes even on errors
B. open() + close()
C. They are the same
D. Neither is safe

3. Inside a with block, when is the file closed?

A. Never
B. After each line
C. Only if an error occurs
D. When the block ends

4. What is the syntax for a with statement?

A. with f = open("f.txt"):
B. with open("f.txt") as f:
C. open("f.txt") with f:
D. with f.open("f.txt"):

Lesson 11.2 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What does the with statement do for files?

A. Opens them faster
B. Automatically closes the file when the block ends
C. Sorts the file
D. Deletes the file

2. Which is the correct with statement for a file?

A. with f = open("f.txt"): B. open("f.txt") with f:
C. with open("f.txt") as f: D. with f as open("f.txt"):

3. Why is with safer than open/close?

A. It runs faster B. You cannot forget to close the file
C. It sorts the data D. It removes errors

4. What happens at the end of a with block?

A. The file is deleted B. The file is closed automatically
C. Nothing D. The program ends

5. Does with close the file if an error occurs inside the block?

A. Yes B. No
C. Only sometimes D. Only for read mode

6. True or False: Using with is the recommended way to work with files.

☐ True ☐ False

7. What does this code print if hello.txt contains the word Hi?

PYTHON

```
with open("hello.txt", "r") as f:
    print(f.read())
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
with open("data.txt", "r") f:
    print(f.read())
```

Bug: _____

9. In one sentence, explain why the with statement is preferred for files.

10. Write a with statement that opens hello.txt for reading and prints its contents.

Lesson 11.3: Reading Files

There are three main ways to read the contents of a file: all at once, line by line, or as a list of lines. Each is useful in different situations.

PYTHON · read_modes.py

```
# Read the entire file as a single string
with open("greeting.txt", "r") as f:
    content = f.read()
    print(content)

# Read line by line (good for big files)
with open("greeting.txt", "r") as f:
    for line in f:
        print(line.strip()) # strip() removes whitespace from both ends

# Read all lines into a list
with open("greeting.txt", "r") as f:
    lines = f.readlines()
    print(f"The file has {len(lines)} lines.")
```

For most programs, reading line by line with for line in f is the best choice. It works for files of any size, because Python only keeps one line in memory at a time.

Lesson 11.3 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What does f.read() do?

A. Reads one line
B. Reads one character
C. Closes the file
D. Reads the entire file as a single string

2. What does for line in f: do?

A. Reads the whole file into a list
B. Reads one line at a time
C. Writes to the file
D. Closes the file

3. What does f.readlines() return?

A. A single string
B. One line
C. A list of lines
D. Nothing

4. Which is most memory-efficient for large files?

A. f.read()
B. for line in f:
C. f.readlines()
D. They are all the same

Lesson 11.3 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What does f.read() do?

A. Reads one line
B. Reads all lines into a list
C. Reads the whole file as one string
D. Closes the file

2. What does f.readline() do?

A. Reads the whole file
B. Closes the file
C. Sorts the file
D. Reads one line from the file

3. What does f.readlines() return?

A. One string
B. A list of lines
C. One number
D. Nothing

4. Which is the most memory-friendly way to loop over a big file?

A. f.read() B. for line in f:
C. f.readlines() D. f.read() in a loop

5. What does each line from a file usually end with?

A. A newline character B. A comma
C. A period D. Nothing

6. True or False: You can loop over a file with for line in f: to read one line at a time.

☐ True ☐ False

7. What does this code print if greeting.txt contains Hello then World on two lines?

PYTHON

```
with open("greeting.txt") as f:
    for line in f:
        print(line.strip())
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
with open("data.txt", "r") as f:
    print(f.real())
```

Bug: _____

9. In one sentence, explain the difference between read() and readlines().

10. Write a with statement that opens nums.txt and prints the number of lines in the file.

Lesson 11.4: A Practical Example: High Score Table

Here is a more realistic example. The program below keeps a high score in a file. Each time the program runs, it reads the previous high score, asks the user for a new score, and updates the file if the new score is higher. This combines everything you have learned about files, conditionals, and input.

PYTHON · highscore.py

```
# Read the previous high score
try:
    with open("highscore.txt", "r") as f:
        high = int(f.read().strip())
except FileNotFoundError:
    high = 0 # no file yet, so the high score is 0

print(f"Current high score: {high}")

# Get a new score
new_score = int(input("What is your score? "))

# Update if higher
if new_score > high:
    print("New high score!")
    with open("highscore.txt", "w") as f:
        f.write(str(new_score))
else:
    print("Not a new high score. Try again!")
```

Try running this program several times. The first time, it creates the file. Each time after that, it reads the saved high score and compares it to the new one. This is exactly how games keep track of high scores between play sessions.

Lesson 11.4 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. In the high score example, why is try/except used?

A. To make the program faster
B. To handle the case where the file does not exist yet
C. To close the file
D. To sort the scores

2. What does FileNotFoundError mean?

A. The file does not exist
B. The file is too big
C. The file is read-only
D. The file is empty

3. When reading a number from a file, what type do you get?

A. int
B. str — you must convert with int() or float()
C. float
D. bool

4. What does \n represent?

A. A backslash and the letter n
B. Nothing
C. An error
D. A newline character

Lesson 11.4 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. Which exception is raised when a file is not found?

A. FileNotFoundError
B. ValueError
C. TypeError
D. IndexError

2. Why wrap file code in try/except?

A. To make it slower
B. To handle missing files gracefully
C. To sort the file
D. To close the file twice

3. What is a high score file used for?

A. Printing all scores
B. Closing the program
C. Sorting names
D. Storing the best score between runs

4. What should you do if a file does not exist when loading a high score?

A. Crash the program
B. Print all errors
C. Delete the file
D. Use a default value like 0

5. Which code safely reads a score from score.txt?

A. try: f = open("score.txt") except FileNotFoundError: score = 0
B. open("score.txt") no try
C. while open("score.txt")
D. if open("score.txt")

6. True or False: Using try/except around file code lets the program keep running even if the file is missing.

☐ True ☐ False

7. What does this code print if data.txt does not exist?

PYTHON

```
try:
    f = open("data.txt")
    print(f.read())
except FileNotFoundError:
    print("No file")
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
f = open("missing.txt")
print(f.read())
```

Bug: _____

9. In one sentence, explain why you would use try/except when opening a file.

10. Write code that tries to open score.txt and print its contents, and prints "No score" if the file is missing.

Mixed Quiz 11

This quiz mixes different question types: multiple choice, True/False, short answer, and code writing. Try to answer all of them without looking back at the chapter first — that is how you really learn. Answers are in the Answer Key at the back of the book.

1. Which mode opens a file for writing and erases its current contents?

A. "r" B. "w"
C. "a" D. "x"

2. Which mode opens a file for appending (adding to the end)?

A. "r" B. "w"
C. "a" D. "i"

3. What does the with statement do?

A. Runs a block of code multiple times B. Automatically closes the file when the block ends
C. Makes the file read-only D. Creates a new file

4. What does the special character \n represent?

A. A backslash followed by n B. A newline
C. The number 0 D. An error

5. Which method reads the entire file as a single string?

A. f.readlines() B. f.readline()
C. f.read() D. f.readall()

6. True or False: You should always close a file when you are done with it.

☐ True ☐ False

7. True or False: The with statement closes the file even if an error occurs inside the block.

☐ True ☐ False

8. True or False: Opening a file in "w" mode keeps its existing contents and lets you add to them.

☐ True ☐ False

9. Which mode should you use to add new content to the end of an existing file without erasing it?

A. "r" B. "w"
C. "a" D. "x"

10. What does for line in f: do when f is an open file?

A. Reads the whole file into one big string B. Reads the file one line at a time, looping over each
C. Closes the file automatically D. Raises an error

11. True or False: When you read a number from a file, it comes back as a string and needs to be converted with int() or float() before you can do maths with it.

☐ True ☐ False

12. True or False: The \n character represents a newline.

☐ True ☐ False

13. Explain the difference between the file modes "r", "w", and "a". What happens to the file in each case?

14. Why is the with statement safer than using open() and close() manually?

15. Write a program that asks the user for their name and appends it to a file called "visitors.txt". Each name should be on its own line.

Chapter 11 Challenges

These questions are harder than the quiz. They ask you to combine ideas from this chapter, think creatively, and apply what you learned in new ways. Do not worry if you cannot solve them all on your first try — the point is to make you think. The full solutions are in the Answer Key at the back of the book.

Challenge 1. Write a program that asks the user for their name, then writes it to a file called "names.txt" (overwriting any existing content).

Hint: Use with open("names.txt", "w") and f.write().

Challenge 2. Write a program that reads a file called "numbers.txt" (one number per line), adds them all up, and prints the total.

Hint: Loop line by line, convert each to int, add to total.

Challenge 3. What happens if you open a file in "w" mode, write some text, and then open the SAME file in "r" mode and read it? Do you get the text you wrote? Explain.

Hint: You need to close the first file before opening it again.

Challenge 4. Write a program that asks the user for 3 lines of text and writes them to a file called "diary.txt", one line per file line. Then read the file back and print its contents to verify.

Hint: Use a loop to write, then read with f.read().

Challenge 5. A program wants to ADD new high scores to an existing file without erasing old ones. Which mode should it use: "w" or "a"? Explain why.

Hint: "w" erases; "a" appends.

Chapter Recap

Here are the most important ideas from this chapter. If you can explain each one in your own words, you are ready to move on.

- open(filename, mode) opens a file. Modes: "r" read, "w" write (overwrites), "a" append.
- Always close files when you are done. The with statement closes them automatically.
- Read files with f.read() (whole file as string), f.readlines() (list of lines), or for line in f (one line at a time).
- Write to files with f.write(text). Remember to add \n for newlines.

Common Mistakes

Every beginner makes these mistakes. Knowing them in advance will save you hours of debugging.

COMMON MISTAKE

Forgetting to close files
If you use open() without with, you must call f.close() yourself. The safer pattern is to always use with.

COMMON MISTAKE

Using "w" when you mean "a"
"w" erases the file before writing. "a" adds to the end without erasing. If you accidentally use "w", you can lose data.

COMMON MISTAKE

Forgetting to convert strings to numbers when reading
Files store text. If you wrote the number 100 to a file and read it back, you get the string "100". Convert it with int() or float() if you need to do maths.

COMMON MISTAKE

Forgetting newlines
f.write("Hello") does not add a newline. If you call write again, the text appears right after "Hello" on the same line. Add "\n" explicitly when you want a new line.

CHAPTER 12

Errors and Exceptions

Every programmer makes mistakes. The good news is that Python tries hard to tell you exactly what went wrong and where. This chapter teaches you how to read error messages, how to handle errors gracefully in your programs using try and except, and how to raise your own errors when something is wrong.

Lesson 12.1: Reading Error Messages

When Python hits a problem it cannot solve, it raises an **exception**. The default behaviour is to print an error message and stop the program. A typical error message looks like this:

OUTPUT · error_message.txt

```
Traceback (most recent call last):
  File "demo.py", line 3, in <module>
    print(10 / 0)
          ~~~^~~
ZeroDivisionError: division by zero
```

The most important parts are the **last line** (which tells you what kind of error occurred) and the line above the error code (which tells you *where* it occurred). When you see an error, read the last line first to find out what is wrong, then look at the line number to find out where to fix it.

Common Error Types

Error	When it happens	Example
`SyntaxError`	Your code breaks Python's grammar rules	`print("hi"`
`NameError`	You used a variable that does not exist	`print(score) (no score)`
`TypeError`	You used the wrong type for an operation	`"a" + 1`
`ValueError`	The value is the right type but wrong	`int("hello")`
`IndexError`	You used a list index that does not exist	`[1,2][5]`
`KeyError`	You looked up a missing dictionary key	`d["nope"]`
`ZeroDivisionError`	You divided by zero	`10 / 0`
`FileNotFoundError`	You tried to open a file that does not exist	`open("x.txt")`

Table 12.1 — The error types you will meet most often.

Lesson 12.1 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What error does int("hello") raise?

A. TypeError
B. NameError
C. SyntaxError
D. ValueError

2. What error does print(10 / 0) raise?

A. ValueError
B. MathError
C. TypeError
D. ZeroDivisionError

3. What error does print(undefined_var) raise?

A. ValueError
B. TypeError
C. SyntaxError
D. NameError

4. What does a SyntaxError mean?

A. A runtime error
B. A missing file
C. Your code breaks Python's grammar rules
D. A division by zero

Lesson 12.1 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What is a traceback?

A. A type of loop
B. A file format
C. A list of function calls leading to an error
D. A print statement

2. Where in the traceback is the actual error type and message?

A. At the top
B. In the middle
C. Nowhere
D. At the bottom (last line)

3. What does a NameError mean?

A. A file is missing
B. A variable was used before it was defined
C. A type is wrong
D. A loop never ends

4. What does a TypeError usually mean?

A. A name is missing
B. A file is missing
C. An operation was used on the wrong type
D. A loop ran too long

5. What does the first line in a traceback show?

A. The error type
B. The file and line number where the error started
C. The fix
D. The print output

6. True or False: The last line of a traceback shows the type of error and a short message.

☐ True ☐ False

7. What error does this code raise?

PYTHON

```
print(5 + "x")
```

Output: _____

8. A student gets a NameError for x. What is the most likely cause?

PYTHON - has a bug

```
print(x)
```

Bug: _____

9. In one sentence, explain what a traceback is.

10. Write code that would raise a ZeroDivisionError when run.

Lesson 12.2: try / except

Sometimes you can predict that an error might occur — for example, the user might type something that is not a number. You can **catch** the error with a try / except block. Python tries to run the code in the try block. If an error occurs, it jumps to the except block instead of crashing.

PYTHON · try_except.py

```
try:
    age = int(input("How old are you? "))
    print(f"Next year you will be {age + 1}.")
except ValueError:
    print("That does not look like a number. Please try again.")
```

Run this program and type hello instead of a number. Instead of crashing, the program prints the friendly message from the except block. This is how user-friendly programs handle bad input.

You can catch different exception types separately by chaining except blocks:

PYTHON · multiple_except.py

```
try:
    n = int(input("Enter a number: "))
    result = 100 / n
    print(f"100 / {n} = {result}")
except ValueError:
    print("That is not a number.")
except ZeroDivisionError:
    print("You cannot divide by zero.")
except:
    print("Some other error occurred.")
```

COMMON MISTAKE

A bare except: (with no exception type) catches *every* error, including ones you did not anticipate. This can hide bugs and make your program hard to debug. Always try to catch specific exception types, and use a bare except: only as a last resort.

Lesson 12.2 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. Which keyword catches an exception?

A. catch
B. handle
C. rescue
D. except

2. What is the syntax for try/except?

A. catch: ... throw: ...
B. try: ... rescue: ...
C. try: ... except: ...
D. handle: ... error: ...

3. What happens if no exception occurs in the try block?

A. The except block is skipped
B. The except block runs anyway
C. An error
D. The program crashes

4. Why should you catch specific exception types?

A. It is faster
B. It uses less memory
C. It is required by Python
D. A bare except can hide bugs you did not anticipate

Lesson 12.2 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What does try/except do?

A. Catches errors so the program can keep running
B. Sorts data
C. Loops
D. Prints files

2. Which keyword starts the block that may cause an error?

A. except
B. catch
C. try
D. error

3. Which keyword starts the block that handles the error?

A. try
B. error
C. except
D. fix

4. What happens if no error occurs in the try block?

A. The except block runs anyway
B. The program crashes
C. Nothing
D. The except block is skipped

5. Why catch a specific exception like ZeroDivisionError?

A. To slow down
B. To print files
C. To handle only that error and let others crash
D. No reason

6. True or False: If an error occurs in the try block, Python jumps to the matching except block.

☐ True ☐ False

7. What does this code print?

PYTHON

```
try:
    print(10 / 0)
except ZeroDivisionError:
    print("Nope")
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
try:
    print(10 / 0)
print("done")
```

Bug: _____

9. In one sentence, explain what try/except does.

10. Write code that tries to print 10 / 0 and prints "Cannot divide by zero" if it fails.

Lesson 12.3: finally and else

A try block can also have an else block (which runs only if no error occurred) and a finally block (which normally runs whether or not an error occurs). The finally block is perfect for cleaning up resources like files.

PYTHON · try_else_finally.py

```
try:
    with open("data.txt", "r") as f:
        content = f.read()
except FileNotFoundError:
    print("The file does not exist.")
    content = ""
else:
    print("File read successfully.")
finally:
    print("Cleanup done.")
    # The with statement already closed the file
```

Lesson 12.3 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. What does the finally block do?

A. Runs only if no error
B. Runs no matter what
C. Runs only if an error occurs
D. Ends the program

2. What does the else block in a try do?

A. Runs if NO error occurs
B. Runs if an error occurs
C. Always runs
D. Never runs

3. Which block is perfect for cleanup code (like closing files)?

A. finally
B. try
C. except
D. else

4. Can you have try/except/else/finally all together?

A. No
B. Yes
C. Only try/except
D. Only in Python 3

Lesson 12.3 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. What does finally do?

A. Runs only on error
B. Runs only on success
C. Skips the try
D. Runs its block normally, error or not

2. What does else do in try/except?

A. Runs only if no error occurred in try
B. Runs only on error
C. Always runs
D. Skips everything

3. When does the else block run?

A. Only when the try block did not raise an error
B. Only when an error occurred
C. Always
D. Never

4. When does the finally block run?

A. Only on error
B. Normally, error or no error
C. Only on success
D. Never

5. Which is a typical use of finally?

A. Looping B. Sorting data

C. Printing numbers D. Closing a file or cleaning up

6. True or False: The finally block runs whether or not an error occurred.

☐ True ☐ False

7. What does this code print?

PYTHON

```
try:
    x = 5
except:
    print("err")
else:
    print("ok")
finally:
    print("done")
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
try:
    x = 5
finally:
    print("done")
else:
    print("ok")
```

Bug: _____

9. In one sentence, explain the difference between else and finally.

10. Write a try/except/else/finally where the try sets x = 1, except prints "err", else prints "ok", and finally prints "done".

Lesson 12.4: Raising Your Own Exceptions

You can also raise exceptions yourself with the raise keyword. This is useful when your code detects a problem that the caller needs to know about — for example, a function that requires a positive number but received a negative one.

PYTHON · raise.py

```
def calculate_sqrt(n):
    if n < 0:
        raise ValueError("Cannot take sqrt of a negative number.")
    return n ** 0.5

try:
    print(calculate_sqrt(25)) # 5.0
    print(calculate_sqrt(-1)) # raises!
except ValueError as e:
    print(f"Error: {e}")
```

TIP

When you raise an exception, always include a helpful message. The message is what the user (or you, three weeks from now) will see when the error occurs. "Cannot take sqrt of a negative number" is much more helpful than just "Error".

Lesson 12.4 Check

Answer these four questions to check that you understood the lesson. If you get one wrong, go back and re-read the relevant part before moving on.

1. Which keyword raises an exception manually?

A. throw
B. raise
C. error
D. fail

2. What is the syntax for raising an exception?

A. throw ValueError("message")
B. raise ValueError("message")
C. error ValueError("message")
D. raise error ValueError("message")

3. Why should you include a message when raising an exception?

A. For speed
B. So the user understands what went wrong
C. It is required
D. For memory

4. What happens after an exception is raised?

A. The exception propagates until caught (or the program crashes)
B. The program continues normally
C. The program restarts
D. Nothing

Lesson 12.4 Practice

10 practice questions — easy to hard. Try them all before checking the Answer Key.

1. Which keyword raises an exception?

A. throw
B. error
C. except
D. raise

2. Which is the correct way to raise a ValueError with a message?

A. raise("ValueError", "bad")
B. raise ValueError("bad")
C. error ValueError("bad")
D. throw ValueError("bad")

3. Why raise your own exception?

A. To signal a problem with a helpful message
B. To slow the program
C. To print nothing
D. To sort data

4. What must follow the raise keyword?

A. A number
B. A string only
C. An exception object (like ValueError("..."))
D. A loop

5. What happens after raise runs?

A. The program stops at that line and looks for a matching except
B. The program continues
C. The program prints and continues
D. Nothing

6. True or False: You can raise a built-in exception like ValueError with your own message.

☐ True ☐ False

7. What does this code print?

PYTHON

```
try:
    raise ValueError("bad input")
except ValueError as e:
    print("caught:", e)
```

Output: _____

8. What is wrong with this code?

PYTHON - has a bug

```
raise "bad input"
```

Bug: _____

9. In one sentence, explain why you would raise your own exception.

10. Write code that raises a ValueError with the message "age cannot be negative" if age is less than 0.

Mixed Quiz 12

This quiz mixes different question types: multiple choice, True/False, short answer, and code writing. Try to answer all of them without looking back at the chapter first — that is how you really learn. Answers are in the Answer Key at the back of the book.

1. What error does int("hello") raise?

A. TypeError
B. ValueError
C. NameError
D. SyntaxError

2. What error does print(10 / 0) raise?

A. ZeroDivisionError
B. ValueError
C. DivisionError
D. MathError

3. What keyword catches an exception?

A. catch
B. handle
C. except
D. rescue

4. What does a finally block do?

A. Runs only if no error occurred
B. Runs only if an error occurred
C. Runs no matter what
D. Ends the program

5. Which keyword raises an exception manually?

A. throw
B. raise
C. error
D. fail

6. True or False: A bare except: catches every type of exception.

☐ True ☐ False

7. True or False: The else block of a try runs only if no exception occurred.

☐ True ☐ False

8. True or False: A SyntaxError means Python found a typo in your grammar, like a missing colon.

☐ True ☐ False

9. Which exception is raised by [1, 2][5]?

A. ValueError
B. IndexError
C. KeyError
D. TypeError

10. Which exception is raised by {"a": 1}["b"]?

A. ValueError
B. IndexError
C. KeyError
D. AttributeError

11. True or False: The finally block normally runs, whether or not an error occurred.

☐ True ☐ False

12. True or False: A program can continue running after an exception has been caught by except.

☐ True ☐ False

13. Explain why you should usually catch specific exception types instead of using a bare except:.

14. What is the difference between the else and finally blocks in a try statement?

15. Write a program that asks the user for a number and prints its square root. Use a try/except block to handle the case where the user enters a negative number, and print a friendly message instead of crashing.

Chapter 12 Challenges

These questions are harder than the quiz. They ask you to combine ideas from this chapter, think creatively, and apply what you learned in new ways. Do not worry if you cannot solve them all on your first try — the point is to make you think. The full solutions are in the Answer Key at the back of the book.

Challenge 1. Write a program that asks the user for a number and prints 100 divided by that number. Use try/except to handle division by zero and invalid input (non-numbers). Print friendly messages instead of crashing.

Hint: Two except blocks: ZeroDivisionError and ValueError.

Challenge 2. What is the difference between an error and an exception in Python? (Hint: are all errors exceptions? Are all exceptions errors?)

Hint: Think about SyntaxError vs ValueError.

Challenge 3. Write a function called get_age() that asks the user for their age. If they enter something that is not a number, keep asking until they enter a valid integer. Use a while loop with try/except.

Hint: Use while True with try/except; break on success.

Challenge 4. What does this code do? try: x = 1/0 except: print("Error") Is it good practice? Why or why not?

Hint: Bare except catches everything, including bugs.

Challenge 5. Write a program that asks the user for a filename and tries to open it. If the file does not exist, ask again. Give the user at most 3 tries.

Hint: Use a counter with while, and try/except FileNotFoundError.

Chapter Recap

Here are the most important ideas from this chapter. If you can explain each one in your own words, you are ready to move on.

- Python reports many program failures using **exceptions**. The last line of a traceback usually identifies the exception type and message.
- Common types: SyntaxError, NameError, TypeError, ValueError, IndexError, KeyError, ZeroDivisionError, FileNotFoundError.
- try / except lets you catch exceptions and keep running. Catch specific exception types when possible.
- else runs only if no exception occurred; finally normally runs whether or not an exception occurs, making it useful for cleanup. It is not guaranteed to run if the program is terminated abruptly.
- Use raise to raise an exception deliberately. When appropriate, include a clear message that explains the problem.

Common Mistakes

Every beginner makes these mistakes. Knowing them in advance will save you hours of debugging.

COMMON MISTAKE

Using a bare except:
Catching too broadly hides bugs. Use except ValueError: instead of except: whenever you can. Save the bare form for genuine last-resort cases.

COMMON MISTAKE

Not reading the full error message
The traceback shows the line number and the error type. Skim it carefully — most of the time, the message tells you exactly what is wrong.

COMMON MISTAKE

Forgetting to convert input before validation
If you call int(input()) and the user types text, the ValueError happens before your validation. Wrap the int() in a try block to handle this.

COMMON MISTAKE

Catching an exception but not handling it
except: pass silently swallows the error. This makes bugs invisible. At minimum, print a message so you know what happened.

MINI-PROJECT

Mini-Project 4: Word Frequency Analyzer

You now know about strings, dictionaries, file handling, and functions. Let us build a program that reads a text file, counts how many times each word appears, and prints the top 5 most common words. This is a real-world task that data analysts and writers use, and it brings together many of the skills you have learned.

How It Works

The program asks the user for a filename, reads the file, splits the text into words, and uses a dictionary to count each word. Then it sorts the words by count and prints the top 5. It uses try/except to handle the case where the file does not exist.

The Code

Here is the complete word frequency analyzer. Create a small text file called sample.txt with a few sentences, then run this program to see it in action.

PYTHON · word_count.py

```
# Word Frequency Analyzer

def count_words(filename):
    """Read a file and count word frequencies."""
    try:
        with open(filename, "r", encoding="utf-8") as f:
            text = f.read()
    except FileNotFoundError:
        print(f"Error: {filename} not found.")
        return {}

    # Split into words and count
    words = text.lower().split()
    counts = {}
    for word in words:
        # Remove common punctuation
        word = word.strip(".,!?;:'"()")
        if word:
            if word in counts:
                counts[word] += 1
            else:
                counts[word] = 1
    return counts

def show_top(counts, n=5):
    """Show the top n most common words."""
    # Sort by count (highest first)
    sorted_words = sorted(counts.items(),
                          key=lambda x: x[1],
                          reverse=True)
    print(f"\nTop {n} most common words:")
    for word, count in sorted_words[:n]:
        print(f" {word}: {count}")

# Main program
print("=== Word Frequency Analyzer ===")
filename = input("Enter filename: ")
counts = count_words(filename)
if counts:
    total = sum(counts.values())
    unique = len(counts)
    print(f"\nTotal words: {total}")
    print(f"Unique words: {unique}")
    show_top(counts, 5)
```

Things to Try

Once you have the analyzer working, try these extensions:

- Ignore common words like "the", "a", "and", "is", "to" by keeping a list of stop words and skipping them.
- Write the results to a file called "results.txt" instead of just printing them.

- Ask the user how many top words to show (instead of always 5).
- Count the frequency of each letter instead of each word, and display a simple bar chart using asterisks (e.g., e: **********).

What You Have Learned

This mini-project brings together almost everything in the book: functions, dictionaries, string methods (split(), lower(), strip()), file handling with with and encoding, exception handling with try/except, and the sorted() built-in. If you can understand and modify this program, you are ready for the final project — the quiz game.

FINAL PROJECT

Final Project: Build a Quiz Game

Congratulations — you have learned all the fundamentals of Python. Now it is time to put everything together by building a real, working program: a multiple-choice quiz game. The game will ask the player a series of questions, keep track of their score, and tell them how they did at the end. By the time you finish this project, you will have used every major concept from this book: variables, input, output, conditionals, loops, functions, lists, dictionaries, and exception handling.

Step 1: Design the Quiz

Before we write any code, let us think about what the quiz needs to do. A good design saves time later. We need a way to store questions, a way to ask each question, a way to check the answer, and a way to keep score. We will represent each question as a dictionary with four pieces of information:

- The **question text** — what we want to ask.
- A list of **choices** — the possible answers.
- The **correct answer** — the index of the right choice.
- An optional **explanation** — shown after the answer.

We will store all the questions in a list. This is a great use of lists and dictionaries together: the list gives us an ordered collection of questions, and each dictionary holds the details of one question.

Step 2: Define the Questions

PYTHON · questions.py

```
questions = [
    {
        "question": "What is the capital of France?",
        "choices": ["Berlin", "Paris", "Rome", "Madrid"],
        "answer": 1,
        "explanation": "Paris has been the capital since 987 AD."
    },
    {
        "question": "Which planet is closest to the Sun?",
        "choices": ["Venus", "Earth", "Mercury", "Mars"],
        "answer": 2,
        "explanation": "Mercury is the closest planet to the Sun."
    },
    {
        "question": "Who wrote 'Romeo and Juliet'?",
        "choices": ["Dickens", "Shakespeare", "Austen", "Twain"],
        "answer": 1,
        "explanation": "Shakespeare wrote it around 1594-96."
    },
    {
        "question": "What is 7 times 8?",
        "choices": ["54", "56", "58", "64"],
        "answer": 1,
        "explanation": "7 times 8 equals 56."
    },
    {
        "question": "Which gas do plants absorb?",
        "choices": ["Oxygen", "Nitrogen", "CO2", "Hydrogen"],
        "answer": 2,
        "explanation": "Plants use CO2 in photosynthesis."
    },
]
```

Step 3: Write the Asking Function

We will write a function that takes a single question dictionary and asks it to the player. The function returns True if the player answered correctly, and False otherwise. Using a function keeps the main loop clean and lets us test the asking logic on its own.

You will notice a **docstring** on the second line — the text inside triple quotes """...""" right after the def line. A docstring is a special kind of comment that describes what a function does. Unlike a regular # comment, Python can read docstrings programmatically, and tools like IDLE can display them when you ask for help. It is good practice to always include a docstring in your functions.

You will also see enumerate(questions, 1) in the main function. This is the same enumerate() you learned in Lesson 8.4, but with a second argument: the starting number. By default, enumerate() starts counting from 0, but passing 1 as the second argument makes it start from 1 instead. This is useful for displaying "Question 1", "Question 2", etc. to the player.

PYTHON · ask_question.py

```
def ask_question(question):
    """Ask one question. Return True if correct."""
    print("\n" + question["question"])
    for i, choice in enumerate(question["choices"]):
        print(f" {i + 1}. {choice}")

    num_choices = len(question["choices"])
    while True:
        answer = input("Your answer: ")
        try:
            choice_num = int(answer) - 1
            if 0 <= choice_num < num_choices:
                break
            print(f"Enter 1 to {num_choices}.")
        except ValueError:
            print("Not a number. Try again.")

    if choice_num == question["answer"]:
        print("Correct! " + question["explanation"])
        return True
    else:
        correct = question["choices"][question["answer"]]
        print(f"Sorry, the answer was: {correct}")
        print(question["explanation"])
        return False
```

Notice how the while True loop keeps asking until the player enters a valid number. The try / except catches the case where they type something that is not a number, and the if 0 <= choice_num < num_choices check makes sure the number is in range. Both checks together make the program **robust** — it never crashes on bad input.

Step 4: Put It All Together

Note: Put all the code from Steps 2, 3, and 4 into a single file called quiz_game.py. The questions list goes at the top, then the ask_question() function, then the main() function, and finally the main() call at the very bottom. Run the program with **F5** in IDLE. The main() function is a common pattern in Python: it holds the "top-level" logic of the program — the code that ties everything together. By putting it in a function instead of writing it at the top level, you make the program easier to test and reuse. The line main() at the very bottom is what actually starts the program.

PYTHON · quiz_game.py

```
def main():
    print("=" * 40)
    print(" Welcome to the Quiz Game!")
    print("=" * 40)
    total = len(questions)
    print(f"There are {total} questions.\n")

    score = 0
    for i, question in enumerate(questions, 1):
        print(f"\n--- Question {i} of {total} ---")
        if ask_question(question):
            score += 1

    print("\n" + "=" * 40)
    print(f" You scored {score} out of {total}.")
    pct = (score / total) * 100
    print(f" That is {pct:.0f}%.")
    if pct == 100:
        print(" Perfect score!")
    elif pct >= 60:
        print(" Well done!")
    else:
        print(" Keep practising!")
    print("=" * 40)

# Run the quiz
main()
```

The main() function ties everything together. It prints a welcome message, loops through every question, calls ask_question() for each, and keeps track of the score. At the end, it calculates a percentage and prints a message based on how well the player did. Try running this program — it should feel like a real quiz game!

Step 5: Extension Challenges

Once you have the basic quiz working, try these extensions. Each one will teach you something new and make your program more interesting.

1. Add more questions to the questions list. The longer the quiz, the better.
2. Save the player's score to a file at the end, and read it back at the start to show their best score so far. (Use the pattern from Chapter 11.)
3. Let the player choose a category at the start (e.g. Science, History, Maths) and only ask questions from that category. (Hint: add a "category" key to each question dictionary.)
4. Add a timer that shows how long the player took to answer each question. (Hint: look up the time module.)
5. Make the quiz multiple-player: ask how many people are playing, then run the quiz once for each player, keeping separate scores.
6. Write the questions in a separate text file and read them in at the start, instead of having them in the code. This is how real quizzes work — you can change the questions without changing the program.

> **TIP**
>
> The best way to learn programming is to build things. Once you have the quiz working, change it. Add features that you think would be fun. Break it on purpose and figure out how to fix it. Every minute you spend experimenting teaches you more than ten minutes of reading.

REFERENCE

Glossary

This glossary explains the technical terms used in the book. Words are listed in alphabetical order. If you meet a word you do not recognise while reading, look it up here.

Algorithm
A step-by-step set of instructions for solving a problem. A recipe is an algorithm for cooking a dish.

Argument
A value you pass to a function when you call it. In print("hi"), the string "hi" is the argument.

Assignment
The act of storing a value in a variable using the = operator. x = 5 assigns the value 5 to x.

Boolean
A value that is either True or False. Named after the mathematician George Boole.

Bug
A mistake in a program that causes it to behave incorrectly. A famous early hardware bug was an actual moth found in a relay in 1947.

Call
To run a function by writing its name followed by parentheses, e.g. print().

Comment
A note in the code that Python ignores. Comments start with # and are for humans, not the computer.

Comparison operator
An operator that compares two values and returns a boolean: == != < > <= >=.

Compilation
The process of translating source code into another form before the program runs. In Python, source code is compiled into bytecode, which is then executed by the Python interpreter. This is different from languages like C, which compile directly to machine code.

Concatenation
Joining two strings together with the + operator. "hello" + " world" gives "hello world".

Condition
An expression that evaluates to True or False, used to control if statements and while loops.

Dictionary

A collection of key-value pairs. Look up a key to get its value. Created with curly braces: {"name": "Alex"}.

Exception
An object representing an exceptional condition in Python. Many exceptions occur while a program runs; SyntaxError can be raised while Python compiles or parses code. If not caught, an exception stops the program.

Expression
Any piece of code that produces a value. 2 + 3 is an expression; its value is 5.

Float
A number with a decimal point, like 3.14 or -0.001. Short for "floating-point number".

f-string
A string prefixed with f that lets you insert expressions in curly braces: f"Hello, {name}".

Function
A named block of code that does one specific job. Defined with def, called by name.

Global variable
A variable defined outside any function. It can be read from inside functions but should not be modified there.

IDLE
Python's built-in development environment. Stands for Integrated Development and Learning Environment.

Immutable
Cannot be changed after creation. Strings and tuples are immutable; lists and dictionaries are mutable.

Index
The position of an item in a list or string. Python indices start at 0.

Infinite loop
A loop whose condition never becomes False, so it runs forever. Stop with Ctrl+C.

Input
Data that comes into the program from the user (via input()) or from a file.

Integer
A whole number, like 42 or -7. The Python type is int.

Interpreter
A program that runs source code one line at a time. Python is usually interpreted.

Iteration
One pass through the body of a loop. A loop that runs 5 times has 5 iterations.

Key
The "look-up" part of a dictionary entry. Each key maps to one value.

Keyword
A reserved word in Python with a special meaning, like if, for, def. You cannot use keywords as variable names.

List
An ordered, mutable collection of values, created with square brackets: [1, 2, 3].

Local variable
A variable defined inside a function. It only exists while the function is running.

Loop
A block of code that runs repeatedly. Python has for loops and while loops.

Method
A function attached to an object, called with a dot: "hi".upper().

Modulo
The remainder after division. The operator is %. 10 % 3 is 1.

Module
A file containing Python code that can be imported into other programs with import.

Mutable
Can be changed after creation. Lists and dictionaries are mutable.

Parameter
A variable listed in a function definition that receives an argument when the function is called.

Precedence
The order in which operators are evaluated in an expression. Multiplication has higher precedence than addition.

Return
To send a value back from a function to the caller using the return keyword.

Scope
The part of the program where a variable is visible. Local scope is inside a function; global scope is outside all functions.

Set
An unordered collection of unique items, created with curly braces: {1, 2, 3}.

Slice
A sub-list or sub-string extracted using [start:stop] notation.

String
A sequence of characters — a piece of text. The Python type is str.

Syntax
The grammar rules of a programming language. A SyntaxError means you broke the rules. In Python 3, SyntaxError is technically an exception, but it is caught at compile time (before your program runs), so it usually feels different from runtime errors like ValueError.

Tuple
An ordered, immutable collection, created with parentheses: (1, 2, 3).

Type
The kind of value a variable holds. Common types are int, float, str, bool, list, dict.

Value
The actual data stored in a variable. The value of x = 5 is 5.

Variable
A named container for a value. Created with assignment: name = value.

While loop
A loop that repeats as long as its condition is True.

Argument
A value you pass to a function when you call it. In print("Hi"), the string "Hi" is the argument.

Debugging
The process of finding and fixing errors (bugs) in a program. Common techniques include reading error messages, adding print() statements, and testing with small inputs.

Indentation
The spaces at the start of a line. Python uses indentation (usually 4 spaces) to define which lines belong to a block, such as the body of an if statement or a for loop.

Index
The position of an item in a list, string, or tuple. Python indices start at 0, so the first item is at index 0.

Library
A collection of modules that you can import into your program. The Python Standard Library includes modules like math, random, and os.

Operator
A symbol that performs an operation on one or more values. Examples: + - * / == and or.

Statement
A single instruction in Python. x = 5 is a statement. print("Hi") is a statement. A program is a sequence of statements.

Traceback
The error message Python prints when an uncaught exception occurs. It shows the sequence of function calls that led to the error, with the error type and message on the last line.

REFERENCE

Index

Use this index to quickly find where a topic, function, or keyword is explained in the book.

A

abs() — Lesson 4.5
append() — Lesson 8.3
Arguments — Lesson 7.2
Arithmetic operators — Lesson 3.1
Assignment — Lesson 2.1
Augmented assignment — Lesson 3.2

B

Boolean — Lesson 2.4
break — Lesson 6.3
Bug — Glossary
Built-in functions — Lesson 4.5

C

Challenges — End of each chapter

Comments — Lesson 1.3
Comparison operators — Lesson 3.3
Compilation — Glossary
Concatenation — Lesson 2.3
Conditionals — Chapter 5
continue — Lesson 6.3
Converting types — Lesson 2.5

D

Debugging — Glossary
def — Lesson 7.1
Dictionaries — Chapter 9
docstrings — Final Project, Step 3
Dictionaries, methods — Lesson 9.2

E

elif — Lesson 5.3
else — Lesson 5.2
Escape sequences — Lesson 4.5
Exceptions — Chapter 12
Expressions — Chapter 3
extend() — Lesson 4.5

F

f-strings — Lesson 4.3
File handling — Chapter 11
finally — Lesson 12.3
Float — Lesson 2.2
for loops — Lesson 6.1
Functions — Chapter 7

G

Glossary — After Chapter 12
Global variables — Lesson 7.5

I

IDLE — Lesson 1.2
if statements — Lesson 5.1
Immutable — Lesson 8.5, 10.3
import — Lesson 4.5
Indentation — Lesson 5.1
Index (list) — Lesson 8.1
Index (book) — This section
Input — Lesson 4.1
Integer — Lesson 2.2

L

len() — Lesson 4.5
Lists — Chapter 8
Local variables — Lesson 7.5
Logical operators — Lesson 3.4
Loops — Chapter 6

M

max() — Lesson 4.5
min() — Lesson 4.5
Modules — Lesson 4.5
Mutable — Lesson 8.3

N

Nested conditionals — Lesson 5.4
Nested loops — Lesson 6.4
None — Lesson 7.3

O

Operator precedence — Lesson 3.2
Operators — Chapter 3
Output — Lesson 4.2

P

Parameters — Lesson 7.2
pop() — Lesson 8.3
print() — Lesson 1.3
Python (history) — Lesson 1.1

R

raise — Lesson 12.4
random module — Lesson 4.5
range() — Lesson 6.1
return — Lesson 7.3
round() — Lesson 4.5

S

Slicing — Lesson 8.2, 10.2
sorted() — Lesson 4.5
Strings — Lesson 2.3, Chapter 10
String methods — Lesson 10.1
sum() — Lesson 4.5
SyntaxError — Lesson 12.1

T

Tables — Lesson 2.5, 3.1, 12.1
Tuples — Lesson 8.5
Traceback — Lesson 12.1, Glossary
try/except — Lesson 12.2
type() — Lesson 2.2

V

Variables — Lesson 2.1

W

while loops — Lesson 6.2
with statement — Lesson 11.2

REFERENCE

Answer Key

This Answer Key gives detailed explanations for every question in the book. For each chapter, you will find: (1) the Lesson Check answers that follow each lesson, (2) the Mixed Quiz answers with full explanations, and (3) the Challenge solutions with detailed step-by-step reasoning. Try each question on your own first, then read the explanation carefully.

Lesson Check Answers

These are the answers to the 4-question checks that appear after each lesson. After each chapter's lesson check answers, you will find that chapter's Challenge Solutions in a outlined box — easy to spot. If you got one wrong, re-read the lesson before moving on.

Chapter 1 — Introduction to Python

Lesson 1.1 Check

Correct answer: D. Guido van Rossum

Why: Guido van Rossum created Python and first released it in 1991.

Why the other options are wrong:

- **A.** Linus Torvalds — Created the Linux kernel, not Python.
- **B.** Bill Gates — Co-founded Microsoft, unrelated to Python.
- **C.** Dennis Ritchie — Created the C programming language.

Correct answer: D. The Monty Python comedy group

Why: Python was named after Monty Python, not the snake.

Why the other options are wrong:

- **A.** A species of snake — This is a common misconception.
- **B.** An ancient Greek word — Not the origin of the name.
- **C.** A type of computer — Unrelated.

Correct answer: A. It only runs on Windows

Why: Python runs on Windows, macOS, and Linux, so this is NOT a reason (it is false).

Why the other options are wrong:

- **B.** It is easy to read — This IS a reason — Python is readable.
- **C.** It is free and open — This IS a reason — Python is free.
- **D.** It has a huge community — This IS a reason — huge community.

Correct answer: D. The Netherlands

Why: Guido van Rossum is Dutch.

Why the other options are wrong:

- **A.** The United States — He is European, not American.
- **B.** Germany — He is from the Netherlands, not Germany.
- **C.** Japan — Incorrect.

Lesson 1.1 Practice

Q1. Answer: D. Guido van Rossum

Guido van Rossum created Python in 1991.

Q2. Answer: D. The Monty Python comedy group

Python was named after Monty Python, the British comedy group.

Q3. Answer: D. Only runs on Windows

Python runs on Windows, macOS, and Linux, so "only Windows" is false.

Q4. Answer: C. 1991

Python was first released in February 1991.

Q5. Answer: C. Netherlands

He is Dutch, from the Netherlands.

Q6. Answer: False

Python was named after the Monty Python comedy group, not the snake.

Q7. Output: Hello, World!

print() displays the text inside the quotes.

Q8. Bug: The text is not in quotation marks, so Python treats Hello and World as variables.

Strings must be wrapped in quotes.

Q9. Answer:

A programming language is a way to write instructions that a computer can understand and follow.

Q10. Answer:

print("Python is awesome!")

Lesson 1.2 Check

Correct answer: C. Integrated Development and Learning Environment

Why: IDLE stands for Integrated Development and Learning Environment.

Why the other options are wrong:

- **A.** Interactive Direct Language Editor — Not the correct expansion.
- **B.** Integer Decimal Logic Engine — Not correct.
- **D.** Internal Data Link Environment — Not correct.

Correct answer: D. The box that says "Add Python to PATH"

Why: Without this, Windows cannot find Python from the command line.

Why the other options are wrong:

- **A.** "Install for all users" — Optional, not critical.
- **B.** "Add shortcuts" — Optional.
- **C.** "Install pip" — pip is installed automatically.

Correct answer: B. Windows, macOS, and Linux

Why: Python runs on all three major operating systems.

Why the other options are wrong:

- **A.** Windows only — Python is cross-platform.
- **C.** macOS only — Python is cross-platform.
- **D.** Linux only — Python is cross-platform.

Correct answer: B. Use a free online Python environment like replit.com

Why: Online environments let you code without installing anything.

Why the other options are wrong:

A. Give up — Never give up — there are alternatives!

C. Buy a new computer — Not necessary.

D. Python cannot be used without installing it — Online environments prove this wrong.

Lesson 1.2 Practice

Q1. Answer: D. python.org

The official site is python.org.

Q2. Answer: C. Integrated Development and Learning Environment

IDLE is the Integrated Development and Learning Environment.

Q3. Answer: C. Telling the OS where to find programs

PATH tells the operating system where to find executable programs like Python.

Q4. Answer: D. Windows, macOS, and Linux

Python runs on all major operating systems.

Q5. Answer: C. A .exe installer downloaded from python.org

You download an .exe installer from python.org.

Q6. Answer: True

IDLE is included with the standard Python installation.

Q7. Output: Python 3.15.0

The --version flag prints the installed Python version (exact numbers vary).

Q8. Bug: Python was not added to the PATH during installation.

Reinstall and check "Add Python to PATH".

Q9. Answer:

IDLE is a simple editor that comes with Python for writing and running Python code.

Q10. Answer:

python --version

Lesson 1.3 Check

Correct answer: D. Hello, World!

Why: print() displays the text inside the quotation marks.

Why the other options are wrong:

A. print("Hello, World!") — This is the code, not the output.

B. Hello World — The output includes the comma and exclamation mark.

C. An error — The code is correct.

Correct answer: D. #

Why: The hash symbol # starts a comment in Python.

Why the other options are wrong:

A. // — This is used in C, Java, and JavaScript.

B. /* — This is used in C and Java for block comments.

C. -- — This is used in SQL and Haskell.

Correct answer: C. Python raises an error because Hello is not a defined variable

Why: Without quotes, Python treats Hello as a variable name, but no variable called Hello exists.

Why the other options are wrong:

A. It prints Hello — Only if Hello were a defined variable.

B. It prints the word Hello without quotes — Same issue — Hello is not defined.

D. It prints nothing — It raises an error, not silence.

Correct answer: B. .py

Why: Python files use the .py extension.

Why the other options are wrong:

A. .txt — This is for plain text files.

C. .python — Not a real extension.

D. .p — Not the standard Python extension.

Lesson 1.3 Practice

Q1. Answer: B. print()

print() displays output.

Q2. Answer: D. #

The # symbol starts a comment.

Q3. Answer: C. .py

Python files use the .py extension.

Q4. Answer: C. Quotation marks

Strings need quotes (single or double).

Q5. Answer: C. Python runs the code line by line

Python executes the code in the file.

Q6. Answer: True

Comments are for humans; Python skips them.

Q7. Output: Hello World

Two print statements produce two lines.

Q8. Bug: print is missing parentheses.

In Python 3, print is a function and needs ().

Q9. Answer:

A comment is a note in the code that Python ignores, used to explain the code to humans.

Q10. Answer:

print("My name is Sam")

Chapter 1 Challenges - Solutions

Challenge 1

Question: Write a Python program that prints your name on one line and your favourite hobby on the next, with a comment above each line.

Solution:

You need two print() calls and two comment lines (starting with #). Each comment explains what the next line does.

Print my name print("Sarah") # Print my favourite hobby print("Programming")

Challenge 2

Question: What happens if you run print("Hello") and then print("World")?

Solution:

You get two lines of output: Hello World

Each print() call adds a newline character at the end by default. So "Hello" is printed, then the cursor moves to a new line, then "World" is printed on that new line. The result is two separate lines of text.

Challenge 3

Question: A friend says comments slow down your program. Is this correct?

Solution:

No, this is incorrect. Python completely ignores comments when the program runs. When Python compiles your code, it strips out all comments before executing. They have zero effect on the speed of your program.

Comments exist only for humans reading the code. They are useful for explaining what the code does, leaving notes for yourself or others, and temporarily disabling lines of code while testing. Adding comments never makes your program slower.

Challenge 4

Question: How do you print: She said "Python is fun!" with the quotation marks?

Solution:

If you use double quotes on the outside, Python thinks the string ends at the first internal double quote, causing an error. The trick is to use SINGLE quotes on the outside so the double quotes inside are treated as regular characters:

print('She said "Python is fun!"')

This outputs: She said "Python is fun!" — with the quotation marks visible. You could also use double quotes outside and escape the inner ones with backslashes: print("She said \"Python is fun!\""), but the single-quote approach is cleaner.

Challenge 5

Question: Write a program that uses exactly one print() call to display three lines: "Line 1", "Line 2", and "Line 3".

Solution:

The special character \n means "newline". By putting \n inside the string, you tell print to start a new line at that point, all within a single print() call:

print("Line 1\nLine 2\nLine 3")

This outputs: Line 1 Line 2 Line 3

The \n is called an "escape sequence" — a backslash followed by a letter that means something special. You will learn more about these in later chapters.

Chapter 2 — Variables and Data Types

Lesson 2.1 Check

Correct answer: B. _score

Why: Underscores are allowed in variable names, and _score does not start with a digit or contain spaces.

Why the other options are wrong:

- **A.** 2nd_place — Cannot start with a digit.
- **C.** player score — Cannot contain spaces.
- **D.** for — "for" is a Python keyword.

Correct answer: B. It adds 5 to score and stores the result back in score

Why: Python evaluates the right side first (score + 5), then assigns the result back to score.

Why the other options are wrong:

- **A.** Creates a new variable — It updates the existing variable, not a new one.
- **C.** Compares — Comparison uses ==, not =.
- **D.** Error — This is valid Python.

Correct answer: D. 20

Why: The second assignment overwrites the first. x is now 20.

Why the other options are wrong:

- **A.** 10 — The old value was replaced.
- **B.** 30 — Assignment is not addition.
- **C.** Error — Reassignment is valid.

Correct answer: A. player-name

Why: The hyphen is not allowed in variable names. Python treats it as a minus sign.

Why the other options are wrong:

- **B.** player1 — Valid — digits are allowed after the first character.
- **C.** player_name — Valid — underscores are allowed.
- **D.** PlayerName — Valid — capital letters are allowed.

Lesson 2.1 Practice

Q1. Answer: D. name_2

Variable names can have letters, digits, and underscores, but cannot start with a digit or use reserved words.

Q2. Answer: C. x = 5

A single = assigns a value.

Q3. Answer: B. 10

Reassignment replaces the old value; x is now 10.

Q4. Answer: B. 2nd_place

Variable names cannot start with a digit.

Q5. Answer: D. Creates a variable called name holding the string Ana

It creates a variable and stores the value.

Q6. Answer: True

Variables can be reassigned to new values.

Q7. Output: 7

x is 4, so y is 4 + 3 = 7.

Q8. Bug: Variable names cannot start with a digit.

Start variable names with a letter or underscore.

Q9. Answer:

A variable is a named place in memory that stores a value you can use and change later.

Q10. Answer:

age = 12

Lesson 2.2 Check

Correct answer: C. int

Why: 42 is a whole number, so its type is int (integer).

Why the other options are wrong:

A. float — Floats have decimal points, like 3.14.

B. str — Strings are text in quotation marks.

D. bool — Booleans are True or False.

Correct answer: C. 3

Why: // is floor division. 10 // 3 = 3 (it rounds down to the nearest integer).

Why the other options are wrong:

A. 3.333... — This is 10 / 3 (true division).

B. 1 — This is 10 % 3 (the remainder).

D. 4 — This would be 12 // 3.

Correct answer: A. 1

Why: % is modulo. 10 divided by 3 is 3 with 1 left over.

Why the other options are wrong:

B. 3 — This is the quotient (10 // 3).

C. 3.333... — This is true division.

D. 0 — This would mean 10 is divisible by 3, which it is not.

Correct answer: D. 8

Why: ** is exponentiation. 2 ** 3 = 2 to the power of 3 = 8.

Why the other options are wrong:

A. 6 — This is 2 * 3.

B. 9 — This is 3 ** 2.

C. 23 — This is just the digits 2 and 3 put together.

Lesson 2.2 Practice

Q1. Answer: B. int

Whole numbers are int (integer).

Q2. Answer: B. float

Numbers with a decimal point are float.

Q3. Answer: B. 3

// is floor division; it drops the decimal, giving 3.

Q4. Answer: C. 1

% is the modulo operator; it gives the remainder: 10 divided by 3 leaves remainder 1.

Q5. Answer: B. 8

** is exponentiation; 2 to the power 3 is 8.

Q6. Answer: True

The / operator always returns a float.

Q7. Output: 1

7 divided by 3 is 2 with remainder 1.

Q8. Bug: There is no second operand for the + operator.

Addition needs two numbers.

Q9. Answer:

An int is a whole number, while a float is a number that can have a decimal part.

Q10. Answer:

print(17 % 5)

Lesson 2.3 Check

Correct answer: D. +

Why: The + operator joins (concatenates) two strings together.

Why the other options are wrong:

A. * — * repeats a string, like "ha" * 3 = "hahaha".

B. & — This is not a string operator in Python.

C. . — This is used for method calls, not concatenation.

Correct answer: C. "hahaha"

Why: Multiplying a string by an integer repeats the string.

Why the other options are wrong:

A. "ha ha ha" — Multiplication does not add spaces.

B. "ha3" — It repeats, not concatenates the number.

D. An error — String * int is valid in Python.

Correct answer: C. Both A and B work

Why: Single quotes outside let you use apostrophes inside. Double quotes outside with a backslash escape also works.

Why the other options are wrong:

A. 'It's a lovely day' — This works — single quotes outside.

B. "It\'s a lovely day" — This also works — the backslash escapes the apostrophe.

D. Neither works — Both A and B are valid.

Correct answer: C. str

Why: "Hello" is text in quotation marks, so its type is str (string).

Why the other options are wrong:

A. int — int is for whole numbers.

B. float — float is for decimal numbers.

D. bool — bool is for True/False.

Lesson 2.3 Practice

Q1. Answer: A. With the + operator

The + operator concatenates strings.

Q2. Answer: C. HaHaHa

The * operator repeats a string.

Q3. Answer: A. 'Hello'

Strings must be wrapped in quotes (single or double).

Q4. Answer: D. Python

Concatenation joins them into "Python".

Q5. Answer: B. Both single and double quotes

Python allows both single and double quotes.

Q6. Answer: True

"ab" * 3 gives "ababab".

Q7. Output: LaLaLa

The * operator repeats the string 3 times.

Q8. Bug: The apostrophe inside ends the string early.

Use double quotes around the string instead of single quotes.

Q9. Answer:

String concatenation means joining two or more strings end to end using +.

Q10. Answer:

print("Hello" * 4)

Lesson 2.4 Check

Correct answer: A. True and False

Why: Booleans can only be True or False (with capital letters).

Why the other options are wrong:

B. Yes and No — Python uses True/False, not Yes/No.

C. 1 and 0 — Although booleans act like 1 and 0, their actual values are True and False.

D. On and Off — Not used in Python.

Correct answer: D. True

Why: 5 is equal to 5, so 5 == 5 evaluates to True.

Why the other options are wrong:

A. 5 — == returns a boolean, not the number.

B. False — 5 IS equal to 5.

C. "yes" — Python returns True/False, not strings.

Correct answer: B. True

Why: != means "not equal to". 5 is not equal to 4, so the result is True.

Why the other options are wrong:

A. False — 5 IS not equal to 4, so True.

C. An error — != is a valid operator.

D. 5 — != returns a boolean.

Correct answer: A. x >= 10

Why: >= means "greater than or equal to".

Why the other options are wrong:

B. x > 10 — This is strictly greater than, not >=.

C. x => 10 — The order is wrong. It is >=, not =>.

D. x >> 10 — >> is the bitwise right shift operator.

Lesson 2.4 Practice

Q1. Answer: D. True and False

Python uses True and False (capitalized).

Q2. Answer: D. True

5 is greater than 3, so the result is True.

Q3. Answer: A. True

4 equals 4, so the comparison is True.

Q4. Answer: D. False

10 is equal to 10, so "not equal" is False.

Q5. Answer: A. bool

True and False are of type bool.

Q6. Answer: False

Python uses capitalized True and False.

Q7. Output: False

7 is not greater than 9, so the result is False.

Q8. Bug: It uses = (assignment) instead of == (comparison).

Use == to check equality.

Q9. Answer:

A Boolean value is a truth value that is either True or False.

Q10. Answer:

print(12 > 5)

Lesson 2.5 Check

Correct answer: B. The integer 42

Why: int() converts a string to an integer. int("42") = 42.

Why the other options are wrong:

A. The string "42" — int() converts to an integer, not a string.

C. The float 42.0 — int() returns an int, not a float.

D. An error — "42" can be converted to an integer.

Correct answer: B. 3

Why: int() on a float chops off the decimal part. int(3.99) = 3 (it does NOT round).

Why the other options are wrong:

A. 4 — int() truncates, it does not round. 3.99 becomes 3, not 4.

C. 3.99 — int() returns an integer, not a float.

D. An error — Converting a float to int is valid.

Correct answer: C. The string "42"

Why: str() converts a value to a string. str(42) = "42".

Why the other options are wrong:

A. The integer 42 — str() returns a string, not an int.

B. The float 42.0 — str() returns a string.

D. An error — Converting an int to a string is valid.

Correct answer: B. It raises a ValueError

Why: "hello" cannot be interpreted as a number, so int() raises a ValueError.

Why the other options are wrong:

A. It returns 0 — Python does not guess 0 for non-numeric strings.

C. It returns None — It raises an error, not None.

D. It returns the string "hello" — int() returns an int, not the original string.

Lesson 2.5 Practice

Q1. Answer: C. 5

int() converts the string to the integer 5.

Q2. Answer: D. "7"

str() converts the number to the string "7".

Q3. Answer: B. 3.5

float() converts the string to the float 3.5.

Q4. Answer: C. False

0 is falsy, so bool(0) is False.

Q5. Answer: D. 3

int() of a float truncates toward zero, giving 3.

Q6. Answer: True

"5" + 5 raises a TypeError; convert one side first.

Q7. Output: 10

int("8") becomes 8, then 8 + 2 = 10.

Q8. Bug: You cannot concatenate a string with an int.

Convert with str(): "Age: " + str(10).

Q9. Answer:

Type conversion changes a value from one type (like str) to another (like int) using functions like int(), float(), or str().

Q10. Answer:

print(int("15") + 5)

Chapter 2 Challenges - Solutions

Challenge 1

Question: What is the value of x after this code runs? x = 5 x = x + 3 x = x * 2 x = x - 4 Explain each step.

Solution:

Work through each line, updating x each time:

x = 5 → x is 5 x = x + 3 → x = 5 + 3 = 8 x = x * 2 → x = 8 * 2 = 16 x = x - 4 → x = 16 - 4 = 12

The final value of x is 12.

The key insight is that each line uses the CURRENT value of x to compute the new value, then replaces x with that new value. This is a very common pattern in programming — updating a variable based on its own previous value.

Challenge 2

Question: Ask the user for their birth year and print how old they will be in 2030.

Solution:

You need to: (1) use input() to read the birth year as a string, (2) convert it to an integer with int(), (3) subtract from 2030 to get the age, (4) print the result.

birth_year = int(input("What year were you born? ")) age_in_2030 = 2030 - birth_year print(f"In 2030, you will be {age_in_2030} years old.")

The int() conversion is essential. Without it, Python would try to subtract a string from an integer, which raises a TypeError.

Challenge 3

Question: What is the type of 10 / 2? Is it int or float? Explain why.

Solution:

The result is a float: 5.0, not an int.

In Python 3, the / operator ALWAYS returns a float, even when the result is a whole number. So 10 / 2 = 5.0 (float), not 5 (int). This is different from some other languages and from Python 2, where / would return an int if both operands were ints.

If you specifically need an integer result, use // (floor division): 10 // 2 = 5 (int). The / operator always gives a float so that division never loses precision. Even though 10 / 2 is exactly 5, Python returns 5.0 to keep the behaviour consistent: division always returns float, period.

Challenge 4

Question: age holds the string "14". Add 1 and print "Next year you will be 15" using an f-string.

Solution:

You need to: (1) convert the string "14" to an integer with int(), (2) add 1, (3) use an f-string to print the result.

age = "14" # a string next_year = int(age) + 1 # convert to int, then add 1 print(f"Next year you will be {next_year}")

This prints: Next year you will be 15

The int() conversion is the key step. Without it, age + 1 would try to add a string and an integer, which raises a TypeError. The f-string then inserts the integer value of next_year into the text automatically.

Challenge 5

Question: What does print(type(type(42))) print?

Solution:

It prints: <class 'type'>

Here is the reasoning, step by step: 1. type(42) returns <class 'int'> — the type of 42 is int. 2. <class 'int'> is itself a value — it is a "type object". 3. type(type(42)) asks: "what is the type of the int type itself?" 4. The type of a type is called "type" in Python. 5. So type(type(42)) = <class 'type'>.

This is a deeper idea: in Python, everything is an object, including types themselves. The type of int is "type", the type of str is "type", and the type of "type" is also "type". It is types all the way down. You do not need to understand this fully yet — just know that type() can be applied to anything, even to its own result.

Chapter 3 — Operators and Expressions

Lesson 3.1 Check

Correct answer: C. 1

Why: 10 % 3 = 1 (the remainder when 10 is divided by 3).

Why the other options are wrong:

- **A.** 3 — This is the quotient (10 // 3).
- **B.** 3.33 — This is true division (10 / 3).
- **D.** 0 — 10 is not divisible by 3.

Correct answer: C. 16

Why: 2 ** 4 = 2 to the power of 4 = 16.

Why the other options are wrong:

A. 8 — This is 2 ** 3.

B. 6 — This is 2 * 3.

D. 24 — This is 2 * 3 * 4.

Correct answer: C. //

Why: // is floor division — it gives the integer part, dropping the remainder.

Why the other options are wrong:

A. / — / gives a float result.

B. % — % gives the remainder.

D. ** — ** is exponentiation.

Correct answer: B. -4

Why: Floor division rounds DOWN (towards negative infinity). -7 / 2 = -3.5, and floor of -3.5 is -4.

Why the other options are wrong:

A. -3 — This would be truncation (towards zero), but Python floor division goes down.

C. -3.5 — // returns an integer, not a float.

D. -2 — Incorrect.

Lesson 3.1 Practice

Q1. Answer: A. Regular division (returns float)

/ always returns a float result.

Q2. Answer: D. Floor (whole-number) division

// divides and drops the decimal part.

Q3. Answer: A. The remainder

% returns the remainder of a division.

Q4. Answer: A. Exponentiation (power)

** raises a number to a power.

Q5. Answer: A. 3.0

/ always returns a float, so 3.0.

Q6. Answer: True

For example, 7 % 3 is 1.

Q7. Output: 16

2 to the power 4 is 16.

Q8. Bug: You cannot divide by zero.

Division by zero raises a ZeroDivisionError.

Q9. Answer:

/ returns a float (regular division), while // returns the whole-number part only (floor division).

Q10. Answer:

print(5 ** 2)

Lesson 3.2 Check

Correct answer: D. 14

Why: Multiplication first: 3 * 4 = 12, then 2 + 12 = 14.

Why the other options are wrong:

A. 20 — This would be (2 + 3) * 4.

B. 24 — This would be 2 * 3 * 4.

C. 9 — This is 2 + 3 + 4.

**Correct answer: A. **

Why: Exponentiation (**) has the highest precedence of these four.

Why the other options are wrong:

B. + — Addition has low precedence.

C. * — Multiplication is below **.

D. // — Floor division is at the same level as *.

Correct answer: B. 20

Why: Parentheses override: (2 + 3) = 5 first, then 5 * 4 = 20.

Why the other options are wrong:

A. 14 — This is without parentheses: 2 + 3 * 4.

C. 24 — This is 2 * 3 * 4.

D. 9 — Unrelated.

Correct answer: D. It is the same as score = score + 10

Why: += is augmented assignment. score += 10 is equivalent to score = score + 10.

Why the other options are wrong:

A. Creates a new variable — It updates the existing variable.

B. Adds without changing — It DOES change score.

C. Error — += is valid Python.

Lesson 3.2 Practice

Q1. Answer: B. 3 * 4

* has higher precedence than +, so 3 * 4 is computed first.

Q2. Answer: A. 14

3 * 4 = 12, then 2 + 12 = 14.

Q3. Answer: D. Use parentheses: (2 + 3) * 4

Parentheses override the default precedence.

Q4. Answer: C. Adds then assigns

x += 5 is shorthand for x = x + 5.

Q5. Answer: D. 20

Parentheses first: 2 + 3 = 5, then 5 * 4 = 20.

Q6. Answer: True

Parentheses have the highest precedence.

Q7. Output: 5

Subtraction is left to right: (10 - 2) - 3 = 5.

Q8. Bug: Multiplication happens before addition, so 3 * 4 = 12 is computed first, giving 2 + 12 = 14.

Use parentheses: print((2 + 3) * 4) to get 20.

Q9. Answer:

Operator precedence is the set of rules that decides which operations happen first in an expression.

Q10. Answer:

print((4 + 8 + 12) / 3)

Lesson 3.3 Check

Correct answer: C. True

Why: 5 is equal to 5, so 5 == 5 is True.

Why the other options are wrong:

A. 5 — == returns a boolean, not the number.

B. False — 5 IS equal to 5.

D. "equal" — Python returns True/False.

Correct answer: B. !=

Why: != means "not equal to".

Why the other options are wrong:

A. =! — The order is wrong. It is !=, not =!.

C. ==! — This is not a valid operator.

D. <> — Some languages use this, but Python does not.

Correct answer: C. True

Why: Strings are compared alphabetically. "apple" comes before "banana", so "apple" < "banana" is True.

Why the other options are wrong:

A. False — "apple" DOES come before "banana".

B. An error — String comparison is valid.

D. None — Comparison returns a boolean.

Correct answer: B. True

Why: 5 <= 5 means "5 is less than or equal to 5". Since 5 equals 5, this is True.

Why the other options are wrong:

A. False — 5 IS equal to 5.

C. An error — <= is a valid operator.

D. 5 — <= returns a boolean.

Lesson 3.3 Practice

Q1. Answer: A. ==

== checks equality; = assigns.

Q2. Answer: C. Not equal to

!= means "not equal to".

Q3. Answer: C. True

5 is equal to 5, so <= is True.

Q4. Answer: D. False

8 is not greater than or equal to 10.

Q5. Answer: D. True

3 is less than 7, so True.

Q6. Answer: True

Comparison operators always return a Boolean.

Q7. Output: False

10 is equal to 10, so "not equal" is False.

Q8. Bug: It uses = (assignment) instead of == (comparison).

Use == inside conditions.

Q9. Answer:

= assigns a value to a variable, while == checks whether two values are equal.

Q10. Answer:

print(9 > 4)

Lesson 3.4 Check

Correct answer: C. False

Why: and requires BOTH sides to be True. Since False is one side, the result is False.

Why the other options are wrong:

A. True — Both sides must be True for "and".

B. None — and returns a boolean.

D. An error — This is valid.

Correct answer: A. True

Why: or requires at least ONE side to be True. True is one side, so the result is True.

Why the other options are wrong:

B. False — At least one side is True.

C. None — or returns a boolean.

D. An error — This is valid.

Correct answer: B. False

Why: not flips the boolean. not True = False.

Why the other options are wrong:

A. True — not flips it to False.

C. None — not returns a boolean.

D. An error — not is a valid operator.

Correct answer: A. False

Why: (5 > 3) is True, (10 > 20) is False. True and False = False.

Why the other options are wrong:

B. True — One side is False, so "and" gives False.

C. An error — This is valid.

D. None — and returns a boolean.

Lesson 3.4 Practice

Q1. Answer: B. and

and, or, and not are the logical operators.

Q2. Answer: A. False

and needs both sides True; one is False, so the result is False.

Q3. Answer: A. True

or needs only one side True, so the result is True.

Q4. Answer: A. False

not flips the value, so not True is False.

Q5. Answer: C. True

Both comparisons are True, so the and result is True.

Q6. Answer: True

not flips False to True.

Q7. Output: True

or returns True if at least one side is True.

Q8. Bug: The and operator needs an expression on its right side.

Add another comparison after and.

Q9. Answer:

The and operator returns True only when both sides are True.

Q10. Answer:

print(10 > 5 and 3 < 7)

Chapter 3 Challenges - Solutions

Challenge 1

Question: What is the value of 2 + 3 * 4 - 1?

Solution:

The answer is 13.

Python evaluates this in the following order: 1. Multiplication first: 3 * 4 = 12 2. Now the expression is: 2 + 12 - 1 3. Addition and subtraction have the same precedence, so they are evaluated left to right: 2 + 12 = 14 4. Then: 14 - 1 = 13

So the final answer is 13. The key rule is that multiplication and division always happen before addition and subtraction, and when operators have the same precedence (like + and -), they are evaluated from left to right.

Challenge 2

Question: Write an expression that is True if n is divisible by both 3 and 5.

Solution:

A number is divisible by another if the remainder is 0 (using %). To check divisibility by BOTH 3 AND 5, combine the two checks with the "and" operator:

n % 3 == 0 and n % 5 == 0

For example: - n = 15: 15 % 3 == 0 (True) and 15 % 5 == 0 (True) → True ✓ - n = 9: 9 % 3 == 0 (True) and 9 % 5 == 0 (False) → False - n = 10: 10 % 3 == 0 (False) and 10 % 5 == 0 (True) → False - n = 30: 30 % 3 == 0 (True) and 30 % 5 == 0 (True) → True ✓

Numbers divisible by both 3 and 5 are also divisible by 15 (since 3 × 5 = 15), so you could also write: n % 15 == 0. Both approaches work, but the first one shows the logic more clearly.

Challenge 3

Question: What does 3 > 2 > 1 evaluate to?

Solution:

It evaluates to True.

Python supports chained comparisons. The expression 3 > 2 > 1 is equivalent to (3 > 2) and (2 > 1). Python breaks it into two comparisons and joins them with "and": - 3 > 2 is True - 2 > 1 is True - True and True = True

This is a feature unique to Python — most other languages do not allow chained comparisons and would interpret 3 > 2 > 1 differently (e.g., as (3 > 2) > 1, which is True > 1, which is 1 > 1 = False). Python's chained comparison is more natural and matches how you would read it in mathematics.

Challenge 4

Question: What does print(2 ** 3 ** 2) output?

Solution:

It outputs 512.

The ** operator is right-associative, which means when there are two ** operators, the RIGHT one is evaluated first. So: 2 ** 3 ** 2 = 2 ** (3 ** 2) = 2 ** 9 = 512

Step by step: 1. The right side is done first: 3 ** 2 = 9 2. Then the left side: 2 ** 9 = 512

If ** were left-associative (like + and -), it would be (2 ** 3) ** 2 = 8 ** 2 = 64. But Python does the right side first, giving 512. This matches the mathematical convention for exponentiation, where 2^3^2 means 2^(3^2) = 2^9 = 512.

Challenge 5

Question: Compute the final price: 10% discount if total > 100, otherwise just total. Use a single expression (the ternary operator).

Solution:

Python has an inline conditional called the "ternary operator". The syntax is:

value_if_true if condition else value_if_false

So for the discount problem:

final_price = total * 0.9 if total > 100 else total

How this works: - If total > 100 is True, the expression evaluates to total * 0.9 (10% off). - If total > 100 is False, the expression evaluates to total (no discount).

This is the standard, readable way to write a conditional expression in Python. It is much clearer than tricks like using True/False as 1/0 in arithmetic, and it is what professional Python programmers use.

You can also use it inside other expressions. For example: print(f"Price: ${total * 0.9 if total > 100 else total:.2f}")

The ternary operator is the Pythonic way to do "if/else in one line." It keeps code compact without sacrificing readability.

Chapter 4 — Input and Output

Lesson 4.1 Check

Correct answer: A. str

Why: input() always returns a string, even if the user types a number.

Why the other options are wrong:

- **B.** int — input() never returns an int.
- **C.** float — input() never returns a float.
- **D.** It depends — It always returns str, regardless of input.

Correct answer: A. int(input())

Why: Wrap input() inside int() to convert the string to an integer.

Why the other options are wrong:

- **B.** int_input() — This function does not exist.
- **C.** input().int() — Strings do not have an int() method.
- **D.** input(int) — This passes int as a prompt, which is wrong.

Correct answer: D. The prompt then waits for input

Why: input() displays the prompt, then waits for the user to type and press Enter.

Why the other options are wrong:

A. Nothing — It DOES display the prompt.

B. An error — This is valid.

C. The user's name — It displays the PROMPT, not the user's input.

Correct answer: C. str

Why: Even though 42 looks like a number, input() returns it as the string "42".

Why the other options are wrong:

A. int — input() always returns str.

B. float — input() always returns str.

D. bool — input() always returns str.

Lesson 4.1 Practice

Q1. Answer: C. str

input() always returns a string, even if you type a number.

Q2. Answer: D. Pass it to input(): input("Name: ")

The string inside input() becomes the prompt.

Q3. Answer: C. Stops and waits for the user to type a line

input() reads a line of typed text from the user.

Q4. Answer: C. str

The result is the string "10", not the int 10.

Q5. Answer: D. int(input())

Wrap input() with int() to get an integer.

Q6. Answer: True

You must convert with int() or float() if you need a number.

Q7. Output: Hi Bob

input() returns "Bob", and print() joins with a space.

Q8. Bug: age is a string, so you cannot add 1 to it.

Convert with int(): age = int(input()).

Q9. Answer:

input() returns a string, so to do math with a typed number you must convert it to int first.

Q10. Answer:

```
age = int(input("How old are you? "))
print(age + 1)
```

Lesson 4.2 Check

Correct answer: B. A space

Why: The default separator is a single space.

Why the other options are wrong:

A. A comma — The default is a space, not a comma.

C. A newline — Newline is the default for end, not sep.

D. No separator — There IS a separator: a space.

Correct answer: B. Use the end argument

Why: The end argument controls what is printed after the last argument.

Why the other options are wrong:

A. sep argument — sep controls what goes BETWEEN arguments.
C. stop argument — There is no "stop" argument.
D. Cannot change — You can change it with end.

Correct answer: A. a-b-c

Why: sep="-" puts a hyphen between each argument.

Why the other options are wrong:

B. a b c — This is the default (sep=" ").
C. abc — This would require sep="".
D. a, b, c — This would require sep=", ".

Correct answer: B. It does not add a newline at the end

Why: end="" means nothing is added after "Hello", so the next print continues on the same line.

Why the other options are wrong:

A. Prints nothing — It DOES print "Hello".
C. Extra space — end="" adds nothing, not a space.
D. Error — end="" is valid.

Lesson 4.2 Practice

Q1. Answer: D. Sets the separator between arguments

sep sets the string placed between arguments.

Q2. Answer: B. Sets what is printed at the end (default newline)

end sets the final character; default is a newline.

Q3. Answer: A. Space

By default, arguments are separated by a space.

Q4. Answer: B. a-b

sep="-" places a dash between the arguments.

Q5. Answer: A. Prints Hi with no newline after

end="" replaces the newline with an empty string.

Q6. Answer: True

end lets you replace the default newline.

Q7. Output: a-b-c

sep="-" places a dash between each argument.

Q8. Bug: There is no comma between the string and the sep= argument.

Separate arguments with commas: print("Hi", sep="-").

Q9. Answer:

The sep parameter sets the string that print() places between its arguments.

Q10. Answer:

print(1, 2, 3, sep=",")

Lesson 4.3 Check

Correct answer: B. f"Hello {name}"

Why: An f-string is prefixed with f, and expressions go in curly braces.

Why the other options are wrong:

- **A.** "Hello {name}"f — The f must be at the START.
- **C.** "f-Hello {name}" — This is just a regular string.
- **D.** format(...) — This is the old method, not an f-string.

Correct answer: D. It prints: My name is Alex

Why: The f-string inserts the value of name into the text.

Why the other options are wrong:

- **A.** My name is {name} — Only without the f prefix.
- **B.** Error — This is valid.
- **C.** My name is + Alex — f-strings do not add +.

Correct answer: D. 3.14

Why: .2f means "format as a float with 2 decimal places".

Why the other options are wrong:

- **A.** 3.14159 — This is without formatting.
- **B.** 3.1 — This would be .1f.
- **C.** 3 — This would be .0f.

Correct answer: A. Yes, any valid Python expression

Why: f-strings can contain arithmetic, function calls, method calls, and more inside the braces.

Why the other options are wrong:

- **B.** Only variable names — f-strings support full expressions.
- **C.** Only addition — Any expression works.
- **D.** Only string operations — Any expression works.

Lesson 4.3 Practice

Q1. Answer: A. f"Hello {name}"

An f-string starts with f before the quote.

Q2. Answer: D. Prints Hi Sam

The {name} part is replaced with the value of name.

Q3. Answer: A. "7"

Expressions inside {} are evaluated, so it is the string "7".

Q4. Answer: B. Formats to 2 decimal places: "3.50"

:.2f formats a float with 2 decimal places.

Q5. Answer: A. f"Hello {name}"

Variable names go inside curly braces {}.

Q6. Answer: True

f"...{expr}..." evaluates the expression and inserts it.

Q7. Output: Hi Ana!

The {name} is replaced with the value of name.

Q8. Bug: There is no f before the string, so {name} is not replaced.

Use f"Hi {name}".

Q9. Answer:

An f-string is a string prefixed with f that lets you insert variables and expressions inside {} directly.

Q10. Answer:

```
age = 12
print(f"My age is {age}")
```

Lesson 4.4 Check

Correct answer: A. It reads a string and converts it to an integer in one step

Why: int(input(...)) combines reading input and converting to int.

Why the other options are wrong:

B. Reads an integer directly — input() always returns str.

C. Prints an integer — It reads, not prints.

D. Error — This is a valid and common pattern.

Correct answer: A. It raises a ValueError

Why: "hello" cannot be converted to an integer.

Why the other options are wrong:

B. Returns 0 — Python does not guess 0.

C. Returns None — It raises an error.

D. Returns "hello" — int() returns an int, not a string.

Correct answer: B. The sum, inserted into the string

Why: The expression 2026 + years_to_100 is evaluated and the result is inserted.

Why the other options are wrong:

A. The literal string — f-strings evaluate expressions.

C. Error — This is valid.

D. Nothing — It produces a value.

Correct answer: D. Because input() returns a string and you cannot subtract a string from an integer

Why: Without int(), you would be trying to do "14" - 2026, which raises a TypeError.

Why the other options are wrong:

A. Python requires all variables to be integers — Python is dynamically typed.

B. Makes it faster — It is about correctness, not speed.

C. Just style — It is necessary for the program to work.

Lesson 4.4 Practice

Q1. Answer: B. input()

input() reads the user input.

Q2. Answer: A. Because input() returns a string

input() returns a string, so int() is needed for math.

Q3. Answer: B. 10

int() turns the input into 5, then 5 * 2 = 10.

Q4. Answer: D. name = input("Name: ")

input() with a prompt asks the user.

Q5. Answer: D. Print, then input, then respond

A bot greets, reads input, then responds.

Q6. Answer: True

input() reads; print() responds.

Q7. Output: 10

n is 3, so 3 + 7 = 10.

Q8. Bug: age is a string, so age + 1 fails.

Use int(input()) to get a number.

Q9. Answer:

input() reads what the user types and print() sends the bot reply back to the screen.

Q10. Answer:

```
name = input("What is your name? ")
print("Hello, " + name)
```

Lesson 4.5 Check

Correct answer: C. import

Why: The import keyword loads a module so you can use its functions.

Why the other options are wrong:

A. load — Not a Python keyword.

B. include — Not a Python keyword.

D. module — Not a keyword; it is a noun.

Correct answer: C. math.sqrt(25)

Why: After importing math, you access its functions with module.function().

Why the other options are wrong:

A. sqrt(25) — You need the module prefix.

B. import sqrt(25) — Not valid syntax.

D. math:sqrt(25) — Use a dot, not a colon.

Correct answer: C. 5

Why: len() counts the number of items. "Hello" has 5 characters.

Why the other options are wrong:

A. "Hello" — len returns a number, not the string.

B. 6 — There are 5 letters, not 6.

D. H — That is the first character, not the length.

Correct answer: B. A random integer between 1 and 6 (inclusive)

Why: randint(a, b) returns a random integer from a to b, including both endpoints.

Why the other options are wrong:

A. A random float between 0 and 1 — That is random.random().

C. The number 6 — It is random, not always 6.

D. A random string — randint returns an integer.

Lesson 4.5 Practice

Q1. Answer: B. import

import loads a module into your program.

Q2. Answer: B. math.sqrt(25)

Use module.function() syntax: math.sqrt(25).

Q3. Answer: B. 4

len() returns 4 because there are 4 items.

Q4. Answer: B. 4

round(3.7) rounds to 4.

Q5. Answer: C. 60

sum() adds up all items: 10 + 20 + 30 = 60.

Q6. Answer: True

random is a module, so you need import random first.

Q7. Output: 4.0

sqrt(16) returns 4.0 (a float).

Q8. Bug: Nothing is wrong; this code is correct.

It imports random, calls randint, and prints. No bug.

Q9. Answer:

A module is a file containing Python code that you can load into your program using import.

Q10. Answer:

```
import random
roll = random.randint(1, 6)
print(f"You rolled a {roll}")
```

Chapter 4 Challenges - Solutions

Challenge 1

Question: Ask for two numbers, add them, and print "5 + 3 = 8" using an f-string.

Solution:

You need two input() calls (both converted to int), then an f-string that shows the full equation.

a = int(input("Enter first number: ")) b = int(input("Enter second number: ")) total = a + b print(f"{a} + {b} = {total}")

If the user enters 5 and 3, the output is: 5 + 3 = 8

The f-string inserts all three values: a, b, and the sum. The int() conversions are necessary because input() returns strings, and you cannot add two strings together to get their numeric sum (you would get "53" instead of 8).

Challenge 2

Question: What does print("a", "b", "c", sep="", end="!") output?

Solution:

The output is: abc!

Here is why: - sep="" means no separator is placed between the arguments. So instead of "a b c" (with spaces), you get "abc" (no spaces). - end="!" means "!" is printed after the last argument, instead of the default newline.

So the full output is "abc!" with no newline at the end. If another print() call followed, its output would appear right after the "!" on the same line.

This shows how sep and end give you fine control over the output format. The default is sep=" " (space) and end="\n" (newline).

Challenge 3

Question: Ask for name and age, print "Hi Alex! You are 14 years old. In 10 years you will be 24."

Solution:

You need two inputs: name (a string, no conversion needed) and age (needs int conversion for arithmetic). Then use an f-string to format the output.

name = input("What is your name? ") age = int(input("How old are you? ")) future_age = age + 10 print(f"Hi {name}! You are {age} years old. In 10 years you will be {future_age}.")

If the user enters "Alex" and "14", the output is: Hi Alex! You are 14 years old. In 10 years you will be 24.

The key points: name does not need conversion because we only display it (no arithmetic). age needs int() because we add 10 to it. The f-string inserts all the values cleanly.

Challenge 4

Question: If x = input("Enter a number: ") and the user types 5, what does print(x * 3) print?

Solution:

It prints: 555 (not 15).

Here is why: input() ALWAYS returns a string. So x holds the string "5", not the integer 5. When you multiply a string by an integer in Python, the string is REPEATED that many times. So "5" * 3 = "555" (the string "5" repeated three times).

This is very different from 5 * 3 = 15 (integer multiplication). The type of the value determines what the * operator does: - int * int → multiplication → 5 * 3 = 15 - str * int → repetition → "5" * 3 = "555"

To get 15, you would need to convert first: int(x) * 3 = 15. This is a classic beginner trap — the code looks correct but produces unexpected output because of the type difference.

Challenge 5

Question: Write a single print() call that outputs: Hello***World (using sep).

Solution:

You need two string arguments with sep="***":

print("Hello", "World", sep="***")

This outputs: Hello***World

The sep argument tells print() what to put BETWEEN the arguments. By default it is a space, but here we set it to "***", so the three asterisks go between "Hello" and "World".

Note that sep goes BETWEEN arguments, not after each one. So you do not get "Hello***World***". If you wanted that, you would need to add an empty string argument: print("Hello", "World", "", sep="***").

Chapter 5 — Making Decisions — Conditionals

Lesson 5.1 Check

Correct answer: B. :

Why: Every if line must end with a colon.

Why the other options are wrong:

- **A.** ; — Python does not use semicolons.
- **C.** , — Commas separate items, not end statements.
- **D.** -> — Used for type hints, not if.

Correct answer: C. Indentation

Why: Python uses indentation to define blocks.

Why the other options are wrong:

- **A.** Curly braces — Used in C/Java, not Python.
- **B.** Parentheses — Used for grouping, not blocks.
- **D.** The word "end" — Used in Ruby/Lua, not Python.

Correct answer: C. The indented block is skipped

Why: If the condition is False, Python skips the indented block.

Why the other options are wrong:

- **A.** Crashes — False conditions do not crash.
- **B.** Runs anyway — The block only runs if True.
- **D.** Asks user — Python does not ask.

Correct answer: C. Consistent

Why: The standard is 4 spaces, but any consistent amount works. PEP 8 recommends 4.

Why the other options are wrong:

A. 2 — Acceptable but not standard.

B. 4 — Standard, but consistency is the real rule.

D. 8 — Too much.

Lesson 5.1 Practice

Q1. Answer: C. A colon

An if statement ends with a colon.

Q2. Answer: A. By indentation

Indented lines belong to the if block.

Q3. Answer: D. The block is skipped

A False condition means the block does not run.

Q4. Answer: A. if x > 5:

An if needs a condition and a colon.

Q5. Answer: C. 4

PEP 8 recommends 4 spaces.

Q6. Answer: True

Indentation marks the block that belongs to the if.

Q7. Output: big

x is 10, which is greater than 5, so the block runs.

Q8. Bug: The if line is missing a colon.

Add a colon: if x > 5:

Q9. Answer:

An if statement runs a block of code only when its condition is True.

Q10. Answer:

```
if age >= 18:
print("Adult")
```

Lesson 5.2 Check

Correct answer: D. else

Why: else is the keyword for the "otherwise" branch.

Why the other options are wrong:

A. elseif — Python uses elif, not elseif.

B. otherwise — Not a Python keyword.

C. elif — elif is "else if", not just "else".

Correct answer: D. Exactly 1

Why: In an if/else, exactly one of the two blocks runs.

Why the other options are wrong:

A. 0 — One always runs.

B. 2 — Only one runs, not both.

C. Both — They are mutually exclusive.

Correct answer: C. The else block runs

Why: When if is False, the else block runs.

Why the other options are wrong:

A. Neither — The else runs.

B. The if block — The if is False, so it does not run.

D. Both — Only one runs.

Correct answer: D. Yes, the else is optional

Why: An if without else is perfectly valid.

Why the other options are wrong:

A. Must come in pairs — else is optional.

B. Only inside functions — Valid anywhere.

C. Only in Python 3 — Valid in all Python versions.

Lesson 5.2 Practice

Q1. Answer: C. The block that runs when the if is False

else runs when the if condition is False.

Q2. Answer: A. Zero or one

An if can have at most one else.

Q3. Answer: D. A colon

else ends with a colon, no condition.

Q4. Answer: D. Exactly one

Only one of the two blocks runs.

Q5. Answer: A. if x: ... else: ...

Both if and else need colons.

Q6. Answer: True

If the condition is True, the if block runs; otherwise the else block runs.

Q7. Output: small

x is 3, not greater than 5, so the else runs.

Q8. Bug: The else line is missing a colon.

Add a colon after else.

Q9. Answer:

The else block runs only when the if condition is False.

Q10. Answer:

```
if score >= 50:
print("Pass")
else:
print("Fail")
```

Lesson 5.3 Check

Correct answer: D. elif

Why: elif is short for "else if".

Why the other options are wrong:

A. elseif — Not used in Python.

B. else if — Two words do not work.

C. elsf — Not a keyword.

Correct answer: D. At most one

Why: Python runs the first matching branch and skips the rest.

Why the other options are wrong:

A. All — Only the first match runs.

B. Two — At most one.

C. Depends — Always at most one.

Correct answer: B. No, it skips them

Why: Once the if is True, the elifs are not checked (first-match branching).

Why the other options are wrong:

A. Yes, always — First-match branching means they are skipped.

C. Only if no else — Unrelated to else.

D. Only first elif — None are checked.

Correct answer: A. Because if we reach elif, the if was False

Why: First-match branching means each elif can assume all previous conditions were False.

Why the other options are wrong:

B. Python adds it — Python does not add it; it is implied.

C. Does not matter — It does matter for clarity.

D. You DO need it — You do not need it.

Lesson 5.3 Practice

Q1. Answer: B. Else if

elif means "else if".

Q2. Answer: C. Zero or more

You can have many elif branches.

Q3. Answer: C. Only if all earlier conditions were False and the elif condition is True

elif checks only after the previous conditions were False.

Q4. Answer: C. A condition and a colon

elif needs a condition and a colon.

Q5. Answer: D. At most one

Only the first matching branch runs.

Q6. Answer: True

Python stops checking after the first True branch.

Q7. Output: b

x is not > 10, but it is > 3, so "b" prints.

Q8. Bug: The elif line is missing a colon.

Add a colon: elif x > 3:

Q9. Answer:

Python checks each condition top to bottom and runs only the first branch whose condition is True.

Q10. Answer:

```
if temp > 30:
print("hot")
elif temp > 20:
print("warm")
else:
print("cold")
```

Lesson 5.4 Check

Correct answer: A. An if inside another if

Why: Nesting means putting one if inside the block of another.

Why the other options are wrong:

B. Many elif — That is a chain, not nesting.

C. No else — That is just an optional else.

D. Never runs — Nesting is about structure.

Correct answer: C. They make code hard to read

Why: Deep nesting makes code hard to follow.

Why the other options are wrong:

A. Faster — Nesting does not affect speed.

B. Errors — Nesting is valid.

D. Not allowed — Nesting is allowed.

Correct answer: D. Combining conditions with and/or in a single if

Why: Using and/or in a single if is often cleaner than nesting.

Why the other options are wrong:

A. For loop — Loops are for repetition, not for replacing nesting.

B. While loop — Loops are for repetition, not for replacing nesting.

C. No alternative — There are alternatives to nesting.

Correct answer: B. Only when the outer if is True

Why: The inner if only runs if the outer if block is entered.

Why the other options are wrong:

A. Always — Only if the outer is True.

C. Outer is False — Then the inner never runs.

D. Never — It runs when outer is True.

Lesson 5.4 Practice

Q1. Answer: A. An if inside another if

A nested conditional is an if inside another if.

Q2. Answer: B. To check a second condition only when the first is True

They let you check further conditions after the first is met.

Q3. Answer: D. By indentation

Deeper indentation shows nesting.

Q4. Answer: A. The code becomes hard to read

Deep nesting is hard to follow; often you can flatten with and.

Q5. Answer: C. Only when the outer if is True

The inner block only runs if the outer condition is True.

Q6. Answer: True

That is exactly a nested conditional.

Q7. Output: very big

Both conditions are True, so "very big" prints.

Q8. Bug: The inner if is not indented under the outer if.

Indent the inner if to nest it.

Q9. Answer:

A nested conditional is an if statement placed inside the block of another if statement.

Q10. Answer:

```
if age >= 13:
if age <= 100:
print("valid")
```

Lesson 5.5 Check

Correct answer: C. That age is at least 18 AND the person has ID

Why: and requires BOTH conditions to be True.

Why the other options are wrong:

- **A.** OR — and is not or.
- **B.** Exactly 18 — >= means 18 or more.
- **D.** Has ID — Both conditions are checked.

Correct answer: C. It is NOT raining

Why: not flips the boolean. not is_raining = not True = False if raining, or not False = True if not raining.

Why the other options are wrong:

- **A.** Always raining — not means the opposite.
- **B.** Might rain — not gives a definite answer.
- **D.** Heavily — not does not add intensity.

Correct answer: B. True

Why: or requires at least one True. True or anything = True.

Why the other options are wrong:

- **A.** False — At least one is True.
- **C.** None — or returns a boolean.
- **D.** Error — Valid expression.

Correct answer: A. Using and is usually clearer

Why: For independent conditions, and is cleaner than nesting.

Why the other options are wrong:

- **B.** Nesting always clearer — Often the opposite.
- **C.** Equally clear — and is usually clearer.
- **D.** Neither clear — One is usually better.

Lesson 5.5 Practice

Q1. Answer: B. and

and requires both sides True.

Q2. Answer: D. or

or needs at least one True side.

Q3. Answer: A. x is between 5 and 10

It checks that x is greater than 5 and less than 10.

Q4. Answer: D. x is 5 or less

not flips the result, so it checks x is 5 or less.

Q5. Answer: A. True

At least one side is True, so or gives True.

Q6. Answer: True

if x > 5 and x < 10: is valid.

Q7. Output: in range

7 is between 5 and 10, so the block runs.

Q8. Bug: The right side of and has no left operand.

Write x < 10 instead of < 10.

Q9. Answer:

and needs both sides True, while or needs only one side True.

Q10. Answer:

if age >= 13 and age <= 19:
print("teen")

Chapter 5 Challenges - Solutions

Challenge 1

Question: Ask for a number and print "Positive", "Negative", or "Zero".

Solution:

Three branches: > 0, < 0, and the remaining case (== 0).

n = int(input("Enter a number: ")) if n > 0: print("Positive") elif n < 0: print("Negative") else: print("Zero")

The else catches the case where n is exactly 0 (neither > 0 nor < 0). You could also write "elif n == 0:" but the else is cleaner because there are no other possibilities.

Challenge 2

Question: What is wrong with: if age = 18: print("Adult")?

Solution:

The problem is that the code uses a single = (assignment) instead of == (comparison).

A single = means "put the value on the right into the variable on the left". So age = 18 tries to assign 18 to age, which is not a valid condition for an if statement. Python raises a SyntaxError.

The fix is to use ==: if age == 18: print("Adult")

This is one of the most common beginner mistakes. Get into the habit early: use = for assignment, == for comparison.

Challenge 3

Question: Ask for a score (0-100) and print A/B/C/D/F. Why does order matter?

Solution:

You must check from highest to lowest. If you check 60+ first, everyone with 60 or above gets D, and the higher branches never run.

score = int(input("Enter your score (0-100): ")) if score >= 90: grade = "A" elif score >= 80: grade = "B" elif score >= 70: grade = "C" elif score >= 60: grade = "D" else: grade = "F" print(f"Your grade is {grade}.")

The order matters because of first-match branching. When we reach "elif score >= 80", we already know score < 90 (because the if was False). So we do not need to write "score >= 80 and score < 90" — the < 90 is implied. If we reversed the order and checked >= 60 first, a score of 95 would match >= 60 and get a D, never reaching the A branch.

Challenge 4

Question: Print "Yes" only when x > 0 AND x < 10, using nested ifs (no "and").

Solution:

Put one if inside another. The inner if only runs if the outer if is True.

x = int(input("Enter a number: ")) if x > 0: if x < 10: print("Yes")

This is equivalent to "if x > 0 and x < 10: print('Yes')" but uses nesting instead of the "and" operator. The inner if (x < 10) is only checked when the outer if (x > 0) is True. If x <= 0, the inner if is never reached, and "Yes" is not printed.

In practice, using "and" is cleaner: if x > 0 and x < 10: print("Yes"). But knowing how to nest is important for cases where the second condition only makes sense after the first passes.

Challenge 5

Question: Movie theater: $10 adult (18+), $7 child (under 13), $8 teen (13-17). Ask age, print price.

Solution:

Three age ranges. The key is to check in the right order and use elif so only one branch runs.

age = int(input("Enter your age: ")) if age >= 18: price = 10 elif age >= 13: price = 8 else: price = 7 print(f"Your ticket costs ${price}.")

The logic: if age is 18 or above, it is an adult ($10). Otherwise, if age is 13 or above (but we know it is under 18 because the if was False), it is a teen ($8). Otherwise (under 13), it is a child ($7). The else catches everyone under 13.

This uses first-match branching: when we check "elif age >= 13", we already know age < 18, so we do not need to write "age >= 13 and age < 18".

Chapter 6 — Loops

Lesson 6.1 Check

Correct answer: C. 5

Why: range(5) produces 5 numbers: 0,1,2,3,4.

Why the other options are wrong:

- **A.** 4 — This would be range(4).
- **B.** 6 — This would be range(6).
- **D.** 0 — range(5) runs 5 times.

Correct answer: C. 0, 1, 2

Why: range(3) starts at 0 and stops before 3.

Why the other options are wrong:

- **A.** 1, 2, 3 — range starts at 0, not 1.
- **B.** 0, 1, 2, 3 — range stops BEFORE the argument.
- **D.** 1, 2 — range starts at 0.

Correct answer: D. 1, 2, 3

Why: range(1, 4) starts at 1, stops before 4.

Why the other options are wrong:

- **A.** 1, 2, 3, 4 — range stops before the second argument.
- **B.** 0, 1, 2, 3 — The start is 1, not 0.
- **C.** 2, 3, 4 — Wrong start and end.

Correct answer: D. 0, 2, 4, 6, 8

Why: range(0, 10, 2) starts at 0, stops before 10, step 2.

Why the other options are wrong:

A. 0, 2, 4, 6, 8, 10 — 10 is not included.

B. 2, 4, 6, 8, 10 — Starts at 0, not 2.

C. 1, 3, 5, 7, 9 — This would be range(1, 10, 2).

Lesson 6.1 Practice

Q1. Answer: D. 0, 1, 2, 3, 4

range(5) gives 0 through 4 (five numbers starting at 0).

Q2. Answer: D. 2, 3, 4, 5

range(a, b) goes from a up to but not including b.

Q3. Answer: D. 1, 3, 5, 7, 9

The third argument is the step, so it goes 1, 3, 5, 7, 9.

Q4. Answer: A. 3

range(3) gives 0, 1, 2 — three values.

Q5. Answer: C. for

for starts the loop.

Q6. Answer: True

It produces 0, 1, 2, 3, 4.

Q7. Output: 0 1 2

Each value 0, 1, 2 is printed on its own line.

Q8. Bug: The for line is missing a colon.

Add a colon: for i in range(5):

Q9. Answer:

range(a, b) produces numbers starting at a and going up to but not including b.

Q10. Answer:

```
for i in range(1, 6):
print(i)
```

Lesson 6.2 Check

Correct answer: A. while

Why: The while keyword creates a while loop.

Why the other options are wrong:

B. for — for creates a for loop.

C. loop — Not a Python keyword.

D. repeat — Not a Python keyword.

Correct answer: C. When its condition becomes False

Why: A while loop runs as long as its condition is True.

Why the other options are wrong:

A. After 10 iterations — There is no fixed limit.

B. When user presses Enter — Not how it works.

D. Never stops — It stops when the condition is False.

Correct answer: A. Update the variable that the condition depends on

Why: If the condition variable never changes, the loop runs forever.

Why the other options are wrong:

B. Use break — break works but is not the main solution.

C. Use for instead — Sometimes, but not the answer.

D. Always stop — They do not stop automatically.

Correct answer: A. Press Ctrl+C

Why: Ctrl+C sends a KeyboardInterrupt to stop the program.

Why the other options are wrong:

B. Close the window — This works but is not the shortcut.

C. Ctrl+Z — This is undo, not interrupt.

D. Restart — Too extreme.

Lesson 6.2 Practice

Q1. Answer: B. A True condition

The loop runs while the condition is True.

Q2. Answer: A. Update the variable used in the condition

You must change the variable so the condition can become False.

Q3. Answer: B. A loop that never ends

An infinite loop never stops because the condition never becomes False.

Q4. Answer: D. while x < 5:

A while needs a condition and a colon.

Q5. Answer: D. Before each iteration

The condition is checked before each pass.

Q6. Answer: True

If the condition is False initially, the body never runs.

Q7. Output: 0 1 2

n starts at 0 and increases until it reaches 3.

Q8. Bug: n is never updated, so the loop runs forever.

Add n = n + 1 inside the loop.

Q9. Answer:

A while loop stops when its condition becomes False.

Q10. Answer:

```
n = 5
while n > 0:
print(n)
n = n - 1
```

Lesson 6.3 Check

Correct answer: B. Exits the loop immediately

Why: break causes Python to leave the loop right away.

Why the other options are wrong:

A. Skips to next iteration — This is continue.

C. Restarts — break does not restart.

D. Pauses — break exits, not pauses.

Correct answer: A. Skips the rest of this iteration and moves to the next

Why: continue jumps back to the top of the loop.

Why the other options are wrong:

B. Exits the loop — This is break.

C. Restarts program — continue does not restart.

D. Nothing — continue does something.

Correct answer: D. break

Why: break exits the loop entirely.

Why the other options are wrong:

A. continue — continue skips to the next iteration.

B. stop — Not a keyword.

C. exit — exit() is a function, not a loop keyword.

Correct answer: B. No, use them sparingly

Why: break and continue make loops harder to follow. Use them only when they genuinely make the code clearer.

Why the other options are wrong:

A. Yes, always — Overuse makes code hard to read.

C. Only in for loops — They work in both.

D. Not allowed — They are allowed.

Lesson 6.3 Practice

Q1. Answer: B. Stops the loop immediately

break exits the loop right away.

Q2. Answer: B. Skips the rest of this iteration and goes to the next

continue skips to the next iteration.

Q3. Answer: D. In both for and while loops

break works in any loop.

Q4. Answer: B. The loop ends and code after the loop runs

Control jumps to the line after the loop.

Q5. Answer: D. continue

continue skips the rest of the current iteration.

Q6. Answer: True

break ends the loop; continue goes to the next pass.

Q7. Output: 0 1 2

The loop stops when i becomes 3, before printing 3.

Q8. Bug: The if line is missing a colon.

Add a colon: if i == 2:

Q9. Answer:

break exits the loop entirely, while continue skips the rest of the current iteration and moves to the next one.

Q10. Answer:

```
for i in range(10):
if i == 5:
break
print(i)
```

Lesson 6.4 Check

Correct answer: A. A loop inside another loop

Why: Nesting means putting one loop inside another.

Why the other options are wrong:

B. Forever — That is an infinite loop.
C. No body — That would be a syntax error.
D. Calls itself — That is recursion.

Correct answer: B. It depends on the inner loop's range

Why: The inner loop runs its full range for each outer iteration.

Why the other options are wrong:

A. Once — Usually more than once.
C. Never — It does run.
D. Twice — Depends on the range.

Correct answer: C. 12

Why: 3 outer × 4 inner = 12 total iterations of the inner body.

Why the other options are wrong:

A. 7 — This is 3 + 4, not 3 × 4.
B. 3 — This is just the outer count.
D. 4 — This is just the inner count.

Correct answer: A. Prints a blank line

Why: print() with no arguments just prints a newline.

Why the other options are wrong:

B. Nothing — It prints a newline.
C. Error — print() is valid.
D. "None" — It prints nothing, just a newline.

Lesson 6.4 Practice

Q1. Answer: B. A loop inside another loop

A nested loop is a loop inside another loop.

Q2. Answer: B. The outer loop

The outer loop runs once per full pass of the inner loop.

Q3. Answer: C. 12

3 times 4 = 12 inner iterations.

Q4. Answer: A. Printing a table or grid

Nested loops are great for grids and tables.

Q5. Answer: A. With deeper indentation

The inner loop is indented more than the outer.

Q6. Answer: True

For each outer iteration, the inner loop runs from start to end.

Q7. Output: 0 0 0 1 1 0 1 1

For each i, j goes through 0 and 1.

Q8. Bug: The inner for is not indented under the outer for.

Indent the inner loop to nest it.

Q9. Answer:

A nested loop is a loop placed inside the body of another loop.

Q10. Answer:

```
for i in range(3):
for j in range(3):
print("*", end="")
print()
```

Lesson 6.5 Check

Correct answer: D. Before the loop starts

Why: The total variable must be created before the loop so it persists across iterations.

Why the other options are wrong:

A. Inside the loop — This resets it every iteration.

B. After the loop — Too late.

C. Does not matter — It matters a lot.

Correct answer: B. 0

Why: Starting at 0 means adding the first number gives that number.

Why the other options are wrong:

A. 1 — Starting at 1 would add an extra 1.

C. 100 — Wrong starting value.

D. Depends — For sums, always start at 0.

Correct answer: B. It is the same as total = total + n

Why: += is augmented assignment.

Why the other options are wrong:

A. Creates new variable — It updates the existing one.

C. Compares — It is assignment, not comparison.

D. Error — += is valid.

Correct answer: A. total is reset to 0 on every iteration

Why: Putting total = 0 inside the loop means it is reset each time, so the sum never accumulates.

Why the other options are wrong:

B. Works fine — It does NOT work.

C. Error — No error, just wrong result.

D. Infinite loop — It does not affect the loop.

Lesson 6.5 Practice

Q1. Answer: D. A variable that builds up a total

An accumulator stores a running total.

Q2. Answer: D. 0

Sums usually start at 0.

Q3. Answer: A. 0

Counts usually start at 0.

Q4. Answer: A. Adds 5 to total

It is shorthand for total = total + 5.

Q5. Answer: C. total = 0; for i in range(1, 6): total += i

Start at 0 and add each i from 1 to 5.

Q6. Answer: True

That is the counting pattern.

Q7. Output: 6

1 + 2 + 3 = 6.

Q8. Bug: total is reset to 0 inside the loop, so it never builds up.

Move total = 0 above the loop.

Q9. Answer:

You start a variable at 0 (or another base) and update it each loop iteration to build a total or count.

Q10. Answer:

```
total = 0
for i in range(1, 11):
total += i
print(total)
```

Chapter 6 Challenges - Solutions

Challenge 1

Question: Print the first 10 square numbers: 1, 4, 9, 16, 25, ...

Solution:

Loop from 1 to 10 and print i ** 2 for each.

for i in range(1, 11): print(i ** 2)

Output: 1 4 9 16 25 36 49 64 81 100

range(1, 11) gives the numbers 1 through 10. For each number i, i ** 2 gives its square (i to the power of 2).

Challenge 2

Question: Predict the output of: for i in range(1, 4): for j in range(1, 4): print(i, j)

Solution:

The outer loop runs 3 times (i = 1, 2, 3). For each value of i, the inner loop runs 3 times (j = 1, 2, 3). So the inner body runs 3 × 3 = 9 times total.

Output: 1 1 1 2 1 3 2 1 2 2 2 3 3 1 3 2 3 3

Notice the pattern: the first number (i) stays the same while the second number (j) goes through 1, 2, 3. Then i increases and j starts over. This is how nested loops work — the inner loop completes its full range for each iteration of the outer loop.

Challenge 3

Question: Ask for a number and countdown from that number to 0 using while.

Solution:

Use a while loop that runs while n >= 0. Print n, then subtract 1.

n = int(input("Enter a number: ")) while n >= 0: print(n) n -= 1

If the user enters 5, the output is: 5 4 3 2 1 0

The loop runs while n is 0 or more. Each iteration prints the current value, then subtracts 1. When n becomes -1, the condition n >= 0 is False, and the loop stops. The key is the n -= 1 line — without it, the loop would run forever printing the same number.

Challenge 4

Question: Add up all even numbers from 1 to 100 and print the total.

Solution:

Loop from 1 to 100, check if each number is even (i % 2 == 0), and add it to a total.

total = 0 for i in range(1, 101): if i % 2 == 0: total += i print(f"The sum of even numbers from 1 to 100 is {total}")

The output is: The sum of even numbers from 1 to 100 is 2550

The total variable is created BEFORE the loop (starting at 0). Inside the loop, we check if i is even using i % 2 == 0. If it is, we add it to total with total += i. After the loop finishes, total holds the sum of all even numbers from 2 to 100, which is 2550.

A clever shortcut: you could also use range(2, 101, 2) to only loop over even numbers, skipping the if check entirely.

Challenge 5

Question: Password checker: ask for password, allow 3 tries. Use while loop.

Solution:

Use a counter that tracks the number of tries. The while condition checks if the user has tries remaining.

password = "python123" tries = 0 while tries < 3: guess = input("Enter password: ") if guess == password: print("Access granted!") break print("Wrong password.") tries += 1 if tries == 3: print("Too many wrong attempts. Access denied.")

How it works: - tries starts at 0. - The while loop runs as long as tries < 3 (i.e., 3 attempts). - Each iteration asks for the password. - If correct, it prints "Access granted!" and breaks out of the loop. - If wrong, it increments tries. - After the loop, if tries == 3, all attempts were used and access is denied.

This combines a while loop, a counter, a break, and a condition after the loop — a realistic example of how loops are used in real programs.

Chapter 7 — Functions

Lesson 7.1 Check

Correct answer: D. def

Why: def starts a function definition.

Why the other options are wrong:

- **A.** function — Not used in Python.
- **B.** func — Not a keyword.
- **C.** define — Python uses def.

Correct answer: C. Call it by name with parentheses

Why: You call a function by writing its name followed by ().

Why the other options are wrong:

- **A.** Just define it — Defining does not run it.
- **B.** Press F5 — F5 runs the whole program.
- **D.** Runs automatically — Functions never run automatically.

Correct answer: C. Missing colon

Why: def greet(): needs a colon before the body.

Why the other options are wrong:

- **A.** Quotation marks — The quotes are fine.
- **B.** Capitalize print — print is lowercase.
- **D.** Nothing wrong — The colon is missing.

Correct answer: A. The function body never runs

Why: Defining a function just tells Python what it does. The body runs only when called.

Why the other options are wrong:

B. Runs once — It never runs if not called.

C. Warning — No warning.

D. Crash — No crash.

Lesson 7.1 Practice

Q1. Answer: B. def

def starts a function definition.

Q2. Answer: A. Parentheses ()

The parameter list goes in parentheses.

Q3. Answer: C. A colon

A def line ends with a colon.

Q4. Answer: C. Call it by name with parentheses

Call it like greet().

Q5. Answer: B. The indented lines that run when called

The body is the indented block under the def.

Q6. Answer: True

def only creates the function; calling it runs the body.

Q7. Output: Hi

Calling greet() runs its body, which prints Hi.

Q8. Bug: The def line is missing a colon.

Add a colon: def greet():

Q9. Answer:

A function is a named block of code you can run by calling its name.

Q10. Answer:

```
def hello():
print("Hello there!")
hello()
```

Lesson 7.2 Check

Correct answer: D. A parameter

Why: Parameters are in the definition; arguments are in the call.

Why the other options are wrong:

A. Argument — Arguments are in the call.

B. Variable — It is a parameter, which is a kind of variable.

C. Keyword — name is not a keyword.

Correct answer: C. An argument

Why: Arguments are the actual values you pass when calling.

Why the other options are wrong:

A. Parameter — Parameters are in the definition.

B. Keyword — It is a value.

D. Function — It is an argument.

Correct answer: C. Any number

Why: Functions can have 0, 1, 2, or any number of parameters.

Why the other options are wrong:

A. Exactly 1 — Can be 0 or many.

B. At most 2 — No limit.

D. Exactly 3 — No limit.

Correct answer: C. Two arguments

Why: Two parameters means two arguments required.

Why the other options are wrong:

A. One — Two parameters.

B. Three — Two parameters.

D. None — Two required.

Lesson 7.2 Practice

Q1. Answer: D. A name in the def line that receives a value

A parameter is the variable name in the function definition.

Q2. Answer: C. The value passed to a function when calling it

An argument is the actual value you pass.

Q3. Answer: C. A parameter

x is a parameter in the definition.

Q4. Answer: A. An argument

5 is the argument passed in.

Q5. Answer: A. It raises a TypeError

Python requires the right number of arguments.

Q6. Answer: True

They are often mixed up but are different.

Q7. Output: 5

n becomes 4, so 4 + 1 = 5.

Q8. Bug: add was called with no argument but expects one.

Pass a value: add(4).

Q9. Answer:

A parameter is the name in the function definition; an argument is the actual value you pass when calling.

Q10. Answer:

```
def square(n):
print(n * n)
square(6)
```

Lesson 7.3 Check

Correct answer: A. Sends a value back to the caller

Why: return hands a value back to the code that called the function.

Why the other options are wrong:

B. Prints — print displays; return sends back.

C. Restarts — return exits.

D. Ends program — return ends the function, not the program.

Correct answer: B. The function exits immediately

Why: return immediately exits the function. No more lines run.

Why the other options are wrong:

A. Continues — It exits immediately.

C. Pauses — It exits.

D. Error — No error.

Correct answer: D. None

Why: A function with no return returns None by default.

Why the other options are wrong:

A. 0 — Not 0.

B. Error — No error.

C. Last variable — Not automatic.

Correct answer: C. print displays text; return sends a value back

Why: print is for the user; return is for the program.

Why the other options are wrong:

A. Same — Different purposes.

B. Reversed — Opposite.

D. Faster — Speed is not the difference.

Lesson 7.3 Practice

Q1. Answer: A. Sends a value back to the caller and stops the function

return sends a value back and ends the function.

Q2. Answer: A. print shows on screen; return sends a value back to the code

print displays; return gives a value to the caller.

Q3. Answer: A. None

Functions with no return give back None.

Q4. Answer: D. Store it in a variable, use it in math, pass to another function

Returned values can be used anywhere a value is needed.

Q5. Answer: C. Only one

A basic return sends one value (tuples can package more).

Q6. Answer: True

return exits the function immediately.

Q7. Output: 10

double(5) returns 10, which print displays.

Q8. Bug: double prints instead of returning, so result is None and None + 1 fails.

Use return x * 2 inside the function.

Q9. Answer:

print shows a value on the screen; return sends a value back to the caller so it can be used in code.

Q10. Answer:

```
def add(a, b):
return a + b
print(add(3, 4))
```

Lesson 7.4 Check

Correct answer: C. A parameter with a default value

Why: Default arguments are used when the caller does not provide a value.

Why the other options are wrong:

- **A.** Always 0 — Not necessarily 0.
- **B.** Cannot be changed — They can be overridden.
- **D.** First argument — Position is separate.

Correct answer: B. greeting uses the default "Hello"

Why: Since no second argument is given, the default is used.

Why the other options are wrong:

- **A.** Error — The default prevents an error.
- **C.** Empty — It uses "Hello".
- **D.** name ignored — name is "Alex".

Correct answer: D. After parameters without defaults

Why: Defaults must come last so Python can match arguments unambiguously.

Why the other options are wrong:

- **A.** Before — This causes a SyntaxError.
- **B.** Any order — Order matters.
- **C.** Only at start — Must be at the end.

Correct answer: B. Yes, by providing a different argument

Why: greet("Alex", "Hi") overrides the default.

Why the other options are wrong:

- **A.** Fixed — Defaults can be overridden.
- **C.** Only inside — From the call.
- **D.** Only Python 3 — Always.

Lesson 7.4 Practice

Q1. Answer: A. A value used when the caller does not pass one

A default is used if no argument is given.

Q2. Answer: C. def f(x = 5):

Use = inside the parameter list.

Q3. Answer: D. 3

x uses its default value of 3.

Q4. Answer: D. 7

The passed argument overrides the default.

Q5. Answer: C. Last (after non-default ones)

Default parameters must come after non-default ones.

Q6. Answer: True

Passing an argument overrides the default.

Q7. Output: Hi friend

No argument is passed, so name uses the default "friend".

Q8. Bug: A parameter with a default cannot come before one without.

Put y first: def f(y, x=5):

Q9. Answer:

A default argument is a value used automatically when the caller does not pass one for that parameter.

Q10. Answer:

```
def greet(name="there"):
print("Hello, " + name)
greet()
```

Lesson 7.5 Check

Correct answer: C. Local

Why: Variables created inside a function are local.

Why the other options are wrong:

A. Global — Globals are outside functions.

B. Static — Not a Python concept here.

D. Constant — Not the right term.

Correct answer: A. No, it only exists inside the function

Why: Local variables are destroyed when the function ends.

Why the other options are wrong:

B. Yes — They are destroyed.

C. Same file — Still local.

D. If returned — You can get the value, but the variable name does not exist outside.

Correct answer: A. It is destroyed

Why: Local variables cease to exist when the function ends.

Why the other options are wrong:

B. Stays in memory — It is destroyed.

C. Becomes global — It does not.

D. Error — No error.

Correct answer: B. Global

Why: Variables outside all functions are global.

Why the other options are wrong:

A. Local — Locals are inside functions.

C. Static — Not applicable.

D. Temporary — Not the right term.

Lesson 7.5 Practice

Q1. Answer: A. A variable created inside a function

Local variables live inside a function.

Q2. Answer: D. A variable created outside all functions, usable throughout the file

Globals are defined at the top level.

Q3. Answer: D. Yes

Functions can read globals.

Q4. Answer: B. It creates a new local variable

Assignment inside a function creates a local variable unless you use global.

Q5. Answer: C. global

global tells Python to use the global variable.

Q6. Answer: True

Local variables only exist inside the function.

Q7. Output: 5

The function can read the global x, which is 5.

Q8. Bug: y is local to f, so it cannot be used outside.

Return y from f and store the result.

Q9. Answer:

A local variable exists only inside its function, while a global variable is defined outside functions and can be read throughout the file.

Q10. Answer:

```
def show():
message = "hi"
print(message)
show()
```

Lesson 7.6 Check

Correct answer: D. Makes code run slower

Why: Functions do not make code slower; they make it more organized.

Why the other options are wrong:

A. Code reuse — This IS a benefit.

B. Better readability — This IS a benefit.

C. Easier testing — This IS a benefit.

Correct answer: B. More readable

Why: The name explains what the code does.

Why the other options are wrong:

A. Faster — Speed is not the main benefit.

C. Less memory — Not the main point.

D. Required — Not required, just good practice.

Correct answer: D. Yes

Why: Functions can call other functions.

Why the other options are wrong:

A. No — They can.

B. Only Python 3 — Always.

C. Only if return — No restriction.

Correct answer: B. Writing once, calling many times

Why: Reuse means you write the function once and call it wherever needed.

Why the other options are wrong:

A. Fast as possible — Not about speed.

C. From internet — Not what it means.

D. Recompiling — Unrelated.

Lesson 7.6 Practice

Q1. Answer: B. Code reuse

Functions let you reuse code.

Q2. Answer: A. They hide details behind a name

A good function name explains what the code does.

Q3. Answer: D. You can test one task in isolation

Each function can be tested by itself.

Q4. Answer: B. Using the same function in many places instead of copying code

You call a function instead of duplicating code.

Q5. Answer: C. The name describes what it does

A good name describes the task.

Q6. Answer: True

Functions organize code into named tasks.

Q7. Output: Hi Sam Hi Ana

The same function is reused for two calls.

Q8. Bug: They duplicated code instead of using a function; if they need a change they must fix it in three places.

Put the code in a function and call it three times.

Q9. Answer:

Functions let you reuse code, make programs easier to read, and make code easier to test.

Q10. Answer:

```
def cheer():
print("Go team!")
cheer()
cheer()
```

Chapter 7 Challenges - Solutions

Challenge 1

Question: Write a function square(n) that returns n * n. Call it with 7.

Solution:

Define the function with def, then call it with the argument 7.

def square(n): return n * n

result = square(7) print(result) # prints 49

The function takes a parameter n, returns n * n, and the caller stores the result in a variable. square(7) returns 7 * 7 = 49.

Challenge 2

Question: What does print(f(f(3))) output? Explain step by step.

Solution:

The output is 12.

Step by step: 1. The inner call is f(3). The function returns 3 * 2 = 6. 2. The outer call is f(6). The function returns 6 * 2 = 12. 3. print(12) displays 12.

This is called "function composition" — using the result of one function call as the argument to another. Python evaluates from the inside out: first f(3) = 6, then f(6) = 12.

Challenge 3

Question: Write is_even(n) and use it to print all even numbers from 1 to 20.

Solution:

The function returns True if n % 2 == 0. The loop calls the function for each number.

def is_even(n): return n % 2 == 0

for i in range(1, 21): if is_even(i): print(i)

Output: 2, 4, 6, 8, 10, 12, 14, 16, 18, 20

The function encapsulates the "is this even?" logic. The loop calls is_even(i) for each number, and if it returns True, prints the number. This is cleaner than writing "if i % 2 == 0" directly in the loop — the function name makes the intent obvious.

Challenge 4

Question: What happens with mystery("Hello", " World") and mystery(5, 3)?

Solution:

Both work, but they do different things:

- mystery("Hello", " World") returns "Hello World" (string concatenation, because + joins strings) - mystery(5, 3) returns 8 (integer addition, because + adds numbers)

The same function works for both because the + operator behaves differently depending on the types of its operands. For strings, + means concatenation. For integers, + means addition. Python decides which behaviour to use based on the types of the arguments at runtime.

This is called "operator overloading" — the same operator (+) does different things for different types. It is a powerful feature of Python, but it can also be surprising if you are not expecting it. Always be aware of what types your function receives.

Challenge 5

Question: Write calculate_bmi(weight, height), ask user, print result rounded to 1 decimal.

Solution:

BMI = weight / (height squared). The function computes this and returns it. The main program asks for input, calls the function, and formats the output.

def calculate_bmi(weight, height): return weight / (height ** 2)

weight = float(input("Enter your weight in kg: ")) height = float(input("Enter your height in meters: ")) bmi = calculate_bmi(weight, height) print(f"Your BMI is {bmi:.1f}")

If the user enters 70 kg and 1.75 m, the output is: Your BMI is 22.9

The function does the calculation. The main program handles input and output. This separation of concerns — calculation in the function, I/O in the main program — is a key principle of good program design. The .1f format specifier rounds to 1 decimal place.

Chapter 8 — Lists and Tuples

Lesson 8.1 Check

Correct answer: B. [1, 2, 3]

Why: Lists use square brackets.

Why the other options are wrong:

A. (1, 2, 3) — This is a tuple.
C. {1, 2, 3} — This is a set.
D. <...> — Not valid.

Correct answer: A. 0

Why: Python lists are zero-indexed.

Why the other options are wrong:

B. 1 — Counting starts at 0.
C. -1 — This is the last item.
D. Always 0 — Always 0.

Correct answer: C. 20

Why: Index 1 is the second item.

Why the other options are wrong:

A. 10 — This is index 0.
B. 30 — This is index 2.
D. 1 — The question asks for the value.

Correct answer: C. 30

Why: Index -1 is the last item.

Why the other options are wrong:

A. 10 — This is index 0 or -3.
B. -1 — The question asks for the value.
D. Error — Negative indices are valid.

Lesson 8.1 Practice

Q1. Answer: B. With []

Lists use square brackets.

Q2. Answer: C. 0

Python lists start at index 0.

Q3. Answer: B. 10

Index 0 is the first item, 10.

Q4. Answer: B. 30

Negative index -1 is the last item, 30.

Q5. Answer: B. 3

len returns the number of items, which is 3.

Q6. Answer: True

The first item is at index 0.

Q7. Output: 20

Index 1 is the second item, 20.

Q8. Bug: Index 3 is out of range; valid indices are 0, 1, 2.

Use a valid index like a[2].

Q9. Answer:

A list is an ordered collection of items stored in square brackets that you can access by index.

Q10. Answer:

```
colors = ["red", "green", "blue"]
print(colors[1])
```

Lesson 8.2 Check

Correct answer: B. [2, 3]

Why: Slice [1:3] gives indices 1 and 2 (stops before 3).

Why the other options are wrong:

A. [1, 2, 3] — This would be [0:3].
C. [2, 3, 4] — This would be [1:4].
D. [1, 2] — This would be [0:2].

Correct answer: A. [1, 2]

Why: [:2] means from the start to index 2 (exclusive).

Why the other options are wrong:

B. [1, 2, 3] — This would be [:3] or [:].
C. [2, 3] — This would be [1:].
D. [] — It is not empty.

Correct answer: B. [3, 2, 1]

Why: [::-1] reverses the list.

Why the other options are wrong:

A. [1, 2, 3] — Same order.
C. [1, 3] — Not a valid slice result.
D. Error — Valid.

Correct answer: A. No, it stops before stop

Why: Slicing is exclusive of the stop index.

Why the other options are wrong:

B. Yes — It is exclusive.
C. Sometimes — Always exclusive.
D. Only if start is 0 — Always exclusive.

Lesson 8.2 Practice

Q1. Answer: A. [20, 30, 40]

Slice 1:4 gives items at indices 1, 2, 3 — [20, 30, 40].

Q2. Answer: C. The list reversed

A negative step reverses the list.

Q3. Answer: A. [10, 20, 30]

Missing start means from the beginning, up to but not including index 3.

Q4. Answer: B. [30, 40]

Missing end means from index 2 to the end.

Q5. Answer: B. start

The start is included; stop is excluded.

Q6. Answer: True

Slices create a copy of the chosen part.

Q7. Output: [2, 3]

Slice 1:3 gives items at indices 1 and 2 — [2, 3].

Q8. Bug: They started at index 3 (the last item) instead of index -2.

Use a[-2:] to get the last two items.

Q9. Answer:

List slicing returns a new list containing a portion of the original, chosen by start and stop indices.

Q10. Answer:

```
nums = [10, 20, 30, 40, 50]
print(nums[-3:])
```

Lesson 8.3 Check

Correct answer: C. append()

Why: append() adds to the end.

Why the other options are wrong:

A. add() — Sets use add().

B. insert() — insert() adds at a specific position.

D. push() — Not a Python list method.

Correct answer: C. [1, 2, 3, 4]

Why: append(4) adds 4 to the end of the list. The list is modified in place.

Why the other options are wrong:

A. [4, 1, 2, 3] — This would be insert(0, 4).

B. [1, 2, 3] — It does add the item.

D. None — append() modifies the list; it does not return None as the list value.

Correct answer: A. Removes and returns the last item

Why: pop() removes and returns the last item.

Why the other options are wrong:

B. First item — pop() removes the last.

C. Adds — pop() removes.

D. Sorts — pop() does not sort.

Correct answer: D. [1, 2, 3]

Why: sort() sorts the list in ascending order, in place. The list itself is now [1, 2, 3].

Why the other options are wrong:

A. [3, 2, 1] — This would be sort(reverse=True).

B. [3, 1, 2] — The list IS sorted, so this is wrong.

C. None — sort() modifies the list in place; the list is not None.

Lesson 8.3 Practice

Q1. Answer: C. Adds an item to the end of the list

append adds to the end.

Q2. Answer: A. Inserts "x" at index 1

insert(i, x) puts x at index i.

Q3. Answer: C. Removes the first 20 found

remove deletes the first matching value.

Q4. Answer: B. Removes and returns the last item

pop() removes and returns the last item.

Q5. Answer: A. Sorts the list in place (lowest to highest)

sort() arranges items in order, in place.

Q6. Answer: True

It grows the list by one at the end.

Q7. Output: [1, 2, 3]

append adds 3 to the end.

Q8. Bug: sort() returns None, so b is None, not a sorted list.

Use sorted(a) to get a new sorted list, or call a.sort() then print a.

Q9. Answer:

append() adds a single item to the end of a list.

Q10. Answer:

```
nums = [3, 1, 2]
nums.sort()
print(nums)
```

Lesson 8.4 Check

Correct answer: A. for item in list:

Why: This is the standard way to loop over a list.

Why the other options are wrong:

- **B.** for item = list — Invalid syntax.
- **C.** foreach — Not Python.
- **D.** loop — Not Python.

Correct answer: A. Index-value pairs

Why: enumerate() gives (index, value) pairs.

Why the other options are wrong:

- **B.** Just items — That is "for item in list".
- **C.** Just indices — That is range(len(list)).
- **D.** Reverse — Not what enumerate does.

Correct answer: B. The actual value from the list

Why: item is the value, not the index.

Why the other options are wrong:

- **A.** Index — The value.
- **C.** The list — Individual items.
- **D.** Count — The value.

Correct answer: D. Yes, but be careful

Why: You can modify, but adding/removing while looping can cause bugs.

Why the other options are wrong:

- **A.** No — You can.
- **B.** Only while — For loops too.
- **C.** Only range() — Any loop.

Lesson 8.4 Practice

Q1. Answer: B. for item in list

for item in list is the standard pattern.

Q2. Answer: C. Both the index and the item

enumerate yields (index, item) pairs.

Q3. Answer: D. 1

x takes each item in turn, starting with 1.

Q4. Answer: B. 3

Once per item, so 3 times.

Q5. Answer: C. for i, x in enumerate(lst):

enumerate gives (index, item) pairs.

Q6. Answer: True

That gives each item one by one.

Q7. Output: 10 20 30

Each item is printed on its own line.

Q8. Bug: The for line is missing a colon.

Add a colon: for x in [1, 2, 3]:

Q9. Answer:

enumerate() gives you both the index and the item as you loop through a list.

Q10. Answer:

```
fruits = ["apple", "banana", "cherry"]
for f in fruits:
print(f)
```

Lesson 8.5 Check

Correct answer: D. (1, 2, 3)

Why: Tuples use parentheses.

Why the other options are wrong:

- **A.** [...] — Lists.
- **B.** {...} — Sets or dicts.
- **C.** <...> — Not valid.

Correct answer: C. No, tuples are immutable

Why: Once created, a tuple cannot be changed.

Why the other options are wrong:

- **A.** Yes — They are immutable.
- **B.** Only first — No item can change.
- **D.** Only Python 3 — Always immutable.

Correct answer: A. Lists are mutable; tuples are immutable

Why: This is the key difference.

Why the other options are wrong:

- **B.** Only numbers — Both hold any type.
- **C.** Speed — Not the main difference.
- **D.** No difference — There is.

Correct answer: B. When the data should not change

Why: Use tuples for fixed collections like coordinates.

Why the other options are wrong:

- **A.** Need to add items — Use a list.
- **C.** Need to sort — Use a list.
- **D.** Never — Tuples are useful.

Lesson 8.5 Practice

Q1. Answer: B. ()

Tuples use parentheses.

Q2. Answer: D. No, tuples are immutable

Tuples cannot be changed (immutable).

Q3. Answer: C. 10

Index 0 is the first item, 10.

Q4. Answer: C. Storing fixed data like coordinates (x, y)

Tuples suit fixed groups like coordinates.

Q5. Answer: A. 3

The tuple has 3 items.

Q6. Answer: True

Tuples are immutable.

Q7. Output: 2

Index 1 is the second item, 2.

Q8. Bug: Tuples are immutable, so you cannot assign to an index.

Use a list if you need to change values.

Q9. Answer:

A list can be changed (mutable) and uses [], while a tuple cannot be changed (immutable) and uses ().

Q10. Answer:

```
point = (5, 7)
print(point[0])
```

Chapter 8 Challenges - Solutions

Challenge 1

Question: Create a list of 5 numbers, print the largest and smallest.

Solution:

Use the built-in max() and min() functions.

numbers = [42, 17, 89, 3, 56] print(f"Largest: {max(numbers)}") # 89 print(f"Smallest: {min(numbers)}") # 3

max() and min() are built-in functions that work on any iterable (list, tuple, etc.). They return the largest and smallest items respectively. No need to write your own loop — Python does it for you.

Challenge 2

Question: What does [1, 2, 3] + [4, 5] produce? What about [1, 2] * 3?

Solution:

[1, 2, 3] + [4, 5] produces [1, 2, 3, 4, 5] (concatenation). [1, 2] * 3 produces [1, 2, 1, 2, 1, 2] (repetition).

The + operator on lists creates a new list with all the items from both lists. The * operator repeats the list that many times. These are the same operators used for strings (+ joins, * repeats), and they work the same way for lists.

Note that + creates a NEW list. The original lists are not changed. If you want to modify a list in place, use .extend() or .append() instead.

Challenge 3

Question: Ask for 3 names, store in a list, print in reverse order.

Solution:

Build the list with append(), then reverse with [::-1].

names = [] for i in range(3): name = input(f"Enter name {i + 1}: ") names.append(name)

for name in names[::-1]: print(name)

If the user enters Alex, Maya, Sam, the output is: Sam Maya Alex

names[::-1] creates a reversed copy of the list. You could also use names.reverse() (which reverses in place) and then loop, but [::-1] is cleaner for a one-time reversal.

Challenge 4

Question: Write distance(point1, point2) for two (x, y) tuples.

Solution:

Unpack the tuples inside the function, then apply the distance formula.

def distance(point1, point2): x1, y1 = point1 x2, y2 = point2 return ((x2 - x1) ** 2 + (y2 - y1) ** 2) ** 0.5

p1 = (0, 0) p2 = (3, 4) d = distance(p1, p2) print(f"Distance: {d}") # Distance: 5.0

The line "x1, y1 = point1" is called tuple unpacking — it extracts the two values from the tuple into separate variables. The distance formula is the Pythagorean theorem: the square root of the sum of squared differences. ** 0.5 is the same as taking the square root.

Challenge 5

Question: Add up [10, 20, 30, 40, 50] without using sum().

Solution:

Create a total variable before the loop, then add each item.

numbers = [10, 20, 30, 40, 50] total = 0 for n in numbers: total += n print(f"Total: {total}") # Total: 150

The total variable starts at 0 (before the loop). Each iteration adds the current number to total. After the loop, total holds the sum of all numbers: 10 + 20 + 30 + 40 + 50 = 150.

In practice, you would just use sum(numbers), but writing it yourself helps you understand how sum() works internally. This pattern — initialize a variable before a loop, update it inside — is one of the most common patterns in programming.

Chapter 9 — Dictionaries and Sets

Lesson 9.1 Check

Correct answer: C. {"a": 1}

Why: Dictionaries use curly braces with key: value pairs.

Why the other options are wrong:

A. [...] — Lists.

B. (...) — Tuples.

D. <...> — Not valid.

Correct answer: A. Looks up the value for "name"

Why: Square brackets look up by key.

Why the other options are wrong:

B. Adds — Assignment adds: d["name"] = value.

C. Removes — del or .pop() removes.

D. Sorts — Dicts are not sorted.

Correct answer: D. The old value is replaced

Why: Keys are unique; assigning overwrites.

Why the other options are wrong:

A. Error — No error; it overwrites.

B. New key — The key already exists.

C. Nothing — It replaces.

Correct answer: D. [1, 2]

Why: Lists are mutable, so they cannot be keys.

Why the other options are wrong:

A. "name" — Strings are immutable — valid key.

B. 42 — Numbers are immutable — valid key.

C. (1, 2) — Tuples are immutable — valid key.

Lesson 9.1 Practice

Q1. Answer: C. {}

Dictionaries use curly braces.

Q2. Answer: C. Key-value pairs

A dict maps keys to values.

Q3. Answer: B. 2

The key "b" maps to 2.

Q4. Answer: A. d["x"] = 5

Use square bracket assignment.

Q5. Answer: D. Raises KeyError

A missing key raises KeyError (unless you use get).

Q6. Answer: True

Each key appears only once; assigning again updates the value.

Q7. Output: Ana

The key "name" maps to "Ana".

Q8. Bug: The key "b" is not in the dictionary, so it raises KeyError.

Use d.get("b") or add the key first.

Q9. Answer:

A dictionary is a collection of key-value pairs where you look up a value using its key.

Q10. Answer:

```
scores = {"math": 90}
print(scores["math"])
```

Lesson 9.2 Check

Correct answer: B. All the keys

Why: .keys() returns all the keys.

Why the other options are wrong:

A. Values — That is .values().
C. Pairs — That is .items().
D. First key — Not what it does.

Correct answer: D. Key-value pairs as tuples

Why: .items() returns (key, value) pairs.

Why the other options are wrong:

A. Keys — That is .keys().
B. Values — That is .values().
C. Count — That is len().

Correct answer: B. None

Why: .get() returns None for missing keys (no error).

Why the other options are wrong:

A. Error — Only square brackets raise errors.
C. 0 — It returns None, not 0.
D. "missing" — It returns None.

Correct answer: D. d.get("key")

Why: get() does not crash on missing keys.

Why the other options are wrong:

A. d["key"] — This raises KeyError on missing keys.
B. Same — They differ on missing keys.
C. Neither — get() is safer.

Lesson 9.2 Practice

Q1. Answer: D. The keys

keys() returns the dictionary keys.

Q2. Answer: D. The values

values() returns the dictionary values.

Q3. Answer: B. (key, value) pairs

items() returns key-value pairs.

Q4. Answer: C. None (or a default)

get returns None by default for missing keys.

Q5. Answer: B. for k, v in d.items():

items() gives (key, value) pairs to unpack.

Q6. Answer: True

That is the safer way to look up a key.

Q7. Output: ['a', 'b']

keys() returns the keys a and b.

Q8. Bug: You cannot unpack a dict directly; you need d.items().

Use for k, v in d.items():

Q9. Answer:

d.get(key) returns None (or a default) for missing keys, while d[key] raises a KeyError.

Q10. Answer:

```
d = {"x": 1, "y": 2}
for k, v in d.items():
print(k, v)
```

Lesson 9.3 Check

Correct answer: D. A phone book

Why: Dictionaries map keys (names) to values (numbers).

Why the other options are wrong:

- **A.** List of numbers — Use a list.
- **B.** Single number — Use a variable.
- **C.** True/false — Use a bool.

Correct answer: D. Using parallel lists

Why: A dict is cleaner than two parallel lists for key-value data.

Why the other options are wrong:

- **A.** Too many variables — Possible but not the main fix.
- **B.** Too many comments — Unrelated.
- **C.** Too many loops — Unrelated.

Correct answer: D. Keys

Why: Keys must be unique; values can repeat.

Why the other options are wrong:

- **A.** Values — Values can repeat.
- **B.** Both — Only keys.
- **C.** Neither — Keys are unique.

Correct answer: A. Yes

Why: Values can be any type, including lists.

Why the other options are wrong:

- **B.** No — They can.
- **C.** Only strings — Any type.
- **D.** Only Python 3 — Always.

Lesson 9.3 Practice

Q1. Answer: A. When you look up values by a key often

Dicts are great for key-based lookups.

Q2. Answer: A. Two lists where index i in one matches index i in the other

Parallel lists keep related items at the same index.

Q3. Answer: C. Keys are easier to read than index numbers

You look up by name, not by a number that could be wrong.

Q4. Answer: A. A phone book (name to number)

Name-to-number mappings are perfect for dicts.

Q5. Answer: A. Easy to get the indices out of sync

If lists get out of order, data is mismatched.

Q6. Answer: True

Key-based lookup is the main use of a dict.

Q7. Output: 123

Looking up "Bob" returns "123".

Q8. Bug: The two lists become out of sync, so names and ages no longer match.

Use a dictionary mapping name to age instead.

Q9. Answer:

Use a dictionary when you need to look up values by a key, since it keeps related data together and avoids index mistakes.

Q10. Answer:

```
fruit = {"apple": 3, "banana": 5}
print(fruit["apple"])
```

Lesson 9.4 Check

Correct answer: C. {1, 2, 3}

Why: Sets use curly braces (but no colons).

Why the other options are wrong:

A. [...] — Lists.

B. (...) — Tuples.

D. <...> — Not valid.

Correct answer: A. They only hold unique items

Why: Sets automatically remove duplicates.

Why the other options are wrong:

B. Ordered — Sets are unordered.

C. Only numbers — Any hashable type.

D. Mutable like lists — Sets are mutable but not ordered.

Correct answer: D. {1, 2, 3}

Why: Duplicates are removed.

Why the other options are wrong:

A. [1, 2, 3] — set() returns a set, not a list.

B. {1, 1, 2, 2, 3} — Sets cannot have duplicates.

C. Error — Valid.

Correct answer: C. {2}

Why: & is intersection — items in BOTH sets.

Why the other options are wrong:

A. {1, 2, 3} — This is union (|).

B. {1, 3} — This is symmetric difference (^).

D. Error — Valid.

Lesson 9.4 Practice

Q1. Answer: B. It only stores unique items

A set has no duplicates.

Q2. Answer: C. {}

Sets use curly braces (like dicts, but with single items).

Q3. Answer: B. {1, 2, 3}

Duplicates are removed, leaving {1, 2, 3}.

Q4. Answer: A. Union (items in either)

| is union.

Q5. Answer: B. Intersection

& is intersection.

Q6. Answer: True

Sets only keep unique items.

Q7. Output: {1, 2, 3}

Duplicates are removed.

Q8. Bug: {} creates an empty dictionary, not an empty set.

Use set() to create an empty set.

Q9. Answer:

A set is an unordered collection that stores only unique items.

Q10. Answer:

```
a = {1, 2, 3}
b = {2, 3, 4}
print(a | b)
```

Chapter 9 Challenges - Solutions

Challenge 1

Question: Count how many times each word appears in a string.

Solution:

Split the string into words, then use a dictionary to count each word. For each word, either increment its count or set it to 1 if it is new.

text = "the cat sat on the mat the cat" words = text.split() counts = {} for word in words: if word in counts: counts[word] += 1 else: counts[word] = 1 for word, count in counts.items(): print(f"{word}: {count}")

Output: the: 3 cat: 2 sat: 1 on: 1 mat: 1

This is called a "word count" and is a classic programming exercise. The dictionary acts as a counter: each word is a key, and its count is the value. The "if word in counts" check determines whether we have seen this word before.

Challenge 2

Question: What is the output of d = {}; d["a"] = 1; d["a"] = 2; print(d)?

Solution:

The output is: {'a': 2}

Dictionary keys are unique. When you assign d["a"] = 2, Python checks if the key "a" already exists. It does, so Python REPLACES the old value (1) with the new value (2). A new key-value pair is not added.

This is different from lists, where assigning to an index replaces the item at that position. With dictionaries, assigning to a key either adds a new pair (if the key is new) or updates the existing value (if the key already exists). There is never a duplicate key.

Challenge 3

Question: Ask for 5 numbers, store in a set, print how many UNIQUE numbers.

Solution:

A set automatically removes duplicates. len(set) gives the number of unique items.

numbers = set() for i in range(5): n = int(input(f"Enter number {i + 1}: ")) numbers.add(n) print(f"You entered {len(numbers)} unique numbers.")

If the user enters 5, 3, 5, 7, 3, the set becomes {3, 5, 7} (duplicates removed), and len() returns 3.

Sets are perfect for this task because they automatically enforce uniqueness. You do not need to check for duplicates yourself — the set does it for you.

Challenge 4

Question: Map student names to scores, print students above 80.

Solution:

Create a dictionary, then loop with .items() and filter with an if.

scores = {"Alex": 85, "Maya": 72, "Sam": 91, "Lisa": 68} for name, score in scores.items(): if score > 80: print(f"{name}: {score}")

Output: Alex: 85 Sam: 91

.items() gives us both the key (name) and the value (score) for each entry. The if statement filters to only those with score > 80. This pattern — loop over a dictionary and filter — is very common in real programs.

Challenge 5

Question: What does dict.get("missing", "N/A") do? How is it different from dict.get("missing")?

Solution:

dict.get("missing", "N/A") returns "N/A" if the key "missing" does not exist. dict.get("missing") (without a second argument) returns None if the key is missing.

The second argument to get() is a CUSTOM DEFAULT. Instead of returning None for missing keys, you can return any value you choose:

d = {"name": "Alex"} print(d.get("name", "Unknown")) # "Alex" (key exists) print(d.get("email", "N/A")) # "N/A" (key missing, custom default) print(d.get("phone")) # None (key missing, default default)

This is useful when None is not a helpful default. For example, if you are displaying a user profile and the email is missing, "N/A" or "Not provided" is more user-friendly than "None".

Chapter 10 — Working with Strings

Lesson 10.1 Check

Correct answer: D. "HELLO"

Why: upper() converts all letters to uppercase.

Why the other options are wrong:

- **A.** "hello" — This is lower().
- **B.** "Hello" — This is capitalize() or title().
- **C.** Error — Valid.

Correct answer: C. "hi"

Why: strip() removes leading and trailing whitespace.

Why the other options are wrong:

- **A.** " hi " — Original unchanged.
- **B.** " hi" — This is rstrip() only.
- **D.** "hi " — This is lstrip() only.

Correct answer: C. ["a", "b", "c"]

Why: split(",") splits at each comma.

Why the other options are wrong:

- **A.** "abc" — split returns a list.
- **B.** ["abc"] — No commas to split on in "abc".
- **D.** "a b c" — split returns a list.

Correct answer: A. "a-b-c"

Why: join() glues the list items with the separator.

Why the other options are wrong:

B. "abc-" — join puts separator BETWEEN items.

C. ["a-b-c"] — join returns a string.

D. "a b c" — This would need " ".join().

Lesson 10.1 Practice

Q1. Answer: C. "HI"

upper() makes all letters uppercase.

Q2. Answer: C. "hi"

lower() makes all letters lowercase.

Q3. Answer: B. "hi"

strip() removes leading and trailing whitespace.

Q4. Answer: D. ['a', 'b', 'c']

split() breaks the string into a list by the separator.

Q5. Answer: D. "a-b-c"

join() glues list items with the separator.

Q6. Answer: True

replace returns a new string; it does not change the original.

Q7. Output: HELLO

upper() makes all letters uppercase.

Q8. Bug: upper() returns a new string; the original s is not changed.

Store the result: s = s.upper().

Q9. Answer:

split() breaks a string into a list of parts using a separator you choose.

Q10. Answer:

```
s = " Python "
print(s.strip())
```

Lesson 10.2 Check

Correct answer: A. "P"

Why: Index 0 is the first character.

Why the other options are wrong:

B. "ython" — This is [1:].

C. 0 — The value, not the index.

D. Error — Valid.

Correct answer: B. "yth"

Why: Slice [1:4] gives characters at indices 1, 2, 3.

Why the other options are wrong:

A. "Pyt" — This is [0:3].

C. "ython" — This is [1:].

D. "Pyth" — This is [0:4].

Correct answer: A. "n"

Why: Index -1 is the last character.

Why the other options are wrong:

B. "P" — This is index 0.

C. "-1" — The value, not the index.

D. Error — Valid.

Correct answer: A. "nohtyP"

Why: [::-1] reverses the string.

Why the other options are wrong:

B. "Python" — Same order.

C. "Pytoh" — Not a valid slice.

D. Error — Valid.

Lesson 10.2 Practice

Q1. Answer: B. "h"

Index 0 is the first character, "h".

Q2. Answer: D. "el"

Slice 1:3 gives characters at indices 1 and 2 — "el".

Q3. Answer: D. "o"

Negative index -1 is the last character, "o".

Q4. Answer: D. Reverses the string

A negative step reverses the string.

Q5. Answer: A. "he"

Missing start means from the beginning, up to but not including index 2.

Q6. Answer: True

Strings are sequences, just like lists.

Q7. Output: ell

Slice 1:4 gives characters at indices 1, 2, 3 — "ell".

Q8. Bug: Index 5 is out of range; valid indices are 0 to 4.

Use s[-1] or s[4] to get the last character.

Q9. Answer:

It returns a new string with the characters from index start up to but not including stop.

Q10. Answer:

```
s = "abcdef"
print(s[-2:])
```

Lesson 10.3 Check

Correct answer: C. Immutable

Why: Strings cannot be changed after creation.

Why the other options are wrong:

A. Mutable — Lists are mutable; strings are not.

B. Sometimes — Always immutable.

D. Only in functions — Always.

Correct answer: A. It raises a TypeError

Why: You cannot assign to a string index.

Why the other options are wrong:

B. It works — Strings are immutable.

C. Changes s — No, error.

D. Nothing — Error.

Correct answer: A. Returns "HELLO" but s is still "hello"

Why: Methods return new strings; they do not change the original.

Why the other options are wrong:

B. Changes s — Strings are immutable.

C. Error — Valid.

D. Nothing — It returns a new string.

Correct answer: D. Create a new one and reassign

Why: s = s.upper() creates a new string and reassigns.

Why the other options are wrong:

A. modify() — No such method.

B. change() — No such function.

C. Directly — Cannot.

Lesson 10.3 Practice

Q1. Answer: C. Strings cannot be changed after creation

Immutable means cannot be changed.

Q2. Answer: B. Creates a new uppercase string and rebinds s to it

upper() returns a new string; s now points to it.

Q3. Answer: A. Raises TypeError

Strings are immutable; you cannot assign to an index.

Q4. Answer: B. Because strings are immutable

Methods cannot change the original, so they return new ones.

Q5. Answer: B. "bat"

replace returns a new string "bat".

Q6. Answer: True

The original string stays the same.

Q7. Output: hi HI

s is unchanged; t holds the uppercase version.

Q8. Bug: You cannot assign to a string index because strings are immutable.

Build a new string: s = "H" + s[1:].

Q9. Answer:

Strings cannot be changed after they are created; operations like upper() return a new string instead.

Q10. Answer:

```
s = "hello"
new_s = s[0].upper() + s[1:]
print(new_s)
```

Lesson 10.4 Check

Correct answer: B. True

Why: All characters in "123" are digits.

Why the other options are wrong:

- **A.** False — They ARE digits.
- **C.** "123" — isdigit() returns a boolean.
- **D.** Error — Valid.

Correct answer: A. False

Why: "Hello" has uppercase letters, so islower() is False.

Why the other options are wrong:

- **B.** True — "Hello" is not all lowercase.
- **C.** None — Returns a boolean.
- **D.** Error — Valid.

Correct answer: A. True

Why: "fox" appears inside "The quick brown fox".

Why the other options are wrong:

- **B.** False — It does appear.
- **C.** 3 — in returns a boolean.
- **D.** Error — Valid.

Correct answer: C. True

Why: "Hello" starts with "He".

Why the other options are wrong:

- **A.** False — It does start with "He".
- **B.** "He" — startswith() returns a boolean.
- **D.** Error — Valid.

Lesson 10.4 Practice

Q1. Answer: A. True

All characters are letters, so True.

Q2. Answer: C. True

All characters are digits, so True.

Q3. Answer: B. True

The string starts with "He", so True.

Q4. Answer: B. True

The character "a" is in "cat", so True.

Q5. Answer: C. False

It has digits, so isalpha() is False.

Q6. Answer: True

"ab" in "abc" is True.

Q7. Output: True

The string starts with "Py".

Q8. Bug: isalpha() takes no arguments.

Call it as s.isalpha() with nothing inside.

Q9. Answer:

isdigit() returns True if every character in the string is a digit.

Q10. Answer:

```
s = "abc123"
print(s.isalpha())
```

Chapter 10 Challenges - Solutions

Challenge 1

Question: Ask for a sentence; print word count, character count, and title case.

Solution:

Use split() for words, len() for characters, and .title() for title case.

sentence = input("Enter a sentence: ") word_count = len(sentence.split()) char_count = len(sentence) title_case = sentence.title() print(f"Words: {word_count}") print(f"Characters: {char_count}") print(f"Title case: {title_case}")

If the user enters "hello world", the output is: Words: 2 Characters: 11 Title case: Hello World

split() with no arguments splits on any whitespace. len(sentence) counts all characters including spaces. .title() capitalizes the first letter of each word.

Challenge 2

Question: What does "-".join("hello") return?

Solution:

It returns: "h-e-l-l-o"

join() can take any iterable as its argument. A string is an iterable of characters. So "-".join("hello") joins each CHARACTER of "hello" with hyphens: h, then -, then e, then -, etc.

This works because Python treats strings as sequences of characters. When join() receives "hello", it iterates over the characters h, e, l, l, o and inserts "-" between each pair. The result is "h-e-l-l-o".

This is a nice trick to know, but in practice you usually join lists of strings, not individual characters.

Challenge 3

Question: Check if a password is valid: 8+ chars, has a digit, has uppercase.

Solution:

Check three conditions and combine with "and".

password = input("Enter password: ") has_length = len(password) >= 8 has_digit = False has_upper = False for char in password: if char.isdigit(): has_digit = True if char.isupper(): has_upper = True if has_length and has_digit and has_upper: print("Valid") else: print("Invalid")

The loop checks each character. If any character is a digit, has_digit becomes True. If any is uppercase, has_upper becomes True. The final if combines all three checks with "and". All three must be True for the password to be valid.

Challenge 4

Question: What does "Hello".replace("l", "w") output?

Solution:

It outputs: "Hewwo"

replace() returns a NEW string with ALL occurrences of the first argument replaced by the second. In "Hello", there are two "l" characters (at indices 2 and 3). Both are replaced with "w", giving "Hewwo".

The original string "Hello" is NOT changed (strings are immutable). replace() returns a new string. If you want to keep the result, you must assign it: s = s.replace("l", "w").

You can also limit the number of replacements: "Hello".replace("l", "w", 1) replaces only the FIRST "l", giving "Hewlo".

Challenge 5

Question: Check if a word is a palindrome (reads the same forwards and backwards).

Solution:

Compare the word to its reverse. If they are the same, it is a palindrome.

word = input("Enter a word: ") if word == word[::-1]: print(f"{word} is a palindrome!") else: print(f"{word} is NOT a palindrome.")

If the user enters "racecar", the output is: racecar is a palindrome!

word[::-1] creates a reversed copy of the string. If the original and the reversed version are identical, the word reads the same forwards and backwards, which is the definition of a palindrome. Other palindromes: "level", "radar", "madam".

This is a classic programming challenge that shows the power of slicing — one line of code solves the problem.

Chapter 11 — File Handling

Lesson 11.1 Check

Correct answer: A. "r"

Why: "r" opens for reading.

Why the other options are wrong:

- **B.** "w" — Write mode.
- **C.** "a" — Append mode.
- **D.** "x" — Exclusive creation.

Correct answer: A. "w"

Why: "w" opens for writing and erases existing content.

Why the other options are wrong:

- **B.** "r" — Read mode.
- **C.** "a" — Append keeps existing.
- **D.** "read" — Not a valid mode.

Correct answer: C. Writes "Hello" to the file

Why: write() puts text into the file.

Why the other options are wrong:

- **A.** Reads — read() reads.
- **B.** Closes — close() closes.
- **D.** Prints — print() prints to screen.

Correct answer: D. Close it when done

Why: Always close files to ensure data is saved.

Why the other options are wrong:

A. Nothing — You must close.

B. Print — Unrelated.

C. Rename — Unrelated.

Lesson 11.1 Practice

Q1. Answer: A. 'r'

'r' is read mode.

Q2. Answer: B. 'w'

'w' writes (and overwrites).

Q3. Answer: C. 'a'

'a' appends.

Q4. Answer: B. open()

open() opens a file.

Q5. Answer: A. Close it with close()

Always close files to free resources.

Q6. Answer: True

Closing frees resources and saves data.

Q7. Output: Hi

read() returns the file contents, which is "Hi".

Q8. Bug: The file is never closed.

Add f.close() at the end.

Q9. Answer:

open() opens a file in a chosen mode (like read or write) and returns a file object you can use.

Q10. Answer:

```
f = open("notes.txt", "w")
f.write("Hello")
f.close()
```

Lesson 11.2 Check

Correct answer: C. Automatically closes the file

Why: with closes the file when the block ends.

Why the other options are wrong:

A. Multiple times — That is a loop.

B. Creates — Depends on the mode.

D. Read-only — Mode determines access.

Correct answer: A. with open(...)

Why: with closes even if an error occurs.

Why the other options are wrong:

B. open + close — Forgetting close causes bugs.

C. Same — with is safer.

D. Neither — with is safe.

Correct answer: D. When the block ends

Why: The file is closed automatically at the end of the with block.

Why the other options are wrong:

A. Never — It IS closed.

B. After each line — At the end of the block.

C. Only on error — Always.

Correct answer: B. with open("f.txt") as f:

Why: This is the correct syntax.

Why the other options are wrong:

A. with f = open(...) — Wrong syntax.

C. open(...) with f — Wrong syntax.

D. with f.open(...) — Wrong syntax.

Lesson 11.2 Practice

Q1. Answer: B. Automatically closes the file when the block ends

with ensures the file is closed automatically.

Q2. Answer: C. with open("f.txt") as f:

Use "with open(...) as f:".

Q3. Answer: B. You cannot forget to close the file

with closes the file even if an error happens.

Q4. Answer: B. The file is closed automatically

with closes the file at the end of the block.

Q5. Answer: A. Yes

with guarantees closing, even on errors.

Q6. Answer: True

It guarantees the file is closed properly.

Q7. Output: Hi

read() returns the contents, which is "Hi".

Q8. Bug: The as keyword is missing between open(...) and f.

Use: with open("data.txt", "r") as f:

Q9. Answer:

The with statement automatically closes the file when the block ends, even if an error occurs.

Q10. Answer:

```
with open("hello.txt", "r") as f:
print(f.read())
```

Lesson 11.3 Check

Correct answer: D. Reads the entire file as a single string

Why: f.read() with no argument reads everything.

Why the other options are wrong:

A. One line — That is readline().

B. One character — Not what read() does.

C. Closes — close() closes.

Correct answer: B. Reads one line at a time

Why: for line in f iterates over lines.

Why the other options are wrong:

A. Whole file into list — That is readlines().

C. Writes — for loop reads.

D. Closes — No.

Correct answer: C. A list of lines

Why: readlines() returns a list of strings.

Why the other options are wrong:

A. Single string — That is read().

B. One line — That is readline().

D. Nothing — It returns a list.

Correct answer: B. for line in f:

Why: Reading line by line only keeps one line in memory.

Why the other options are wrong:

A. f.read() — Reads the entire file at once.

C. f.readlines() — Reads all lines into a list.

D. Same — for line is most efficient.

Lesson 11.3 Practice

Q1. Answer: C. Reads the whole file as one string

read() returns the entire contents as one string.

Q2. Answer: D. Reads one line from the file

readline() returns one line at a time.

Q3. Answer: B. A list of lines

readlines() returns a list of all lines.

Q4. Answer: B. for line in f:

Looping directly reads one line at a time.

Q5. Answer: A. A newline character

Lines end with a newline character.

Q6. Answer: True

This is the best way for large files.

Q7. Output: Hello World

Each line is printed with the newline removed by strip().

Q8. Bug: real() is not a file method; it should be read().

Use f.read() to read the file.

Q9. Answer:

read() returns the whole file as one string, while readlines() returns a list of the lines.

Q10. Answer:

```
with open("nums.txt") as f:
print(len(f.readlines()))
```

Lesson 11.4 Check

Correct answer: B. To handle the case where the file does not exist yet

Why: The first time the program runs, there is no high score file.

Why the other options are wrong:

- **A.** Faster — Not about speed.
- **C.** Close file — with does that.
- **D.** Sort — Not about sorting.

Correct answer: A. The file does not exist

Why: FileNotFoundError means the file was not found.

Why the other options are wrong:

- **B.** Too big — Not what this error means.
- **C.** Read-only — Different error.
- **D.** Empty — Empty files are valid.

Correct answer: B. str — you must convert

Why: Files store text; numbers come back as strings.

Why the other options are wrong:

- **A.** int — Must convert.
- **C.** float — Must convert.
- **D.** bool — Must convert.

Correct answer: D. A newline character

Why: \n represents a new line.

Why the other options are wrong:

- **A.** Backslash and n — It is interpreted as a single newline.
- **B.** Nothing — It means newline.
- **C.** Error — Valid escape sequence.

Lesson 11.4 Practice

Q1. Answer: A. FileNotFoundError

Opening a missing file raises FileNotFoundError.

Q2. Answer: B. To handle missing files gracefully

try/except lets you handle missing files.

Q3. Answer: D. Storing the best score between runs

A high score file saves the best score so it lasts between runs.

Q4. Answer: D. Use a default value like 0

Fall back to a sensible default.

Q5. Answer: A. try: f = open("score.txt") except FileNotFoundError: score = 0

Wrap the open in try and handle FileNotFoundError.

Q6. Answer: True

You can fall back to a default value.

Q7. Output: No file

The FileNotFoundError is caught and "No file" is printed.

Q8. Bug: If the file is missing, the program crashes with FileNotFoundError.

Wrap it in try/except FileNotFoundError.

Q9. Answer:

It lets your program handle a missing file gracefully instead of crashing.

Q10. Answer:

```
try:
with open("score.txt") as f:
print(f.read())
except FileNotFoundError:
print("No score")
```

Chapter 11 Challenges - Solutions

Challenge 1

Question: Ask for name, write to "names.txt" (overwriting).

Solution:

Open in "w" mode and write.

name = input("Enter your name: ") with open("names.txt", "w") as f: f.write(name)

The "w" mode erases any existing content and writes the new text. The with statement ensures the file is closed automatically. If "names.txt" does not exist, Python creates it.

Note: write() does NOT add a newline. If you want the name on its own line, use f.write(name + "\n").

Challenge 2

Question: Read "numbers.txt" (one number per line), sum them, print total.

Solution:

Read line by line, convert each to int, add to total.

total = 0 with open("numbers.txt", "r") as f: for line in f: number = int(line.strip()) # strip removes the newline total += number print(f"Total: {total}")

Each line from the file includes the newline character at the end. strip() removes it. Then int() converts the string to an integer. The total variable is created BEFORE the loop (starting at 0) and accumulates the sum.

If numbers.txt contains: 10 20 30

The output is: Total: 60

Challenge 3

Question: Open in "w", write text, then open in "r" and read. Do you get the text?

Solution:

Yes, you get the text — BUT only if you CLOSE the first file before opening it again.

This works: with open("test.txt", "w") as f: f.write("Hello!") # File is automatically closed here with open("test.txt", "r") as f: print(f.read()) # prints "Hello!"

But this might NOT work: f1 = open("test.txt", "w") f1.write("Hello!") # f1 is NOT closed yet — data may still be in the buffer f2 = open("test.txt", "r") print(f2.read()) # might print nothing!

The reason: Python buffers writes in memory. The data is only guaranteed to be on disk after the file is closed. The with statement closes the file automatically, which is why it is the recommended approach.

Challenge 4

Question: Ask for 3 lines of text, write to "diary.txt", read back and print.

Solution:

Use a loop to collect 3 lines, write them with newlines, then read back.

Write 3 lines to the file with open("diary.txt", "w") as f: for i in range(3): line = input(f"Line {i + 1}: ") f.write(line + "\n")

Read the file back and print print("\n--- File contents ---") with open("diary.txt", "r") as f: content = f.read() print(content)

Each line is written with a \n at the end so they appear on separate lines in the file. The second with block reads the entire file and prints it, verifying that the write was successful.

Challenge 5

Question: Should you use "w" or "a" to ADD new high scores without erasing?

Solution:

You must use "a" (append mode).

"w" (write mode) ERASES the existing content of the file before writing. If you use "w" to add a new high score, all previous scores would be lost.

"a" (append mode) KEEPS the existing content and adds new content to the end. This is exactly what you want for a high score file:

with open("highscores.txt", "a") as f: f.write(f"{new_score}\n")

Each new score is added to the end of the file without erasing the old ones. This is a common mistake — using "w" when you mean "a" can cause data loss, so always double-check the mode.

Chapter 12 — Errors and Exceptions

Lesson 12.1 Check

Correct answer: D. ValueError

Why: "hello" cannot be converted to an integer.

Why the other options are wrong:

- **A.** TypeError — Wrong type for the operation.
- **B.** NameError — Undefined variable.
- **C.** SyntaxError — Grammar error.

Correct answer: D. ZeroDivisionError

Why: Dividing by zero is not allowed.

Why the other options are wrong:

- **A.** ValueError — Wrong value type.
- **B.** MathError — No such error in Python.
- **C.** TypeError — Wrong type.

Correct answer: D. NameError

Why: Using a variable that does not exist raises NameError.

Why the other options are wrong:

- **A.** ValueError — Wrong value.
- **B.** TypeError — Wrong type.
- **C.** SyntaxError — Grammar error.

Correct answer: C. Your code breaks Python's grammar rules

Why: SyntaxError means the code is syntactically invalid.

Why the other options are wrong:

A. Runtime error — Syntax errors are caught before running.

B. Missing file — That is FileNotFoundError.

D. Division by zero — That is ZeroDivisionError.

Lesson 12.1 Practice

Q1. Answer: C. A list of function calls leading to an error

A traceback shows the call chain where the error happened.

Q2. Answer: D. At the bottom (last line)

The last line shows the error type and message.

Q3. Answer: B. A variable was used before it was defined

NameError means a name is not defined.

Q4. Answer: C. An operation was used on the wrong type

TypeError is about wrong types, like adding int and str.

Q5. Answer: B. The file and line number where the error started

It shows the file and line number.

Q6. Answer: True

That is the most important line to read.

Q7. Output: TypeError

You cannot add int and str; Python raises TypeError.

Q8. Bug: The variable x was used before being defined.

Define x first: x = 5.

Q9. Answer:

A traceback is a report showing the line of code and the chain of calls that led to an error.

Q10. Answer:

print(10 / 0)

Lesson 12.2 Check

Correct answer: D. except

Why: except catches exceptions.

Why the other options are wrong:

A. catch — Used in Java/JS, not Python.

B. handle — Not a keyword.

C. rescue — Used in Ruby.

Correct answer: C. try: ... except: ...

Why: This is the correct syntax.

Why the other options are wrong:

A. catch/throw — Not Python syntax.

B. try/rescue — Not Python.

D. handle/error — Not Python.

Correct answer: A. The except block is skipped

Why: If no error, except does not run.

Why the other options are wrong:

B. Runs anyway — Only runs on error.

C. Error — No error.

D. Crash — No crash.

Correct answer: D. A bare except can hide bugs

Why: Catching specific types lets unexpected errors surface.

Why the other options are wrong:

A. Faster — Not about speed.

B. Less memory — Not about memory.

C. Required — Not required, but recommended.

Lesson 12.2 Practice

Q1. Answer: A. Catches errors so the program can keep running

try/except catches and handles errors.

Q2. Answer: C. try

try starts the risky block.

Q3. Answer: C. except

except handles the error.

Q4. Answer: D. The except block is skipped

except only runs if an error occurred.

Q5. Answer: C. To handle only that error and let others crash

Specific catches let other errors surface.

Q6. Answer: True

That is how errors are caught.

Q7. Output: Nope

The division raises ZeroDivisionError, which is caught and "Nope" is printed.

Q8. Bug: There is no except block to handle the error, so the program will crash.

Add an except block: except ZeroDivisionError: print("error").

Q9. Answer:

try/except lets you run risky code and catch errors so the program can keep going instead of crashing.

Q10. Answer:

```
try:
print(10 / 0)
except ZeroDivisionError:
print("Cannot divide by zero")
```

Lesson 12.3 Check

Correct answer: B. Runs no matter what

Why: finally normally runs whether or not an exception occurs. It is not guaranteed to run if the program is terminated abruptly.

Why the other options are wrong:

A. No error — That is else.

C. Error — finally runs in both cases.

D. Ends program — It runs code, not ends.

Correct answer: A. Runs if NO error occurs

Why: else runs only on success.

Why the other options are wrong:

B. Error — That is except.
C. Always — That is finally.
D. Never — It runs on success.

Correct answer: A. finally

Why: finally is for cleanup that normally runs whether or not an error occurs.

Why the other options are wrong:

B. try — try is the risky code.
C. except — except handles errors.
D. else — else is for success code.

Correct answer: B. Yes

Why: You can have all four blocks.

Why the other options are wrong:

A. No — You can.
C. Only try/except — You can have else and finally too.
D. Only Python 3 — Always.

Lesson 12.3 Practice

Q1. Answer: D. Runs its block normally, error or not

finally normally runs whether or not an error occurs (not guaranteed to run if the program is terminated abruptly).

Q2. Answer: A. Runs only if no error occurred in try

else runs when try succeeds with no error.

Q3. Answer: A. Only when the try block did not raise an error

else is for the success case.

Q4. Answer: B. Normally, error or no error

finally normally runs whether or not an error occurs (not guaranteed to run if the program is terminated abruptly).

Q5. Answer: D. Closing a file or cleaning up

finally is great for cleanup that normally runs whether or not an error occurs.

Q6. Answer: True

finally normally executes whether or not an error occurs.

Q7. Output: ok done

No error, so else runs; finally normally runs.

Q8. Bug: else must come before finally in a try block.

Order: try, except, else, finally.

Q9. Answer:

else runs only when the try block succeeded; finally normally runs whether or not an error occurred.

Q10. Answer:

```
try:
x = 1
except:
print("err")
else:
print("ok")
finally:
print("done")
```

Lesson 12.4 Check

Correct answer: B. raise

Why: raise throws an exception.

Why the other options are wrong:

A. throw — Used in Java/JS.

C. error — Not a keyword.

D. fail — Not a keyword.

Correct answer: B. raise ValueError("message")

Why: This is the correct syntax.

Why the other options are wrong:

A. throw — Not Python.

C. error — Not Python.

D. raise error — Wrong syntax.

Correct answer: B. So the user understands what went wrong

Why: A helpful message explains the problem.

Why the other options are wrong:

A. Speed — Not about speed.

C. Required — Not required, but strongly recommended.

D. Memory — Not about memory.

Correct answer: A. The exception propagates until caught

Why: If not caught, the program crashes with a traceback.

Why the other options are wrong:

B. Continues normally — Only if caught.

C. Restarts — Does not restart.

D. Nothing — Something happens.

Lesson 12.4 Practice

Q1. Answer: D. raise

raise throws an exception in Python.

Q2. Answer: B. raise ValueError("bad")

raise ExceptionType("message") is the form.

Q3. Answer: A. To signal a problem with a helpful message

Custom raises communicate problems clearly.

Q4. Answer: C. An exception object (like ValueError("..."))

raise needs an exception instance.

Q5. Answer: A. The program stops at that line and looks for a matching except

raise jumps to the nearest matching except, or stops the program.

Q6. Answer: True

raise ValueError("message") is common.

Q7. Output: caught: bad input

The raised ValueError is caught and its message is printed.

Q8. Bug: raise needs an exception object, not a plain string.

Use raise ValueError("bad input").

Q9. Answer:

You raise your own exception to signal a problem with a helpful message so the caller can handle it.

Q10. Answer:

if age < 0:
raise ValueError("age cannot be negative")

Chapter 12 Challenges - Solutions

Challenge 1

Question: Ask for a number, print 100 / number. Handle zero and invalid input.

Solution:

Use two except blocks: one for ZeroDivisionError (user enters 0) and one for ValueError (user enters text).

try: n = float(input("Enter a number: ")) result = 100 / n print(f"100 / {n} = {result}") except ZeroDivisionError: print("You cannot divide by zero!") except ValueError: print("That is not a valid number!")

If the user enters 0: "You cannot divide by zero!" If the user enters "hello": "That is not a valid number!" If the user enters 5: "100 / 5.0 = 20.0"

Each except block handles a specific error type. The user never sees a traceback — just a friendly message.

Challenge 2

Question: What is the difference between an error and an exception?

Solution:

In Python, the word "error" is an informal everyday term for anything that goes wrong in a program. An "exception" is a specific Python mechanism: Python represents many program failures as exception objects that you can catch and handle with try/except.

Most runtime failures are exceptions: ValueError, TypeError, ZeroDivisionError, KeyError, FileNotFoundError. These happen when the program is running, and you can handle them gracefully.

SyntaxError is a special case. In Python 3, SyntaxError IS technically a subclass of Exception, so it can be caught with try/except in some situations (for example, when using eval() or importing a module with a syntax error). However, in normal code you write directly, a SyntaxError is caught at compile time — before your program starts running — so a try/except block around the code with the error will NOT catch it. This is why beginners usually think of SyntaxError as different from runtime exceptions: it stops the program before it even begins.

Another special case is SystemExit (raised by sys.exit()), which bypasses most exception handling.

In everyday programming, you mostly deal with runtime exceptions — errors you can catch and handle. The distinction between compile-time and runtime errors is subtle and mostly matters in advanced cases.

Challenge 3

Question: Write get_age() that keeps asking until a valid integer is entered.

Solution:

Use while True with try/except. Break out of the loop on success.

def get_age(): while True: try: age = int(input("Enter your age: ")) return age except ValueError: print("That is not a number. Try again.")

my_age = get_age() print(f"Your age is {my_age}.")

The loop runs forever (while True) until the user enters a valid integer. If int() raises a ValueError, the except block prints a message and the loop repeats. If int() succeeds, the return statement exits the function (and the loop) with the valid age.

This pattern — while True with try/except and return or break on success — is the standard way to validate user input in Python.

Challenge 4

Question: Is try: x = 1/0 except: print("Error") good practice?

Solution:

No, it is NOT good practice.

The problem is the BARE except (no exception type specified). A bare except catches EVERY kind of exception, including: - ZeroDivisionError (which you expected) - NameError (if you misspelled a variable) - TypeError (if you used the wrong type) - KeyboardInterrupt (if the user pressed Ctrl+C to stop the program) - Any other bug in the try block

This hides bugs. If there is a typo in the try block, you would see "Error" instead of the actual problem, making debugging very difficult.

Better practice — catch the specific exception: try: x = 1 / 0 except ZeroDivisionError: print("Cannot divide by zero!")

Now only ZeroDivisionError is caught. Any other error (like a NameError from a typo) will still crash the program with a helpful traceback, so you can find and fix the bug.

Challenge 5

Question: Ask for a filename, try to open it, give 3 tries. Use FileNotFoundError.

Solution:

Use a counter and a while loop with try/except.

tries = 0 while tries < 3: filename = input("Enter filename: ") try: with open(filename, "r") as f: content = f.read() print(content) break # success — exit the loop except FileNotFoundError: print(f"File '{filename}' not found.") tries += 1 if tries == 3: print("Too many failed attempts. Goodbye.")

The loop runs at most 3 times. Each iteration asks for a filename and tries to open it. If the file exists, it reads and prints the content, then breaks out. If the file does not exist, FileNotFoundError is caught, a message is printed, and the counter increments. After 3 failures, the program gives up.

Lesson 4.5 Challenges - Solutions

Challenge 1

Question: Roll two dice and print both rolls and their sum.

Solution:

Import random, call randint(1, 6) twice, then add.

import random dice1 = random.randint(1, 6) dice2 = random.randint(1, 6) total = dice1 + dice2 print(f"You rolled {dice1} and {dice2}") print(f"Total: {total}")

Each call to random.randint(1, 6) simulates one die. The sum gives a value between 2 and 12, just like rolling two real dice.

Challenge 2

Question: What does round(math.pi, 2) print?

Solution:

It prints: 3.14

math.pi is a constant in the math module that holds the value of pi (approximately 3.14159265...). The round() function takes two arguments: the number to round, and the number of decimal places. round(3.14159..., 2) rounds to 2 decimal places, giving 3.14.

This shows how you can combine a module constant (math.pi) with a built-in function (round) in a single expression.

Challenge 3

Question: Ask for a number, print its square root, handle negatives.

Solution:

Use math.sqrt() for the calculation, and check if the number is negative before calling it.

import math n = float(input("Enter a number: ")) if n < 0: print("Cannot take the square root of a negative number.") else: result = math.sqrt(n) print(f"The square root of {n} is {result}")

math.sqrt() raises a ValueError if you give it a negative number, so the if statement prevents that error from occurring. This is a good example of defensive programming: checking for a problem before it happens.

Challenge 4

Question: Difference between random.random() and random.randint(1, 10)?

Solution:

random.random() returns a random floating-point number between 0.0 and 1.0 (never exactly 1.0). For example: 0.7234 or 0.0015.

random.randint(1, 10) returns a random integer between 1 and 10 (inclusive of both). For example: 3 or 7 or 10.

The key differences: - random() returns a float; randint() returns an int. - random() always ranges from 0 to 1; randint() lets you choose the range. - random() is useful for probabilities (like a 50% chance); randint() is useful for games (like dice rolls).

Challenge 5

Question: Pick a random name from 5 friends and print the winner.

Solution:

Create a list of names, then use random.choice() to pick one.

import random friends = ["Alex", "Maya", "Sam", "Lisa", "Bob"] winner = random.choice(friends) print(f"The winner is: {winner}!")

random.choice() takes a list (or any sequence) and returns one randomly selected item. It is perfect for this kind of task because you do not need to generate a random index yourself Python does it for you.

Mixed Quiz Answers

These are the answers to the Mixed Quizzes at the end of each chapter. For each multiple-choice question, you will see the correct answer, why it is correct, and why every other option is wrong.

Chapter 1 — Introduction to Python

Question 1

Correct answer: B. Guido van Rossum

Why: Guido van Rossum, a Dutch programmer, created Python and first released it in 1991. He remained its "Benevolent Dictator For Life" until 2018.

Why the other options are wrong:

A. Linus Torvalds — Linus Torvalds created the Linux kernel, not Python.

C. Bill Gates — Bill Gates co-founded Microsoft; he had nothing to do with creating Python.

D. Ada Lovelace — Ada Lovelace is considered the first computer programmer (in the 1840s), long before Python existed.

Question 2

Correct answer: C. Displays text on the screen

Why: The print() function takes whatever is inside the parentheses and shows it on the screen. For example, print("Hi") displays Hi.

Why the other options are wrong:

A. Sends a document to a printer — Despite the name, print() has nothing to do with paper printers. It prints to the screen.

B. Reads input from the keyboard — Reading input is the job of the input() function, not print().

D. Saves a file to disk — Saving files uses the open() function with mode "w", covered in Chapter 11.

Question 3

Correct answer: A. It marks the start of a comment

Why: The # symbol tells Python to ignore everything from there to the end of the line. This lets you write notes to yourself.

Why the other options are wrong:

B. Creates a hashtag for social media — Python does not know about social media. The # symbol is purely for comments.

C. Multiplies two numbers — Multiplication uses the * operator, not #.

D. Marks an error in your code — # is a normal, useful symbol in Python. It does not mean anything is wrong.

Question 4

Correct answer: D. .p

Why: Python files use the .py extension (for example, hello.py). This tells the computer the file contains Python code.

Why the other options are wrong:

A. .pyt — There is no standard .pyt extension for Python files.

C. .python — Although it sounds logical, Python files never use this extension.

B. .py — The .p extension is not used for Python source files.

Question 5

Correct answer: C. IDLE

Why: IDLE (Integrated Development and Learning Environment) is the simple editor that comes bundled with Python when you install it.

Why the other options are wrong:

A. Python Studio — There is no tool called "Python Studio" bundled with Python.

B. PyCharm — PyCharm is a separate, more advanced editor made by JetBrains. It is not bundled with Python.

D. Visual Studio — Visual Studio is Microsoft's large development environment. It is not bundled with Python.

Question 6

Correct answer: D. 2010

Why: Python was first released in February 1991 by Guido van Rossum.

Why the other options are wrong:

A. 1985 — In 1985, Guido was still working on the ABC language. Python did not exist yet.

C. 2000 — By 2000, Python was already 9 years old. Python 2.0 was released in October 2000.

B. 1991 — Python was already 19 years old by 2010.

Question 7

Correct answer: D. Ctrl+Enter

Why: In IDLE, pressing F5 runs the program currently open in the editor window. You can also click Run > Run Module.

Why the other options are wrong:

A. F1 — F1 opens the help system in most programs, including IDLE.

C. Ctrl+R — Ctrl+R is not the run shortcut in IDLE.

B. F5 — Ctrl+Enter does not run programs in IDLE.

Question 8

Correct answer: False

Why: Python was named after the British comedy group Monty Python. Guido van Rossum is a fan of the show and wanted a name that was short, unique, and slightly mysterious.

Why the other answer is wrong: The snake connection is a coincidence. The language is named after Monty Python's Flying Circus, the British comedy series.

Question 9

Correct answer: True

Why: Yes. Everything from the # to the end of the line is completely ignored by Python. Comments exist only for humans reading the code.

Question 10

Correct answer: True

Why: Yes. Text (strings) must be wrapped in quotation marks so Python knows it is text and not a variable name or keyword.

Question 11

Correct answer: False

Why: A single print() call prints on one line by default. To print on multiple lines, you can include \n in the string, or use the end argument, or use multiple print() calls.

Question 12

Model answer:

A programming language is a way of writing instructions that a computer can understand and follow. Just as a recipe tells a cook what steps to follow to bake a cake, a program tells the computer what steps to follow to accomplish a task.

Python is one particular programming language, designed to be easy for humans to read and write. Other examples of programming languages include JavaScript, Java, C++, and Scratch. Each has its own rules and its own strengths, but they all share the same basic purpose: giving instructions to a computer.

Question 13

Model answer:

Comments are useful for two main reasons:

First, they let you explain what your code does. When you write a program, the code itself shows the computer what to do, but it does not always show why. A comment like "# Calculate the average of the student's scores" tells a future reader (including yourself, three weeks later) what the next few lines are for. This makes the code much easier to understand and maintain.

Second, comments let you temporarily turn off a line of code without deleting it. Programmers call this "commenting out" a line. It is very useful when you are testing things: if you suspect a particular line is causing a bug, you can comment it out, run the program, and see if the bug disappears. If it does, you have found the problem.

Question 14

Model answer:

Explanation: The program should use three print() calls, each with a different string. Each call prints one line.

PYTHON · model_answer.py

```
# A simple introduction program
print("My name is Alex.")
print("My favourite subject is Maths.")
print("I hope to learn how to build my own game.")
```

Question 15

Model answer:

Explanation: A single line can have both code and a comment. Python runs the code and ignores the comment.

PYTHON · model_answer.py

```
print("Hello!") # This line prints a greeting to the screen
```

Chapter 2 — Variables and Data Types

Question 1

Correct answer: C. player_score

Why: player_score uses only letters and an underscore, does not start with a digit, and is not a Python keyword. It is a valid variable name.

Why the other options are wrong:

A. 2cool — Variable names cannot start with a digit. The rule is: letters, digits, or underscores only, but the first character cannot be a digit.

B. player score — Variable names cannot contain spaces. Use an underscore instead: player_score.

D. for — for is a Python keyword (used in for loops). You cannot use keywords as variable names.

Question 2

Correct answer: D. bool

Why: 3.14 has a decimal point, which makes it a floating-point number (float). Whole numbers are int; text is str.

Why the other options are wrong:

A. int — int is for whole numbers like 3 or 42. Because 3.14 has a decimal point, it is a float, not an int.

C. str — str is for text wrapped in quotation marks. 3.14 is a number, not text.

B. float — bool is for True/False values only. 3.14 is a number.

Question 3

Correct answer: A. 3

Why: 10 % 3 means "the remainder when 10 is divided by 3". 10 divided by 3 is 3 with 1 left over, so the answer is 1.

Why the other options are wrong:

B. 1 — This is the quotient (10 // 3 = 3), not the remainder.

C. 3.33 — This is the result of true division (10 / 3), not modulo.

D. 0 — This would only be the answer if 10 were exactly divisible by 3, which it is not.

Question 4

Correct answer: A. "hahaha"

Why: Multiplying a string by an integer repeats the string that many times. So "ha" * 3 gives "ha" + "ha" + "ha" = "hahaha".

Why the other options are wrong:

B. "ha ha ha" — Multiplication does not add spaces. It just repeats the string exactly.

C. an error — This operation is perfectly valid in Python. String * integer is a built-in feature.

D. "ha3" — Multiplication does not concatenate the number to the string. It repeats the string.

Question 5

Correct answer: D. ===

Why: The double equals sign == means "is equal to". It compares two values and returns True or False.

Why the other options are wrong:

A. = — A single = is the assignment operator. It puts a value into a variable (x = 5), it does not compare.

C. != — != means "not equal to", the opposite of ==.

B. == — Python does not use === (that is JavaScript). In Python, == is the comparison operator.

Question 6

Correct answer: C. bool

Why: True and False are the only two values of the boolean type (bool). Note the capital letters: True, not true.

Why the other options are wrong:

A. str — Strings are text in quotation marks. True is not a string; it is a special keyword.

B. int — Although booleans can act like 0 and 1 in some contexts, their type is bool, not int.

D. truth — "truth" is not a Python type. The type is called "bool".

Question 7

Correct answer: B. <class 'float'>

Why: 3.14 is a float, so type(3.14) returns <class 'float'>. The type() function tells you what type a value is.

Why the other options are wrong:

A. <class 'int'> — This would be the answer for a whole number like 42.

C. <class 'str'> — This would be the answer for text like "hello".

D. <class 'num'> — There is no type called "num" in Python. Numbers are either int or float.

Question 8

Correct answer: False

Why: Python is case-sensitive. age, Age, and AGE are three completely different variables. This is why consistent naming matters.

Why the other answer is wrong: They are three separate variables. Mixing them up is a common source of bugs, so pick a style (usually lowercase) and stick to it.

Question 9

Correct answer: True

Why: 10 // 3 is floor (integer) division. It gives the integer part of the quotient, which is 3. (10 / 3 would give 3.333...; 10 // 3 rounds down.)

Question 10

Correct answer: False

Why: int("hello") raises a ValueError because "hello" cannot be interpreted as a number. Only strings that look like integers (like "42") can be converted with int().

Why the other answer is wrong: The conversion fails. Python does not know how to turn the word "hello" into a number, so it raises an error.

Question 11

Correct answer: True

Why: Underscores are allowed in variable names. _score, player_name, and __init__ are all valid. Underscores are often used to separate words in multi-word names.

Question 12

Model answer:

A single equals sign = is the assignment operator. It puts a value into a variable. For example, x = 5 means "store the value 5 in the variable x".

A double equals sign == is the comparison operator. It asks "are these two things equal?" and returns True or False. For example, x == 5 means "is x equal to 5?" and does not change x.

Mixing them up is one of the most common beginner mistakes. Get into the habit early: use = when you want to set a value, and == when you want to compare two values.

Question 13

Model answer:

Names like x and y are bad because they do not describe what the variable holds. When you read the code later (or when someone else reads it), you have no idea what x is supposed to be — a score? a coordinate? a counter? This makes the code hard to understand and easy to break.

Instead, use descriptive names that tell the reader what the variable represents. For example: score, player_name, total_price, number_of_students. These names are self-documenting: you can read the code and understand what each variable means without having to trace through every line.

Long, clear names are better than short, mysterious ones. The extra typing is worth it.

Question 14

Model answer:

Explanation: Create three variables of different types, then print each one.

PYTHON · model_answer.py

```
name = "Alex" # string
age = 14 # integer
height = 1.65 # float

print(name)
print(age)
print(height)
```

Question 15

Model answer:

Explanation: Convert the string "42" to an integer using int(), then do maths with it. The string must be converted first because Python cannot add an integer to a string directly.

PYTHON · model_answer.py

```
s = "42" # a string
n = int(s) # convert to integer
result = n + 8 # now we can do maths: 42 + 8
print(result) # prints 50
```

Chapter 3 — Operators and Expressions

Question 1

Correct answer: D. 9

Why: Multiplication has higher precedence than addition, so Python computes 3 * 4 = 12 first, then 2 + 12 = 14.

Why the other options are wrong:

A. 20 — This would be the answer if you did addition first: (2 + 3) * 4 = 20. But multiplication comes first.

C. 24 — This would be 2 * 3 * 4, but the operator is +, not *.

B. 14 — This is 2 + 3 + 4, ignoring the multiplication entirely.

Question 2

Correct answer: A. 3

Why: 17 % 5 means "the remainder when 17 is divided by 5". 17 divided by 5 is 3 with 2 left over, so the answer is 2.

Why the other options are wrong:

B. 2 — This is the quotient (17 // 5 = 3), not the remainder.

C. 3.4 — This is the result of true division (17 / 5), not modulo.

D. 12 — This is 17 - 5, which is unrelated to the modulo operation.

Question 3

Correct answer: D. None

Why: The and operator returns True only if both sides are True. Since False is one of the sides, the whole expression is False.

Why the other options are wrong:

A. True — For "and" to return True, both sides must be True. Here, one side is False, so the answer cannot be True.

C. An error — True and False is a perfectly valid expression. No error occurs.

B. False — and always returns a boolean (True or False), never None.

Question 4

Correct answer: D. 36

Why: The ** operator is right-associative, so 2 ** 3 ** 2 is evaluated as 2 ** (3 ** 2) = 2 ** 9 = 512.

Why the other options are wrong:

A. 64 — This would be (2 ** 3) ** 2 = 8 ** 2 = 64. But ** associates right-to-left, so the right side is done first.

C. 12 — This would be 2 * 3 * 2, but ** is exponentiation, not multiplication.

B. 512 — This is (2+3)**2 or similar, none of which match the actual expression.

Question 5

Correct answer: D. score =+ 5

Why: The += operator is augmented assignment. score += 5 is exactly equivalent to score = score + 5.

Why the other options are wrong:

A. score + 5 — This is just an expression that produces a value. It does not change score, and it is not a statement.

C. score++ 5 — Python does not have a ++ operator. (Languages like C++ and Java do, but Python does not.)

B. score += 5 — This is valid Python, but it means score = (+5), i.e. score = 5. It does not add 5 to the old value.

Question 6

Correct answer: True

Why: Python allows chained comparisons. 5 > 3 > 1 is equivalent to (5 > 3) and (3 > 1), which is True and True = True.

Question 7

Correct answer: False

Why: 5 > 3 is True. The not operator flips the boolean, so not True = False. The parentheses do not change this.

Why the other answer is wrong: not (5 > 3) evaluates to not True, which is False.

Question 8

Correct answer: False

Why: Exponentiation (**) has higher precedence than multiplication (*). So 2 * 3 ** 2 is 2 * 9 = 18, not (2 * 3) ** 2 = 36.

Why the other answer is wrong: ** is done before *. This matches the maths convention that powers bind tighter than multiplication.

Question 9

Correct answer: A. 13

Why: Following precedence: 3 * 4 = 12 first, then 2 + 12 - 1 = 13.

Why the other options are wrong:

B. 14 — This would be 2 + 3 * 4, ignoring the - 1 at the end.

C. 23 — This would be (2 + 3) * 4 - 1, but addition is not done before multiplication.

D. 11 — This would be 2 + 3 * (4 - 1) or similar, but subtraction is done last.

Question 10

Correct answer: A. True

Why: and has higher precedence than or. So True or False and False is evaluated as True or (False and False) = True or False = True.

Why the other options are wrong:

B. False — This would be the answer if it were evaluated as (True or False) and False = True and False = False. But "and" binds tighter than "or".

C. An error — The expression is valid. No error occurs.

D. None — Logical operators always return booleans here.

Question 11

Correct answer: True

Why: 10 != 10 asks "is 10 not equal to 10?" Since 10 is equal to 10, the answer is False.

Question 12

Correct answer: True

Why: 5 is less than 10, so 5 < 10 evaluates to True.

Question 13

Model answer:

The modulo operator % gives the remainder after division. For example, 10 % 3 is 1, because 10 divided by 3 equals 3 with 1 left over. Similarly, 17 % 5 is 2, because 17 = 3 * 5 + 2.

A very common real-world use of modulo is checking whether a number is even or odd. A number is even if it divides exactly by 2, which means number % 2 == 0. A number is odd if number % 2 == 1. For example, 8 % 2 == 0 (even) and 7 % 2 == 1 (odd).

Another use is in clocks and calendars: to convert 27 hours into days and hours, you would use 27 // 24 = 1 day and 27 % 24 = 3 hours.

Question 14

Model answer:

Python has rules of precedence so that every expression has exactly one unambiguous meaning. Without these rules, an expression like 2 + 3 * 4 could be interpreted in two ways: as (2 + 3) * 4 = 20, or as 2 + (3 * 4) = 14. Different programmers might get different results, and the language would be unusable.

Precedence rules give a single, consistent answer: multiplication is done before addition, so the result is always 14. These rules match the conventions of mathematics, so most expressions behave the way you would expect.

When you want a different order, you can use parentheses to override the default. (2 + 3) * 4 forces the addition first, giving 20. Using parentheses also makes your intent clear to anyone reading the code, which is why many programmers add them even when they are not strictly necessary.

Question 15

Model answer:

Explanation: The number must be odd (n % 2 == 1) and greater than 10 (n > 10). Both conditions are combined with and.

PYTHON · model_answer.py

```
n = int(input("Enter a number: "))
print(n % 2 == 1 and n > 10)
```

Chapter 4 — Input and Output

Question 1

Correct answer: C. str

Why: input() always returns a string, even if the user types a number. You must convert with int() or float() if you need a number.

Why the other options are wrong:

A. int — input() never returns an int directly. You must wrap it in int() to convert.

B. float — input() never returns a float directly. You must wrap it in float() to convert.

D. Whatever the user typed — The user's input is always returned as a string, regardless of what they typed.

Question 2

Correct answer: C. int(input())

Why: Wrap input() inside int() to convert the string the user typed into an integer. For example: age = int(input("Age? ")).

Why the other options are wrong:

A. int_input() — There is no function called int_input(). The correct pattern is to nest int() around input().

B. input(int) — This tries to pass the int function as a prompt to input(), which is not what you want.

D. input().to_int() — Strings in Python do not have a to_int() method. Use the int() function instead.

Question 3

Correct answer: C. a-b-c

Why: The sep argument tells print() what to put between the arguments. sep="-" puts a hyphen between them, giving a-b-c.

Why the other options are wrong:

A. a b c — This is the default output with sep=" " (a space).

B. abc — This would require sep="", an empty separator.

D. a, b, c — This would require sep=", ".

Question 4

Correct answer: A. f"Hello {name}"

Why: An f-string is a normal string prefixed with the letter f. Expressions inside curly braces { } are evaluated and inserted.

Why the other options are wrong:

B. "Hello {name}"f — The f must come before the opening quote, not after the closing quote.

C. "f-Hello {name}" — This is just a regular string starting with the letter f. The braces are not evaluated.

D. format("Hello {name}") — The older .format() method works, but the syntax shown here is wrong. It would be "Hello {name}".format(name=name).

Question 5

Correct answer: D. 3

Why: The format specifier .2f means "format as a float with 2 decimal places". 3.14159 rounded to 2 decimals is 3.14.

Why the other options are wrong:

A. 3.14159 — This would be the output without any format specifier.

C. 3.1 — This would be the output with .1f (one decimal place).

B. 3.14 — This would be the output with .0f (no decimal places).

Question 6

Correct answer: True

Why: The end argument controls what is printed after the last argument. By default it is a newline ("\n"), but you can change it to anything, including an empty string.

Question 7

Correct answer: False

Why: The default separator is a single space (" "), not a comma. You can change it with the sep argument.

Why the other answer is wrong: The default is a space. To use a comma, you would write sep=", ".

Question 8

Correct answer: True

Why: Yes. Any valid Python expression can go inside the curly braces of an f-string, including arithmetic, function calls, and method calls.

Question 9

Correct answer: D. None of the above

Why: input() returns the text the user typed (without the prompt). The prompt is shown on the screen but is not part of the returned value.

Why the other options are wrong:

A. The integer 4 — input() never returns an integer. It always returns a string.

C. The string "Name: Maya" — The prompt is not included in the returned value. Only what the user typed is returned.

B. The string "Maya" — Option B is correct.

Question 10

Correct answer: C. f"${price:.2f}"

Why: The .2f format specifier formats a number as a float with exactly 2 decimal places, which is what you want for money.

Why the other options are wrong:

A. f"${price}" — This prints the number with whatever decimal places it happens to have, which may be 0 or many.

B. f"${price:2}" — The :2 specifier sets a minimum width of 2 characters; it does not control decimal places.

D. f"${price:float.2}" — This is not valid f-string syntax. The correct specifier is :.2f.

Question 11

Correct answer: True

Why: Yes. When Python reaches input(), it stops and waits for the user to type a line and press Enter. Only then does the program continue.

Question 12

Correct answer: True

Why: Yes. You can pass any number of arguments to print(), separated by commas. They are printed on the same line, separated by the sep value (a space by default).

Question 13

Model answer:

input() always returns a string. So if the user types 14, the variable age holds the string "14", not the integer 14. When you then try age + 1, Python tries to add a string and an integer, which it cannot do, so it raises a TypeError.

The fix is to convert the string to an integer before doing arithmetic. Wrap input() in int(): age = int(input("Age? ")). Now age holds the integer 14, and age + 1 gives 15 with no error.

Question 14

Model answer:

You use the end argument when you want the next print() call to continue on the same line, instead of starting a new line.

For example, without end, two print() calls produce two lines: print("Loading") # prints "Loading" then a newline print("Done") # prints "Done" on the next line

With end="", the first print does not add a newline, so "Done" appears right after "Loading" on the same line: print("Loading", end="") # prints "Loading" with no newline print("Done") # prints "Done" on the same line

Result: "LoadingDone".

You can also use end="... " to add specific text after a print, which is useful for building progress bars or prompts.

Question 15

Model answer:

Explanation: Read the price (as a float) and quantity (as an integer), then multiply them and format the total with two decimal places using an f-string.

PYTHON · model_answer.py

```
price = float(input("Price per item: "))
qty = int(input("Quantity: "))
total = price * qty
print(f"Total: ${total:.2f}")
```

Chapter 5 — Making Decisions — Conditionals

Question 1

Correct answer: B. the colon :

Why: Every if, elif, and else line must end with a colon. The colon tells Python "the condition is over, the block starts on the next indented line".

Why the other options are wrong:

- **A.** ; — Python does not use semicolons to end statements (unlike C or Java).
- **C.** , — A comma is used to separate items in a list or arguments to a function, not to end an if line.
- **D.** -> — The -> symbol is used for function return type annotations in modern Python, not for if statements.

Question 2

Correct answer: C. indentation

Why: Python uses indentation (usually 4 spaces) to define which lines belong to a block. All lines indented the same amount are part of the same block.

Why the other options are wrong:

- **A.** curly braces { } — Languages like C, Java, and JavaScript use curly braces. Python does not.
- **B.** parentheses ( ) — Parentheses are used for grouping expressions and function calls, not for defining blocks.
- **D.** the word "end" — Some languages (like Ruby or Lua) use explicit end keywords. Python uses indentation instead.

Question 3

Correct answer: D. elsf

Why: elif is short for "else if". It lets you check multiple conditions in a chain without nesting if statements.

Why the other options are wrong:

A. elseif — Python does not use "elseif". Other languages like PHP do, but Python uses the shortened "elif".

C. else if — Two words do not work in Python. You must use the single keyword "elif".

B. elif — This is not a Python keyword at all.

Question 4

Correct answer: A. A

Why: 5 > 3 is True, so the if block runs and prints "A". The else block is skipped entirely.

Why the other options are wrong:

B. B — This would be printed only if the condition were False. 5 > 3 is True.

C. AB — Only one block runs in an if/else. You cannot get both.

D. Nothing — The if block always runs when the condition is True. Something is always printed.

Question 5

Correct answer: A. 1 <= x <= 10

Why: Python allows chained comparisons. 1 <= x <= 10 is equivalent to (1 <= x) and (x <= 10), and it is the cleanest way to express "x is between 1 and 10 inclusive".

Why the other options are wrong:

B. x > 1 and x < 10 — This excludes 1 and 10. The question asks for "inclusive", so endpoints should be allowed.

C. x >= 1 or x <= 10 — Using "or" here is wrong. Almost every number satisfies this (e.g. -100 is <= 10, 1000 is >= 1). You need "and".

D. 1 < x < 10 — Like option B, this excludes the endpoints 1 and 10.

Question 6

Correct answer: True

Why: You can have as many elif blocks as you need. For example, a grading program might have elif for grades B, C, D, and F.

Question 7

Correct answer: False

Why: The else block only runs if NO if or elif condition was True. If an elif block runs, the else is skipped.

Why the other answer is wrong: Once any branch (if or elif) matches and runs, the entire if statement is finished. The else is only a fallback when nothing else matched.

Question 8

Correct answer: False

Why: Indentation is required in Python and is part of the language syntax. Wrong indentation causes IndentationError or changes the meaning of your code.

Why the other answer is wrong: In Python, indentation IS the syntax for blocks. The interpreter uses it to decide which lines belong to which block.

Question 9

Correct answer: A. A

Why: 7 > 5 is True, so the if block runs and prints "A". The elif and else are not even evaluated (first-match branching).

Why the other options are wrong:

- **B.** B — The elif is only checked if the if is False. Since 7 > 5 is True, the elif is skipped.
- **C.** C — The else only runs if both the if and elif are False. Here the if is True.
- **D.** AB — Only one branch of an if/elif/else chain can run.

Question 10

Correct answer: C. Only when both a and b are True

Why: The and operator returns True only when both operands are True. If either is False, the whole expression is False.

Why the other options are wrong:

- **A.** Only when a is True — This describes "if a:" — "and b" would be ignored. With "and", both must be True.
- **B.** Only when b is True — Same issue: both sides must be True for "and" to return True.
- **D.** When either a or b is True — This describes the "or" operator, not "and".

Question 11

Correct answer: True

Why: Yes, you can nest if statements inside other if statements. This is useful when you need to check a second condition only if the first one is true.

Question 12

Correct answer: True

Why: An if without an else is perfectly valid. Sometimes you only want to do something when a condition is true, and do nothing otherwise.

Question 13

Model answer:

Nesting if statements means putting one if inside another. This is useful when the second condition only matters if the first one is true. For example:

if age >= 18: if has_id == "yes": print("Welcome in!")

Here, the second check (has_id) is only done if the first (age) passes.

Using "and" combines multiple conditions into a single if statement. This is cleaner when both conditions are independent and must both be true:

if age >= 18 and has_id == "yes": print("Welcome in!")

As a rule of thumb: if the conditions are independent, use "and" for clarity. If the second condition only makes sense after the first one is true (for example, checking a value that only exists if the first check passed), use nesting.

Question 14

Model answer:

Short-circuit evaluation means Python stops evaluating a Boolean expression as soon as its result is determined. With "and", if the first operand is False, Python does not need to evaluate the second operand. With "or", if the first operand is True, Python does not need to evaluate the second operand.

An if/elif/else chain follows a related but different rule: Python checks conditions from top to bottom and executes only the first branch whose condition is True. Once that branch runs, the remaining branches are skipped.

For example:

if score >= 90: grade = "A" elif score >= 80: grade = "B"

Inside the elif branch, we do not need to add "score < 90". If the score had been 90 or higher, the first branch would already have run. This is called first-match branching, not short-circuit evaluation.

Question 15

Model answer:

Explanation: Use three branches: one for positive (> 0), one for negative (< 0), and one for the remaining case (== 0).

PYTHON · model_answer.py

```
n = int(input("Enter a number: "))
if n > 0:
    print("positive")
elif n < 0:
    print("negative")
else:
    print("zero")
```

Chapter 6 — Loops

Question 1

Correct answer: D. 0

Why: range(5) produces 5 numbers: 0, 1, 2, 3, 4. The loop body runs once for each number, so 5 times.

Why the other options are wrong:

A. 4 — This would be range(4), which gives 0,1,2,3.

C. 6 — This would be range(6), which gives 0,1,2,3,4,5.

B. 5 — range(5) gives 5 values starting from 0, so the loop definitely runs.

Question 2

Correct answer: D. 1, 2, 3, 4

Why: range(1, 5) starts at 1 and stops BEFORE 5, so it produces 1, 2, 3, 4 (four numbers).

Why the other options are wrong:

A. 1, 2, 3, 4, 5 — range stops before the second argument, so 5 is not included.

B. 2, 3, 4, 5 — This would be range(2, 6). The start and end are both wrong.

C. 0, 1, 2, 3, 4 — This would be range(5) or range(0, 5). The start here is 1, not 0.

Question 3

Correct answer: B. Exits the loop immediately

Why: break causes Python to immediately leave the loop and continue with the code after the loop. No more iterations run.

Why the other options are wrong:

A. Skips to the next iteration — This is what "continue" does, not "break".

C. Restarts the loop from the beginning — break does not restart anything; it ends the loop.

D. Pauses the loop for one second — There is no pause concept. Use time.sleep() for that, not break.

Question 4

Correct answer: A. Skips the rest of this iteration and moves to the next

Why: continue causes Python to skip the remaining lines in the loop body for this iteration and jump back to the top of the loop for the next iteration.

Why the other options are wrong:

B. Ends the loop — This is what "break" does. "continue" only skips the current iteration.

C. Restarts the program — continue has nothing to do with restarting the program.

D. Nothing, it is a comment — continue is a keyword, not a comment. Comments start with #.

Question 5

Correct answer: A. 0 1 2 3 4 5 6 7 8 9

Why: range(0, 10, 2) starts at 0, stops before 10, with a step of 2. So it produces 0, 2, 4, 6, 8 (five numbers).

Why the other options are wrong:

B. 0 2 4 6 8 — This would be range(0, 10, 1) or just range(10). The step here is 2.

C. 2 4 6 8 10 — range stops BEFORE the end value, so 10 is not included. Also, the start is 0.

D. 0 2 4 6 8 10 — 10 is not included because range stops before the end value.

Question 6

Correct answer: True

Why: If the condition of a while loop never becomes False, the loop runs forever. This is called an infinite loop. Stop it in IDLE with Ctrl+C.

Question 7

Correct answer: True

Why: You can nest loops: a for loop inside another for loop, a while inside a for, or any combination. This is useful for working with 2D structures like grids.

Question 8

Correct answer: False

Why: range(10) produces the numbers 0 through 9 (ten numbers, starting from 0). It does not include 10.

Why the other answer is wrong: range(10) gives 0,1,2,3,4,5,6,7,8,9. To include 10, use range(1, 11).

Question 9

Correct answer: A. 6

Why: range(2, 8) produces 2, 3, 4, 5, 6, 7 (six numbers). It starts at 2 and stops before 8.

Why the other options are wrong:

- **B.** 7 — This counts the numbers 2,3,4,5,6,7,8 (7 numbers) but 8 is not included.
- **C.** 8 — This would be the end value, not the count.
- **D.** 10 — Unrelated to the actual range.

Question 10

Correct answer: C. 15

Why: The loop adds 1, 2, 3, 4, 5 to total. 1+2+3+4+5 = 15. range(1, 6) gives 1,2,3,4,5.

Why the other options are wrong:

- **A.** 5 — This is just the last value of n, not the sum.
- **B.** 10 — This is the sum if range stopped at 4 (1+2+3+4).
- **D.** 21 — This is the sum if range went to 6 (1+2+3+4+5+6 = 21). But range(1, 6) excludes 6.

Question 11

Correct answer: True

Why: A for loop can iterate over any sequence directly: a list, a string, a tuple, or any other iterable. For example: for fruit in ["apple", "banana"]: print(fruit).

Question 12

Correct answer: True

Why: break only exits the innermost loop. If you have nested loops and break inside the inner loop, the outer loop continues normally.

Question 13

Model answer:

An infinite loop is a loop whose condition never becomes False, so it runs forever (or until the program is killed). The most common cause is forgetting to update the variable that the condition depends on.

For example, this is an infinite loop:

count = 1 while count <= 5: print(count) # Forgot to write count += 1!

Because count never changes, count <= 5 is always True, and the loop prints 1 forever.

In IDLE, you can stop an infinite loop by pressing Ctrl+C. This raises a KeyboardInterrupt and stops the program. In other environments, you may need to close the window or kill the process.

Question 14

Model answer:

Use a for loop when you know in advance how many times to repeat, or when you are iterating over a known sequence (a list, a string, a range of numbers). For example, printing the numbers 1 to 10, or processing each item in a shopping list, are perfect for a for loop.

Use a while loop when the number of repetitions is not known in advance and depends on a condition that changes during the loop. For example, asking the user to guess a number until they get it right: you cannot know in advance how many guesses they will need, so a while loop is the right choice:

answer = 7 guess = int(input("Guess: ")) while guess != answer: print("Wrong, try again!") guess = int(input("Guess: ")) print("Correct!")

Question 15

Model answer:

Explanation: Use a for loop with range(1, 11) to print each row of the 7 times table. f-strings make the output neat.

PYTHON · model_answer.py

```
for i in range(1, 11):
    print(f"7 x {i} = {7 * i}")
```

Chapter 7 — Functions

Question 1

Correct answer: D. define

Why: The def keyword starts a function definition. The syntax is def name(parameters): followed by an indented body.

Why the other options are wrong:

A. function — Python does not use "function" as a keyword. Some languages (like JavaScript) use "function", but Python uses "def".

C. func — This is not a Python keyword.

B. def — Python uses "def", not "define".

Question 2

Correct answer: B. Parameters are in the definition; arguments are in the call

Why: A parameter is the variable listed in the function definition. An argument is the actual value you pass when calling the function. For example, in def greet(name):, name is a parameter. In greet("Alex"), "Alex" is an argument.

Why the other options are wrong:

A. They are the same thing — They are related but different concepts. Parameters are placeholders; arguments are the actual values.

C. Arguments are in the definition; parameters are in the call — This is the opposite of the correct answer.

D. Parameters are optional; arguments are required — Both can be optional or required, depending on the function.

Question 3

Correct answer: C. Sends a value back to the caller

Why: The return keyword sends a value back to the code that called the function. The caller can then store or use that value.

Why the other options are wrong:

A. Prints a value — Printing is done by print(), not return. return does not display anything.

B. Restarts the function — return exits the function; it does not restart it.

D. Ends the program — return only ends the current function. The rest of the program continues.

Question 4

Correct answer: B. 10

Why: f(5) calls the function with x = 5. The function returns x * 2 = 5 * 2 = 10. print(10) displays 10.

Why the other options are wrong:

A. 5 — This would be the output if the function just returned x without multiplying.

C. 25 — This would be x * x (x squared), not x * 2.

D. None — The function has a return statement, so it returns a value, not None.

Question 5

Correct answer: B. local

Why: A variable created inside a function is called a local variable. It only exists inside that function and is destroyed when the function ends.

Why the other options are wrong:

A. Global — Global variables are defined outside any function and can be accessed from anywhere.

C. Static — Python does not have "static" variables in the same sense as C or Java.

D. Constant — A constant is a value that does not change. Local variables can change.

Question 6

Correct answer: True

Why: A function can have multiple return statements, often in different if/elif branches. The first one reached during execution runs, and the function exits immediately.

Question 7

Correct answer: False

Why: When return is executed, the function exits immediately. No lines after the return in the same block will run.

Why the other answer is wrong: return immediately ends the function. Any code after it in the same block is unreachable.

Question 8

Correct answer: False

Why: Parameters with default values must come AFTER parameters without defaults. Otherwise Python cannot match arguments to parameters unambiguously.

Why the other answer is wrong: For example, def f(a, b=10): is valid. def f(a=10, b): is a SyntaxError.

Question 9

Correct answer: C. 12

Why: The inner call f(3) returns 3 * 2 = 6. The outer call f(6) returns 6 * 2 = 12. So f(f(3)) = f(6) = 12.

Why the other options are wrong:

A. 6 — This is f(3), but the outer f() call is also applied.

B. 9 — This would be 3 squared, but the function multiplies by 2, not by itself.

D. 8 — This would be 3 + 5 or similar, not related to the actual function.

Question 10

Correct answer: C. The function body never runs

Why: Defining a function just tells Python what the function does. The body only runs when you call the function by name.

Why the other options are wrong:

A. The program crashes — Defining an unused function is perfectly fine. No crash.

B. Python prints a warning — Python does not warn about unused functions.

D. The function runs automatically when the program starts — Functions never run automatically. They must be called explicitly.

Question 11

Correct answer: False

Why: Local variables only exist inside their function. Once the function ends, they are destroyed. To get a value out, use return.

Why the other answer is wrong: Local variables are scoped to their function. Trying to use them outside raises a NameError.

Question 12

Correct answer: True

Why: If a function has no return statement (or just "return" with no value), it returns None by default. None is Python's way of representing "no value".

Question 13

Model answer:

print and return are easy to confuse but do very different things.

print displays text on the screen for the user to read. It does not give anything back to the program. For example:

def bad_double(x): print(x * 2)

result = bad_double(5) # prints "10" to the screen print(result + 1) # ERROR: result is None

return sends a value back to the caller, who can store it in a variable or use it in further calculations. Nothing is printed unless you explicitly call print. For example:

def good_double(x): return x * 2

result = good_double(5) # nothing printed print(result + 1) # prints 11

The key difference: print is for the user; return is for the program. A function that only prints cannot be used in expressions, because it returns None. A function that returns can be used anywhere — in expressions, as arguments to other functions, in conditions, and so on. This is why almost all useful functions use return, not print.

Question 14

Model answer:

Breaking a long program into many small functions is useful for two main reasons.

First, functions let you reuse code instead of copying it. If you need to calculate an average in five different places in your program, you write the calculation once in a function called calculate_average(), and then call that function five times. The program is shorter, and if you find a bug in the calculation, you fix it in one place instead of five.

Second, well-named functions make the code easier to read. A line like total = calculate_average(scores) communicates what is happening much more clearly than ten lines of arithmetic. The function name acts as a label, so the reader can understand the flow of the program without getting bogged down in details. When they want the details, they can look inside the function.

Question 15

Model answer:

Explanation: Define is_even to return True if n % 2 == 0 (the number is divisible by 2 with no remainder), False otherwise. Then call it with 7 and print the result.

PYTHON · model_answer.py

```
def is_even(n):
    return n % 2 == 0

result = is_even(7)
print(result) # prints False, because 7 is odd
```

Chapter 8 — Lists and Tuples

Question 1

Correct answer: A. "a"

Why: Python lists are zero-indexed, so index 1 is the SECOND item. ["a", "b", "c"][1] = "b".

Why the other options are wrong:

B. "b" — This is index 0, the first item.

C. "c" — This is index 2, the third item.

D. 1 — The question asks what [1] returns, not what index 1 is. The value at index 1 is "b".

Question 2

Correct answer: A. 1

Why: Negative indices count from the end. -1 is the last item, which in [1, 2, 3, 4] is 4.

Why the other options are wrong:

B. 4 — This would be the first item (index 0 or -4).

C. -1 — The question asks for the VALUE returned, not the index used. The value at index -1 is 4.

D. An error — Negative indices are valid in Python. -1 is the most common one.

Question 3

Correct answer: C. append()

Why: append() adds an item to the end of a list. For example: lst.append(5) adds 5 to the end of lst.

Why the other options are wrong:

A. add() — Lists do not have an add() method. Sets use add(), but lists use append().

B. insert() — insert() adds an item at a specific position, not necessarily the end. insert(len(lst), x) would work but append(x) is the standard way.

D. push() — Python lists do not have a push() method. (Some other languages do.) Use append() instead.

Question 4

Correct answer: D. [1, 2]

Why: Slicing [1:3] gives items at indices 1 and 2 (it stops before index 3). For [1, 2, 3, 4, 5], indices 1 and 2 are 2 and 3, so the result is [2, 3].

Why the other options are wrong:

A. [1, 2, 3] — This would be slice [0:3], giving indices 0, 1, 2.

C. [2, 3, 4] — This would be slice [1:4], giving indices 1, 2, 3.

B. [2, 3] — This would be slice [0:2], giving indices 0, 1.

Question 5

Correct answer: B. immutable

Why: Tuples are immutable: once you create a tuple, you cannot change, add, or remove its items. This is the main difference from lists.

Why the other options are wrong:

A. Mutable — This describes lists, not tuples. Tuples cannot be changed.

C. Can only hold numbers — Tuples can hold any type: (1, "hello", True) is a valid tuple.

D. Same as lists — Tuples and lists are similar but differ in mutability. Lists are mutable; tuples are not.

Question 6

Correct answer: True

Why: Lists are mutable. You can change an item by assigning to its index: lst[0] = "new". You can also add and remove items.

Question 7

Correct answer: False

Why: Tuples are immutable. You cannot change, add, or remove items after a tuple is created. Trying to do so raises a TypeError.

Why the other answer is wrong: Once a tuple is created, it cannot be changed. If you need to modify the contents, use a list instead.

Question 8

Correct answer: True

Why: The slice [::-1] means "start to end, step -1", which reverses the order. [1, 2, 3][::-1] gives [3, 2, 1].

Question 9

Correct answer: D. 0

Why: len() returns the number of items in a list. [1, 2, 3, 4, 5] has 5 items.

Why the other options are wrong:

A. 4 — This is the highest index, not the count. Indices go 0, 1, 2, 3, 4 for a 5-item list.

C. 6 — This would be the count if there were 6 items.

B. 5 — The list clearly has items, so len() cannot be 0.

Question 10

Correct answer: A. [1, 2, 3, 4, 5]

Why: The + operator concatenates two lists, creating a new list with all the items from both. [1, 2, 3] + [4, 5] = [1, 2, 3, 4, 5].

Why the other options are wrong:

B. [5, 7, 3] — This looks like element-wise addition, but + on lists does not do that. It concatenates.

C. [[1, 2, 3], [4, 5]] — This would be nesting the lists, not concatenating. You would write [[1,2,3], [4,5]] to get this.

D. An error — Adding two lists with + is perfectly valid. It concatenates them.

Question 11

Correct answer: True

Why: append() adds an item to the end of the list. It modifies the list in place and returns None.

Question 12

Correct answer: True

Why: Python lists can hold items of any type, including mixed types. For example: [42, "hello", True, 3.14] is a valid list with an int, a string, a bool, and a float.

Question 13

Model answer:

The main difference between a list and a tuple is mutability. A list is mutable: you can change its items, add new items, and remove items after it is created. A tuple is immutable: once created, it cannot be changed.

Use a tuple when you have a small, fixed collection of related values that should not change. Common examples include: - Coordinates: point = (3, 4) - RGB colours: orange = (255, 128, 0) - Dates: birthday = (15, 3, 2009)

Using a tuple here signals to anyone reading the code that these values are meant to stay together and should not be modified.

Use a list when you need a collection that can grow, shrink, or be modified. Common examples include a shopping list (you add and remove items), a list of student names (new students may join), or a sequence of quiz questions (you might add more later). Lists are the right choice whenever the data changes during the program.

Question 14

Model answer:

lst.append(5) modifies the list IN PLACE. It adds 5 to the end of the existing list and returns None. The original list object is changed; no new list is created. This is more memory-efficient.

lst = lst + [5] creates a BRAND NEW list with all the old items plus 5, and then reassigns the variable lst to point to this new list. The old list is unchanged (and may be garbage-collected if nothing else refers to it). This uses more memory and is slower.

The first form (append) is usually what you want, because it is faster and does not create extra copies. However, if other variables refer to the original list and you do NOT want them to see the change, the second form (+) is safer because it leaves the original list untouched.

Question 15

Model answer:

Explanation: Create a list of three favourite foods, then loop over it with a for loop. Each item is printed with "Love:" in front.

PYTHON · model_answer.py

```
foods = ["pizza", "sushi", "tacos"]
for food in foods:
    print("Love:", food)
```

Chapter 9 — Dictionaries and Sets

Question 1

Correct answer: A. { }

Why: An empty dictionary is created with empty curly braces: {}. (Note: empty curly braces create a dict, not a set. To create an empty set, use set().)

Why the other options are wrong:

B. [ ] — Square brackets create an empty list, not a dictionary.

C. ( ) — Parentheses create an empty tuple, not a dictionary.

D. dict[] — This is not valid syntax. The correct way is {} or dict().

Question 2

Correct answer: B. Looks up the value associated with the key "age"

Why: Square brackets on a dictionary look up the value for the given key. student["age"] returns the value stored under the key "age".

Why the other options are wrong:

A. Adds a new key called age — Adding a key uses assignment: student["age"] = 14. Just reading student["age"] does not add anything.

C. Removes the key "age" — Removing a key uses del or .pop(). Just reading does not remove.

D. Returns the index of "age" — Dictionaries do not have indices; they have keys. Square brackets look up by key.

Question 3

Correct answer: C. None

Why: get() returns None if the key is not found, instead of raising an error. You can also provide a default: d.get("missing", "default").

Why the other options are wrong:

- **A.** An error — This is what happens with square brackets (d["missing"]). get() is the safe way that returns None.
- **B.** The string "missing" — get() does not return the key you looked up. It returns None or your custom default.
- **D.** False — get() returns None (a special value), not the boolean False.

Question 4

Correct answer: A. [1, 2, 3}

Why: Sets automatically remove duplicates. set([1, 1, 2, 2, 3]) keeps only unique items, giving {1, 2, 3}.

Why the other options are wrong:

- **B.** {1, 2, 3] — This is a list, not a set. set() returns a set, which uses curly braces.
- **C.** {1, 1, 2, 2, 3} — Sets cannot contain duplicates. The repeats are removed.
- **D.** [3, 2, 1] — Sets are unordered, and the result is a set with curly braces, not a list.

Question 5

Correct answer: D. [1, 2]

Why: Dictionary keys must be immutable. Lists are mutable, so they cannot be used as keys. Strings, numbers, and tuples can.

Why the other options are wrong:

- **A.** "name" — Strings are immutable, so they can be keys.
- **B.** 42 — Numbers are immutable, so they can be keys.
- **C.** (1, 2) — Tuples are immutable, so they can be keys.
- **D.** [1, 2] — Lists are mutable, so they CANNOT be keys. This would raise a TypeError.

Question 6

Correct answer: True

Why: Dictionary keys must be unique. If you assign to a key that already exists, the old value is replaced with the new one.

Question 7

Correct answer: False

Why: Sets are UNORDERED. The items have no specific order, and the order may even change between Python versions. You cannot index a set like you can a list.

Why the other answer is wrong: Sets are unordered collections. If you need items in a specific order, use a list instead.

Question 8

Correct answer: True

Why: Dictionary values can be any type, including lists, other dictionaries, or any object. For example: d = {"scores": [90, 85, 92]} has a list as a value.

Question 9

Correct answer: D. An error

Why: len() on a dictionary returns the number of key-value pairs. The dictionary has 3 pairs: "a":1, "b":2, "c":3.

Why the other options are wrong:

A. 1 — This might be the value of one of the keys, but len() counts pairs.

C. 6 — This counts all keys and values separately, but len() counts pairs.

B. 3 — len() works on dictionaries and returns the number of pairs.

Question 10

Correct answer: D. An error

Why: The & operator is set INTERSECTION: it returns only the items that are in BOTH sets. {"a", "b"} & {"b", "c"} = {"b"}.

Why the other options are wrong:

A. {"a", "b", "c"} — This is the UNION (|), which gives all items from both sets.

C. {"a", "c"} — This is the SYMMETRIC DIFFERENCE, items in one set but not both. The operator for that is ^, not &.

B. {"b"} — The & operator on sets is perfectly valid.

Question 11

Correct answer: True

Why: Using square brackets on a missing key raises KeyError. For example, {"a": 1}["b"] raises KeyError: 'b'. To avoid this, use .get() instead.

Question 12

Correct answer: True

Why: Dictionary values can be any type, including another dictionary. This is called a nested dictionary. Example: student = {"name": "Alex", "address": {"city": "Cairo", "zip": "12345"}}.

Question 13

Model answer:

A phone book is a perfect real-world example where a dictionary is much better than two parallel lists.

With parallel lists, you would write: names = ["Alex", "Maya", "Sam"] numbers = ["555-1234", "555-5678", "555-9012"]

To look up a number, you have to find the index of the name in the first list, then use that index in the second list: i = names.index("Maya") print(numbers[i]) # "555-5678"

This is error-prone (what if the lists get out of sync?) and slow (searching a list takes time proportional to its length).

With a dictionary, the code is simpler and faster: phone = {"Alex": "555-1234", "Maya": "555-5678", "Sam": "555-9012"} print(phone["Maya"]) # "555-5678"

The lookup is a single, clear operation. It is also much faster for large phone books, because dictionaries use a technique called hashing that makes lookups nearly instant regardless of size.

Question 14

Model answer:

d.get("key") returns None (or a default you provide) if the key does not exist. It is the safe way to look up a key that might not be there.

d["key"] raises a KeyError if the key does not exist. This is fine when you are sure the key is present, but it will crash your program if the key is missing.

For example: d = {"name": "Alex"} print(d.get("email")) # None (no error) print(d.get("email", "n/a")) # "n/a" (custom default) print(d["email"]) # KeyError!

Use get() when you are not sure the key exists. Use square brackets when you are sure, or when a missing key would indicate a bug that should crash the program.

Question 15

Model answer:

Explanation: Create a dictionary mapping names to ages. Loop with .items() to get both keys and values at once.

PYTHON · model_answer.py

```
friends = {"Alex": 14, "Maya": 15, "Sam": 14}
for name, age in friends.items():
    print(f"{name} is {age} years old.")
```

Chapter 10 — Working with Strings

Question 1

Correct answer: D. An error

Why: upper() returns a new string with all letters converted to uppercase. "hello".upper() = "HELLO".

Why the other options are wrong:

A. Hello — This would be the output of capitalize(), which capitalises only the first letter.

C. hello — This is the original string. upper() returns a NEW uppercase string; it does not return the original.

B. HELLO — upper() is a valid string method. No error.

Question 2

Correct answer: A. ["a", "b", "c"]

Why: split(",") splits the string at each comma and returns a list of the pieces. "a,b,c".split(",") = ["a", "b", "c"].

Why the other options are wrong:

B. "abc" — This would require join(), not split(). split() returns a list.

C. ["abc"] — This would be the result if there were no commas to split on.

D. a b c — This is a string with spaces, not a list. split() returns a list.

Question 3

Correct answer: B. hi

Why: strip() removes leading and trailing whitespace (spaces, tabs, newlines) from a string. " hi ".strip() = "hi".

Why the other options are wrong:

A. hi — This is the original string. strip() returns a new string without the whitespace.

C. hi — This would be rstrip() (right strip only). strip() removes from both ends.

D. hi — This would be lstrip() (left strip only). strip() removes from both ends.

Question 4

Correct answer: B. nohtyP

Why: The slice [::-1] means "from end to start, step -1", which reverses the string. "Python"[::-1] = "nohtyP".

Why the other options are wrong:

A. Python — This is the original string. [::-1] reverses it.

C. Ptoh — This is not a meaningful slice of "Python".

D. An error — Negative steps are valid in Python slicing.

Question 5

Correct answer: B. immutable

Why: Strings are immutable: once created, they cannot be changed. Methods like upper() return NEW strings; they do not modify the original.

Why the other options are wrong:

A. Mutable — Lists and dictionaries are mutable. Strings are not.

C. Sometimes mutable — Strings are always immutable in Python. There is no exception.

D. Only mutable inside functions — Mutability is a property of the type, not the location. Strings are always immutable.

Question 6

Correct answer: True

Why: The "in" operator checks for substrings. "cat" appears at the start of "catalogue", so "cat" in "catalogue" is True.

Question 7

Correct answer: False

Why: Strings are immutable. name.upper() returns a NEW uppercase string; it does NOT change name. To change name, you must reassign: name = name.upper().

Why the other answer is wrong: upper() returns a new string. The original variable is unchanged unless you capture the result and reassign.

Question 8

Correct answer: True

Why: Because strings are immutable, methods like replace() cannot modify the original. They always return a new string with the changes applied.

Question 9

Correct answer: B. ["hello", "world"]

Why: split() with no argument splits on any whitespace (spaces, tabs, newlines) and removes empty strings. "hello world".split() = ["hello", "world"].

Why the other options are wrong:

- **A.** "hello world" — split() returns a list, not a string.
- **C.** ["h", "e", "l", "l", "o"] — This would be list("hello"), which splits into individual characters.
- **D.** "helloworld" — This would require replace(" ", ""), not split().

Question 10

Correct answer: A. a-b-c

Why: join() takes the separator (the string you call it on) and glues the list items together with that separator. "-".join(["a", "b", "c"]) = "a-b-c".

Why the other options are wrong:

- **B.** abc- — join() puts the separator BETWEEN items, not after each.
- **C.** ["a-b-c"] — join() returns a string, not a list.
- **D.** a b c — This would require " ".join(...), not "-".join(...).

Question 11

Correct answer: True

Why: Strings are zero-indexed. "hello"[0] = "h", "hello"[1] = "e", and so on. So "hello"[1] is "e".

Question 12

Correct answer: True

Why: startswith() returns True if the string begins with the given prefix, False otherwise. So it always returns a boolean.

Question 13

Model answer:

Strings in Python are immutable, which means once a string is created, it can never be changed. You cannot modify individual characters, and you cannot append to a string.

This is why methods like upper(), lower(), replace(), and strip() do not modify the original string. Instead, they return a brand new string with the changes applied. The original stays exactly as it was. For example:

name = "alex" name.upper() # returns "ALEX" but does NOT change name print(name) # still "alex" name = name.upper() # now name is "ALEX" (reassigned)

This matters because it prevents a class of bugs: if two variables refer to the same string, you cannot accidentally change it for one of them. Strings are safe to share.

When you need to "modify" a string, you build a new one based on the old one and assign it back to the variable. The old string is garbage-collected when no longer needed.

Question 14

Model answer:

split() and join() are opposites. split() breaks a string into a list of smaller strings. join() glues a list of strings back into a single string.

split() example: sentence = "apple,banana,cherry" words = sentence.split(",") print(words) # ["apple", "banana", "cherry"]

join() example: words = ["apple", "banana", "cherry"] sentence = ", ".join(words) print(sentence) # "apple, banana, cherry"

Note the syntax difference: split() is called on the string you want to split (with the separator as an argument), while join() is called on the separator string (with the list as an argument). This asymmetry catches many beginners.

Question 15

Model answer:

Explanation: Use upper() to convert to uppercase, split() to count words (len of the resulting list), and [::-1] to reverse the string.

PYTHON · model_answer.py

```
s = input("Enter a sentence: ")
print(s.upper()) # (a) all uppercase
print(len(s.split())) # (b) number of words
print(s[::-1]) # (c) reversed
```

Chapter 11 — File Handling

Question 1

Correct answer: B. "w"

Why: Mode "w" opens the file for writing. If the file already exists, its contents are ERASED. If it does not exist, a new file is created.

Why the other options are wrong:

A. "r" — Mode "r" opens for READING. You cannot write to a file opened in "r" mode.

C. "a" — Mode "a" opens for APPENDING. It keeps the existing contents and adds new content to the end. It does not erase.

D. "x" — Mode "x" is for exclusive creation. It fails if the file already exists. It does not erase an existing file.

Question 2

Correct answer: C. "a"

Why: Mode "a" opens the file for appending. The existing contents are kept, and new content is added to the end.

Why the other options are wrong:

A. "r" — Mode "r" is for reading only. You cannot add anything.

B. "w" — Mode "w" ERASES the file before writing. It does not keep existing content.

D. "i" — There is no "i" mode in Python.

Question 3

Correct answer: B. Automatically closes the file when the block ends

Why: The with statement ensures the file is closed automatically when the block ends, even if an error occurs. This is the safest way to work with files.

Why the other options are wrong:

A. Runs a block of code multiple times — This describes a loop (for or while), not the with statement.

C. Makes the file read-only — The mode argument determines read/write access, not the with statement.

D. Creates a new file — Creating a file depends on the mode ("w" or "x"). with itself does not create files.

Question 4

Correct answer: B. a newline

Why: The \n character is a special escape sequence that represents a newline. When printed or written to a file, it moves the cursor to the next line.

Why the other options are wrong:

A. A backslash followed by n — Although it looks like two characters in the source code, Python interprets \n as a single newline character.

C. The number 0 — \n has nothing to do with the number 0.

D. An error — \n is a valid escape sequence. No error.

Question 5

Correct answer: C. f.read()

Why: f.read() (with no arguments) reads the entire file and returns it as a single string.

Why the other options are wrong:

A. f.readlines() — readlines() returns a LIST of lines, not a single string.

B. f.readline() — readline() reads ONE line (the next one), not the whole file.

D. f.readall() — There is no readall() method on file objects in Python.

Question 6

Correct answer: True

Why: You should always close files when you are done with them. If you forget, data you wrote might not be saved, because Python keeps some data in a buffer in memory.

Question 7

Correct answer: True

Why: One of the main benefits of the with statement is that it closes the file automatically, even if an error occurs inside the block. This is why it is the recommended way to work with files.

Question 8

Correct answer: False

Why: Mode "w" ERASES the existing contents before writing. If you want to keep the existing contents and add to them, use mode "a" (append) instead.

Why the other answer is wrong: "w" overwrites; "a" appends. This is one of the most common and dangerous mistakes with files.

Question 9

Correct answer: C. "a"

Why: Mode "a" (append) opens the file for writing at the end. Existing contents are kept; new content is added after them.

Why the other options are wrong:

A. "r" — Mode "r" is for reading. You cannot write to a file opened in "r" mode.

B. "w" — Mode "w" erases the existing contents. You would lose what was there before.

D. "x" — Mode "x" fails if the file already exists. It is for creating new files only.

Question 10

Correct answer: B. Reads the file one line at a time, looping over each

Why: When you use "for line in f:", Python reads one line from the file on each iteration. This is memory-efficient because only one line is in memory at a time.

Why the other options are wrong:

A. Reads the whole file into one big string — This would be f.read(). The for loop reads one line at a time.

C. Closes the file automatically — You still need to close the file (or use a with statement). The for loop does not close it.

D. Raises an error — Iterating over a file with a for loop is perfectly valid and very common.

Question 11

Correct answer: True

Why: Files always store text. When you read a number back from a file, it comes in as a string. You must convert it with int() or float() before doing arithmetic with it.

Question 12

Correct answer: True

Why: The two-character sequence \n is interpreted by Python as a single newline character. It moves the cursor to the next line when printed or written to a file.

Question 13

Model answer:

The three file modes "r", "w", and "a" differ in what they do to the file and what they allow you to do:

"r" (read): Opens the file for reading. You can read the contents but cannot write to it. The file MUST already exist; otherwise Python raises a FileNotFoundError.

"w" (write): Opens the file for writing. If the file already exists, its contents are ERASED. If it does not exist, a new empty file is created. Be careful: opening a file in "w" mode destroys whatever was there before.

"a" (append): Opens the file for appending. The existing contents are kept, and anything you write is added to the end. If the file does not exist, a new one is created (just like "w").

A common mistake is using "w" when you mean "a". If you want to add to a file without losing what is already there, always use "a".

Question 14

Model answer:

The with statement is safer than using open() and close() manually for two reasons.

First, it closes the file automatically when the block ends. You do not have to remember to call close(), and there is no risk of forgetting. Forgetting to close a file can cause data loss, because Python keeps some data in a buffer in memory and only writes it to disk when the buffer is full or the file is closed.

Second, the with statement closes the file EVEN IF AN ERROR OCCURS inside the block. If you do it manually and an exception is raised before you reach close(), the file stays open. With the with statement, Python guarantees the file is closed no matter what.

For these reasons, the with statement is the recommended way to work with files in Python.

Question 15

Model answer:

Explanation: Open the file in "a" (append) mode so existing visitors are kept. Write the name followed by a newline character.

PYTHON · model_answer.py

```
name = input("Your name? ")
with open("visitors.txt", "a") as f:
    f.write(name + "\n")
```

Chapter 12 — Errors and Exceptions

Question 1

Correct answer: B. ValueError

Why: int("hello") raises a ValueError because "hello" is a string that cannot be interpreted as an integer. ValueError is raised when the type is right but the value is inappropriate.

Why the other options are wrong:

A. TypeError — TypeError is raised when the TYPE is wrong, e.g. trying to add a string and an int. Here, int() expects a string, so the type is correct; the value is the problem.

C. NameError — NameError is raised when you use a variable that does not exist. "hello" is a string literal, not a variable.

D. SyntaxError — SyntaxError is raised when your code breaks Python's grammar rules. int("hello") is syntactically valid; the error happens at runtime.

Question 2

Correct answer: A. ZeroDivisionError

Why: Dividing any number by zero raises ZeroDivisionError. This is always illegal in Python (and in mathematics).

Why the other options are wrong:

B. ValueError — ValueError is for wrong values, like int("hello"). Division by zero is a different kind of error.

C. DivisionError — There is no DivisionError in Python. The correct name is ZeroDivisionError.

D. MathError — There is no MathError in Python.

Question 3

Correct answer: C. except

Why: The except keyword starts a block that catches (handles) an exception. The block runs when an error occurs in the corresponding try block.

Why the other options are wrong:

A. catch — Some languages (like Java and JavaScript) use "catch". Python uses "except".

B. handle — "handle" is not a Python keyword.

D. rescue — Ruby uses "rescue". Python uses "except".

Question 4

Correct answer: C. Runs no matter what

Why: The finally block normally runs whether or not an exception occurred, was caught, or was not caught. It is perfect for cleanup code like closing files. (It is not guaranteed to run if the program is terminated abruptly.)

Why the other options are wrong:

A. Runs only if no error occurred — This describes the "else" block of a try statement.

B. Runs only if an error occurred — finally runs in both cases. That is its purpose.

D. Ends the program — finally does not end the program. After it runs, execution continues normally (or the exception propagates if it was not caught).

Question 5

Correct answer: B. raise

Why: The raise keyword throws an exception manually. For example: raise ValueError("That is not allowed").

Why the other options are wrong:

A. throw — Some languages (like Java and JavaScript) use "throw". Python uses "raise".

C. error — "error" is not a Python keyword.

D. fail — "fail" is not a Python keyword.

Question 6

Correct answer: True

Why: A bare except: (with no exception type) catches EVERY kind of exception. This is generally discouraged because it can hide bugs and make debugging very difficult.

Question 7

Correct answer: True

Why: The else block of a try statement runs ONLY when no exception occurred in the try block. It is useful for code that should only run on success.

Question 8

Correct answer: True

Why: SyntaxError means your code breaks Python's grammar rules, like a missing colon, unmatched parentheses, or a misspelled keyword. These errors are caught when Python compiles your code, before it even starts running.

Question 9

Correct answer: B. IndexError

Why: [1, 2] has only 2 items (indices 0 and 1). Trying to access index 5 is out of range, so Python raises IndexError.

Why the other options are wrong:

A. ValueError — ValueError is for wrong values, like int("hello"). Index out of range is a different kind of error.

C. KeyError — KeyError is raised when a dictionary key is missing, not when a list index is out of range.

D. TypeError — TypeError is for wrong types. The index 5 is a valid integer; it is just out of range.

Question 10

Correct answer: C. KeyError

Why: Looking up a key that does not exist in a dictionary (with square brackets) raises KeyError. Use .get() to avoid this.

Why the other options are wrong:

A. ValueError — ValueError is for wrong values, not for missing keys.

B. IndexError — IndexError is for list indices out of range, not for dictionary keys.

D. AttributeError — AttributeError is raised when you try to access an attribute or method that does not exist on an object.

Question 11

Correct answer: True

Why: The finally block normally runs whether or not an exception occurred, was caught, or was not caught. This is why it is the right place for cleanup code like closing files or releasing resources. (It is not guaranteed to run if the program is terminated abruptly.)

Question 12

Correct answer: True

Why: Once an exception is caught by an except block, the program continues normally with the code after the try/except. The exception does not crash the program. This is the whole point of exception handling.

Question 13

Model answer:

A bare except: catches every kind of exception, including ones you did not anticipate. This is dangerous for several reasons.

First, it hides bugs. If your program has a typo that causes a NameError or AttributeError, a bare except will silently catch it and continue. You will not know anything went wrong, but your program will behave strangely. Debugging becomes a nightmare because the real cause is hidden.

Second, it catches exceptions that should not be caught, like KeyboardInterrupt (raised when the user presses Ctrl+C to stop the program). Catching these makes your program impossible to interrupt.

When you catch specific exception types (like except ValueError:), only the errors you expect are caught. Unexpected errors still propagate up and stop the program, which is what you want: you can then fix the bug, and the program will work correctly.

Question 14

Model answer:

The else and finally blocks serve different purposes in a try statement.

The else block runs ONLY when no exception occurred in the try block. It is useful for code that should only run on success. For example, if you try to open a file and read its contents, the else block might process the contents — you only want to process them if the read succeeded.

The finally block normally runs whether or not an exception occurred, was caught, or was not caught. It is for cleanup code like closing a file or releasing a network connection. It is not guaranteed to run if the program is terminated abruptly, but in normal programs it executes in all cases.

In short: else is for "happy path" code that depends on success; finally is for cleanup code that normally runs whether or not an error occurred.

Question 15

Model answer:

Explanation: Use math.sqrt() to compute the square root. Wrap the input and sqrt call in a try block. If the user enters a negative number, raise a ValueError (or check for negative first). The except block prints a friendly message instead of crashing.

PYTHON · model_answer.py

```
import math

try:
    n = float(input("Enter a number: "))
    if n < 0:
        raise ValueError("Cannot take the square root of a negative number.")
    print(f"The square root of {n} is {math.sqrt(n)}")
except ValueError as e:
    print(f"Error: {e}")
```

○ ABOUT THIS TEXTBOOK

Learn Python the Right Way

A COMPLETE FOUNDATION IN PROGRAMMING

01 Lessons with lots of code examples

02 Mixed quizzes after every chapter

03 Chapter recaps and common mistakes

04 A full glossary of programming terms

05 A complete final project to build yourself

06 Fully explained answer key with reasoning

“The only way to learn a new programming language is by writing programs in it.”

— DENNIS RITCHIE

PYTHON 3 · STUDENT EDITION · 2026

DIRECTED BY
Mohamed Ahmed Fayad